

LỜI CAM ĐOAN

Chúng tôi cam đoan đồ án chuyên ngành với đề tài “Xây dựng hệ thống SMAP – Social Media Analytics Platform” là công trình nghiên cứu độc lập của nhóm tác giả, được thực hiện dưới sự hướng dẫn của thầy Lê Đình Thuận. Toàn bộ nội dung nghiên cứu và các kết quả trình bày trong đồ án đều trung thực, là công sức của chính nhóm thực hiện và chưa từng được công bố trong bất kỳ công trình nào tại các cơ sở đào tạo khác. Mọi số liệu, bảng biểu và thông tin tham khảo từ các nguồn bên ngoài đều được trích dẫn rõ ràng, đầy đủ theo đúng quy định về sở hữu trí tuệ. Chúng tôi xin hoàn toàn chịu trách nhiệm trước hội đồng và nhà trường nếu phát hiện có bất kỳ hành vi gian lận hoặc sao chép không hợp lệ nào liên quan đến đồ án này.

Thành phố Hồ Chí Minh, ngày 21 tháng 12 năm 2025

Sinh viên thực hiện

Nguyễn Tấn Tài

Đặng Quốc Phong

Nguyễn Chánh Tín



LỜI CẢM ƠN

Trước tiên, chúng tôi xin gửi lời cảm ơn chân thành đến quý thầy/cô trong Khoa Khoa học Và Kỹ thuật Máy tính, Trường Đại học Bách Khoa - Đại học Quốc gia TP.HCM, những người đã tận tình truyền đạt kiến thức và tạo điều kiện thuận lợi cho em trong suốt quá trình học tập.

Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc đến thầy Lê Đình Thuận, người đã hướng dẫn, động viên và đóng góp nhiều ý kiến quý báu trong suốt quá trình thực hiện đề tài này. Sự chỉ bảo tận tâm của thầy đã giúp chúng em hoàn thành tốt báo cáo này.

Em xin chân thành cảm ơn!

TÓM TẮT ĐỀ TÀI

Đề tài tập trung xây dựng hệ thống SMAP - Social Media Analytics Platform nhằm hỗ trợ phân tích dữ liệu mạng xã hội phục vụ hoạt động marketing. Trọng tâm của đề án không nằm ở việc nghiên cứu thuật toán học máy chuyên sâu, mà tập trung vào bài toán thiết kế kiến trúc phần mềm, tổ chức luồng xử lý dữ liệu và đảm bảo khả năng vận hành ổn định trong bối cảnh dữ liệu lớn và thay đổi liên tục.

Hệ thống được định hướng như một nền tảng phục vụ nhu cầu vận hành và khai thác thông tin trong phạm vi nội bộ, hỗ trợ thu thập, xử lý, phân tích và tổng hợp dữ liệu từ các nguồn mạng xã hội nhằm cung cấp thông tin có giá trị cho việc theo dõi thương hiệu, nhận diện xu hướng và hỗ trợ ra quyết định. Trên cơ sở đó, đề án tập trung vào việc phân tích yêu cầu, thiết kế kiến trúc và hiện thực các thành phần kỹ thuật chính của hệ thống theo hướng phân tách nhiều dịch vụ với các vai trò chuyên biệt như quản lý ngữ cảnh nghiệp vụ, điều phối thu thập dữ liệu, xử lý phân tích, truy hồi tri thức và thông báo thời gian thực.

Kết quả đạt được là một hệ thống có cấu trúc rõ ràng, có giao diện web được triển khai tại `smap.tantai.dev` để phục vụ kiểm thử và khai thác trên môi trường triển khai thực tế. Đồng thời, báo cáo cũng hệ thống hóa các quyết định thiết kế quan trọng, các thành phần kỹ thuật cốt lõi và các luồng xử lý dữ liệu chính trong hệ thống, tạo nền tảng thuận lợi cho việc phát triển, bảo trì và mở rộng trong các giai đoạn tiếp theo.

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN	1
1.1 Giới thiệu đề tài	1
1.2 Mục tiêu đề tài	1
1.2.1 Mục tiêu tổng quát	1
1.2.2 Mục tiêu cụ thể	1
1.3 Phạm vi đề tài	2
1.3.1 Phạm vi ứng dụng	2
1.3.2 Phạm vi dữ liệu	2
1.3.3 Phạm vi kỹ thuật	2
1.4 Cấu trúc báo cáo	3
CHƯƠNG 2: CÁC HỆ THỐNG LIÊN QUAN	4
2.1 Tổng quan thị trường	4
2.2 Các hệ thống, dịch vụ tại Việt Nam	4
2.2.1 YouNet Group	4
2.2.1.1 Buzzmetrics	4
2.2.1.2 YouNet Media	5
2.2.2 Reputa	6
2.2.3 Kompa	6
2.2.4 Commsights / Andi	7
2.2.5 Nhận xét chung về các hệ thống tại Việt Nam	7
2.3 Các hệ thống, dịch vụ toàn cầu	8
2.3.1 Talkwalker	8
2.3.2 Brandwatch	9
2.3.3 Sprinklr	9
2.3.4 YouScan	10
2.3.5 Meltwater	10
2.3.6 Nhận xét chung về các hệ thống toàn cầu	11
2.4 Ma trận so sánh tính năng	11
2.4.1 Tiêu chí so sánh	11
2.4.2 Bảng so sánh tổng hợp	11
2.4.3 Nhận xét từ ma trận so sánh	13
CHƯƠNG 3: CƠ SỞ KỸ THUẬT VÀ CÔNG NGHỆ SỬ DỤNG	14
3.1 Kiến trúc Microservices	14

3.1.1 Định nghĩa và nguồn gốc	14
3.1.2 Đặc điểm chính	14
3.1.2.1 Loose Coupling	14
3.1.2.2 Triển khai độc lập	14
3.1.2.3 Tổ chức theo khả năng nghiệp vụ	15
3.1.2.4 Quản lý dữ liệu phi tập trung	15
3.1.2.5 Tự động hóa hạ tầng	15
3.1.3 So sánh với kiến trúc Monolithic	15
3.2 Clean Architecture	16
3.2.1 Nguyên tắc thiết kế	16
3.2.2 Các tầng và Dependency Rule	16
3.2.2.1 Entities	17
3.2.2.2 Use Cases	17
3.2.2.3 Interface Adapters	17
3.2.2.4 Frameworks and Drivers	17
3.2.2.5 Dependency Rule	17
3.2.3 So sánh với MVC và Hexagonal Architecture	17
3.3 Message Queue và Event-Driven Architecture	18
3.3.1 Khái niệm Message Queue	18
3.3.2 RabbitMQ	19
3.3.3 Apache Kafka	19
3.3.4 Producer-Consumer Pattern	20
3.3.5 Publish/Subscribe và Event-Driven Architecture	20
3.3.6 Delivery Semantics và các lưu ý thiết kế	20
3.3.7 So sánh các Message Brokers	21
3.4 Real-time Communication	21
3.4.1 WebSocket Protocol	21
3.4.2 Redis Pub/Sub	22
3.4.3 So sánh các Real-time Protocols	23
3.5 Data Storage Technologies	24
3.5.1 PostgreSQL	24
3.5.2 Redis	24
3.5.3 Qdrant	25
3.5.4 MinIO	25
3.5.5 Polyglot Persistence	25

3.6 Authentication và Security	26
3.6.1 JWT	26
3.6.2 HttpOnly Cookie	27
3.6.3 OAuth2	27
3.6.4 Security Practices	28
3.7 Công nghệ Backend	28
3.7.1 Go và Gin Framework	28
3.7.2 SQLBoiler	29
3.7.3 Python và FastAPI	30
3.8 Cơ sở kỹ thuật AI, NLP và khai thác tri thức	30
3.8.1 Xử lý ngôn ngữ tự nhiên trong social listening	30
3.8.2 Tiền xử lý văn bản tiếng Việt	31
3.8.3 Phân tích cảm xúc và mô hình ngôn ngữ tiếng Việt	31
3.8.4 Trích xuất từ khóa, thực thể và khía cạnh	32
3.8.5 Chỉ số ảnh hưởng, rủi ro và khả năng giải thích	32
3.8.6 Embedding, vector search và cơ sở dữ liệu vector	33
3.8.7 Retrieval-Augmented Generation và mô hình ngôn ngữ lớn	33
3.8.8 Giới hạn của AI trong phân tích dữ liệu mạng xã hội	34
3.9 Công nghệ Frontend	34
3.9.1 Next.js	34
3.9.2 React	35
3.9.3 TypeScript	35
3.9.4 Tailwind CSS	36
3.9.5 Electron và khả năng đóng gói desktop	36
3.9.6 Tích hợp analytics API và dashboard	37
3.10 Containerization và Orchestration	37
3.10.1 Docker	37
3.10.2 Docker Compose	37
3.10.3 Kubernetes	38
3.10.4 Auto Scaling và Self-Healing	38
3.11 Tổng kết	39
CHƯƠNG 4: PHÂN TÍCH HỆ THỐNG	40
4.1 Nhóm người dùng và tác nhân hệ thống	40
4.2 Yêu cầu chức năng	41
4.3 Yêu cầu phi chức năng	43

4.3.1	Đặc tính kiến trúc	43
4.3.2	Thuộc tính chất lượng	45
4.3.3	Nhận xét và liên hệ với SMAP	46
4.4	Use Case	46
4.4.1	UC-01: Thiết lập chiến dịch theo dõi	48
4.4.2	UC-02: Vận hành chiến dịch theo dõi	52
4.4.3	UC-03: Tra cứu và hỏi đáp dữ liệu phân tích	55
4.4.4	UC-04: Theo dõi trạng thái và nhận cảnh báo	58
4.4.5	UC-05: Thiết lập và quản lý quy tắc cảnh báo khủng hoảng	60
4.5	Sơ đồ hoạt động	63
4.5.1	Quy trình nghiệp vụ tổng quát	63
4.5.2	UC-01: Thiết lập chiến dịch theo dõi	65
4.5.3	UC-02: Vận hành chiến dịch theo dõi	65
4.5.4	UC-03: Tra cứu và hỏi đáp dữ liệu phân tích	68
4.5.5	UC-04: Theo dõi trạng thái và nhận cảnh báo	69
4.5.6	UC-05: Thiết lập và quản lý quy tắc cảnh báo khủng hoảng	70
CHƯƠNG 5: THIẾT KẾ HỆ THỐNG		72
5.1	Phương pháp tiếp cận	72
5.1.1	Architectural Drivers	72
5.1.2	Nguyên tắc thiết kế	73
5.1.3	Liên hệ với SMAP	73
5.2	Kiến trúc tổng thể	74
5.2.1	Lựa chọn phong cách kiến trúc	74
5.2.1.1	Bối cảnh quyết định	74
5.2.1.2	Đánh giá các lựa chọn	74
5.2.1.3	Kết luận lựa chọn kiến trúc	75
5.2.2	Kiến trúc tổng thể	75
5.2.3	Service Ownership và ranh giới trách nhiệm	76
5.2.4	Technology Stack và các lựa chọn hạ tầng	77
5.2.4.1	Backend và frontend	77
5.2.4.2	Data storage	78
5.2.4.3	Transport và orchestration	78
5.2.5	Tổng kết mục	79
5.3	Thiết kế chi tiết các dịch vụ	79
5.3.1	Identity Service	79

5.3.1.1 Component Diagram - C4 Level 3	80
5.3.1.2 Component Catalog	80
5.3.1.3 Data Flow	81
5.3.1.4 Design Patterns áp dụng	83
5.3.1.5 Key Decisions	84
5.3.1.6 Dependencies	84
5.3.2 Analytics Service	84
5.3.2.1 Component Diagram - C4 Level 3	85
5.3.2.2 Component Catalog	86
5.3.2.3 Data Flow	87
5.3.2.4 Design Patterns áp dụng	91
5.3.2.5 Key Decisions	91
5.3.2.6 Dependencies	91
5.3.3 Project Service	92
5.3.3.1 Component Diagram - C4 Level 3	92
5.3.3.2 Component Catalog	93
5.3.3.3 Data Flow	94
5.3.3.4 Design Patterns áp dụng	98
5.3.3.5 Key Decisions	98
5.3.3.6 Dependencies	98
5.3.4 Ingest Service	98
5.3.4.1 Component Diagram - C4 Level 3	99
5.3.4.2 Data Flow	100
5.3.4.3 Design Patterns áp dụng	104
5.3.4.4 Key Decisions	104
5.3.4.5 Dependencies	104
5.3.5 Notification Service	104
5.3.5.1 Component Diagram - C4 Level 3	105
5.3.5.2 Data Flow	106
5.3.5.3 Design Patterns áp dụng	108
5.3.5.4 Key Decisions	109
5.3.5.5 Dependencies	109
5.3.6 Knowledge Service	109
5.3.6.1 Component Diagram - C4 Level 3	110
5.3.6.2 Data Flow	111

5.3.6.3 Design Patterns áp dụng	118
5.3.6.4 Key Decisions	118
5.3.6.5 Dependencies	119
5.3.7 Scrapper Worker Service	119
5.3.7.1 Component Diagram - C4 Level 3	121
5.3.7.2 Data Flow	122
5.3.7.3 Design Patterns áp dụng	124
5.3.7.4 Key Decisions	124
5.3.7.5 Dependencies	124
5.3.8 Frontend Application	125
5.3.8.1 Component Diagram - C4 Level 3	126
5.3.8.2 Data Flow	127
5.3.8.3 Design Patterns áp dụng	131
5.3.8.4 Key Decisions	132
5.3.8.5 Dependencies	132
5.4 Thiết kế cơ sở dữ liệu	133
5.4.1 Chiến lược lựa chọn Storage	133
5.4.1.1 Tiêu chí đánh giá	133
5.4.1.2 So sánh các phương án lưu trữ	133
5.4.1.3 Tổng quan Storage Stack	135
5.4.1.4 Lý do lựa chọn	136
5.4.1.5 Redis TTL và data retention	136
5.4.2 Identity Service	137
5.4.2.1 Database Schema	138
5.4.2.2 Table Catalog	138
5.4.2.3 Design Decisions	139
5.4.3 Project Service	139
5.4.3.1 Database Schema	139
5.4.3.2 Table Catalog	139
5.4.3.3 Relationship và Design Decisions	140
5.4.4 Analytics Service	141
5.4.4.1 Database Schema	141
5.4.4.2 Table Catalog	141
5.4.4.3 Data Shape và Design Decisions	142
5.4.5 Ingest Service	142

5.4.5.1 Database Schema	143
5.4.5.2 Table Catalog	143
5.4.5.3 Lineage Model và Design Decisions	144
5.4.6 Knowledge Service	144
5.4.6.1 Database Schema	145
5.4.6.2 Table Catalog	145
5.4.6.3 Relational Metadata, Vector Store và Design Decisions	146
5.4.7 Data Management Patterns	146
5.4.7.1 Database per Service và External Reference IDs	146
5.4.7.2 Artifact Reference Pattern	146
5.4.7.3 Vector and Metadata Split	146
5.4.7.4 Async Operational Tracking	147
5.4.8 Tổng kết	147
5.5 Sơ đồ tuần tự	147
5.5.1 UC-01: Thiết lập chiến dịch theo dõi	148
5.5.1.1 Tạo campaign và project	148
5.5.1.2 Khai báo nguồn dữ liệu và mục tiêu thu thập	150
5.5.1.3 Kiểm tra thử và đánh giá mức sẵn sàng	151
5.5.2 UC-02: Vận hành chiến dịch theo dõi	152
5.5.2.1 Kiểm tra mức sẵn sàng và kích hoạt chiến dịch	152
5.5.2.2 Tạm dừng, tiếp tục, lưu trữ và mở lại chiến dịch	154
5.5.3 UC-03: Tra cứu và hỏi đáp dữ liệu phân tích	156
5.5.3.1 Tra cứu theo ngữ cảnh chiến dịch	156
5.5.3.2 Hỏi đáp có dẫn chứng và lưu hội thoại	158
5.5.4 UC-04: Theo dõi trạng thái và nhận cảnh báo	160
5.5.4.1 Quan sát trạng thái hiện tại	160
5.5.4.2 Nhận thông báo và cảnh báo	162
5.5.4.3 Supporting flow: phát hiện khủng hoảng từ dữ liệu crawl	163
5.5.5 UC-05: Thiết lập và quản lý quy tắc cảnh báo khủng hoảng	165
5.5.5.1 Xem và lưu bộ quy tắc cảnh báo khủng hoảng	165
5.5.5.2 Xóa cấu hình cảnh báo khủng hoảng	167
5.5.6 Tổng kết	169
5.6 Mẫu giao tiếp và tích hợp	169
5.6.1 Mẫu giao tiếp	169
5.6.1.1 Tổng quan các mẫu giao tiếp	170

5.6.1.2 Synchronous Request-Response Pattern	171
5.6.1.3 Asynchronous Messaging Pattern	172
5.6.1.4 Realtime Notification Delivery Pattern	172
5.6.1.5 Artifact Reference Pattern	173
5.6.2 RabbitMQ cho execution plane	173
5.6.2.1 Topology dispatch và completion	173
5.6.2.2 Delivery semantics và xử lý lỗi	175
5.6.3 Giao tiếp thời gian thực	175
5.6.3.1 Redis ingress và routing theo scope	176
5.6.3.2 Thiết lập kết nối và vòng đời phiên	177
5.6.3.3 Notification envelope và message families	177
5.6.3.4 Crisis runtime path và notification boundary	178
5.6.4 Giám sát hệ thống	179
5.6.4.1 Logging và ngữ cảnh chẩn đoán	179
5.6.4.2 Health và readiness probes	180
5.6.4.3 Metric instrumentation hiện có	181
5.6.4.4 Trace ID propagation	182
5.6.5 Tổng kết	182
5.7 Thiết kế triển khai hệ thống	183
5.7.1 Mô hình triển khai	183
5.7.2 Cấu hình phần cứng và phần mềm	184
5.7.3 Cấu hình mạng và bảo mật	184
5.7.4 Container Registry và Image Management	185
5.7.5 CI/CD Pipeline	185
5.7.6 Cấu hình Kubernetes Deployments	186
5.7.7 Infrastructure Services	189
5.7.8 Monitoring và Logging	189
5.7.9 Backup và Disaster Recovery	189
5.7.10 Quy trình triển khai	190
5.8 Truy xuất nguồn gốc và xác thực thiết kế	190
5.8.1 Ma trận truy xuất nguồn gốc	190
5.8.1.1 Truy xuất yêu cầu chức năng	190
5.8.1.2 Truy xuất đặc tính kiến trúc	192
5.8.1.3 Truy xuất yêu cầu phi chức năng	193
5.8.1.4 Tổng hợp coverage	194

5.8.2 Hồ sơ quyết định kiến trúc	195
ADR-001: Kiến trúc Microservices theo bounded context	195
ADR-002: Polyglot runtime với Go và Python	196
ADR-003: Chuyên biệt hóa cơ chế giao tiếp theo lane xử lý	196
ADR-004: Polyglot persistence với PostgreSQL, Redis, MinIO và Qdrant ..	197
ADR-005: Redis cho cache, state ngắn hạn và notification ingress	197
ADR-006: Kubernetes cho orchestration theo pod role	198
5.8.3 Phân tích khoảng trống	199
5.8.3.1 Khoảng trống kỹ thuật	199
5.8.3.2 Hạn chế đã biết	200
5.8.3.3 Đánh đổi kiến trúc có chủ đích	201
5.8.4 Tổng kết mục	202
CHƯƠNG 6: KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG	203
6.1 Mục tiêu kiểm thử	203
6.2 Công cụ và phương pháp kiểm thử	204
6.2.1 Công cụ kiểm thử đơn vị	204
6.2.2 Công cụ kiểm thử tích hợp và đầu cuối	205
6.2.3 Công cụ kiểm thử phi chức năng	206
6.3 Kiểm thử đơn vị theo dịch vụ	206
6.3.1 Tổng quan	206
6.3.2 ingest-srv	207
6.3.3 identity-srv	207
6.3.4 project-srv	208
6.3.5 notification-srv	208
6.3.6 analysis-srv	209
6.3.7 shared-lib	209
6.3.8 knowledge-srv, crawler worker và smap-analyse	210
6.4 Kiểm thử chức năng và đầu cuối	211
6.4.1 Phạm vi, môi trường và cách thực hiện	211
6.4.2 Kết quả kiểm thử health và API tổng hợp	211
6.4.3 UC-01: Thiết lập chiến dịch theo dõi	214
6.4.4 UC-02: Vận hành chiến dịch theo dõi	215
6.4.5 UC-03: Tra cứu và hỏi đáp dữ liệu phân tích	217
6.4.6 UC-04: Theo dõi trạng thái và nhận cảnh báo	218
6.4.7 UC-05: Thiết lập và quản lý quy tắc cảnh báo khủng hoảng	219

6.4.8 Crawler worker và notification	220
6.5 Đánh giá phi chức năng	221
6.5.1 Phạm vi và phương pháp đánh giá	221
6.5.2 Performance và stability	222
6.5.3 Availability và recovery	222
6.5.4 Resource và khả năng mở rộng	223
6.5.5 Security	224
6.5.6 Data integrity	224
6.5.7 Modularity và observability	225
6.6 Đối chiếu tổng hợp với yêu cầu và kiến trúc	225
6.7 Phạm vi diễn giải kết quả	227
6.8 Kết luận	228
CHƯƠNG 7: TỔNG KẾT	229
7.1 Tổng kết kết quả thực hiện	229
7.2 Đóng góp chính của đề tài	230
7.3 Hạn chế	230
7.4 Hướng phát triển	231
7.5 Kết luận	232
CHƯƠNG 8: TÀI LIỆU THAM KHẢO	233
CHƯƠNG 9: PHỤ LỤC	237

CHƯƠNG 1: TỔNG QUAN

1.1 Giới thiệu đề tài

Trong bối cảnh chuyển đổi số, dữ liệu mạng xã hội ngày càng trở thành một nguồn thông tin quan trọng cho hoạt động theo dõi thương hiệu, phân tích xu hướng và hỗ trợ ra quyết định. Tuy nhiên, giá trị của nguồn dữ liệu này không chỉ nằm ở việc thu thập số lượng lớn bài đăng hay bình luận, mà còn nằm ở khả năng chuẩn hóa, phân tích và chuyển hóa dữ liệu thành thông tin có thể sử dụng trong thực tế.

Các hệ thống social listening hiện đại vì vậy không còn dừng ở mức hiển thị biểu đồ hay thống kê đơn giản. Chúng cần hỗ trợ một chuỗi xử lý gồm thu thập dữ liệu từ nhiều nguồn, xử lý và phân tích nội dung, lưu trữ kết quả có cấu trúc, khai thác thông tin theo ngữ cảnh, cũng như cung cấp cơ chế thông báo kịp thời khi xuất hiện tín hiệu quan trọng.

Từ nhu cầu đó, SMAP được xây dựng như một hệ thống phân tích dữ liệu mạng xã hội phục vụ mục đích vận hành nội bộ. Hệ thống được tổ chức thành nhiều thành phần với vai trò chuyên biệt, hỗ trợ luồng xử lý từ ngữ cảnh nghiệp vụ, thu thập dữ liệu, phân tích, khai thác thông tin theo ngữ cảnh, đến thông báo thời gian thực. Đây cũng là cơ sở để luận văn tập trung vào việc phân tích kiến trúc và hiện thực của hệ thống thay vì chỉ mô tả bài toán ở mức ý tưởng.

1.2 Mục tiêu đề tài

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống phân tích dữ liệu mạng xã hội phục vụ nhu cầu vận hành nội bộ, có khả năng thu thập dữ liệu từ nhiều nguồn, chuẩn hóa và xử lý dữ liệu, từ đó hỗ trợ phân tích, tổng hợp thông tin và khai thác thông tin theo ngữ cảnh. Trọng tâm chính của đề án là kiến trúc phần mềm, tổ chức luồng dữ liệu và sự phối hợp giữa các thành phần trong hệ thống, thay vì đi sâu vào việc nghiên cứu hoặc đề xuất các mô hình học máy mới.

1.2.2 Mục tiêu cụ thể

Các mục tiêu cụ thể của đề tài bao gồm:

- Hỗ trợ thiết lập và quản lý các đối tượng phân tích cần thiết để làm cơ sở cho quá trình theo dõi và xử lý dữ liệu.
- Hỗ trợ thu thập và chuẩn bị dữ liệu từ các nguồn mạng xã hội theo quy trình có kiểm soát.
- Hỗ trợ xử lý và phân tích dữ liệu theo hướng bất đồng bộ để tạo ra các kết quả có cấu trúc phục vụ tra cứu, tổng hợp thông tin và hỗ trợ ra quyết định.

- Hỗ trợ tìm kiếm, tra cứu và khai thác thông tin theo ngữ cảnh từ dữ liệu và kết quả phân tích đã được xử lý.
- Hỗ trợ cập nhật thông tin và tín hiệu quan trọng theo hướng gần thời gian thực trong quá trình vận hành hệ thống.
- Xây dựng hệ thống theo hướng có cấu trúc rõ ràng, thuận lợi cho vận hành, bảo trì và nâng cấp sau này.

1.3 Phạm vi đề tài

1.3.1 Phạm vi ứng dụng

Hệ thống SMAP được xây dựng nhằm hỗ trợ nhu cầu theo dõi, phân tích và tổng hợp thông tin từ mạng xã hội trong phạm vi vận hành nội bộ. Người dùng đích của hệ thống là các nhóm nội bộ như chuyên viên phân tích dữ liệu truyền thông xã hội, bộ phận theo dõi thương hiệu, marketing, nghiên cứu thị trường, truyền thông và các cấp quản lý có nhu cầu khai thác thông tin để hỗ trợ ra quyết định.

Phạm vi ứng dụng của hệ thống tập trung vào việc hỗ trợ quan sát dữ liệu, nhận diện tín hiệu đáng chú ý, tổng hợp kết quả phân tích và cung cấp thông tin phục vụ công tác theo dõi, đánh giá và ra quyết định.

1.3.2 Phạm vi dữ liệu

Nguồn dữ liệu được sử dụng trong hệ thống bao gồm các bài đăng, bình luận và phản hồi công khai được thu thập từ các nền tảng mạng xã hội như TikTok, YouTube và Facebook.

Trong phạm vi của đề án, dữ liệu được xem là đầu vào phục vụ cho quá trình thu thập, xử lý và phân tích thông tin. Hệ thống tập trung vào việc tổ chức luồng xử lý và khai thác giá trị từ dữ liệu đầu vào, không đặt mục tiêu xác minh tính xác thực tuyệt đối của từng nội dung phát sinh trên các nền tảng mạng xã hội. Do chất lượng kết quả phân tích phụ thuộc trực tiếp vào chất lượng dữ liệu đầu vào, hệ thống không cam kết độ chính xác cao trong mọi trường hợp; các kết quả phân tích cần được hiểu theo nguyên tắc “garbage in, garbage out” và nên được sử dụng như nguồn hỗ trợ tham khảo cho quá trình đánh giá.

1.3.3 Phạm vi kỹ thuật

Về mặt kỹ thuật, đề án tập trung vào việc xây dựng một hệ thống phân tán gồm nhiều thành phần chuyên biệt, trong đó dữ liệu được thu thập từ các nguồn mạng xã hội, chuẩn hóa, xử lý phân tích và cung cấp cho người dùng thông qua các cơ chế lưu trữ, truyền tin và thông báo phù hợp. Phạm vi kỹ thuật của đề tài bao gồm tổ chức kiến trúc hệ thống, thiết kế luồng xử lý dữ liệu và làm rõ cách các thành phần trong hệ thống phối hợp với nhau thông qua cả cơ chế giao tiếp đồng bộ lẫn bất đồng bộ.

Đồ án không đi sâu vào việc nghiên cứu các mô hình học máy mới, tối ưu thuật toán AI hay chứng minh độ chính xác của mô hình ở mức học thuật. Thay vào đó, đồ án tập trung nghiên cứu, lựa chọn và sử dụng các công cụ, thư viện và mô hình phù hợp để giải quyết bài toán trong phạm vi hệ thống. Trọng tâm chính là xây dựng một hệ thống đạt đủ điều kiện phục vụ các luồng nghiệp vụ cốt lõi như thiết lập đối tượng theo dõi, thu thập dữ liệu, xử lý phân tích, khai thác thông tin theo ngữ cảnh và cung cấp thông tin phục vụ công tác theo dõi, đánh giá và ra quyết định.

Ngoài ra, đồ án cũng xem xét các yếu tố hỗ trợ triển khai và vận hành như container hóa, tổ chức môi trường thực thi và khả năng mở rộng của từng thành phần, nhưng không đặt mục tiêu mô tả chi tiết toàn bộ hạ tầng triển khai như một tài liệu vận hành hoàn chỉnh.

1.4 Cấu trúc báo cáo

Báo cáo được tổ chức theo trình tự từ bối cảnh bài toán, cơ sở lý thuyết, phân tích yêu cầu, thiết kế hệ thống, kiểm thử đánh giá đến tổng kết. Cách tổ chức này giúp người đọc trước hết nắm được lý do hình thành đề tài và phạm vi giải quyết, sau đó đi vào các quyết định kiến trúc, bằng chứng hiện thực và kết quả kiểm chứng.

Cấu trúc các chương như sau:

- Chương 1 trình bày tổng quan đề tài, mục tiêu, phạm vi và cấu trúc của báo cáo.
- Chương 2 trình bày cơ sở lý thuyết và các nền tảng kỹ thuật liên quan đến hệ thống social listening, xử lý dữ liệu và kiến trúc phần mềm.
- Chương 3 phân tích bài toán, đối tượng sử dụng, yêu cầu chức năng và yêu cầu phi chức năng của hệ thống.
- Chương 4 mô tả phân tích hệ thống ở mức nghiệp vụ, bao gồm tác nhân, use case và các sơ đồ hoạt động chính.
- Chương 5 trình bày thiết kế hệ thống, kiến trúc dịch vụ, luồng dữ liệu, cơ chế giao tiếp và phương án triển khai.
- Chương 6 trình bày quá trình kiểm thử và đánh giá hệ thống dựa trên unit test, kiểm thử đầu cuối, kiểm thử giao diện và các chỉ số phi chức năng trong phạm vi đã đo.
- Chương 7 tổng kết kết quả đạt được, đóng góp chính, hạn chế và hướng phát triển tiếp theo.
- Chương 8 liệt kê các tài liệu tham khảo được sử dụng trong quá trình thực hiện đề tài.
- Chương 9 cung cấp phụ lục hỗ trợ tra cứu, bao gồm danh sách từ viết tắt và các thông tin bổ sung liên quan.

CHƯƠNG 2: CÁC HỆ THỐNG LIÊN QUAN

2.1 Tổng quan thị trường

Các hệ thống social listening và phân tích dữ liệu truyền thông xã hội hiện nay tạo thành một nhóm giải pháp tương đối rõ ràng trên thị trường phần mềm. Nhóm giải pháp này hướng tới các đối tượng sử dụng như chuyên viên phân tích dữ liệu truyền thông xã hội, bộ phận theo dõi thương hiệu, marketing, nghiên cứu thị trường, truyền thông và các cấp quản lý có nhu cầu khai thác thông tin để hỗ trợ đánh giá và ra quyết định.

Mặc dù khác nhau về mô hình cung cấp và mức độ tự phục vụ, các hệ thống trong nhóm này thường cùng giải quyết một số nhu cầu chính như theo dõi dữ liệu từ nhiều nguồn, phân tích nội dung, tổng hợp kết quả và hỗ trợ khai thác thông tin theo ngữ cảnh. Điều đó cho thấy giá trị của một hệ thống không chỉ nằm ở khả năng thu thập dữ liệu, mà còn ở cách dữ liệu được tổ chức và chuyển hóa thành thông tin có thể sử dụng trong thực tế.

Trong bối cảnh đó, việc khảo sát các hệ thống liên quan có ý nghĩa xác định vị trí của SMAP trong không gian giải pháp hiện có. Dù được xây dựng phục vụ nhu cầu nội bộ, SMAP vẫn hướng tới cùng một nhóm bài toán cốt lõi là theo dõi, phân tích và tổng hợp thông tin từ mạng xã hội cho các nhóm người dùng chuyên môn.

2.2 Các hệ thống, dịch vụ tại Việt Nam

Việc khảo sát các hệ thống tại Việt Nam có ý nghĩa vì đây là nhóm giải pháp gần với ngữ cảnh sử dụng thực tế của SMAP nhất, đặc biệt ở khía cạnh ngôn ngữ, nguồn dữ liệu, hành vi người dùng và nhu cầu theo dõi thương hiệu trên môi trường truyền thông số trong nước. Các hệ thống này cho thấy bài toán social listening tại Việt Nam đã hình thành tương đối rõ, với các hướng tiếp cận khác nhau như dịch vụ phân tích chuyên sâu, nền tảng tự phục vụ, giải pháp giám sát danh tiếng và media intelligence đa nguồn.

2.2.1 YouNet Group

YouNet Group thường được nhắc đến như một trong những đơn vị có hiện diện rõ trong lĩnh vực social listening và phân tích dữ liệu truyền thông xã hội tại Việt Nam. Điểm đáng chú ý là hệ sinh thái này thể hiện hai hướng tiếp cận khác nhau: một hướng nghiêng về dịch vụ phân tích chuyên sâu và một hướng nghiêng về nền tảng cho người dùng tự khai thác.

2.2.1.1 Buzzmetrics

Buzzmetrics được biết đến như một đơn vị nghiên cứu và phân tích dữ liệu truyền thông xã hội theo hướng dịch vụ chuyên sâu. Mô hình này phù hợp với các tổ chức

muốn nhận báo cáo, insight và tư vấn từ đội ngũ chuyên gia, thay vì trực tiếp thao tác trên một nền tảng phần mềm.

Ưu điểm:

- Phù hợp với các nhu cầu cần phân tích sâu, diễn giải kết quả và tư vấn chiến lược.
- Có lợi thế ở cách tiếp cận theo hướng nghiên cứu, giúp dữ liệu được chuyển hóa thành insight có chiều sâu.
- Thích hợp với các tổ chức ưu tiên chất lượng báo cáo hơn là quyền tự thao tác hệ thống.

Hạn chế:

- Mức độ tự phục vụ thấp, người dùng không trực tiếp kiểm soát toàn bộ quy trình khai thác dữ liệu.
- Khả năng mở rộng sử dụng nội bộ theo nhiều nhóm người dùng có thể bị hạn chế do phụ thuộc vào đội ngũ phân tích.
- Không phải là mô hình tối ưu nếu mục tiêu là xây dựng một hệ thống để vận hành thường xuyên trong nội bộ.

Giá trị tham chiếu đối với SMAP:

- Buzzmetrics cho thấy nhu cầu không chỉ dừng ở dữ liệu thô mà còn cần insight có thể sử dụng.
- Điều này củng cố hướng đi của SMAP ở chỗ hệ thống không nên dừng ở việc thu thập dữ liệu, mà cần hỗ trợ tổng hợp, phân tích và cung cấp thông tin có giá trị.

2.2.1.2 YouNet Media

YouNet Media với SocialHeat đại diện cho hướng nền tảng social listening tự phục vụ. Đây là kiểu hệ thống cho phép người dùng trực tiếp theo dõi dữ liệu, xem dashboard, tra cứu kết quả và khai thác thông tin trong quá trình làm việc.

Ưu điểm:

- Phù hợp với nhóm người dùng chuyên môn cần tự thao tác, theo dõi và khai thác dữ liệu thường xuyên.
- Gắn với nhu cầu sử dụng hệ thống trong nội bộ hơn so với mô hình dịch vụ thuần túy.
- Thể hiện rõ giá trị của dashboard, theo dõi thời gian gần thực và tổng hợp thông tin từ nhiều nguồn.

Hạn chế:

- Các khả năng phân tích nâng cao hoặc mô tả kỹ thuật chi tiết thường không được công bố đầy đủ.
- Nếu xét ở góc nhìn kỹ thuật, người đọc khó đánh giá rõ kiến trúc bên trong, luồng xử lý dữ liệu hay mức độ tích hợp sâu giữa các thành phần.
- Mô hình nền tảng hoàn chỉnh thường đi kèm chi phí vận hành và phạm vi chức năng lớn.

Giá trị tham chiếu đối với SMAP:

- SocialHeat là ví dụ gần với hướng sử dụng thực tế của SMAP hơn, vì đều liên quan đến người dùng chuyên môn trực tiếp khai thác dữ liệu.
- Từ đó có thể thấy một hệ thống như SMAP cần quan tâm không chỉ đến xử lý dữ liệu, mà còn đến khả năng hỗ trợ người dùng theo dõi, tra cứu và tổng hợp thông tin.

2.2.2 Reputa

Reputa được biết đến như một giải pháp giám sát và phân tích thông tin trên môi trường mạng, có định hướng rõ về quản lý danh tiếng và phát hiện rủi ro truyền thông. Hướng tiếp cận này cho thấy social listening không chỉ là theo dõi thảo luận, mà còn gắn với nhu cầu cảnh báo và phản ứng trước các tín hiệu bất lợi.

Ưu điểm:

- Nổi bật ở góc nhìn quản lý danh tiếng và theo dõi rủi ro truyền thông.
- Phù hợp với các tổ chức coi cảnh báo sớm và giám sát tín hiệu bất thường là yêu cầu quan trọng.
- Là ví dụ tốt cho việc gắn social listening với hoạt động vận hành và quản trị truyền thông.

Hạn chế:

- Ít thông tin công khai về chiều sâu công nghệ phân tích hoặc mức độ linh hoạt của nền tảng.
- Định vị giải pháp nghiêng về giám sát danh tiếng có thể làm phạm vi sử dụng hẹp hơn so với một nền tảng hỗ trợ nhiều lớp khai thác thông tin.
- Nếu nhìn từ góc độ kỹ thuật, khó đánh giá chi tiết cách hệ thống tổ chức các tầng phân tích và xử lý dữ liệu.

Giá trị tham chiếu đối với SMAP:

- Reputa nhấn mạnh tầm quan trọng của khả năng nhận diện tín hiệu đáng chú ý và hỗ trợ cảnh báo.
- Đây là luận cứ tốt để SMAP quan tâm đến lớp theo dõi trạng thái, thông báo và hỗ trợ đánh giá tình hình trong vận hành nội bộ.

2.2.3 Kompa

Kompa thường được định vị như một đơn vị công nghệ dữ liệu có định hướng mở rộng hơn social listening truyền thống, nhấn mạnh vào dữ liệu lớn, xử lý ngôn ngữ tự nhiên và khai thác thông tin ở quy mô rộng hơn.

Ưu điểm:

- Thể hiện hướng tiếp cận thiên công nghệ dữ liệu và xử lý ngôn ngữ, phù hợp với xu hướng tăng chiều sâu phân tích.
- Gợi ý rằng giá trị của hệ thống không chỉ nằm ở dashboard, mà còn nằm ở khả năng xử lý và cấu trúc hóa dữ liệu.
- Cho thấy nhu cầu gắn social data với các bài toán phân tích rộng hơn là có thật.

Hạn chế:

- Phạm vi định vị rộng có thể làm mờ trọng tâm nếu người dùng chỉ cần một công cụ social listening phục vụ vận hành thường xuyên.
- Mức độ công khai về mô hình hệ thống và chi tiết kỹ thuật không nhiều.
- Nếu xét cho mục tiêu đồ án, đây là một hướng tham chiếu tốt về chiều sâu công nghệ, nhưng ít trực tiếp hơn so với các nền tảng social listening chuyên biệt.

Giá trị tham chiếu đối với SMAP:

- Kompa là một ví dụ cho thấy lớp xử lý dữ liệu và lớp phân tích có thể trở thành điểm khác biệt quan trọng của hệ thống.
- Điều này phù hợp với hướng SMAP xây dựng các luồng xử lý và phân tích thay vì chỉ dừng ở lớp hiển thị.

2.2.4 Commsights / Andi

Commsights và Andi đại diện cho hướng media intelligence rộng hơn, bao phủ không chỉ mạng xã hội mà còn nhiều nguồn truyền thông khác. Đây là một nhánh tham chiếu hữu ích vì nó cho thấy nhu cầu khai thác thông tin thực tế thường không giới hạn trong một loại nguồn dữ liệu duy nhất.

Ưu điểm:

- Cho thấy giá trị của góc nhìn đa nguồn, đặc biệt khi người dùng cần theo dõi sự hiện diện thông tin trên nhiều kênh.
- Phù hợp với các tổ chức muốn tổng hợp dữ liệu truyền thông ở phạm vi rộng.
- Nhấn mạnh vai trò của báo cáo tổng hợp và phân tích định tính.

Hạn chế:

- Có xu hướng nghiêng về dịch vụ hoặc media intelligence tổng hợp hơn là một nền tảng social listening thuần kỹ thuật.
- Mức độ tự phục vụ và chi tiết kỹ thuật của nền tảng không phải lúc nào cũng rõ ràng.
- Ít trực tiếp hơn đối với bài toán social data pipeline nhiều lớp mà SMAP đang hướng tới.

Giá trị tham chiếu đối với SMAP:

- Nhóm giải pháp này cho thấy giá trị của việc tổng hợp thông tin và hỗ trợ báo cáo cho người dùng chuyên môn.
- Tuy nhiên, SMAP vẫn khác ở chỗ trọng tâm nằm ở tổ chức luồng dữ liệu và xử lý hệ thống từ nguồn social data đến phân tích và khai thác thông tin.

2.2.5 Nhận xét chung về các hệ thống tại Việt Nam

Nhìn chung, các hệ thống tại Việt Nam phản ánh ba hướng tiếp cận chính. Hướng thứ nhất là dịch vụ phân tích chuyên sâu, nơi giá trị chính nằm ở chất lượng insight và diễn giải. Hướng thứ hai là nền tảng tự phục vụ, nơi người dùng trực tiếp theo dõi

và khai thác dữ liệu. Hướng thứ ba là các giải pháp giám sát danh tiếng hoặc media intelligence, nơi việc phát hiện tín hiệu và tổng hợp thông tin được đặt lên hàng đầu.

Đối với SMAP, nhóm giải pháp này có giá trị tham chiếu ở chỗ chúng cho thấy bài toán theo dõi, phân tích và tổng hợp thông tin từ mạng xã hội là một nhu cầu rõ ràng, có nhóm người dùng cụ thể và có nhiều cách tiếp cận khác nhau. Từ đó, SMAP có thể được nhìn nhận như một hệ thống phục vụ nhu cầu nội bộ nhưng vẫn nằm trong cùng không gian giải pháp với các nền tảng social listening hiện có.

2.3 Các hệ thống, dịch vụ toàn cầu

Khảo sát các hệ thống toàn cầu giúp mở rộng góc nhìn khỏi bối cảnh địa phương, đồng thời chỉ ra các năng lực thường được xem là chuẩn tham chiếu trong nhóm giải pháp social listening hiện đại. So với các hệ thống trong nước, các nền tảng toàn cầu thường có độ phủ dữ liệu rộng hơn, hệ sinh thái chức năng lớn hơn và nhiều lớp hỗ trợ cho phân tích, cảnh báo, dashboarding và khai thác dữ liệu.

2.3.1 Talkwalker

Talkwalker thường được biết đến như một nền tảng social listening và consumer intelligence quy mô lớn, phục vụ nhu cầu phân tích sâu ở cấp doanh nghiệp.

Ưu điểm:

- Độ phủ nguồn dữ liệu và ngôn ngữ rộng.
- Tập trung mạnh vào phân tích xu hướng, giám sát thương hiệu và các bài toán intelligence.
- Là ví dụ tiêu biểu cho hướng xây dựng một nền tảng tổng thể với nhiều lớp chức năng.

Hạn chế:

- Độ phức tạp cao, không phải lúc nào cũng phù hợp với các nhu cầu sử dụng hẹp hoặc nội bộ.
- Chi phí và phạm vi chức năng lớn khiến đây không phải mô hình tham chiếu trực tiếp cho một đề án phần mềm nội bộ.
- Khả năng thích nghi với các ngữ cảnh ngôn ngữ địa phương như tiếng Việt có thể không phải điểm mạnh nhất.

Giá trị tham chiếu đối với SMAP:

- Talkwalker là chuẩn tham chiếu tốt cho chiều rộng chức năng của một nền tảng social listening hiện đại.
- Tuy nhiên, SMAP nên học ở logic tổ chức chức năng và luồng xử lý, không phải ở quy mô sản phẩm.

2.3.2 Brandwatch

Brandwatch là một nền tảng consumer intelligence và social listening được nhắc đến nhiều ở thị trường toàn cầu, đặc biệt ở khả năng hỗ trợ dashboard, phân tích xu hướng và khai thác insight ở cấp doanh nghiệp.

Ưu điểm:

- Mạnh ở việc tổ chức dữ liệu thành dashboard và insight có thể khai thác.
- Phù hợp với các tổ chức cần kết hợp theo dõi dữ liệu với tổng hợp và báo cáo.
- Là một ví dụ tốt cho hướng gắn social listening với khai thác thông tin phục vụ người dùng chuyên môn.

Hạn chế:

- Quy mô enterprise làm cho nền tảng có độ phức tạp cao hơn nhiều so với nhu cầu nội bộ gọn nhẹ.
- Không phải mọi thành phần kỹ thuật hoặc logic phân tích đều được công khai chi tiết để dùng như chuẩn kỹ thuật trực tiếp.
- Mô hình sản phẩm hoàn chỉnh có nhiều phần vượt ra ngoài phạm vi cốt lõi mà SMAP cần giải quyết.

Giá trị tham chiếu đối với SMAP:

- Brandwatch gợi ý tốt về lớp tổng hợp thông tin, dashboard và hỗ trợ người dùng khai thác kết quả.
- Đây là điểm tham chiếu hữu ích cho việc nghĩ về đầu ra sử dụng được, thay vì chỉ pipeline xử lý bên trong.

2.3.3 Sprinklr

Sprinklr là một nền tảng enterprise rộng, kết hợp social listening với các workflow vận hành và quản lý trải nghiệm khách hàng.

Ưu điểm:

- Cho thấy social listening có thể được tích hợp vào các quy trình vận hành lớn hơn thay vì đứng độc lập.
- Mạnh ở góc nhìn doanh nghiệp, quản trị và khả năng kết nối nhiều nhóm người dùng trong một hệ thống.
- Là ví dụ tốt cho hướng coi social data như một phần của vận hành liên phòng ban.

Hạn chế:

- Phạm vi chức năng rất rộng, dễ vượt xa trọng tâm kỹ thuật của một đề án.
- Khó dùng như chuẩn đối chiếu trực tiếp vì bao gồm nhiều thành phần ngoài bài toán cốt lõi của SMAP.
- Không phù hợp để lấy làm mục tiêu chức năng đầy đủ cho một hệ thống nội bộ có phạm vi gọn hơn.

Giá trị tham chiếu đối với SMAP:

- Sprinklr hữu ích ở góc nhìn cho thấy dữ liệu và tín hiệu từ mạng xã hội có thể trở thành đầu vào cho quy trình vận hành thực tế.
- Với SMAP, giá trị học hỏi nằm ở vai trò hỗ trợ người dùng chuyên môn và thông tin cho ra quyết định, chứ không phải sao chép độ rộng chức năng.

2.3.4 YouScan

YouScan thường được biết đến nhờ khả năng phân tích hình ảnh và trải nghiệm người dùng trực quan, đại diện cho hướng mở rộng social listening vượt ra ngoài văn bản thuần túy.

Ưu điểm:

- Nhấn mạnh giá trị của việc phân tích đa phương thức.
- Cho thấy giao diện và trải nghiệm khai thác dữ liệu cũng là thành phần quan trọng của hệ thống.
- Là ví dụ cho hướng tích hợp thêm khả năng tương tác trực quan với dữ liệu.

Hạn chế:

- Hướng visual listening là một lớp chức năng nâng cao, chưa phải trọng tâm hiện tại của SMAP.
- Nếu dùng làm chuẩn so sánh trực tiếp, có thể kéo luận văn sang những yêu cầu vượt khỏi phạm vi hiện tại.
- Giá trị tham chiếu của nền tảng này phù hợp hơn ở hướng mở rộng tương lai hơn là yêu cầu cốt lõi hiện tại.

Giá trị tham chiếu đối với SMAP:

- YouScan giúp làm rõ rằng các hệ thống social listening có thể phát triển rất xa ở lớp tương tác và phân tích đa phương tiện.
- Với SMAP, đây nên được xem là một mốc tham chiếu về hướng phát triển hơn là tiêu chuẩn bắt buộc trong phạm vi hiện tại.

2.3.5 Meltwater

Meltwater là một nền tảng media intelligence và social listening toàn diện, nổi bật ở hướng hỗ trợ theo dõi truyền thông và quản lý khủng hoảng.

Ưu điểm:

- Mạnh ở bài toán phát hiện tín hiệu bất thường và hỗ trợ giám sát truyền thông.
- Cho thấy giá trị của việc kết hợp social listening với media intelligence rộng hơn.
- Phù hợp với các tổ chức coi cảnh báo và theo dõi khủng hoảng là yêu cầu quan trọng.

Hạn chế:

- Giống nhiều nền tảng enterprise khác, phạm vi quá rộng nếu so với mục tiêu của một hệ thống nội bộ tập trung hơn.
- Một số tính năng phân tích sâu có thể không phải là trọng tâm nổi bật như ở các nền tảng khác.

- Không phải chuẩn phù hợp để sao chép toàn bộ, mà chỉ nên dùng như đối chiếu theo nhóm năng lực.

Giá trị tham chiếu đối với SMAP:

- Meltwater nhấn mạnh vai trò của cảnh báo và giám sát tín hiệu.
- Đây là góc tham chiếu hữu ích cho các chức năng theo dõi trạng thái và hỗ trợ đánh giá tình huống trong SMAP.

2.3.6 Nhận xét chung về các hệ thống toàn cầu

Các hệ thống toàn cầu cho thấy một số năng lực thường được xem là chuẩn tham chiếu trong social listening hiện đại: độ phủ dữ liệu lớn, dashboard mạnh, phân tích xu hướng, cảnh báo tín hiệu bất thường, và các lớp hỗ trợ khai thác thông tin ở mức cao hơn. Tuy nhiên, chúng cũng cho thấy rằng quy mô enterprise thường kéo theo độ phức tạp lớn, phạm vi chức năng rộng và chi phí cao.

Đối với SMAP, giá trị của việc khảo sát các hệ thống toàn cầu không nằm ở mục tiêu tái hiện một sản phẩm thương mại quy mô lớn, mà ở việc xác định những nhóm chức năng nào là quan trọng đối với người dùng chuyên môn. Từ đó, SMAP có thể được định vị như một hệ thống nội bộ tập trung vào các nhu cầu cốt lõi và các luồng xử lý quan trọng, thay vì theo đuổi độ bao phủ sản phẩm ở mức enterprise.

2.4 Ma trận so sánh tính năng

Để tổng hợp các hệ thống đã khảo sát, có thể xây dựng một ma trận so sánh theo các tiêu chí gắn trực tiếp với nhu cầu của người dùng chuyên môn và với định hướng của SMAP. Mục tiêu của bảng này không phải là xếp hạng tuyệt đối giữa các nền tảng, mà là chỉ ra những nhóm năng lực thường gặp trong không gian giải pháp social listening, đồng thời làm rõ các hướng tham chiếu quan trọng đối với việc xây dựng SMAP.

2.4.1 Tiêu chí so sánh

Các tiêu chí được lựa chọn gồm:

- phạm vi dữ liệu và nguồn dữ liệu
- khả năng theo dõi và cập nhật thông tin
- mức độ hỗ trợ phân tích và tổng hợp
- khả năng cảnh báo tín hiệu bất thường
- mức độ tự phục vụ của người dùng
- khả năng hỗ trợ khai thác thông tin theo ngữ cảnh
- mức độ phù hợp với nhu cầu vận hành nội bộ

2.4.2 Bảng so sánh tổng hợp

Bảng 1: Ma trận so sánh tổng hợp các hệ thống liên quan

Hệ thống	Phạm vi dữ liệu	Theo dõi và cập nhật	Phân tích và tổng hợp	Cảnh báo tín hiệu	Mức tự phục vụ	Khai thác thông tin theo ngữ cảnh	Giá trị tham chiếu với SMAP
1. Hệ thống tại Việt Nam							
Buzzmetrics	Mạng xã hội và dữ liệu liên quan theo hướng nghiên cứu	Có, nhưng thiên về dịch vụ	Mạnh ở insight chuyên sâu	Có thể hỗ trợ theo dự án	Thấp	Gián tiếp qua báo cáo chuyên gia	Nhấn mạnh giá trị của insight chất lượng
SocialHeat	Social listening đa nguồn trong nước	Mạnh	Khá mạnh	Có	Cao	Ở mức dashboard và tra cứu	Gần với hướng người dùng trực tiếp khai thác
Reputa	Giám sát thông tin và danh tiếng	Khá mạnh	Phù hợp cho theo dõi rủi ro	Mạnh	Trung bình	Hạn chế thông tin công khai	Tham chiếu tốt cho giám sát và cảnh báo
Kompa	Dữ liệu và phân tích theo hướng rộng	Có	Nhấn mạnh công nghệ dữ liệu	Có đề cập	Trung bình	Có tiềm năng cao	Tham chiếu tốt cho chiều sâu công nghệ phân tích
Commsights / Andi	Media intelligence đa nguồn	Khá mạnh	Mạnh ở tổng hợp báo cáo	Có	Thấp đến trung bình	Chủ yếu qua báo cáo	Tham chiếu cho tổng hợp thông tin đa nguồn
2. Hệ thống toàn cầu							
Talkwalker	Rất rộng, quy mô toàn cầu	Rất mạnh	Rất mạnh	Mạnh	Cao	Khá mạnh	Chuẩn tham chiếu cho nền tảng tổng thể
Brandwatch	Rộng, thiên consumer intelligence	Mạnh	Mạnh	Có	Cao	Khá mạnh	Tham chiếu tốt cho dashboard và insight
Sprinklr	Rộng, gắn với vận hành enterprise	Mạnh	Mạnh	Mạnh	Cao	Tốt trong workflow lớn	Tham chiếu cho vai trò hỗ trợ vận hành
YouScan	Rộng, nổi bật ở hình ảnh	Mạnh	Khá mạnh	Có	Cao	Có yếu tố tương tác trực quan	Tham chiếu cho hướng mở rộng đa phương tiện
Meltwater	Media intelligence	Mạnh	Khá mạnh	Rất mạnh	Cao	Ở mức khai thác	Tham chiếu cho cảnh báo

	và social listening					báo cáo và theo dõi	và quản lý khủng hoảng
--	------------------------	--	--	--	--	------------------------	---------------------------

2.4.3 Nhận xét từ ma trận so sánh

Từ bảng so sánh có thể thấy rằng các hệ thống liên quan tuy khác nhau về mô hình cung cấp và phạm vi chức năng, nhưng đều xoay quanh một số nhu cầu lặp lại: theo dõi dữ liệu, phân tích nội dung, tổng hợp thông tin, hỗ trợ cảnh báo và phục vụ người dùng chuyên môn trong quá trình đánh giá tình hình. Đây cũng chính là những nhóm nhu cầu cốt lõi mà SMAP hướng tới.

So với nhiều nền tảng thương mại lớn, SMAP không đặt mục tiêu bao phủ toàn bộ chiều rộng chức năng của một sản phẩm enterprise. Thay vào đó, hệ thống tập trung vào việc tổ chức luồng dữ liệu, phối hợp các thành phần kỹ thuật và cung cấp thông tin có thể sử dụng cho nhóm người dùng nội bộ. Vì vậy, giá trị của SMAP không nằm ở việc cạnh tranh trực tiếp với các nền tảng thương mại, mà ở việc xây dựng một hệ thống phù hợp với nhu cầu sử dụng cụ thể và có cấu trúc kỹ thuật rõ ràng, có thể kiểm soát và phát triển tiếp trong thực tế.

CHƯƠNG 3: CƠ SỞ KỸ THUẬT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này hệ thống hóa các nền tảng lý thuyết và công nghệ chủ đạo làm cơ sở cho việc thiết kế, hiện thực và đánh giá hệ thống SMAP. Nội dung bao quát các phong cách kiến trúc, cơ chế giao tiếp trong hệ thống phân tán, nhóm công nghệ lưu trữ dữ liệu, các kỹ thuật AI/NLP và khai thác tri thức, cùng các công nghệ triển khai.

3.1 Kiến trúc Microservices

3.1.1 Định nghĩa và nguồn gốc

Kiến trúc Microservices là một phong cách kiến trúc phần mềm trong đó một ứng dụng được xây dựng như một tập hợp các dịch vụ nhỏ, độc lập, mỗi dịch vụ chạy trong tiến trình riêng và giao tiếp với nhau thông qua các cơ chế nhẹ như HTTP API hoặc Message Queue. Theo Martin Fowler, Microservices là cách tiếp cận để phát triển một ứng dụng đơn lẻ như một bộ các dịch vụ nhỏ, mỗi dịch vụ chạy trong tiến trình riêng và giao tiếp với các cơ chế nhẹ, thường là HTTP Resource API.

Sam Newman mở rộng định nghĩa này bằng cách nhấn mạnh rằng Microservices là các dịch vụ có thể triển khai độc lập và được tổ chức xung quanh các khả năng nghiệp vụ. Mỗi dịch vụ thường sở hữu logic xử lý và dữ liệu liên quan của riêng nó, qua đó tạo điều kiện cho việc phát triển, triển khai và mở rộng theo từng thành phần tương đối độc lập.

3.1.2 Đặc điểm chính

3.1.2.1 Loose Coupling

Các vi dịch vụ được thiết kế để có mức độ liên kết thấp với nhau, nghĩa là thay đổi trong một dịch vụ không yêu cầu thay đổi trong các dịch vụ khác. Mỗi dịch vụ có giao diện rõ ràng và giao tiếp thông qua các API được định nghĩa tốt. Điều này cho phép các nhóm phát triển các dịch vụ một cách độc lập mà không cần phối hợp chặt chẽ với các nhóm khác. Liên kết lỏng lẻo cũng giảm thiểu rủi ro khi thay đổi, vì một lỗi trong một dịch vụ không lan truyền sang các dịch vụ khác.

3.1.2.2 Triển khai độc lập

Một trong những lợi ích quan trọng nhất của kiến trúc Microservices là khả năng triển khai từng dịch vụ một cách độc lập mà không cần triển khai lại toàn bộ hệ thống. Điều này cho phép các nhóm phát triển và phát hành tính năng mới nhanh hơn, giảm thời gian ngừng hoạt động và rủi ro khi triển khai. Mỗi dịch vụ có thể có chu kỳ phát hành riêng, cho phép các nhóm làm việc với tốc độ khác nhau tùy theo nhu cầu nghiệp vụ.

3.1.2.3 Tổ chức theo khả năng nghiệp vụ

Các vi dịch vụ được tổ chức xung quanh các khả năng nghiệp vụ thay vì các lớp kỹ thuật. Mỗi dịch vụ đại diện cho một lĩnh vực nghiệp vụ cụ thể và sở hữu toàn bộ ngăn xếp công nghệ cần thiết để thực hiện chức năng đó, từ giao diện người dùng đến cơ sở dữ liệu. Cách tiếp cận này phản ánh cấu trúc tổ chức của doanh nghiệp và cho phép các nhóm đa chức năng làm việc hiệu quả hơn.

3.1.2.4 Quản lý dữ liệu phi tập trung

Trong kiến trúc Microservices, các dịch vụ thường hướng tới việc quản lý dữ liệu theo cách tách biệt tương đối, thay vì phụ thuộc vào một mô hình dữ liệu tập trung duy nhất. Cách tiếp cận này cho phép mỗi dịch vụ lựa chọn công nghệ và cách tổ chức dữ liệu phù hợp hơn với nhu cầu riêng. Tuy nhiên, nó cũng đặt ra thách thức về tính nhất quán dữ liệu và giao dịch phân tán. Vì vậy, các hệ thống microservices thường phải sử dụng những chiến lược như eventual consistency hoặc các cơ chế điều phối phù hợp để kiểm soát nhất quán dữ liệu.

3.1.2.5 Tự động hóa hạ tầng

Kiến trúc Microservices thường kéo theo nhu cầu tự động hóa cao trong triển khai, giám sát và quản lý hạ tầng. Khi số lượng dịch vụ tăng lên, việc vận hành thủ công trở nên khó duy trì và dễ phát sinh sai lệch giữa các môi trường. Vì vậy, microservices thường đi kèm với các công cụ và quy trình hỗ trợ container hóa, điều phối triển khai, giám sát và tái tạo môi trường một cách nhất quán.

3.1.3 So sánh với kiến trúc Monolithic

Bảng 2: So sánh Microservices và Monolithic

Tiêu chí	Microservices	Monolithic
Khả năng mở rộng	Có thể mở rộng từng service độc lập theo nhu cầu cụ thể	Phải mở rộng toàn bộ ứng dụng
Triển khai	Triển khai từng service độc lập, giảm rủi ro	Triển khai toàn bộ ứng dụng, thời gian ngừng hoạt động cao hơn
Phát triển	Nhiều nhóm có thể làm việc song song trên các service khác nhau	Một codebase lớn, dễ xung đột mã nguồn
Gỡ lỗi	Phức tạp hơn do distributed tracing, cần công cụ chuyên dụng	Đơn giản hơn, có thể gỡ lỗi trong một tiến trình
Tính nhất quán dữ liệu	Eventual consistency, cần xử lý distributed transactions	Strong consistency với ACID transactions
Chi phí hạ tầng	Chi phí cao hơn do cần nhiều tài nguyên và công cụ quản lý	Chi phí thấp hơn cho hệ thống nhỏ

Kiến trúc Microservices mang lại nhiều lợi ích cho các hệ thống có phạm vi chức năng lớn hoặc cần tách biệt rõ các khả năng nghiệp vụ. Khả năng mở rộng và triển khai

tương đối độc lập giúp tối ưu hóa tài nguyên và giảm rủi ro khi thay đổi từng phần của hệ thống. Đồng thời, cách tiếp cận này cũng hỗ trợ lựa chọn công nghệ linh hoạt hơn cho từng dịch vụ.

Tuy nhiên, Microservices cũng đi kèm với những thách thức rõ rệt. Độ phức tạp vận hành tăng lên khi phải quản lý nhiều dịch vụ, nhiều điểm giao tiếp và nhiều lớp dữ liệu. Độ trễ mạng, độ tin cậy của các kết nối liên dịch vụ, cũng như bài toán kiểm thử tích hợp và nhất quán dữ liệu đều trở nên khó xử lý hơn so với kiến trúc nguyên khối.

Ngược lại, kiến trúc Monolithic thường phù hợp hơn với các ứng dụng nhỏ hoặc các hệ thống chưa có nhu cầu tách biệt rõ theo nhiều khả năng nghiệp vụ. Việc phát triển, triển khai và gỡ lỗi thường trực tiếp hơn khi toàn bộ mã nguồn và dữ liệu tập trung trong một khối thống nhất.

Những đặc điểm trên là cơ sở để sử dụng kiến trúc Microservices như một nền tảng lý thuyết phù hợp khi phân tích và thiết kế hệ thống SMAP ở các chương sau.

3.2 Clean Architecture

3.2.1 Nguyên tắc thiết kế

Clean Architecture là một kiểu kiến trúc phần mềm được đề xuất bởi Robert C. Martin nhằm tạo ra các hệ thống dễ hiểu, linh hoạt và dễ bảo trì. Ý tưởng cốt lõi của Clean Architecture là tách biệt các mối quan tâm trong ứng dụng thành các tầng riêng biệt, với nguyên tắc các phụ thuộc chỉ trở vào trong. Kiến trúc này đảm bảo rằng logic nghiệp vụ không phụ thuộc vào các chi tiết triển khai như framework, cơ sở dữ liệu, hay giao diện người dùng.

Theo Robert C. Martin, một kiến trúc tốt phải đạt được các mục tiêu sau: độc lập với framework, có khả năng kiểm thử cao, độc lập với giao diện người dùng, độc lập với cơ sở dữ liệu, và độc lập với bất kỳ tác nhân bên ngoài nào. Điều này có nghĩa là các quy tắc nghiệp vụ không nên biết gì về thế giới bên ngoài. Sự độc lập này cho phép thay đổi các chi tiết triển khai mà không ảnh hưởng đến logic nghiệp vụ cốt lõi.

Khả năng kiểm thử là một trong những lợi ích quan trọng nhất của Clean Architecture. Vì logic nghiệp vụ không phụ thuộc vào các phụ thuộc bên ngoài, nó có thể được kiểm thử một cách độc lập mà không cần mock cơ sở dữ liệu, web server, hay bất kỳ dịch vụ bên ngoài nào. Các unit test có thể chạy nhanh và đáng tin cậy vì chúng chỉ kiểm thử logic nghiệp vụ thuần túy.

3.2.2 Các tầng và Dependency Rule

Clean Architecture tổ chức mã nguồn thành các tầng đồng tâm, với logic nghiệp vụ ở trung tâm và các chi tiết triển khai ở bên ngoài. Có bốn tầng chính: Entities, Use Cases, Interface Adapters, và Frameworks and Drivers.

3.2.2.1 Entities

Entities là tầng trong cùng, chứa các quy tắc nghiệp vụ quan trọng nhất và ổn định nhất của ứng dụng. Đây là các mô hình miền đại diện cho các khái niệm nghiệp vụ cốt lõi. Tầng này không phụ thuộc vào các chi tiết triển khai như use case, controller hay cơ sở dữ liệu. Vì vậy, Entities chủ yếu chứa logic nghiệp vụ thuần túy, tức các quy tắc vẫn có ý nghĩa ngay cả khi chưa xét đến cách hệ thống được hiện thực.

3.2.2.2 Use Cases

Tầng Use Cases chứa các quy tắc nghiệp vụ cụ thể của ứng dụng, tức các quy tắc mô tả cách ứng dụng sử dụng entities để đạt được một mục tiêu rõ ràng. Mỗi use case đại diện cho một chức năng nghiệp vụ cụ thể. Tầng này điều phối luồng dữ liệu đến và từ entities, đồng thời xác định các interface cần thiết để các tầng bên ngoài triển khai. Vì vậy, Use Cases vẫn giữ được sự tách biệt với giao diện người dùng, cơ sở dữ liệu hay các dịch vụ bên ngoài.

3.2.2.3 Interface Adapters

Tầng Interface Adapters chịu trách nhiệm chuyển đổi dữ liệu giữa định dạng phù hợp với use case và định dạng phù hợp với các tác nhân bên ngoài như web hay cơ sở dữ liệu. Tầng này thường bao gồm controller nhận request và gọi use case, presenter định dạng dữ liệu để trả về cho giao diện người dùng, cùng gateway hoặc repository để truy cập dữ liệu. Đây cũng là nơi thể hiện rõ nguyên tắc dependency inversion: các use case xác định interface cần thiết, còn adapter ở bên ngoài triển khai các interface đó.

3.2.2.4 Frameworks and Drivers

Tầng ngoài cùng chứa các framework và công cụ như cơ sở dữ liệu, web framework, API bên ngoài. Đây là nơi tất cả các chi tiết triển khai cụ thể nằm. Mã nguồn trong tầng này thường rất ít vì phần lớn công việc được thực hiện bởi các thư viện bên thứ ba. Tầng này giao tiếp với tầng Interface Adapters để tích hợp với phần còn lại của ứng dụng.

3.2.2.5 Dependency Rule

Nguyên tắc quan trọng nhất của Clean Architecture là Dependency Rule: các phụ thuộc mã nguồn chỉ được trỏ vào trong. Nói cách khác, các tầng bên trong không nên phụ thuộc vào chi tiết của các tầng bên ngoài. Điều này thường được thực hiện thông qua dependency inversion, trong đó tầng trong xác định interface cần thiết, còn tầng ngoài cung cấp phần triển khai cụ thể.

3.2.3 So sánh với MVC và Hexagonal Architecture

Bảng 3: So sánh Clean Architecture, MVC và Hexagonal Architecture

Tiêu chí	Clean Architecture	MVC	Hexagonal Architecture
Testability	Rất cao, business logic hoàn toàn độc lập	Trung bình, business logic thường lẫn với controller	Rất cao, tương tự Clean Architecture
Complexity	Cao, nhiều tầng và abstractions	Thấp, cấu trúc đơn giản	Trung bình, ít tầng hơn Clean Architecture
Learning Curve	Dốc, cần hiểu dependency inversion	Thoải, pattern phổ biến	Trung bình, concepts dễ hiểu hơn
Flexibility	Rất cao, dễ thay đổi triển khai	Thấp, tight coupling với framework	Cao, ports và adapters dễ thay đổi
Boilerplate	Nhiều, cần nhiều interfaces và abstractions	Ít, mã nguồn ngắn gọn hơn	Trung bình, cần ports và adapters

Clean Architecture và Hexagonal Architecture có nhiều điểm tương đồng, cả hai đều nhấn mạnh việc tách biệt logic nghiệp vụ khỏi các mối quan tâm bên ngoài thông qua abstraction. Sự khác biệt chính là Clean Architecture có cấu trúc tầng rõ ràng hơn với sự phân biệt giữa Entities và Use Cases, trong khi Hexagonal Architecture tập trung vào khái niệm port và adapter. Clean Architecture có thể được xem như một sự phát triển của Hexagonal Architecture với cấu trúc rõ ràng hơn.

So với MVC, Clean Architecture phức tạp hơn đáng kể nhưng mang lại khả năng kiểm thử và tính linh hoạt cao hơn nhiều. MVC phù hợp cho các ứng dụng nhỏ, đơn giản nơi logic nghiệp vụ không phức tạp và không cần độ bao phủ kiểm thử cao. Tuy nhiên, khi ứng dụng phát triển, MVC thường dẫn đến các controller phình to chứa cả logic nghiệp vụ lẫn logic trình bày, từ đó làm giảm khả năng kiểm thử và bảo trì.

Những nguyên tắc trên là cơ sở để phân tích cách các service trong SMAP tổ chức business logic, delivery layer và infrastructure layer ở các chương sau.

3.3 Message Queue và Event-Driven Architecture

3.3.1 Khái niệm Message Queue

Message Queue là một cơ chế giao tiếp bất đồng bộ cho phép các thành phần trong hệ thống phân tán trao đổi thông tin mà không cần kết nối trực tiếp với nhau. Trong mô hình này, producer đưa message vào queue, còn consumer lấy message ra để xử lý. Queue đóng vai trò như một bộ đệm trung gian, lưu trữ message cho đến khi consumer sẵn sàng tiếp nhận. Nhờ đó, hệ thống đạt được sự tách biệt về thời gian, khi producer và consumer không cần trực tuyến cùng lúc, và tách biệt về không gian, khi chúng không cần biết trực tiếp về địa chỉ hay cách triển khai của nhau.

Message Queue mang lại nhiều lợi ích quan trọng cho hệ thống phân tán. Xử lý bất đồng bộ cho phép producer tiếp tục công việc ngay sau khi gửi message mà không cần

chờ xử lý hoàn tất. Cân bằng tải giúp hệ thống hấp thụ các đợt tăng lưu lượng đột ngột bằng cách xếp hàng và xử lý dần. Độ tin cậy được cải thiện thông qua lưu trữ message và cơ chế thử lại. Khả năng mở rộng cũng tốt hơn vì có thể tăng số lượng consumer để xử lý song song khi cần.

3.3.2 RabbitMQ

RabbitMQ là một message broker mã nguồn mở phổ biến, triển khai AMQP, một giao thức chuẩn cho middleware hướng message. RabbitMQ đóng vai trò trung gian giữa producer và consumer, nhận message từ producer, định tuyến chúng đến đúng queue và chuyển giao đến consumer. RabbitMQ hỗ trợ nhiều mẫu nhắn tin khác nhau thông qua các loại exchange, cơ chế xác nhận và các queue durable để tăng độ tin cậy trong xử lý bất đồng bộ.

Kiến trúc của RabbitMQ bao gồm một số thành phần chính. Exchange nhận message từ producer và định tuyến đến queue dựa trên quy tắc định tuyến. Queue lưu trữ message cho đến khi consumer tiêu thụ. Binding định nghĩa quan hệ giữa exchange và queue. Routing key là thuộc tính của message được exchange sử dụng để quyết định định tuyến. Channel là kết nối ảo trong một kết nối TCP, giúp ghép nhiều thao tác trên cùng một kết nối vật lý.

RabbitMQ hỗ trợ bốn loại exchange chính. Fanout exchange phát sóng message đến tất cả queue được liên kết. Direct exchange định tuyến dựa trên khớp chính xác giữa routing key và binding key. Topic exchange định tuyến dựa trên mẫu với ký tự đại diện, phù hợp cho multicast có chọn lọc. Headers exchange định tuyến dựa trên header của message thay vì routing key. Sự linh hoạt này khiến RabbitMQ đặc biệt phù hợp với các bài toán task dispatch, workflow routing và request-response bất đồng bộ.

3.3.3 Apache Kafka

Apache Kafka là một nền tảng event streaming phân tán được thiết kế cho thông lượng cao, khả năng lưu giữ message theo thời gian và xử lý dữ liệu theo dòng. Không giống với message queue truyền thống vốn nhấn mạnh vào việc chuyển giao message đến consumer rồi xóa đi, Kafka lưu message trong log bất biến theo từng topic và partition, cho phép nhiều consumer group đọc lại cùng một dòng sự kiện ở các tốc độ khác nhau.

Các khái niệm trung tâm của Kafka gồm topic, partition, offset và consumer group. Topic là luồng dữ liệu logic. Partition chia topic thành các đoạn độc lập để tăng khả năng song song hóa. Offset xác định vị trí của từng message trong partition. Consumer group cho phép nhiều consumer cùng phối hợp đọc một topic theo cách phân chia tải giữa các partition. Nhờ đó, Kafka vừa hỗ trợ throughput cao, vừa hỗ trợ replay, retention và tích hợp downstream hiệu quả.

Kafka thường phù hợp với các bài toán event propagation, analytics pipeline, audit stream và downstream processing. Khi dữ liệu cần được phát hành cho nhiều hệ thống

tiêu thụ độc lập, cần giữ lại theo thời gian, hoặc cần replay lại để xử lý sau này, Kafka thường phù hợp hơn RabbitMQ. Tuy nhiên, Kafka cũng kéo theo độ phức tạp vận hành cao hơn và không phải lúc nào cũng là lựa chọn tối ưu cho task queue truyền thống.

3.3.4 Producer-Consumer Pattern

Producer-Consumer pattern là một mẫu nhả tin cơ bản trong đó producer tạo message và đưa vào queue hoặc stream, còn consumer nhận message ra để xử lý. Mẫu này tạo ra sự tách biệt rõ giữa bên phát sinh công việc và bên xử lý công việc. Producer có thể tạo message với tốc độ của mình, consumer có thể tiêu thụ theo tốc độ riêng, còn lớp trung gian giúp hấp thụ sự chênh lệch tải.

Cân bằng tải là một lợi ích quan trọng của mẫu này. Khi có nhiều consumer cùng tiêu thụ từ một queue hoặc cùng tham gia một consumer group, khối lượng công việc có thể được phân chia giữa chúng. Điều này cho phép mở rộng theo chiều ngang, giảm nguy cơ nghẽn cục bộ và giúp hệ thống phản ứng tốt hơn khi lưu lượng tăng cao.

3.3.5 Publish/Subscribe và Event-Driven Architecture

Publish/Subscribe là một mẫu giao tiếp trong đó publisher phát hành message đến một chủ đề hoặc một exchange, còn subscriber nhận các message mà mình quan tâm. Khác với task queue một-một, trong mẫu này cùng một message có thể được nhiều thành phần tiếp nhận và xử lý độc lập. Điều này đặc biệt hữu ích khi một sự kiện cần kích hoạt nhiều hành vi downstream khác nhau.

Publish/Subscribe là một trong những cơ chế quan trọng thường được sử dụng trong các kiến trúc hướng sự kiện. Trong event-driven architecture, các thành phần giao tiếp với nhau thông qua event thay vì gọi trực tiếp lẫn nhau trong mọi tình huống. Khi một thay đổi trạng thái hoặc một hành vi đáng chú ý xảy ra, service phát hành event. Các service khác có thể đăng ký để phản ứng tương ứng mà không cần phụ thuộc trực tiếp vào service phát hành.

Ưu điểm của cách tiếp cận này là tăng loose coupling, hỗ trợ mở rộng hệ thống theo chiều ngang và làm cho downstream integration linh hoạt hơn. Tuy nhiên, event-driven architecture cũng làm tăng độ phức tạp trong việc theo dõi luồng xử lý, đảm bảo thứ tự, kiểm soát trùng lặp và bảo vệ tính nhất quán dữ liệu giữa các service.

3.3.6 Delivery Semantics và các lưu ý thiết kế

Khi thiết kế hệ thống bất đồng bộ, một vấn đề quan trọng là delivery semantics, tức cách hệ thống đảm bảo việc gửi và nhận message. Có ba mức thường gặp. At-most-once nghĩa là message được gửi tối đa một lần, có thể mất message nhưng tránh trùng lặp. At-least-once nghĩa là message sẽ được giao ít nhất một lần, chấp nhận khả năng trùng lặp để đổi lấy độ tin cậy cao hơn. Exactly-once là mục tiêu khó đạt hơn nhiều trong thực tế và thường đòi hỏi cơ chế điều phối phức tạp ở cả producer, broker và consumer.

Ngoài delivery semantics, một số khái niệm khác cũng rất quan trọng. Ordering quyết định message có được giữ đúng thứ tự hay không. Retention xác định message được giữ lại trong bao lâu để hỗ trợ replay hoặc audit. Idempotency giúp consumer xử lý an toàn khi cùng một message được gửi lại nhiều lần. Consumer group giúp nhiều consumer chia sẻ khối lượng xử lý nhưng cũng kéo theo các vấn đề về rebalance và offset management. Những yếu tố này có ảnh hưởng trực tiếp đến độ tin cậy và khả năng mở rộng của hệ thống.

3.3.7 So sánh các Message Brokers

Bảng 4: So sánh các Message Brokers

Tiêu chí	RabbitMQ	Apache Kafka	Redis Streams
Mô hình chính	Task queue và routing linh hoạt	Event streaming và log phân tán	Stream nhẹ trong hệ sinh thái Redis
Throughput	Trung bình	Rất cao	Cao
Latency	Thấp	Trung bình	Rất thấp
Retention	Chủ yếu theo queue	Mạnh, log-based retention	Có, nhưng đơn giản hơn Kafka
Ordering	Theo queue	Theo partition	Theo stream
Use Cases	Task dispatch, RPC, workflow queue	Analytics pipeline, event propagation, replay	Streaming nhẹ, cache-adjacent workloads

RabbitMQ phù hợp cho các trường hợp sử dụng hàng đợi message truyền thống như hàng đợi tác vụ, request-response bất đồng bộ và các tình huống cần định tuyến linh hoạt. Kafka phù hợp hơn cho event propagation, analytics pipeline và các tình huống cần giữ message theo thời gian để downstream có thể tiêu thụ hoặc replay. Redis Streams là một lựa chọn nhẹ hơn khi hệ thống đã có Redis và cần thêm một lớp stream đơn giản, nhưng không thay thế hoàn toàn vai trò của RabbitMQ hay Kafka trong các bài toán lớn hơn.

Những khái niệm và so sánh trên là cơ sở để phân tích các lựa chọn truyền tin trong SMAP. Trong kiến trúc của SMAP, hệ thống không hoàn toàn event-driven end-to-end, vì vẫn tồn tại các lane điều khiển đồng bộ qua internal HTTP. Tuy nhiên, RabbitMQ, Kafka và các mô hình event propagation vẫn đóng vai trò quan trọng ở các luồng xử lý bất đồng bộ và downstream integration.

3.4 Real-time Communication

3.4.1 WebSocket Protocol

WebSocket là một giao thức giao tiếp cung cấp các kênh giao tiếp song công qua một kết nối TCP duy nhất. Khác với HTTP truyền thống theo mô hình request-response,

WebSocket cho phép server đẩy dữ liệu đến client bất cứ lúc nào mà không cần client yêu cầu. Điều này làm cho WebSocket lý tưởng cho các ứng dụng thời gian thực như chat, thông báo, cập nhật trực tiếp, và chỉnh sửa cộng tác. WebSocket được chuẩn hóa bởi IETF theo RFC 6455 và được hỗ trợ rộng rãi bởi tất cả trình duyệt hiện đại.

Giao tiếp song công có nghĩa là cả client và server đều có thể gửi message độc lập với nhau cùng một lúc. Không giống như HTTP polling nơi client phải liên tục gửi request để kiểm tra cập nhật, với WebSocket, server có thể đẩy cập nhật ngay khi có dữ liệu mới. Điều này giảm độ trễ đáng kể và giảm chi phí của việc thiết lập kết nối liên tục. Một khi kết nối WebSocket được thiết lập, nó được giữ mở cho đến khi một trong hai bên đóng nó, cho phép giao tiếp hai chiều với chi phí tối thiểu.

Quá trình bắt tay WebSocket bắt đầu với một HTTP request đặc biệt từ client. Client gửi HTTP GET request với các header đặc biệt: Upgrade websocket và Connection Upgrade, cùng với Sec-WebSocket-Key để bảo mật. Nếu server hỗ trợ WebSocket, nó phản hồi với mã trạng thái 101 Switching Protocols và các header tương ứng. Sau khi bắt tay thành công, kết nối được nâng cấp từ HTTP sang giao thức WebSocket, và cả hai bên có thể bắt đầu gửi WebSocket frame. Frame là đơn vị dữ liệu nhỏ nhất trong WebSocket, có thể chứa dữ liệu văn bản hoặc nhị phân, và có chi phí rất thấp chỉ 2-14 byte mỗi frame.

Độ trễ thấp là một trong những ưu điểm lớn nhất của WebSocket. Vì kết nối được giữ mở, không có chi phí của TCP handshake và HTTP header cho mỗi message. Message có thể được gửi ngay khi có dữ liệu mà không cần polling liên tục. Trong nhiều trường hợp thực tế, WebSocket thường cho độ trễ thấp hơn đáng kể so với polling truyền thống, dù giá trị cụ thể vẫn phụ thuộc vào điều kiện mạng và hạ tầng triển khai.

3.4.2 Redis Pub/Sub

Redis Pub/Sub là một hệ thống nhắn tin được tích hợp sẵn trong Redis, cho phép publisher gửi message đến channel và subscriber nhận message từ channel họ đăng ký. Redis Pub/Sub hoạt động hoàn toàn trong bộ nhớ, không có tính bền vững, và sử dụng ngữ nghĩa “bắn và quên”, message được gửi đến subscriber đang trực tuyến, nếu không có subscriber nào, message bị loại bỏ. Điều này làm cho Redis Pub/Sub rất nhanh nhưng không đáng tin cậy như hàng đợi message truyền thống.

Việc truyền thông điệp trong bộ nhớ là lý do Redis Pub/Sub có độ trễ cực thấp. Vì tất cả thao tác diễn ra trong RAM và không có I/O đĩa, message được gửi rất nhanh. Redis có thể xử lý lượng lớn message trên phần cứng tiêu chuẩn. Điều này làm cho Redis Pub/Sub phù hợp với các trường hợp sử dụng cần độ trễ thấp, như thông báo thời gian thực, dashboard trực tiếp và ứng dụng chat.

Đăng ký theo mẫu là một tính năng mạnh mẽ của Redis Pub/Sub. Thay vì đăng ký đến một channel cụ thể, subscriber có thể đăng ký đến một mẫu sử dụng ký tự đại diện. Điều này cho phép định tuyến linh hoạt mà không cần tạo nhiều đăng ký.

Ngữ nghĩa Fire and Forget có nghĩa là Redis không đảm bảo việc gửi message. Nếu không có subscriber nào đang lắng nghe channel khi message được phát hành, message bị mất. Nếu subscriber đang xử lý message và gặp sự cố, message cũng bị mất. Redis không có tính bền vững message, cơ chế xác nhận hay cơ chế thử lại. Điều này là sự đánh đổi cho hiệu suất, khi hệ thống ưu tiên độ trễ thấp hơn các đảm bảo về độ tin cậy.

3.4.3 So sánh các Real-time Protocols

Bảng 5: So sánh các Real-time Protocols

Tiêu chí	WebSocket	Server-Sent Events	Long Polling	gRPC
Latency	Rất thấp	Thấp	Cao	Rất thấp
Browser Support	Xuất sắc, trình duyệt hiện đại	Tốt, không hỗ trợ IE	Phổ quát	Cần thư viện hỗ trợ
Complexity	Trung bình	Thấp	Thấp	Cao
Bi-directional	Có, full-duplex	Không, chỉ server đến client	Có, mô phỏng	Có, full-duplex

WebSocket là lựa chọn tốt nhất cho giao tiếp thời gian thực hai chiều thực sự. Nó có độ trễ thấp, chi phí thấp, và được hỗ trợ rộng rãi. WebSocket phù hợp cho ứng dụng chat, chỉnh sửa cộng tác, game, và bất kỳ trường hợp sử dụng nào cần client và server gửi message cho nhau thường xuyên. Tuy nhiên, WebSocket có độ phức tạp cao hơn, cần xử lý quản lý kết nối, logic kết nối lại, và heartbeat.

Server-Sent Events là một giải pháp thay thế đơn giản hơn cho các tình huống chỉ cần server đẩy dữ liệu đến client. SSE sử dụng kết nối HTTP thông thường và được tích hợp sẵn trong trình duyệt qua EventSource API. SSE tự động kết nối lại khi kết nối bị ngắt và có ID sự kiện tích hợp sẵn để sắp xếp thứ tự message. Tuy nhiên, SSE không hỗ trợ dữ liệu nhị phân và không có giao tiếp hai chiều.

Long Polling là kỹ thuật cũ hơn, mô phỏng thời gian thực bằng cách client gửi HTTP request và server giữ request mở cho đến khi có dữ liệu hoặc hết thời gian chờ. Khi response được gửi, client sẽ gửi request mới để tiếp tục chờ dữ liệu. So với WebSocket hoặc các cơ chế push chuyên biệt, Long Polling thường có độ trễ cao hơn và chi phí lớn hơn do phải duy trì chu kỳ request-response lặp lại. Tuy nhiên, nó hoạt động với hầu hết hạ tầng HTTP và không gặp nhiều vấn đề tương thích.

gRPC Streaming cung cấp streaming hai chiều qua HTTP/2. Nó có hiệu suất tương đương với WebSocket và hỗ trợ message có kiểu mạnh thông qua Protocol Buffers.

gRPC phù hợp cho giao tiếp service-to-service trong kiến trúc microservice. Tuy nhiên, hỗ trợ trình duyệt yêu cầu gRPC-Web proxy và độ phức tạp cao hơn WebSocket.

Những khái niệm và so sánh trên là cơ sở để phân tích các lựa chọn giao tiếp thời gian thực trong SMAP, đặc biệt ở lớp WebSocket delivery và lớp fanout nội bộ dựa trên Redis Pub/Sub.

3.5 Data Storage Technologies

3.5.1 PostgreSQL

PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, được biết đến với độ tin cậy cao, tính năng phong phú và tuân thủ chuẩn SQL. PostgreSQL hỗ trợ cả truy vấn quan hệ truyền thống lẫn dữ liệu bán cấu trúc thông qua JSONB, nhờ đó phù hợp với nhiều kiểu bài toán trong các hệ thống doanh nghiệp và hệ thống phân tán hiện đại.

Tuân thủ ACID là một trong những điểm mạnh chính của PostgreSQL. ACID là viết tắt của Atomicity, Consistency, Isolation và Durability, giúp đảm bảo các giao dịch được xử lý an toàn và dữ liệu duy trì tính toàn vẹn ngay cả khi hệ thống gặp sự cố. PostgreSQL triển khai ACID thông qua cơ chế kiểm soát đồng thời đa phiên bản, cho phép đồng thời cao mà không cần phụ thuộc hoàn toàn vào khóa truyền thống.

Trong thực tế, PostgreSQL đặc biệt phù hợp cho các dữ liệu nghiệp vụ có cấu trúc rõ ràng, yêu cầu ràng buộc chặt chẽ và cần truy vấn quan hệ ổn định. Đây cũng là lớp lưu trữ quan hệ chủ đạo trong nhiều service của SMAP, nơi các metadata, trạng thái nghiệp vụ, lịch sử thao tác và dữ liệu phân tích được duy trì lâu dài.

3.5.2 Redis

Redis là một hệ thống lưu trữ dữ liệu trong bộ nhớ, thường được sử dụng như cache, session store, coordination layer hoặc message infrastructure nhẹ. Redis hỗ trợ nhiều cấu trúc dữ liệu như string, hash, list, set và sorted set, cho phép thực hiện các thao tác nhanh với độ trễ rất thấp.

Vì dữ liệu được xử lý chủ yếu trong RAM, Redis có hiệu suất rất cao và phù hợp với các trường hợp cần phản hồi nhanh. Tuy nhiên, Redis thường không được xem là lớp lưu trữ nghiệp vụ chính cho dữ liệu quan hệ dài hạn, mà đóng vai trò hỗ trợ cho các nhu cầu như cache, session management, rate limiting, blacklist token hoặc pub/sub thời gian thực.

Trong ngữ cảnh của SMAP, Redis có giá trị ở tính linh hoạt hơn là ở vai trò cơ sở dữ liệu trung tâm. Nó phù hợp cho các luồng cần hiệu năng cao như session/token blacklist trong xác thực, cache kết quả tìm kiếm, và fanout thông điệp thời gian thực cho notification layer.

3.5.3 Qdrant

Qdrant là một vector database được thiết kế để lưu trữ và truy vấn embedding vector với hiệu năng cao. Khác với cơ sở dữ liệu quan hệ truyền thống, Qdrant tối ưu cho các bài toán semantic search, similarity search và retrieval dựa trên khoảng cách vector.

Trong các hệ thống hiện đại có tích hợp xử lý ngôn ngữ tự nhiên hoặc hỏi đáp theo ngữ cảnh, vector database đóng vai trò quan trọng vì nó cho phép ánh xạ văn bản thành vector và tìm các nội dung gần nghĩa thay vì chỉ dựa vào khớp từ khóa. Nhờ đó, hệ thống có thể hỗ trợ các truy vấn ngữ nghĩa, tìm kiếm theo ngữ cảnh và các luồng khai thác thông tin nâng cao.

Đối với SMAP, Qdrant phù hợp với lớp khai thác thông tin theo ngữ cảnh, nơi dữ liệu đã qua xử lý và phân tích cần được lập chỉ mục để phục vụ semantic search, chat và các khả năng truy vấn dựa trên embedding. Vì vậy, Qdrant không thay thế PostgreSQL hay Redis, mà bổ sung một lớp lưu trữ tối ưu cho dữ liệu vector.

3.5.4 MinIO

MinIO là một hệ thống object storage tương thích S3, phù hợp cho việc lưu trữ các file và artifact kích thước lớn như raw payload, file báo cáo, batch outputs hoặc các tài nguyên trung gian trong pipeline dữ liệu. Khác với cơ sở dữ liệu quan hệ hay vector database, MinIO không tối ưu cho truy vấn nghiệp vụ chi tiết, mà tối ưu cho lưu trữ và truy xuất đối tượng.

Object storage đặc biệt hữu ích trong các hệ thống xử lý dữ liệu nhiều giai đoạn vì nó giúp tách phần dữ liệu tải trọng lớn khỏi các message hoặc metadata điều phối. Cách tiếp cận này làm cho các lane xử lý bất đồng bộ gọn hơn, đồng thời giữ lại được artifact gốc để truy vết, kiểm tra hoặc xử lý lại khi cần.

Trong SMAP, MinIO phù hợp với nhu cầu lưu raw crawl artifacts, kết quả trung gian và các tài liệu đầu ra như report bundles. Nó đóng vai trò là lớp lưu trữ file/object, bổ sung cho PostgreSQL, Redis và Qdrant trong kiến trúc lưu trữ tổng thể của hệ thống.

3.5.5 Polyglot Persistence

Polyglot Persistence là phương pháp sử dụng nhiều công nghệ lưu trữ khác nhau trong cùng một hệ thống, trong đó mỗi công nghệ được lựa chọn dựa trên kiểu dữ liệu và nhu cầu truy cập cụ thể. Thay vì cố gắng giải quyết mọi bài toán bằng một cơ sở dữ liệu duy nhất, hệ thống tận dụng điểm mạnh của từng loại lưu trữ để tối ưu cho từng lớp chức năng.

Trong thực tế, một hệ thống có thể dùng cơ sở dữ liệu quan hệ cho metadata và giao dịch, cache hoặc in-memory store cho hiệu năng và điều phối, vector database cho semantic retrieval, và object storage cho các file hoặc artifact dung lượng lớn. Cách tiếp

cận này giúp tối ưu hóa hiệu năng, khả năng mở rộng và tính phù hợp của từng lớp dữ liệu.

Bảng 6: So sánh các công nghệ lưu trữ dữ liệu

Tiêu chí	PostgreSQL	Redis	Qdrant	MinIO
Loại dữ liệu	Dữ liệu quan hệ và metadata	Cache, session, pub/sub	Embedding vector và semantic index	File, object, raw artifacts
Mô hình truy cập	SQL và transaction	Key-value / in-memory operations	Similarity search theo vector	Object read/write
Điểm mạnh	Tính nhất quán và truy vấn quan hệ mạnh	Độ trễ thấp và hiệu năng cao	Tối ưu cho semantic search	Lưu trữ artifact lớn và bền vững
Vai trò trong SMAP	Lưu trữ nghiệp vụ và metadata chính	Session, cache và realtime fanout	Tìm kiếm và khai thác thông tin theo ngữ cảnh	Lưu raw results và outputs

Với SMAP, cách tiếp cận polyglot persistence phản ánh đúng bản chất của hệ thống đa dịch vụ: không có một loại lưu trữ nào tối ưu cho mọi nhu cầu. PostgreSQL phù hợp cho metadata và dữ liệu nghiệp vụ, Redis phù hợp cho cache và coordination nhẹ, Qdrant phù hợp cho semantic retrieval, còn MinIO phù hợp cho object storage. Sự kết hợp này tạo ra một kiến trúc lưu trữ linh hoạt và bám sát nhu cầu xử lý thực tế của hệ thống.

3.6 Authentication và Security

3.6.1 JWT

JSON Web Token là một tiêu chuẩn mở theo RFC 7519 định nghĩa một cách gọn nhẹ và tự chứa để truyền thông tin an toàn giữa các bên dưới dạng đối tượng JSON. JWT được sử dụng rộng rãi cho xác thực và trao đổi thông tin trong các ứng dụng web và API. Một trong những lợi ích chính của JWT là xác thực không trạng thái, server không cần lưu trữ dữ liệu phiên, tất cả thông tin cần thiết được mã hóa trong chính token.

JWT bao gồm ba phần được phân tách bởi dấu chấm: Header, Payload, và Signature. Header chứa siêu dữ liệu về token, thường bao gồm loại và thuật toán ký. Payload chứa các claim, các tuyên bố về thực thể thường là người dùng và siêu dữ liệu bổ sung. Signature được tạo bằng cách mã hóa header và payload, sau đó ký với khóa bí mật sử dụng thuật toán được chỉ định trong header.

Các claim trong JWT payload có thể là các claim đã đăng ký được định nghĩa trong đặc tả như iss (issuer), exp (expiration time), sub (subject), các claim công khai được định nghĩa bởi người dùng, hoặc các claim riêng tư được thỏa thuận giữa các bên. Các

claim đã đăng ký cung cấp khả năng tương tác, trong khi các claim tùy chỉnh cho phép truyền dữ liệu cụ thể của ứng dụng.

Xác thực không trạng thái là lợi ích chính của JWT. Server không cần lưu trữ dữ liệu phiên, tất cả thông tin cần thiết nằm trong token. Điều này cho phép mở rộng theo chiều ngang dễ dàng, bất kỳ server nào cũng có thể xác minh token mà không cần kho lưu trữ phiên chia sẻ. Tuy nhiên, không trạng thái cũng có nhược điểm, không thể vô hiệu hóa token trước thời gian hết hạn mà không có các cơ chế bổ sung như danh sách đen token.

3.6.2 HttpOnly Cookie

HttpOnly Cookie là một thuộc tính cookie được đặt bởi server để ngăn các script phía client truy cập cookie. Khi cookie có cờ HttpOnly, mã JavaScript không thể đọc giá trị cookie thông qua `document.cookie`. Điều này là một biện pháp bảo mật quan trọng để bảo vệ chống lại các cuộc tấn công Cross-Site Scripting. HttpOnly cookie thường được sử dụng để lưu trữ các token xác thực như JWT, ID phiên, và dữ liệu nhạy cảm khác.

Cookie có nhiều thuộc tính kiểm soát hành vi và bảo mật của chúng. HttpOnly ngăn truy cập JavaScript, bảo vệ chống lại XSS. Secure đảm bảo cookie chỉ được gửi qua kết nối HTTPS, bảo vệ chống lại các cuộc tấn công man-in-the-middle. SameSite kiểm soát khi nào cookie được gửi với các request cross-site, có ba giá trị: Strict không bao giờ gửi với request cross-site, Lax gửi với điều hướng cấp cao nhất, và None luôn gửi nhưng yêu cầu Secure.

3.6.3 OAuth2

OAuth2 là một khuôn khổ ủy quyền phổ biến cho phép người dùng xác thực thông qua một nhà cung cấp danh tính bên ngoài mà không cần chia sẻ trực tiếp thông tin xác thực với từng ứng dụng. Trong các hệ thống web hiện đại, OAuth2 thường được dùng để triển khai đăng nhập thông qua các nhà cung cấp như Google, Microsoft hoặc các dịch vụ identity chuyên biệt.

Luồng phổ biến của OAuth2 bắt đầu khi người dùng được chuyển hướng đến nhà cung cấp danh tính để đăng nhập và cấp quyền. Sau khi xác thực thành công, nhà cung cấp chuyển người dùng quay lại ứng dụng cùng với một mã ủy quyền. Ứng dụng sau đó sử dụng mã này để đổi lấy token hoặc thông tin định danh cần thiết. Cách tiếp cận này giúp giảm việc quản lý trực tiếp mật khẩu trong ứng dụng, đồng thời tận dụng được các chính sách bảo mật và quản lý tài khoản của hệ thống identity bên ngoài.

Trong các kiến trúc hiện đại, OAuth2 thường được kết hợp với JWT hoặc session cookie để xây dựng phiên làm việc sau đăng nhập. Nhờ đó, OAuth2 giải quyết bài toán xác thực ban đầu, còn JWT hoặc cookie đóng vai trò duy trì trạng thái truy cập trong quá trình người dùng sử dụng hệ thống.

3.6.4 Security Practices

Xác thực đầu vào là tuyến phòng thủ đầu tiên chống lại nhiều cuộc tấn công. Tất cả đầu vào của người dùng phải được xác thực trước khi xử lý. Xác thực nên kiểm tra loại dữ liệu, định dạng, độ dài, và các giá trị được phép. Phòng chống SQL injection tốt nhất là sử dụng truy vấn có tham số thay vì nối chuỗi.

HTTPS/TLS mã hóa tất cả giao tiếp giữa client và server, bảo vệ chống lại nghe lén và các cuộc tấn công man-in-the-middle. TLS 1.3 là phiên bản mới nhất với bảo mật và hiệu suất được cải thiện. Tất cả các ứng dụng sản xuất nên sử dụng HTTPS độc quyền.

Giới hạn tốc độ hạn chế số lượng request mà một client có thể thực hiện trong một khoảng thời gian, bảo vệ chống lại các cuộc tấn công brute-force và denial-of-service. Giới hạn tốc độ có thể được triển khai ở nhiều cấp độ: theo IP, theo người dùng, theo endpoint.

CORS là một cơ chế bảo mật kiểm soát các request cross-origin. Trình duyệt thực thi chính sách same-origin, ngăn các script từ một origin truy cập tài nguyên từ origin khác. Các header CORS cho phép server chỉ định origin nào được phép truy cập tài nguyên.

Những khái niệm trên là cơ sở để hiểu mô hình xác thực và bảo mật của SMAP, đặc biệt ở các luồng OAuth2, JWT, HttpOnly cookie, token validation nội bộ và bảo vệ kết nối WebSocket trong quá trình vận hành hệ thống.

3.7 Công nghệ Backend

3.7.1 Go và Gin Framework

Go là một ngôn ngữ lập trình mã nguồn mở được phát triển bởi Google, được thiết kế để xây dựng phần mềm đơn giản, đáng tin cậy, và hiệu quả. Go được công bố năm 2009 bởi Robert Griesemer, Rob Pike, và Ken Thompson. Go kết hợp sự đơn giản của các ngôn ngữ kiểu động với tính an toàn và hiệu suất của các ngôn ngữ kiểu tĩnh. Go đã trở thành một trong những ngôn ngữ phổ biến nhất để xây dựng microservice, ứng dụng cloud-native, và hệ thống phân tán.

Hiệu suất là một trong những điểm mạnh chính của Go. Go là ngôn ngữ biên dịch, mã được biên dịch thành mã máy gốc, cho phép tốc độ thực thi gần với C/C++. Trình biên dịch Go rất nhanh, có thể biên dịch các codebase lớn trong vài giây. Go có bộ thu gom rác hiệu quả, tự động quản lý bộ nhớ mà không hy sinh hiệu suất đáng kể. Bộ thu gom rác của Go được tối ưu hóa cho độ trễ thấp, với thời gian tạm dừng thường dưới 1ms.

Đồng thời là tính năng nổi bật nhất của Go, được triển khai thông qua goroutine và channel. Goroutine là các luồng nhẹ được quản lý bởi Go runtime, có thể tạo hàng nghìn hoặc hàng triệu goroutine mà không làm quá tải hệ thống. Mỗi goroutine chỉ tiêu thụ vài KB bộ nhớ ban đầu, so với các luồng truyền thống tiêu thụ MB. Channel là

các kênh có kiểu để giao tiếp giữa các goroutine, triển khai mô hình CSP. Channel cho phép chia sẻ dữ liệu an toàn giữa các goroutine mà không cần khóa rõ ràng.

Sự đơn giản là triết lý thiết kế cốt lõi của Go. Go có cú pháp tối giản với chỉ 25 từ khóa, so với 50+ trong Java hay C++. Go không có class, kế thừa, hay nhiều tính năng phức tạp khác. Thay vào đó, Go tập trung vào composition thông qua interface và struct. Điều này làm cho mã Go dễ đọc và bảo trì.

Gin là một HTTP web framework được viết bằng Go, được biết đến với hiệu suất cao và thiết kế tối giản. Gin được phát triển năm 2014 và đã trở thành một trong những Go web framework phổ biến nhất. Gin cung cấp bộ định tuyến HTTP nhanh, hỗ trợ middleware, xác thực JSON, xử lý lỗi, và nhiều tính năng khác cần thiết để xây dựng REST API.

Bộ định tuyến HTTP nhanh là điểm mạnh cốt lõi của Gin. Gin sử dụng bộ định tuyến dựa trên radix tree để khớp tuyến đường cực nhanh. Bộ định tuyến có thể xử lý hàng triệu request mỗi giây với độ trễ dưới millisecond. Tham số tuyến đường, ký tự đại diện, và nhóm tuyến đường được hỗ trợ với cú pháp đơn giản.

Hỗ trợ middleware cho phép chèn logic vào pipeline xử lý request. Middleware có thể xử lý xác thực, ghi log, CORS, giới hạn tốc độ, và nhiều mối quan tâm xuyên suốt khác. Gin cung cấp nhiều middleware tích hợp sẵn và cho phép tạo middleware tùy chỉnh dễ dàng.

Xác thực JSON được tích hợp thông qua thẻ struct và thư viện validator. Nội dung request có thể được liên kết và xác thực tự động. Lỗi xác thực được trả về với thông báo chi tiết. Điều này giảm mã boilerplate và đảm bảo tính toàn vẹn dữ liệu.

3.7.2 SQLBoiler

SQLBoiler là một công cụ ORM cho Go, khác biệt với các ORM truyền thống bằng cách tạo mã an toàn kiểu từ lược đồ cơ sở dữ liệu. Thay vì định nghĩa model trong mã và đồng bộ với cơ sở dữ liệu, SQLBoiler kiểm tra cơ sở dữ liệu hiện có và tạo mã Go tương ứng. Điều này đảm bảo các model luôn đồng bộ với lược đồ cơ sở dữ liệu và cung cấp tính an toàn kiểu tại thời điểm biên dịch.

ORM an toàn kiểu có nghĩa là tất cả các thao tác cơ sở dữ liệu được kiểm tra kiểu tại thời điểm biên dịch. Không có truy vấn dựa trên chuỗi hay xác nhận kiểu runtime. Nếu cột không tồn tại hoặc kiểu không khớp, mã sẽ không biên dịch được. Điều này phát hiện lỗi sớm và ngăn ngừa nhiều lỗi runtime.

Phương pháp tạo mã có nhiều lợi ích. Mã được tạo có thể đọc và debug được. Hiệu suất tốt hơn các ORM dựa trên reflection. Hỗ trợ IDE tốt với tự động hoàn thành và kiểm tra kiểu. Tuy nhiên, cần tạo lại mã khi lược đồ thay đổi.

3.7.3 Python và FastAPI

Python là một ngôn ngữ lập trình đa năng, được biết đến với cú pháp rõ ràng và hệ sinh thái phong phú. Python đặc biệt mạnh trong khoa học dữ liệu, học máy, và scripting. Trong bối cảnh của SMAP, Python được sử dụng cho analysis-srv và scraper service, nơi cần tận dụng các thư viện xử lý dữ liệu, AI/ML và cơ chế worker bất đồng bộ.

FastAPI là một web framework hiện đại cho Python, được thiết kế để xây dựng API với hiệu suất cao. FastAPI sử dụng gợi ý kiểu Python để tự động xác thực, tuần tự hóa, và tài liệu hóa. FastAPI có hiệu suất tốt nhờ sử dụng Starlette và Pydantic, đồng thời phù hợp với các service cần kết hợp API layer với các luồng xử lý bất đồng bộ.

Hỗ trợ bất đồng bộ trong FastAPI cho phép xử lý các request đồng thời một cách hiệu quả. Python asyncio được tích hợp sẵn, cho phép các thao tác I/O không chặn. Điều này quan trọng đối với các service vừa cung cấp API vừa phải phối hợp với worker runtime hoặc các pipeline xử lý dữ liệu.

Những lựa chọn công nghệ backend trên phản ánh hướng tiếp cận đa ngôn ngữ của SMAP: Go và Gin phù hợp với các service điều phối, API và business control plane, trong khi Python và FastAPI phù hợp hơn với các thành phần thiên về analytics pipeline và scraper runtime.

3.8 Cơ sở kỹ thuật AI, NLP và khai thác tri thức

Trong các hệ thống social listening, trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên được sử dụng nhằm hỗ trợ chuyển đổi dữ liệu văn bản phi cấu trúc thành các tín hiệu có thể phân tích được. Dữ liệu mạng xã hội thường có đặc điểm ngắn, nhiều, không chuẩn hóa, nhiều biểu tượng cảm xúc, từ viết tắt, tiếng lóng và phụ thuộc mạnh vào bối cảnh. Vì vậy, việc phân tích loại dữ liệu này cần kết hợp nhiều kỹ thuật khác nhau như tiền xử lý văn bản, trích xuất từ khóa, phân tích cảm xúc, biểu diễn ngữ nghĩa bằng embedding, tìm kiếm vector và sinh ngôn ngữ có kiểm soát.

Trong phạm vi cơ sở kỹ thuật, phần này tập trung vào các khái niệm và công nghệ nền tảng giúp hiểu vì sao các kỹ thuật AI/NLP phù hợp với bài toán phân tích dữ liệu mạng xã hội. Các kỹ thuật này đóng vai trò hỗ trợ tổng hợp và khai thác thông tin; kết quả của chúng cần được diễn giải trong mối quan hệ với chất lượng dữ liệu đầu vào, phạm vi mô hình và ngữ cảnh nghiệp vụ.

3.8.1 Xử lý ngôn ngữ tự nhiên trong social listening

Xử lý ngôn ngữ tự nhiên, hay Natural Language Processing, là lĩnh vực nghiên cứu và ứng dụng các phương pháp tính toán để máy tính có thể xử lý, phân tích và tạo ra ngôn ngữ của con người. Trong social listening, NLP thường được dùng để làm sạch văn bản, chuẩn hóa nội dung, phát hiện chủ đề, trích xuất từ khóa, nhận diện thực thể, phân tích cảm xúc và hỗ trợ tổng hợp thông tin.

Khác với văn bản báo chí hoặc tài liệu chính thức, nội dung mạng xã hội thường thiếu cấu trúc và có độ biến thiên cao. Một ý kiến có thể được thể hiện bằng câu ngắn, từ lóng, biểu tượng cảm xúc, lỗi chính tả hoặc cách viết pha trộn giữa nhiều ngôn ngữ. Do đó, pipeline NLP cho social listening thường không chỉ dựa vào một mô hình duy nhất, mà kết hợp nhiều bước xử lý nối tiếp. Mỗi bước tạo thêm một lớp tín hiệu để các bước sau có thể phân tích chính xác và có ngữ cảnh hơn.

Một pipeline NLP điển hình có thể bao gồm tiền xử lý văn bản, phân loại nội dung, trích xuất từ khóa hoặc thực thể, phân tích cảm xúc tổng thể, phân tích cảm xúc theo khía cạnh và tổng hợp các chỉ số hỗ trợ đánh giá. Cách tổ chức theo pipeline giúp bài toán phức tạp được chia thành nhiều bài toán nhỏ hơn, đồng thời giúp kết quả trung gian có thể kiểm tra và giải thích được.

3.8.2 Tiền xử lý văn bản tiếng Việt

Tiền xử lý văn bản là bước chuẩn bị dữ liệu trước khi đưa vào các thuật toán hoặc mô hình NLP. Các thao tác thường gặp gồm chuẩn hóa chữ hoa chữ thường, loại bỏ ký tự không cần thiết, xử lý URL, biểu tượng cảm xúc, khoảng trắng, dấu câu, từ lặp, hashtag và các phần nhiễu khác. Với dữ liệu mạng xã hội, bước này đặc biệt quan trọng vì dữ liệu đầu vào thường không tuân theo chuẩn chính tả hoặc cấu trúc câu đầy đủ.

Đối với tiếng Việt, tiền xử lý còn có thêm thách thức về tách từ. Tiếng Việt là ngôn ngữ đơn lập, trong đó một từ có thể gồm nhiều âm tiết được viết cách nhau bằng khoảng trắng. Ví dụ, “chất lượng” là một đơn vị từ vựng nhưng gồm hai tiếng. Nếu không tách từ phù hợp, mô hình hoặc thuật toán phía sau có thể hiểu sai ranh giới từ, làm giảm chất lượng phân tích. Vì vậy, các công cụ tách từ tiếng Việt như PyVi thường được sử dụng như một bước chuẩn bị cho các mô hình ngôn ngữ tiếng Việt.

Tiền xử lý không làm cho dữ liệu trở nên đúng tuyệt đối, nhưng giúp giảm nhiễu và làm cho dữ liệu nhất quán hơn. Đây là điều kiện quan trọng để các bước như trích xuất từ khóa, phân tích cảm xúc và tạo embedding hoạt động ổn định hơn.

3.8.3 Phân tích cảm xúc và mô hình ngôn ngữ tiếng Việt

Phân tích cảm xúc, hay sentiment analysis, là bài toán xác định thái độ hoặc sắc thái cảm xúc trong văn bản. Trong social listening, phân tích cảm xúc thường được dùng để nhận diện nội dung tích cực, tiêu cực hoặc trung tính, từ đó hỗ trợ đánh giá phản ứng của cộng đồng đối với thương hiệu, sản phẩm, sự kiện hoặc chiến dịch truyền thông.

Với tiếng Việt, các mô hình ngôn ngữ tiền huấn luyện như PhoBERT có ý nghĩa quan trọng vì chúng được xây dựng để biểu diễn tốt hơn đặc trưng của tiếng Việt. PhoBERT thuộc nhóm mô hình dựa trên kiến trúc Transformer, có khả năng học biểu diễn ngữ cảnh của từ trong câu. Nhờ đó, mô hình có thể phân biệt một từ hoặc cụm từ trong các ngữ cảnh khác nhau, thay vì chỉ dựa trên khớp từ đơn giản.

Trong bài toán phân loại cảm xúc, văn bản đầu vào thường được token hóa, đưa qua mô hình ngôn ngữ, sau đó ánh xạ sang các nhãn cảm xúc. Đầu ra có thể bao gồm nhãn dự đoán và độ tin cậy. Tuy nhiên, độ tin cậy này chỉ phản ánh mức chắc chắn của mô hình trên đầu vào cụ thể, không phải bảo đảm tuyệt đối về tính đúng sai của kết quả trong thực tế.

Ngoài phân tích cảm xúc tổng thể, một hướng mở rộng quan trọng là phân tích cảm xúc theo khía cạnh, hay Aspect-Based Sentiment Analysis. Thay vì chỉ kết luận toàn bộ bài viết là tích cực hoặc tiêu cực, ABSA cố gắng xác định cảm xúc đối với từng khía cạnh cụ thể, chẳng hạn giá cả, chất lượng, dịch vụ, giao hàng hoặc hỗ trợ khách hàng. Cách tiếp cận này phù hợp với social listening vì một nội dung có thể khen một khía cạnh nhưng chê một khía cạnh khác.

3.8.4 Trích xuất từ khóa, thực thể và khía cạnh

Trích xuất từ khóa là quá trình xác định các từ hoặc cụm từ có khả năng đại diện cho nội dung chính của văn bản. Trong dữ liệu mạng xã hội, từ khóa giúp nhận diện chủ đề đang được thảo luận, các vấn đề nổi bật và những cụm từ lặp lại trong phản hồi của người dùng. Các phương pháp trích xuất từ khóa có thể dựa trên thống kê, luật, từ điển hoặc mô hình học máy.

YAKE là một phương pháp trích xuất từ khóa không giám sát, dựa trên các đặc trưng thống kê trong chính văn bản đầu vào. Điểm mạnh của cách tiếp cận này là không cần tập dữ liệu huấn luyện riêng và có thể áp dụng linh hoạt cho nhiều miền dữ liệu. Tuy nhiên, vì dựa nhiều vào đặc trưng bề mặt của văn bản, kết quả có thể bị ảnh hưởng bởi dữ liệu quá ngắn, nhiễu hoặc thiếu ngữ cảnh.

spaCy là một thư viện NLP cung cấp nhiều khả năng xử lý ngôn ngữ như tokenization, part-of-speech tagging, dependency parsing và named entity recognition tùy theo model ngôn ngữ được sử dụng. Trong bài toán phân tích nội dung, named entity recognition giúp phát hiện tên người, tổ chức, địa điểm, sản phẩm hoặc các thực thể có ý nghĩa. Khi kết hợp với từ điển nghiệp vụ, kết quả từ khóa và thực thể có thể được ánh xạ về các khía cạnh phân tích ổn định hơn.

Khía cạnh, hay aspect, là nhóm ý nghĩa dùng để gom các từ khóa và phản hồi liên quan đến cùng một chủ đề phân tích. Việc ánh xạ từ khóa về aspect giúp kết quả phân tích có cấu trúc hơn, hỗ trợ thống kê, lọc, so sánh và tổng hợp báo cáo. Trong thực tế, aspect extraction thường cần kết hợp giữa mô hình tự động và tri thức miền, vì cùng một từ có thể mang ý nghĩa khác nhau trong các ngữ cảnh khác nhau.

3.8.5 Chỉ số ảnh hưởng, rủi ro và khả năng giải thích

Trong social listening, không phải nội dung nào cũng có mức độ quan trọng như nhau. Một phản hồi tiêu cực từ tài khoản ít tương tác có thể có tác động khác với một phản hồi tiêu cực đang được chia sẻ rộng rãi. Vì vậy, ngoài phân tích nội dung, hệ thống

phân tích mạng xã hội thường cần các chỉ số bổ sung để ước lượng mức độ ảnh hưởng và rủi ro.

Các chỉ số ảnh hưởng có thể dựa trên lượng tương tác như lượt thích, bình luận, chia sẻ, lượt xem, quy mô người theo dõi hoặc mức độ lan truyền. Các chỉ số rủi ro có thể kết hợp tín hiệu cảm xúc tiêu cực, từ khóa nhạy cảm, tốc độ lan truyền và mức độ liên quan đến đối tượng theo dõi. Những chỉ số này không nhất thiết là mô hình AI thuần túy; chúng có thể là sự kết hợp giữa kết quả NLP, luật nghiệp vụ và công thức tính điểm.

Một yêu cầu quan trọng của các chỉ số này là tính giải thích được. Nếu hệ thống chỉ đưa ra một điểm số mà không cho biết yếu tố nào đóng góp vào điểm số đó, người dùng khó đánh giá mức độ tin cậy hoặc hành động phù hợp. Vì vậy, trong các hệ thống phân tích, việc lưu lại các yếu tố như sentiment, keyword, mức tương tác, nền tảng và lý do đánh giá rủi ro giúp kết quả dễ kiểm tra hơn.

3.8.6 Embedding, vector search và cơ sở dữ liệu vector

Embedding là kỹ thuật biểu diễn văn bản dưới dạng vector số học trong không gian nhiều chiều. Ý tưởng chính là các văn bản có ý nghĩa gần nhau sẽ có vector gần nhau hơn so với các văn bản ít liên quan. Nhờ đó, hệ thống có thể thực hiện tìm kiếm ngữ nghĩa, tức tìm nội dung gần nghĩa với câu truy vấn, thay vì chỉ tìm các văn bản khớp chính xác từ khóa.

Tìm kiếm vector thường sử dụng các độ đo khoảng cách hoặc độ tương đồng như cosine similarity, dot product hoặc Euclidean distance. Trong các hệ thống có số lượng văn bản lớn, việc tìm kiếm tuyến tính qua toàn bộ vector là không hiệu quả, vì vậy các cơ sở dữ liệu vector được sử dụng để lưu trữ, lập chỉ mục và truy vấn embedding. Qdrant là một ví dụ của vector database, hỗ trợ lưu vector kèm payload metadata và truy vấn kết hợp giữa similarity search với các điều kiện lọc.

Trong bối cảnh social listening, vector search hữu ích khi người dùng muốn tìm các phản hồi có cùng ý nghĩa nhưng không dùng cùng từ khóa. Ví dụ, các câu diễn đạt sự không hài lòng có thể khác nhau về từ ngữ nhưng gần nhau về sắc thái. Khi kết hợp embedding với metadata như thời gian, nền tảng, sentiment hoặc aspect, hệ thống có thể hỗ trợ các truy vấn vừa theo ngữ nghĩa vừa theo điều kiện nghiệp vụ.

3.8.7 Retrieval-Augmented Generation và mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn, hay Large Language Model, có khả năng sinh văn bản, tóm tắt, trả lời câu hỏi và diễn giải thông tin bằng ngôn ngữ tự nhiên. Tuy nhiên, nếu chỉ dựa vào tri thức nội tại của mô hình, câu trả lời có thể không phản ánh đúng dữ liệu cụ thể của hệ thống. Đây là lý do Retrieval-Augmented Generation được sử dụng trong nhiều ứng dụng hỏi đáp và tổng hợp tri thức.

Retrieval-Augmented Generation kết hợp hai bước: truy hồi tài liệu liên quan và sinh câu trả lời dựa trên tài liệu đã truy hồi. Ở bước đầu, hệ thống tìm các đoạn dữ liệu phù hợp với câu hỏi bằng tìm kiếm từ khóa, tìm kiếm vector hoặc kết hợp cả hai. Ở bước sau, các tài liệu liên quan được đưa vào prompt như context để LLM tạo câu trả lời. Cách này giúp câu trả lời bám sát dữ liệu hơn và có thể kèm trích dẫn nguồn.

RAG không loại bỏ hoàn toàn rủi ro sinh sai thông tin. Nếu tài liệu truy hồi không liên quan, thiếu dữ liệu hoặc chứa thông tin sai, câu trả lời vẫn có thể bị ảnh hưởng. Vì vậy, các hệ thống RAG thường cần cơ chế giới hạn context, đánh giá độ liên quan, yêu cầu trích dẫn, từ chối trả lời khi không đủ dữ liệu và phân biệt rõ giữa dữ liệu quan sát được với nhận định suy luận.

LLM cũng có thể được sử dụng để hỗ trợ sinh báo cáo. Trong trường hợp này, mô hình không chỉ trả lời một câu hỏi ngắn mà tổng hợp nhiều kết quả phân tích thành các phần nội dung có cấu trúc. Để hạn chế rủi ro vượt quá giới hạn context hoặc tạo báo cáo quá chung chung, dữ liệu đầu vào thường được tổng hợp, chọn mẫu và chia thành từng phần trước khi sinh nội dung.

3.8.8 Giới hạn của AI trong phân tích dữ liệu mạng xã hội

Các kỹ thuật AI/NLP phụ thuộc mạnh vào chất lượng dữ liệu đầu vào. Nếu dữ liệu bị thiếu, nhiễu, sai ngữ cảnh, không đại diện hoặc chứa nội dung mơ hồ, kết quả phân tích cũng có thể sai lệch. Nguyên tắc này thường được gọi là “garbage in, garbage out”: đầu vào kém chất lượng sẽ dẫn đến đầu ra kém chất lượng, dù mô hình hoặc thuật toán phía sau có phức tạp đến đâu.

Ngoài chất lượng dữ liệu, các giới hạn khác còn đến từ phạm vi ngôn ngữ, miền dữ liệu, tập huấn luyện của mô hình, độ dài văn bản, khả năng hiểu ngữ cảnh xã hội và sự thay đổi liên tục của ngôn ngữ mạng. Một mô hình hoạt động tốt trên miền dữ liệu này chưa chắc giữ nguyên chất lượng khi chuyển sang miền khác. Các từ lóng, meme, mĩa mai, châm biếm hoặc nội dung đa nghĩa cũng là những trường hợp khó đối với hệ thống tự động.

Vì vậy, AI trong social listening nên được xem là công cụ hỗ trợ phân tích, không phải nguồn kết luận tuyệt đối. Giá trị chính của các kỹ thuật này nằm ở khả năng xử lý lượng lớn dữ liệu, phát hiện tín hiệu đáng chú ý, hỗ trợ tìm kiếm và tổng hợp thông tin. Đối với các quyết định quan trọng, kết quả tự động cần được đặt trong quy trình đánh giá có kiểm chứng và có thể cần sự xem xét của con người.

3.9 Công nghệ Frontend

3.9.1 Next.js

Next.js là một React framework được phát triển bởi Vercel, cung cấp nhiều tính năng để xây dựng các ứng dụng web sẵn sàng sản xuất. Next.js hỗ trợ Server-Side Rendering,

Static Site Generation, và Client-Side Rendering, cho phép chọn chiến lược render phù hợp cho từng trang. Next.js đã trở thành một trong những React framework phổ biến nhất cho các ứng dụng doanh nghiệp.

Server-Side Rendering cho phép render các React component trên server và gửi HTML đến client. Điều này cải thiện thời gian tải trang ban đầu và SEO vì các công cụ tìm kiếm có thể lập chỉ mục nội dung. SSR đặc biệt quan trọng cho các trang có nhiều nội dung và ứng dụng cần SEO tốt.

Next.js hỗ trợ định tuyến dựa trên hệ thống file và có thể được tổ chức theo nhiều chiến lược khác nhau tùy theo giai đoạn phát triển của dự án. Trong frontend của SMAP, có thể thấy cả hướng tổ chức Pages Router lẫn App Router, cho thấy lớp giao diện có khả năng mở rộng và tái cấu trúc dần theo nhu cầu thực tế. Điều này phản ánh tính linh hoạt của Next.js trong việc hỗ trợ nhiều cách tổ chức routing trong cùng một hệ thống.

API Routes cho phép tạo các endpoint API trong cùng ứng dụng Next.js. Điều này hữu ích cho mẫu backend-for-frontend, nơi frontend cần lớp API để tổng hợp dữ liệu từ nhiều service. API Routes có thể xử lý xác thực, chuyển đổi dữ liệu, và bộ nhớ đệm.

3.9.2 React

React là một thư viện JavaScript để xây dựng giao diện người dùng, được phát triển bởi Facebook. React sử dụng kiến trúc dựa trên component, nơi UI được chia thành các component có thể tái sử dụng. React đã trở thành một trong những thư viện frontend phổ biến nhất với hệ sinh thái rộng lớn.

Kiến trúc dựa trên component cho phép xây dựng UI phức tạp từ các phần nhỏ, độc lập. Mỗi component quản lý trạng thái và render của riêng nó. Các component có thể được kết hợp để tạo UI phức tạp. Điều này tăng khả năng tái sử dụng và bảo trì.

Virtual DOM là một trong những đổi mới của React. Thay vì thao tác DOM trực tiếp, React tạo biểu diễn ảo của DOM trong bộ nhớ. Khi trạng thái thay đổi, React tính toán các thay đổi tối thiểu cần thiết và cập nhật DOM theo lô. Điều này cải thiện hiệu suất đáng kể.

Hook là tính năng được giới thiệu trong React 16.8, cho phép sử dụng trạng thái và tính năng vòng đời trong các functional component. useState cho trạng thái cục bộ, useEffect cho tác dụng phụ, useContext cho tiêu thụ context. Hook làm cho mã ngắn gọn và dễ kiểm thử hơn class component.

3.9.3 TypeScript

TypeScript là một tập cha của JavaScript thêm kiểu tĩnh. TypeScript được phát triển bởi Microsoft và đã trở thành tiêu chuẩn cho các ứng dụng JavaScript quy mô lớn. Mã TypeScript được biên dịch thành JavaScript, có thể chạy trên bất kỳ runtime JavaScript nào.

Kiểu tĩnh là lợi ích chính của TypeScript. Các kiểu được kiểm tra tại thời điểm biên dịch, phát hiện lỗi trước khi mã chạy. Hỗ trợ IDE tốt hơn với tự động hoàn thành, tái cấu trúc, và điều hướng. Tài liệu được nhúng trong mã thông qua các kiểu.

Suy luận kiểu giảm mã boilerplate. TypeScript có thể suy luận kiểu từ ngữ cảnh, không cần chú thích mọi biến. Điều này cân bằng giữa tính an toàn kiểu và năng suất của nhà phát triển.

Interface và Type cho phép định nghĩa hình dạng của các đối tượng. Interface có thể mở rộng và triển khai. Union type và intersection type cho phép biểu đạt các mối quan hệ kiểu phức tạp. Generic cho phép mã có thể tái sử dụng và an toàn kiểu.

3.9.4 Tailwind CSS

Tailwind CSS là một CSS framework utility-first, cung cấp các class tiện ích cấp thấp thay vì các component được thiết kế sẵn. Tailwind cho phép xây dựng thiết kế tùy chỉnh mà không cần viết CSS tùy chỉnh. Tailwind đã trở thành một trong những CSS framework phổ biến nhất cho phát triển web hiện đại.

Phương pháp utility-first có nghĩa là styling được thực hiện bằng cách kết hợp các class tiện ích. Thay vì viết CSS tùy chỉnh cho mỗi component, các nhà phát triển sử dụng các class được định nghĩa sẵn như flex, pt-4, text-center. Điều này tăng tốc độ phát triển và tính nhất quán.

Thiết kế responsive được tích hợp sẵn với các tiền tố responsive. Các class như md:flex chỉ áp dụng trên màn hình trung bình trở lên. Điều này làm cho thiết kế responsive dễ dàng và có thể dự đoán được.

Tùy chỉnh thông qua file cấu hình cho phép định nghĩa màu sắc, khoảng cách, phong chữ, và breakpoint tùy chỉnh. Tailwind có thể được điều chỉnh cho hệ thống thiết kế của dự án, đồng thời hỗ trợ tốt cho việc xây dựng các giao diện có nhiều biến thể thành phần.

3.9.5 Electron và khả năng đóng gói desktop

Electron là một nền tảng cho phép xây dựng ứng dụng desktop đa nền tảng bằng công nghệ web như HTML, CSS và JavaScript. Electron kết hợp Chromium để hiển thị giao diện và Node.js để cung cấp khả năng truy cập hệ thống ở phía desktop runtime. Nhờ đó, cùng một nền mã giao diện có thể được sử dụng để phát hành ứng dụng desktop bên cạnh phiên bản web.

Trong SMAP, frontend không chỉ dừng ở web application thuần túy mà còn có khả năng đóng gói desktop thông qua Electron. Điều này đặc biệt hữu ích khi hệ thống cần một hình thức triển khai thuận tiện cho người dùng cuối, đồng thời vẫn tận dụng được toàn bộ stack giao diện dựa trên Next.js, React và TypeScript.

3.9.6 Tích hợp analytics API và dashboard

Trong các hệ thống phân tích dữ liệu, lớp giao diện không chỉ đảm nhiệm hiển thị trang nghiệp vụ mà còn thường tích hợp các API phân tích hoặc dashboard chuyên dụng để hỗ trợ truy vấn và trực quan hóa. Một hướng tiếp cận phổ biến là frontend cung cấp lớp tích hợp trung gian để truy xuất dữ liệu từ service phân tích và đưa kết quả vào trải nghiệm sử dụng chung của hệ thống.

Đối với SMAP, frontend tích hợp analytics API thông qua các hook và route handler chuyên biệt. Điều này cho thấy frontend không chỉ đóng vai trò giao diện thao tác, mà còn là nơi kết nối giữa trải nghiệm người dùng với các lớp báo cáo và khai thác dữ liệu có cấu trúc hơn.

Những công nghệ trên là cơ sở để hình thành frontend hiện tại của SMAP: một lớp giao diện dựa trên Next.js, React, TypeScript và Tailwind CSS, có hỗ trợ i18n, có khả năng tích hợp analytics dashboard/API, và có hướng triển khai trên cả web lẫn desktop thông qua Electron.

3.10 Containerization và Orchestration

3.10.1 Docker

Docker là một nền tảng phục vụ việc phát triển, đóng gói và chạy ứng dụng trong môi trường container. Container là các gói thực thi nhẹ và độc lập, bao gồm toàn bộ thành phần cần thiết để chạy ứng dụng như mã nguồn, runtime, công cụ hệ thống, thư viện và cấu hình. Docker cho phép đóng gói ứng dụng cùng toàn bộ phụ thuộc của nó thành các image container, có thể chạy nhất quán trên nhiều môi trường triển khai khác nhau.

Công nghệ container giải quyết vấn đề khác biệt môi trường chạy giữa máy phát triển, máy kiểm thử và máy triển khai. Khi ứng dụng và các phụ thuộc của nó được đóng gói trong cùng một image, hành vi của ứng dụng trở nên nhất quán hơn và ít phụ thuộc vào cấu hình thủ công của từng máy. Nhờ đó, container đặc biệt phù hợp với các hệ thống có nhiều thành phần hoặc nhiều service cần được triển khai độc lập.

Docker image là các template chỉ đọc chứa hướng dẫn để tạo container. Image thường được xây dựng từ Dockerfile, trong đó mô tả rõ các bước cài đặt môi trường, sao chép mã nguồn, cấu hình runtime và lệnh khởi động. Cấu trúc phân lớp của image giúp tối ưu không gian lưu trữ và tăng tốc quá trình build lại khi chỉ một phần của ứng dụng thay đổi.

3.10.2 Docker Compose

Docker Compose là công cụ cho phép định nghĩa và khởi động nhiều container như một tập hợp thống nhất. Thay vì chạy từng container riêng lẻ bằng lệnh thủ công, Docker Compose cho phép mô tả service, network, volume và dependency trong một file cấu

hình. Cách tiếp cận này đặc biệt hữu ích trong môi trường phát triển, kiểm thử tích hợp hoặc mô phỏng các hệ thống nhiều thành phần ở quy mô gọn nhẹ.

Trong các hệ thống nhiều service, Docker Compose giúp đơn giản hóa quá trình khởi động đồng thời nhiều thành phần như API service, cache, message broker hay cơ sở dữ liệu. Điều này không thay thế cho orchestration ở quy mô lớn, nhưng là bước đệm quan trọng để duy trì tính nhất quán môi trường trong quá trình phát triển và kiểm thử.

3.10.3 Kubernetes

Kubernetes là một nền tảng orchestration cho container, được thiết kế để tự động hóa việc triển khai, mở rộng và quản lý các ứng dụng được đóng gói trong container. Hệ thống này cho phép phân phối workload lên nhiều node, điều phối vòng đời của container, và duy trì trạng thái mong muốn của ứng dụng trong môi trường triển khai.

Khả năng orchestration của Kubernetes được xây dựng trên nhiều thành phần cơ bản. Pod là đơn vị triển khai nhỏ nhất, có thể chứa một hoặc nhiều container. Deployment quản lý việc phát hành phiên bản mới và hỗ trợ rollback. Service cung cấp lớp địa chỉ ổn định và cân bằng tải cho các pod. ConfigMap và Secret hỗ trợ tách cấu hình khỏi image triển khai. Ingress cho phép định tuyến lưu lượng từ bên ngoài vào các service trong cụm.

Một lợi ích quan trọng của Kubernetes là khả năng duy trì trạng thái mong muốn của hệ thống. Các controller liên tục so sánh trạng thái thực tế với cấu hình khai báo và tự động điều chỉnh khi có sai lệch. Nhờ đó, việc triển khai trở nên có thể tái tạo, dễ kiểm soát bằng mã cấu hình và phù hợp với các hệ thống cần vận hành ổn định ở quy mô nhiều service.

3.10.4 Auto Scaling và Self-Healing

Trong các hệ thống phân tán, tải xử lý giữa các thành phần thường không đồng đều và có thể thay đổi theo thời gian. Vì vậy, orchestration không chỉ dừng ở việc khởi động container mà còn cần hỗ trợ mở rộng và phục hồi tự động. Auto scaling cho phép tăng hoặc giảm số lượng bản sao của một thành phần dựa trên tín hiệu như CPU, bộ nhớ hoặc tải xử lý. Điều này giúp hệ thống sử dụng tài nguyên linh hoạt hơn và tránh lãng phí trong các thời điểm tải thấp.

Self-healing là khả năng tự khôi phục khi container hoặc pod gặp sự cố. Nếu một tiến trình bị treo, crash hoặc không còn đáp ứng kiểm tra sức khỏe, hệ thống orchestration có thể tự động khởi động lại hoặc thay thế instance tương ứng. Cơ chế này đặc biệt quan trọng với các hệ thống nhiều thành phần, nơi việc can thiệp thủ công vào từng service sẽ nhanh chóng trở nên khó quản lý.

Các cơ chế auto scaling và self-healing không thay thế cho thiết kế hệ thống tốt, nhưng chúng là những lớp hỗ trợ vận hành quan trọng giúp tăng tính sẵn sàng và ổn định của hệ thống khi được triển khai ở môi trường thực tế.

Những khái niệm về containerization và orchestration là nền tảng phù hợp để hiểu cách các hệ thống nhiều service như SMAP có thể được đóng gói, triển khai và vận hành theo hướng nhất quán hơn, đồng thời tạo tiền đề cho việc mở rộng và quản lý các thành phần độc lập ở các chương sau.

3.11 Tổng kết

Chương này đã trình bày các nền tảng lý thuyết cần thiết để hiểu và đánh giá hệ thống SMAP. Các kiến trúc Microservices và Clean Architecture cung cấp khung tổng thể cho việc tổ chức hệ thống. Message Queue và giao tiếp thời gian thực là cơ sở cho các cơ chế xử lý bất đồng bộ và realtime communication. Các công nghệ lưu trữ dữ liệu cùng cách tiếp cận polyglot persistence cho phép tổ chức và tối ưu hóa nhiều lớp dữ liệu khác nhau trong hệ thống. Xác thực và bảo mật là nền tảng cho việc bảo vệ truy cập và duy trì an toàn thông tin. Các công nghệ backend như Go, Gin, SQLBoiler, Python và FastAPI cung cấp nền tảng để xây dựng các service. Bên cạnh đó, các kỹ thuật AI/NLP, embedding, vector search, RAG và LLM generation cung cấp cơ sở để hiểu các bài toán phân tích cảm xúc, trích xuất từ khóa, lập chỉ mục ngữ nghĩa, hỏi đáp theo ngữ cảnh và sinh báo cáo trong hệ thống phân tích dữ liệu mạng xã hội. Các công nghệ frontend như Next.js, React, TypeScript và Tailwind CSS tạo nền cho giao diện người dùng, đồng thời hỗ trợ mở rộng theo hướng analytics dashboard/API integration và đóng gói desktop khi cần. Cuối cùng, containerization và orchestration cung cấp cơ sở cho việc đóng gói, triển khai và vận hành hệ thống theo hướng nhất quán hơn.

Những kiến thức này sẽ là nền tảng cho Chương 4, nơi phân tích các yêu cầu hệ thống của SMAP, và Chương 5, nơi phân tích thiết kế chi tiết hệ thống, áp dụng các nguyên tắc và công nghệ đã được trình bày trong chương này.

CHƯƠNG 4: PHÂN TÍCH HỆ THỐNG

4.1 Nhóm người dùng và tác nhân hệ thống

Trong phạm vi của SMAP, việc xác định tác nhân hệ thống không dựa trên kết quả khảo sát người dùng trực tiếp, mà được tổng hợp từ phạm vi ứng dụng đã nêu ở Chương 1, từ nhóm chức năng của hệ thống, và từ các luồng tương tác. Cách tiếp cận này giúp xác định rõ ai là người sử dụng chính của hệ thống, đâu là các tác nhân bên ngoài, và mỗi tác nhân tương tác với SMAP theo vai trò nào.

Nhìn ở mức tổng quát, SMAP phục vụ trước hết cho nhóm người dùng chuyên môn nội bộ có nhu cầu theo dõi, phân tích và tổng hợp thông tin từ mạng xã hội. Trong các use case ở mục 4.4, nhóm này được dùng như actor chính đại diện cho các vai trò nghiệp vụ cụ thể như chuyên viên phân tích truyền thông xã hội, người vận hành chiến dịch theo dõi, bộ phận marketing, truyền thông, nghiên cứu thị trường hoặc quản lý thương hiệu. Bên cạnh đó, hệ thống còn phụ thuộc vào một số tác nhân bên ngoài như nền tảng mạng xã hội là nguồn dữ liệu đầu vào và nhà cung cấp định danh hỗ trợ xác thực người dùng.

Bảng 7: Danh sách tác nhân hệ thống

Actor ID	Tác nhân	Vai trò chính
A-01	Nhóm người dùng chuyên môn nội bộ	Đây là nhóm người dùng chính của hệ thống, bao gồm các vai trò như chuyên viên phân tích dữ liệu truyền thông xã hội, bộ phận theo dõi thương hiệu, marketing, nghiên cứu thị trường, truyền thông và các cấp quản lý. Nhóm tác nhân này sử dụng SMAP để: - cấu hình và quản lý project theo dõi; - theo dõi trạng thái xử lý và kết quả phân tích; - tra cứu, tổng hợp và khai thác thông tin theo ngữ cảnh; - nhận thông báo hoặc cảnh báo phục vụ đánh giá và ra quyết định.
A-02	Nền tảng mạng xã hội	Đây là tác nhân bên ngoài cung cấp dữ liệu đầu vào cho hệ thống. Vai trò của tác nhân này gồm:

		• là nguồn phát sinh bài đăng, bình luận và tín hiệu mạng xã hội; • tạo ra các ràng buộc truy cập dữ liệu như giới hạn truy cập, thay đổi cấu trúc nền tảng hoặc giới hạn kỹ thuật khác; • ảnh hưởng trực tiếp đến khả năng thu thập, cập nhật và chuẩn hóa dữ liệu của hệ thống.
A-03	Nhà cung cấp định danh	Đây là tác nhân hỗ trợ xác thực người dùng thông qua cơ chế OAuth2. Vai trò của tác nhân này gồm: • xác nhận danh tính người dùng trong bước đăng nhập ban đầu; • cung cấp thông tin định danh cần thiết để thiết lập phiên truy cập; • làm đầu vào cho lớp xác thực của hệ thống trước khi người dùng sử dụng các chức năng nghiệp vụ.

4.2 Yêu cầu chức năng

Căn cứ vào nhóm người dùng và tác nhân hệ thống đã xác định ở mục 4.1, phần này tổng hợp các yêu cầu chức năng cốt lõi của SMAP. Thay vì mô tả hệ thống theo từng màn hình hay theo chi tiết hiện thực, các yêu cầu ở đây được viết ở mức capability để có thể ánh xạ sang use case nghiệp vụ ở mục 4.4 và thiết kế kỹ thuật ở Chương 5.

Các yêu cầu chức năng được chia thành ba nhóm lớn. Nhóm thứ nhất là các capability phục vụ thiết lập và quản lý ngữ cảnh nghiệp vụ, bao gồm xác thực, campaign/project, nguồn dữ liệu, mục tiêu thu thập, kiểm tra thử và cấu hình cảnh báo khủng hoảng. Nhóm thứ hai là các capability điều phối và xử lý dữ liệu, bao gồm điều khiển vòng đời vận hành, điều phối thu thập dữ liệu và xử lý phân tích. Nhóm thứ ba là các capability phục vụ khai thác kết quả, bao gồm tìm kiếm, hỏi đáp theo ngữ cảnh, thông báo kịp thời và các cơ chế kiểm tra nội bộ phục vụ liên thông an toàn giữa các thành phần.

Bảng 8: Yêu cầu chức năng

ID	Tên yêu cầu	Mô tả	Mức ưu tiên
FR-01	Xác thực người dùng	Hệ thống phải cho phép người dùng đăng nhập bằng cơ chế xác thực hiện có, duy trì phiên truy cập và truy xuất thông tin người dùng hiện tại để sử dụng các chức năng nghiệp vụ.	Cao
FR-02	Quản lý campaign và project	Hệ thống phải cho phép tạo, xem, cập nhật và xóa campaign/project để thiết lập ngữ cảnh nghiệp vụ cho toàn bộ quá trình theo dõi và xử lý dữ liệu.	Cao
FR-03	Điều khiển vòng đời project	Hệ thống phải cho phép kiểm tra điều kiện sẵn sàng và thay đổi trạng thái vận hành của project như kích hoạt, tạm dừng, tiếp tục, lưu trữ và mở lại theo quy tắc nghiệp vụ hiện tại.	Cao
FR-04	Quản lý cấu hình cảnh báo khủng hoảng	Hệ thống phải cho phép cấu hình, xem và xóa các rule hoặc cấu hình giám sát khủng hoảng gắn với từng project.	Cao
FR-05	Quản lý nguồn dữ liệu	Hệ thống phải cho phép tạo và quản lý nguồn dữ liệu theo project, bao gồm xem chi tiết, cập nhật và duy trì trạng thái cấu hình phục vụ bước vận hành.	Cao
FR-06	Quản lý mục tiêu thu thập	Hệ thống phải cho phép tạo và quản lý mục tiêu thu thập theo các kiểu đầu vào mà hệ thống hiện hỗ trợ, làm cơ sở cho việc thu thập dữ liệu từ các nền tảng mạng xã hội.	Cao
FR-07	Kiểm tra thử đầu vào	Hệ thống phải cho phép thực hiện kiểm tra thử để đánh giá đầu vào thu thập dữ liệu và truy xuất kết quả kiểm tra gần nhất hoặc lịch sử kiểm tra phục vụ quyết định trước khi vận hành chính thức.	Cao
FR-08	Điều phối thu thập dữ liệu	Hệ thống phải hỗ trợ khởi tạo, theo dõi và ghi nhận kết quả của các hoạt động thu thập dữ liệu bất đồng bộ sau khi chiến dịch được đưa vào trạng thái vận hành.	Cao
FR-09	Xử lý phân tích	Hệ thống phải tiếp nhận dữ liệu đầu vào đã chuẩn hóa, thực hiện xử lý phân tích và tạo ra	Cao

		các kết quả có cấu trúc để phục vụ tra cứu, hỏi đáp, theo dõi trạng thái và cảnh báo.	
FR-10	Tra cứu và hỏi đáp dữ liệu	Hệ thống phải hỗ trợ tìm kiếm, tra cứu và hỏi đáp theo ngữ cảnh trên dữ liệu và kết quả phân tích đã sẵn sàng cho việc khai thác.	Cao
FR-11	Gửi thông báo	Hệ thống phải hỗ trợ gửi các loại thông báo cần thiết như tiến độ xử lý, sự kiện chiến dịch hoặc cảnh báo tới người dùng phù hợp một cách kịp thời.	Trung bình
FR-12	Kiểm tra liên thông nội bộ	Hệ thống phải hỗ trợ các cơ chế kiểm tra nội bộ để bảo đảm các thành phần của hệ thống liên thông an toàn khi thực hiện các capability nghiệp vụ được bảo vệ.	Trung bình

4.3 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống SMAP còn cần đáp ứng một nhóm yêu cầu phi chức năng nhằm bảo đảm tính ổn định, an toàn và khả năng vận hành của toàn bộ kiến trúc. Đối với một hệ thống gồm nhiều dịch vụ phối hợp, các yêu cầu phi chức năng không chỉ đóng vai trò tiêu chí đánh giá chất lượng, mà còn là cơ sở chi phối các quyết định về lưu trữ, truyền thông, xác thực, triển khai và quan sát vận hành.

Trong phạm vi của luận văn, các yêu cầu phi chức năng dưới đây được trình bày ở mức hệ thống. Mục tiêu là xác định các thuộc tính chất lượng cần được bảo đảm trong quá trình thiết kế và đánh giá, đồng thời tránh đưa ra các cam kết định lượng khi chưa có cơ sở thực nghiệm tương ứng.

4.3.1 Đặc tính kiến trúc

Phần này trình bày các đặc tính kiến trúc được xem là quan trọng đối với SMAP ở mức thiết kế hệ thống. Đây là các thuộc tính mang tính định hướng, dùng để đánh giá vì sao kiến trúc được tổ chức theo nhiều dịch vụ, vì sao cần các luồng xử lý bất đồng bộ, và vì sao việc tách các miền trách nhiệm là cần thiết đối với hệ thống.

Bảng 9: Đặc tính kiến trúc chính

AC	Đặc tính	Ý nghĩa đối với hệ thống	Chỉ báo đánh giá / định hướng
AC-01	Modularity	Hệ thống cần được tách thành các thành phần có trách nhiệm tương đối độc lập, giúp giới hạn blast radius khi thay đổi hoặc	Tách rõ miền trách nhiệm theo dịch vụ hoặc mô-đun; giảm phụ thuộc chéo không

		lỗi xảy ra ở một lane xử lý cụ thể.	cần thiết giữa các thành phần.
AC-02	Scalability	Các nhóm tải có tính chất khác nhau như tương tác người dùng, thu thập dữ liệu, phân tích hoặc thông báo cần có khả năng được mở rộng linh hoạt theo nhu cầu riêng.	Có thể mở rộng theo vai trò xử lý; không buộc mọi thành phần phải mở rộng đồng thời.
AC-03	Performance	Các luồng đồng bộ và bất đồng bộ phải được tổ chức sao cho hệ thống phản hồi ổn định với người dùng trong khi vẫn xử lý được các tác vụ nền nặng.	Tách luồng phản hồi trực tiếp khỏi luồng xử lý nền; ưu tiên xử lý bất đồng bộ cho các tác vụ tốn tài nguyên.
AC-04	Security	Hệ thống cần có ranh giới bảo mật rõ ràng cho xác thực người dùng, kiểm tra giao tiếp giữa các dịch vụ và bảo vệ các kênh thông báo.	Sử dụng lớp xác thực riêng, kiểm tra token, chính sách phiên truy cập và cơ chế kiểm tra nội bộ phù hợp.
AC-05	Availability	Các thành phần quan trọng phải hỗ trợ cơ chế phát hiện lỗi trong quá trình vận hành và khả năng khôi phục phù hợp để duy trì tính sẵn sàng.	Health check, probe, chính sách khởi động lại và tách luồng xử lý để giảm ảnh hưởng dây chuyền.
AC-06	Data Integrity	Các luồng bất đồng bộ cần đảm bảo liên kết đúng giữa nhiệm vụ xử lý, trạng thái hoàn tất và dữ liệu thu thập gốc để tránh sai lệch dữ liệu.	Khóa tương quan, khả năng xử lý lặp an toàn và hợp đồng dữ liệu nhất quán giữa các thành phần.
AC-07	Observability	Hệ thống cần hỗ trợ nhật ký, chỉ số hoặc các chỉ báo vận hành đủ để theo dõi và chẩn đoán lỗi ở các luồng xử lý chính.	Có nhật ký có cấu hình, chỉ số vận hành hoặc chỉ báo trạng thái ở các nhóm xử lý quan trọng.

4.3.2 Thuộc tính chất lượng

Các thuộc tính chất lượng dưới đây được viết ở mức yêu cầu hệ thống. Phần này tập trung vào những nhóm chất lượng cần thiết để hệ thống vận hành ổn định, an toàn và có khả năng mở rộng trong phạm vi nghiệp vụ đã xác định.

Bảng 10: Yêu cầu phi chức năng cốt lõi

ID	Nhóm	Yêu cầu	Chỉ báo / Bằng chứng định hướng
NFR-01	Performance	Hệ thống phải hỗ trợ xử lý bất đồng bộ cho các lane phân tích và xử lý nền thay vì ép toàn bộ logic vào request-response đồng bộ.	Xử lý nền theo luồng sự kiện; tách luồng phân tích và xử lý nặng khỏi luồng phản hồi trực tiếp.
NFR-02	Security	Hệ thống phải hỗ trợ xác thực người dùng, quản lý phiên truy cập và bảo vệ giao tiếp giữa các dịch vụ bằng các cơ chế xác thực phù hợp.	OAuth2, JWT, HttpOnly cookie, internal auth, token validation.
NFR-03	Availability	Các nhóm xử lý quan trọng phải có cơ chế phát hiện lỗi và hỗ trợ khôi phục để duy trì vận hành ổn định.	Probe, health check, chiến lược khởi động lại hoặc triển khai lại, tách nhóm xử lý theo vai trò.
NFR-04	Scalability	Các luồng xử lý có tải biến động phải có khả năng mở rộng theo vai trò thay vì phụ thuộc vào mở rộng đồng loạt toàn hệ thống.	Mở rộng theo vai trò xử lý khi nhu cầu tải thay đổi.
NFR-05	Data Integrity	Các luồng bất đồng bộ phải duy trì được tính nhất quán giữa nhiệm vụ xử lý, trạng thái hoàn tất, dữ liệu thu thập và kết quả xử lý tiếp theo.	Khóa tương quan, xử lý lặp an toàn, hợp đồng dữ liệu thống nhất và khả năng truy vết dữ liệu nguồn.
NFR-06	Modularity	Các miền trách nhiệm chính phải được tách thành dịch vụ hoặc mô-đun tương đối độc lập để dễ thay đổi, bảo trì và kiểm thử.	Service boundaries rõ, ownership dữ liệu rõ, module tách theo capability.

NFR-07	Observability	Hệ thống phải hỗ trợ đủ nhật ký hoặc chỉ báo vận hành để theo dõi lỗi và giám sát các luồng xử lý quan trọng.	Nhật ký có cấu trúc, chỉ số vận hành và chỉ báo trạng thái ở các nhóm xử lý chính.
--------	---------------	---	--

4.3.3 Nhận xét và liên hệ với SMAP

Đối với SMAP, các yêu cầu phi chức năng trên không tồn tại độc lập mà gắn trực tiếp với kiến trúc của hệ thống. Việc tách các luồng xử lý phân tích, sử dụng nhiều lớp lưu trữ, áp dụng cơ chế xác thực người dùng và tổ chức truyền tin bất đồng bộ cho thấy thiết kế hệ thống cần đồng thời đáp ứng các yêu cầu về hiệu năng, bảo mật, khả năng mở rộng, tính nhất quán dữ liệu và khả năng quan sát vận hành.

Vì vậy, phần NFR trong chương này không nên được hiểu như một danh sách cam kết số liệu tuyệt đối, mà là bộ tiêu chí chất lượng cốt lõi dùng để định hướng thiết kế và đánh giá hệ thống ở các chương sau. Các chỉ số định lượng chi tiết, nếu cần, chỉ nên được xác định khi có kết quả thực nghiệm hoặc kết quả vận hành tương ứng.

4.4 Use Case

Sau khi xác định các yêu cầu chức năng và phi chức năng, phần này mô tả các use case cốt lõi của hệ thống SMAP theo góc nhìn mục tiêu sử dụng của người dùng nghiệp vụ. Use case trong phần này không được tách theo tên thành phần kỹ thuật, giao diện kỹ thuật, module hay luồng xử lý nội bộ, mà được tổ chức theo kết quả có giá trị quan sát được đối với tác nhân chính. Cách tiếp cận này giúp phân phân tích hệ thống tập trung vào nhu cầu sử dụng, còn chi tiết hiện thực sẽ được trình bày ở Chương 5.

Trong phạm vi này, các thao tác như tạo campaign, tạo project, khai báo nguồn dữ liệu, cấu hình mục tiêu thu thập và chạy kiểm tra thử được xem là các bước trong cùng một mục tiêu nghiệp vụ: thiết lập một chiến dịch theo dõi đủ điều kiện vận hành. Tương tự, các xử lý nền như thu thập dữ liệu, phân tích dữ liệu hoặc chuẩn bị dữ liệu phục vụ tra cứu không được tách thành use case độc lập cho người dùng, vì đây là hành vi hệ thống hỗ trợ cho các mục tiêu khai thác kết quả. Các cơ chế xác thực, phân quyền và kiểm tra nội bộ cũng được xem là điều kiện tiên quyết hoặc supporting concern cho các use case được bảo vệ, thay vì là một use case nghiệp vụ riêng.

Từ góc nhìn đó, các use case chính của SMAP được gom thành năm nhóm mục tiêu người dùng: thiết lập chiến dịch theo dõi, vận hành chiến dịch, khai thác dữ liệu phân tích, theo dõi trạng thái và nhận cảnh báo, cùng với thiết lập quy tắc cảnh báo khủng hoảng.

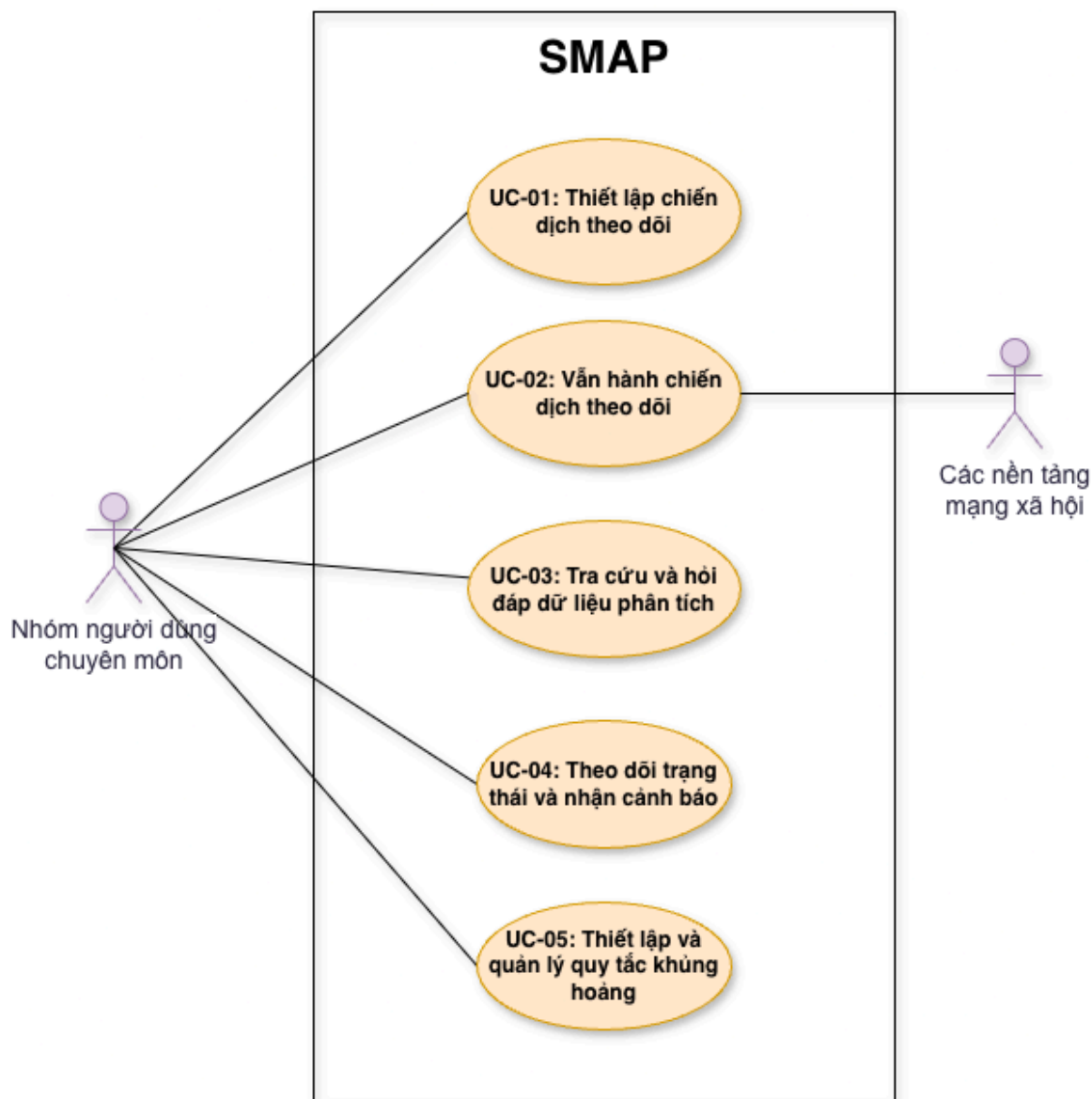
Bảng 11: Danh sách Use Case

ID	Tên Use Case	Actor chính	FR nghiệp vụ chính	Mục tiêu nghiệp vụ
UC-01	Thiết lập chiến dịch theo dõi	A-01: Nhóm người dùng chuyên môn nội bộ	FR-02, FR-05, FR-06, FR-07	Tạo ngữ cảnh theo dõi và hoàn tất cấu hình đầu vào cần thiết, bao gồm campaign, project, nguồn dữ liệu, mục tiêu thu thập và kiểm tra thử, để chiến dịch đủ điều kiện bước sang giai đoạn vận hành.
UC-02	Vận hành chiến dịch theo dõi	A-01: Nhóm người dùng chuyên môn nội bộ	FR-03, FR-08	Kiểm tra mức độ sẵn sàng và điều khiển trạng thái vận hành của chiến dịch như kích hoạt, tạm dừng, tiếp tục hoặc lưu trữ theo nhu cầu nghiệp vụ.
UC-03	Tra cứu và hỏi đáp dữ liệu phân tích	A-01: Nhóm người dùng chuyên môn nội bộ	FR-10	Khai thác dữ liệu đã được hệ thống xử lý thông qua tìm kiếm, hỏi đáp theo ngữ cảnh, câu trả lời có dẫn chứng và các gợi ý truy vấn tiếp theo.
UC-04	Theo dõi trạng thái và nhận cảnh báo	A-01: Nhóm người dùng chuyên môn nội bộ	FR-11	Nhận các thông báo liên quan đến trạng thái xử lý, sự kiện chiến dịch hoặc cảnh báo quan trọng để kịp thời theo dõi và phản ứng.
UC-05	Thiết lập và quản lý quy tắc cảnh báo khủng hoảng	A-01: Nhóm người dùng chuyên môn nội bộ	FR-04	Thiết lập và quản lý các quy tắc giám sát khủng hoảng như keyword, volume, sentiment hoặc influencer để hệ thống có cơ sở phát hiện và cảnh báo ở các luồng xử lý phía sau.

Các yêu cầu FR-01, FR-09 và FR-12 đóng vai trò hỗ trợ cho các use case được bảo vệ: FR-01 là điều kiện xác thực trước khi người dùng thực hiện thao tác nghiệp vụ, FR-09 cung cấp dữ liệu phân tích cho các mục tiêu khai thác và cảnh báo, còn FR-12 bảo vệ

liên thông nội bộ giữa các thành phần hệ thống. Vì vậy, các yêu cầu này không được tách thành use case người dùng độc lập trong bảng trên.

Thứ tự đánh số use case trong bảng trên phản ánh nhóm mục tiêu nghiệp vụ, không nhất thiết là thứ tự thời gian tuyệt đối. Riêng UC-05 có thể được thực hiện trước hoặc trong quá trình vận hành chiến dịch để tạo rule context cho các cảnh báo mà người dùng quan sát ở UC-04.



Hình 1: Sơ đồ use case tổng quan cho các nghiệp vụ chính của người dùng chuyên môn

4.4.1 UC-01: Thiết lập chiến dịch theo dõi

Bảng 12: Use Case UC-01

Use Case name	UC-01: Thiết lập chiến dịch theo dõi
Actors	Primary Actor: A-01 - Nhóm người dùng chuyên môn nội bộ

	Secondary Actor: A-02 - Nền tảng mạng xã hội
Description	Người dùng thiết lập một chiến dịch theo dõi bằng cách tạo hoặc chọn campaign, tạo project trong campaign đó, khai báo nguồn dữ liệu, cấu hình mục tiêu thu thập và thực hiện kiểm tra thử khi cần để đánh giá khả năng thu thập dữ liệu trước khi vận hành chính thức. Use case này tập trung vào mục tiêu hoàn tất cấu hình đầu vào cho chiến dịch, không bao gồm bước kích hoạt vận hành hay xử lý phân tích phía sau.
Trigger	User muốn tạo một chiến dịch theo dõi mới hoặc bổ sung một project theo dõi vào campaign đã tồn tại.
Preconditions	- User đã đăng nhập và có quyền tạo hoặc quản lý campaign/project trong phạm vi làm việc tương ứng. - User đã xác định được mục tiêu theo dõi cơ bản như thương hiệu, chủ đề, đối thủ, nền tảng hoặc phạm vi dữ liệu cần quan sát. - Hệ thống có thể lưu cấu hình và kiểm tra nguồn dữ liệu khi kiểm tra thử được yêu cầu.
Postconditions	- Campaign và project được tạo mới hoặc project được liên kết với campaign đã tồn tại. - Nguồn dữ liệu và một hoặc nhiều mục tiêu thu thập được lưu theo project tương ứng. - Nếu tổ hợp nguồn dữ liệu và mục tiêu thu thập yêu cầu kiểm tra, kết quả kiểm tra thử mới nhất và lịch sử kiểm tra được ghi nhận. - Trạng thái cấu hình của project hoặc nguồn dữ liệu phản ánh mức độ sẵn sàng để chuyển sang bước vận hành. - Nếu cấu hình hoặc kiểm tra thử không đạt yêu cầu, chiến dịch vẫn chưa đủ điều kiện vận hành chính thức cho đến khi user điều chỉnh lại.
Normal Flows	- User bắt đầu luồng thiết lập chiến dịch theo dõi. - User tạo campaign mới hoặc chọn một campaign đã tồn tại làm ngữ cảnh nghiệp vụ. - User nhập thông tin project như tên, mô tả, đối tượng theo dõi, domain hoặc phạm vi dữ liệu cần quan sát. - Hệ thống kiểm tra dữ liệu đầu vào và lưu campaign/project tương ứng. - User khai báo nguồn dữ liệu cho project, bao gồm loại nguồn dữ liệu, nền tảng và các thông tin cấu hình cần thiết.

	6. User thêm một hoặc nhiều mục tiêu thu thập phù hợp với nguồn dữ liệu, chẳng hạn keyword, profile hoặc post URL. 7. Hệ thống kiểm tra loại mục tiêu, dữ liệu đầu vào và lưu mục tiêu ở trạng thái phù hợp trước khi vận hành. 8. Nếu tổ hợp nguồn dữ liệu và mục tiêu thu thập cần kiểm tra, user yêu cầu chạy kiểm tra thử. 9. Hệ thống thực hiện kiểm tra thử, thu thập dữ liệu mẫu hoặc xác nhận khả năng truy cập nguồn dữ liệu. 10. Hệ thống lưu kết quả kiểm tra thử, cập nhật chỉ báo sẵn sàng và hiển thị kết quả để user đánh giá. 11. User xác nhận cấu hình đã đủ dùng hoặc quay lại chỉnh sửa các thông tin chưa đạt yêu cầu.
Alternative Flows	Tạo project trong campaign đã có Tại Bước 2 của luồng cơ bản: • 2A.1. User chọn một campaign đã tồn tại thay vì tạo campaign mới. • 2A.2. Hệ thống sử dụng campaign đó làm ngữ cảnh nghiệp vụ cho project mới. • 2A.3. Luồng tiếp tục tại bước nhập thông tin project. Cấu hình nhiều nguồn dữ liệu hoặc nhiều mục tiêu thu thập Tại Bước 5 hoặc Bước 6 của luồng cơ bản: • 6A.1. User khai báo thêm nguồn dữ liệu hoặc mục tiêu thu thập khác trong cùng project. • 6A.2. Hệ thống kiểm tra và lưu từng cấu hình hợp lệ. • 6A.3. User có thể chạy kiểm tra thử cho từng tổ hợp cần kiểm tra. Tổ hợp nguồn dữ liệu không yêu cầu kiểm tra thử Tại Bước 8 của luồng cơ bản: • 8A.1. Hệ thống xác định tổ hợp nguồn dữ liệu và mục tiêu thu thập hiện tại không yêu cầu kiểm tra thử bắt buộc. • 8A.2. User có thể bỏ qua bước kiểm tra thử và tiếp tục hoàn tất cấu hình. • 8A.3. Mức độ sẵn sàng được xác định dựa trên các điều kiện cấu hình còn lại. • Kết thúc use case.

Exceptions	User chưa được xác thực hoặc không đủ quyền Trước hoặc trong Bước 1 của luồng cơ bản, nếu user chưa có phiên hợp lệ hoặc không có quyền thao tác: • 1.E.1. Hệ thống từ chối truy cập chức năng thiết lập chiến dịch. • 1.E.2. User cần đăng nhập lại hoặc sử dụng tài khoản có quyền phù hợp. • Kết thúc use case. Dữ liệu campaign, project, nguồn dữ liệu hoặc mục tiêu thu thập không hợp lệ Tại Bước 4, Bước 5 hoặc Bước 7 của luồng cơ bản, nếu dữ liệu đầu vào thiếu hoặc sai định dạng: • 7.E.1. Hệ thống từ chối lưu phần cấu hình không hợp lệ. • 7.E.2. Hệ thống hiển thị lỗi tại các trường liên quan. • 7.E.3. User chỉnh sửa dữ liệu và gửi lại. • Tiếp tục tại bước tương ứng. Kiểm tra thử thất bại hoặc không đạt yêu cầu Tại Bước 9 hoặc Bước 10 của luồng cơ bản, nếu không thu thập được dữ liệu mẫu hoặc kết quả không usable: • 10.E.1. Hệ thống ghi nhận kết quả kiểm tra thử thất bại hoặc cảnh báo tương ứng. • 10.E.2. Cấu hình chưa được xem là đủ điều kiện vận hành đối với tổ hợp yêu cầu kiểm tra thử. • 10.E.3. User cần điều chỉnh nguồn dữ liệu hoặc mục tiêu thu thập trước khi chạy lại kiểm tra thử. • Kết thúc use case. Lỗi lưu cấu hình hoặc xử lý kiểm tra thử Tại Bước 4, Bước 7 hoặc Bước 10 của luồng cơ bản, nếu hệ thống không thể lưu dữ liệu hoặc hoàn tất kiểm tra thử: • 10.E.4. Hệ thống thông báo thao tác thiết lập chưa hoàn tất. • 10.E.5. User có thể thử lại sau hoặc lưu cấu hình nháp nếu hệ thống cho phép. • Kết thúc use case.
Phạm vi	Use case này có ý gom campaign, project, nguồn dữ liệu, mục tiêu thu thập và kiểm tra thử thành một mục tiêu nghiệp vụ thống nhất theo góc nhìn người dùng. Xác thực và phân quyền là điều kiện tiên quyết; kích hoạt vận hành, thu thập dữ liệu

	chính thức và xử lý phân tích thuộc các use case hoặc luồng hệ thống khác.
--	--

4.4.2 UC-02: Vận hành chiến dịch theo dõi

Bảng 13: Use Case UC-02

Use Case name	UC-02: Vận hành chiến dịch theo dõi
Actors	- Primary Actor: A-01 - Nhóm người dùng chuyên môn nội bộ - Secondary Actor: Không có
Description	Người dùng vận hành một chiến dịch theo dõi đã được thiết lập bằng cách kiểm tra mức độ sẵn sàng và thực hiện các thao tác như kích hoạt, tạm dừng, tiếp tục hoặc lưu trữ. Use case này bắt đầu khi cấu hình đầu vào đã có đủ thông tin cần thiết và tập trung vào việc chuyển chiến dịch từ trạng thái cấu hình sang trạng thái vận hành có kiểm soát.
Trigger	User muốn kiểm tra mức độ sẵn sàng hoặc thay đổi trạng thái vận hành của một chiến dịch theo dõi đã được thiết lập.
Preconditions	- User đã đăng nhập và có quyền quản lý chiến dịch theo dõi tương ứng. - Campaign và project liên quan đã tồn tại trong hệ thống. - Chiến dịch đã có cấu hình đầu vào cần thiết như nguồn dữ liệu, mục tiêu thu thập và bằng chứng kiểm tra nếu luồng vận hành yêu cầu. - User đã chọn thao tác vận hành cần thực hiện.
Postconditions	- Nếu thao tác hợp lệ và thành công, trạng thái vận hành của chiến dịch được cập nhật theo yêu cầu của user. - Nếu user chỉ kiểm tra mức độ sẵn sàng, hệ thống trả về kết quả đánh giá mà không thay đổi trạng thái vận hành. - Hoạt động thu thập dữ liệu của chiến dịch được bắt đầu, tạm dừng, tiếp tục hoặc giữ nguyên phù hợp với trạng thái mới. - Nếu thao tác không hợp lệ hoặc mức độ sẵn sàng không đạt, trạng thái chiến dịch không bị thay đổi ngoài các thông tin lỗi hoặc cảnh báo cần hiển thị cho user.
Normal Flows	- User mở chiến dịch hoặc project cần vận hành. - User yêu cầu kiểm tra mức độ sẵn sàng hoặc chọn thao tác kích hoạt chiến dịch.

	3. Hệ thống kiểm tra quyền thao tác và trạng thái hiện tại của chiến dịch. 4. Hệ thống đánh giá mức độ sẵn sàng dựa trên cấu hình nguồn dữ liệu, mục tiêu thu thập, trạng thái cấu hình và kết quả kiểm tra thử cần thiết. 5. Nếu mức độ sẵn sàng đạt yêu cầu, user xác nhận thao tác kích hoạt. 6. Hệ thống chuyển chiến dịch sang trạng thái hoạt động và bắt đầu hoạt động thu thập dữ liệu tương ứng. 7. Hệ thống ghi nhận trạng thái vận hành mới và trả kết quả thành công cho user. 8. User có thể theo dõi trạng thái xử lý hoặc tiếp tục khai thác các thông tin liên quan ở các use case sau.
Alternative Flows	Chỉ kiểm tra mức độ sẵn sàng Thay cho Bước 5 của luồng cơ bản: • R1. User chỉ yêu cầu xem chiến dịch đã đủ điều kiện vận hành hay chưa. • R2. Hệ thống trả về danh sách điều kiện đạt hoặc chưa đạt. • R3. Không có thay đổi trạng thái vận hành nào được thực hiện. • Kết thúc use case. Tạm dừng chiến dịch đang hoạt động Thay cho Bước 2 của luồng cơ bản: • P1. User chọn thao tác tạm dừng cho chiến dịch đang hoạt động. • P2. Hệ thống kiểm tra trạng thái hiện tại và quyền thao tác. • P3. Hệ thống chuyển chiến dịch sang trạng thái tạm dừng và dừng hoạt động thu thập dữ liệu phù hợp. • P4. Hệ thống trả kết quả tạm dừng thành công cho user. • Kết thúc use case. Tiếp tục chiến dịch đã tạm dừng Thay cho Bước 2 của luồng cơ bản: • RS1. User chọn thao tác tiếp tục cho chiến dịch đang tạm dừng. • RS2. Hệ thống kiểm tra lại mức độ sẵn sàng trước khi cho phép vận hành tiếp. • RS3. Nếu đạt yêu cầu, hệ thống chuyển chiến dịch về trạng thái hoạt động.

	• RS4. Hệ thống trả kết quả tiếp tục thành công cho user. • Kết thúc use case. Lưu trữ chiến dịch Thay cho Bước 2 của luồng cơ bản: • A1. User chọn lưu trữ một chiến dịch không còn cần vận hành thường xuyên. • A2. Hệ thống chuyển chiến dịch sang trạng thái lưu trữ sau khi xử lý các hoạt động đang chạy nếu cần. • A3. Hệ thống trả kết quả lưu trữ thành công cho user. • Kết thúc use case. Mở lại chiến dịch đã lưu trữ Thay cho Bước 2 của luồng cơ bản: • U1. User chọn mở lại một chiến dịch đã được lưu trữ. • U2. Hệ thống đưa chiến dịch về trạng thái cho phép kiểm tra trước khi vận hành lại. • U3. Hệ thống trả kết quả mở lại thành công cho user. • Kết thúc use case.
Exceptions	Mức độ sẵn sàng không đạt yêu cầu Tại Bước 4 của luồng cơ bản hoặc RS2 của luồng tiếp tục, nếu chiến dịch chưa đủ điều kiện vận hành: • 4.E.1. Hệ thống từ chối kích hoạt hoặc tiếp tục chiến dịch. • 4.E.2. Hệ thống trả về các điều kiện chưa đạt như thiếu nguồn dữ liệu, thiếu mục tiêu thu thập phù hợp, cấu hình chưa sẵn sàng hoặc kiểm tra thử không đạt. • 4.E.3. User cần quay lại use case thiết lập chiến dịch để điều chỉnh cấu hình. • Kết thúc use case. Thao tác không hợp lệ với trạng thái hiện tại Tại Bước 3 của luồng cơ bản hoặc các luồng thay thế, nếu trạng thái chiến dịch không cho phép thao tác được chọn: • 3.E.1. Hệ thống từ chối thao tác vận hành. • 3.E.2. Hệ thống hiển thị trạng thái hiện tại và thao tác hợp lệ có thể thực hiện. • Kết thúc use case. User không đủ quyền vận hành chiến dịch Tại Bước 3 của luồng cơ bản, nếu user không có quyền quản lý chiến dịch: • 3.E.3. Hệ thống từ chối yêu cầu thay đổi trạng thái.

	3.E.4. Không có thay đổi nào được áp dụng lên chiến dịch. - Kết thúc use case. Lỗi áp dụng hoặc lưu trạng thái vận hành Tại Bước 6 hoặc Bước 7 của luồng cơ bản, nếu hệ thống không thể áp dụng trạng thái vận hành hoặc không thể lưu trạng thái mới: 7.E.1. Hệ thống thông báo thao tác vận hành chưa hoàn tất. 7.E.2. Trạng thái chiến dịch được giữ nguyên hoặc được đưa về trạng thái an toàn gần nhất. 7.E.3. User có thể tải lại trạng thái và thử lại sau. - Kết thúc use case.
Phạm vi	Use case này mô tả mục tiêu vận hành chiến dịch theo góc nhìn user, không mô tả chi tiết cách hệ thống hiện thực các thao tác vận hành. Các chi tiết kỹ thuật đó thuộc phần thiết kế hệ thống và luồng xử lý ở Chương 5.

4.4.3 UC-03: Tra cứu và hỏi đáp dữ liệu phân tích

Bảng 14: Use Case UC-03

Use Case name	UC-03: Tra cứu và hỏi đáp dữ liệu phân tích
Actors	- Primary Actor: A-01 - Nhóm người dùng chuyên môn nội bộ - Secondary Actor: Không có
Description	Người dùng tra cứu dữ liệu phân tích đã được hệ thống xử lý hoặc đặt câu hỏi theo ngữ cảnh chiến dịch để nhận câu trả lời có dẫn chứng. Use case này giúp người dùng khai thác dữ liệu đã thu thập và phân tích, thay vì mô tả cách hệ thống chuẩn bị hoặc lưu trữ dữ liệu ở phía sau.
Trigger	User nhập truy vấn tìm kiếm, đặt câu hỏi trong trợ lý dữ liệu hoặc mở lại một hội thoại phân tích đã có.
Preconditions	- User đã đăng nhập và có quyền truy cập chiến dịch hoặc project tương ứng. - Phạm vi tra cứu đã được xác định theo campaign, project hoặc bộ lọc tương đương. - Hệ thống có dữ liệu phân tích phù hợp để truy vấn hoặc có khả năng phản hồi rõ ràng chưa có dữ liệu liên quan. - Nội dung truy vấn của user không rỗng và nằm trong giới hạn hợp lệ.

Postconditions	1. User nhận được danh sách kết quả tra cứu hoặc câu trả lời phân tích phù hợp với phạm vi được chọn. 2. Nếu là hỏi đáp, hệ thống lưu hội thoại và các nội dung trao đổi mới khi thao tác hoàn tất thành công. 3. Nếu không có dữ liệu phù hợp, hệ thống trả thông báo hoặc kết quả rỗng có kiểm soát thay vì suy diễn không có căn cứ. 4. Dữ liệu campaign, project và kết quả phân tích gốc không bị thay đổi bởi thao tác tra cứu hoặc hỏi đáp.
Normal Flows	1. User mở chức năng tra cứu hoặc trợ lý dữ liệu trong phạm vi một campaign hay project. 2. Hệ thống hiển thị ô nhập truy vấn, bộ lọc và ngữ cảnh hiện tại. 3. User nhập câu hỏi hoặc từ khóa cần tra cứu, có thể bổ sung bộ lọc như thời gian, nền tảng hoặc chủ đề. 4. Hệ thống kiểm tra quyền truy cập, nội dung truy vấn và phạm vi dữ liệu được phép sử dụng. 5. Hệ thống tìm các kết quả phân tích liên quan trong phạm vi đã chọn. 6. Hệ thống sắp xếp, tổng hợp và chuẩn bị phần dẫn chứng hoặc metadata cần thiết để user kiểm chứng kết quả. 7. Nếu user đang hỏi đáp, hệ thống tạo câu trả lời dựa trên các kết quả liên quan đã truy xuất. 8. Hệ thống trả kết quả tra cứu hoặc câu trả lời cho user kèm dẫn chứng, gợi ý hoặc thông tin bổ trợ nếu có. 9. Nếu là hội thoại, hệ thống lưu câu hỏi của user, câu trả lời và ngữ cảnh liên quan để user có thể tiếp tục hỏi sau đó.
Alternative Flows	Tra cứu không qua hội thoại Thay cho Bước 7 của luồng cơ bản: • S1. User chỉ yêu cầu tìm kiếm danh sách kết quả, không yêu cầu hệ thống sinh câu trả lời tổng hợp. • S2. Hệ thống trả về danh sách kết quả, điểm liên quan, nguồn và các thống kê tổng hợp nếu có. • Kết thúc use case. Tiếp tục hội thoại đã tồn tại Tại Bước 1 của luồng cơ bản: • C1. User mở một hội thoại phân tích đã có. • C2. Hệ thống nạp lịch sử hội thoại trong giới hạn cho phép.

	• C3. Câu hỏi mới của user được xử lý cùng ngữ cảnh hội thoại trước đó. • Tiếp tục tại bước 3. Tinh chỉnh truy vấn sau khi xem kết quả Sau Bước 8 của luồng cơ bản: • F1. User thay đổi từ khóa, bộ lọc hoặc phạm vi campaign/project để thu hẹp kết quả. • F2. Hệ thống thực hiện lại truy vấn với điều kiện mới. • Tiếp tục tại bước 4.
Exceptions	Truy vấn không hợp lệ Tại Bước 4 của luồng cơ bản, nếu câu hỏi rỗng, quá ngắn, quá dài hoặc chứa bộ lọc không hợp lệ: • 4.E.1. Hệ thống từ chối yêu cầu tra cứu. • 4.E.2. Hệ thống hiển thị lý do để user chỉnh sửa truy vấn. • Tiếp tục tại bước 3. User không có quyền truy cập phạm vi dữ liệu Tại Bước 4 của luồng cơ bản, nếu user không có quyền với campaign hoặc project được chọn: • 4.E.3. Hệ thống từ chối yêu cầu. • 4.E.4. Không có dữ liệu phân tích nào được trả về cho user. • Kết thúc use case. Không có dữ liệu phù hợp Tại Bước 5 hoặc Bước 6 của luồng cơ bản, nếu hệ thống không tìm được dữ liệu liên quan: • 6.E.1. Hệ thống trả kết quả rỗng hoặc thông báo chưa có đủ ngữ cảnh. • 6.E.2. Hệ thống không sinh câu trả lời khẳng định khi thiếu căn cứ dữ liệu. • Kết thúc use case. Không thể hoàn tất hỏi đáp hoặc lưu hội thoại Tại Bước 7 hoặc Bước 9 của luồng cơ bản, nếu hệ thống không tạo được câu trả lời hoặc không lưu được hội thoại: • 9.E.1. Hệ thống thông báo thao tác chưa hoàn tất. • 9.E.2. Nếu kết quả tra cứu đã có, hệ thống có thể vẫn hiển thị phần kết quả không phụ thuộc vào hội thoại. • Kết thúc use case.
Phạm vi	Use case này chỉ mô tả việc user khai thác dữ liệu phân tích. Các bước chuẩn bị dữ liệu phục vụ tra cứu, truy xuất ngữ cảnh

	hoặc sinh câu trả lời là chi tiết thiết kế kỹ thuật và không được tách thành use case người dùng riêng trong mục này.
--	---

4.4.4 UC-04: Theo dõi trạng thái và nhận cảnh báo

Bảng 15: Use Case UC-04

Use Case name	UC-04: Theo dõi trạng thái và nhận cảnh báo
Actors	- Primary Actor: A-01 - Nhóm người dùng chuyên môn nội bộ - Secondary Actor: Không có
Description	Người dùng theo dõi trạng thái vận hành của chiến dịch, project hoặc nguồn dữ liệu và nhận các thông báo quan trọng khi có thay đổi trạng thái, lỗi xử lý hoặc cảnh báo nghiệp vụ. Use case này tập trung vào việc giúp user nắm tình hình và phản ứng kịp thời, không mô tả chi tiết cơ chế truyền sự kiện nội bộ.
Trigger	User mở màn hình theo dõi trạng thái hoặc hệ thống phát sinh thông báo, cảnh báo liên quan đến phạm vi user được phép theo dõi.
Preconditions	- User đã đăng nhập và có quyền xem campaign, project hoặc phạm vi cảnh báo tương ứng. - Hệ thống có thông tin trạng thái hoặc sự kiện cảnh báo cần hiển thị cho user. - Nếu user muốn nhận thông báo trong phiên đang hoạt động, phiên làm việc của user đang hợp lệ.
Postconditions	- User nhìn thấy trạng thái hiện tại, lịch sử gần nhất hoặc thông báo liên quan đến phạm vi được chọn. - Cảnh báo hoặc thông báo phù hợp được hiển thị cho user nếu có sự kiện tương ứng. - Nếu không có sự kiện mới, hệ thống vẫn hiển thị trạng thái hiện tại hoặc thông báo không có cảnh báo mới. - User có đủ thông tin để quyết định thao tác tiếp theo nếu cần.
Normal Flows	- User mở dashboard hoặc khu vực theo dõi của một campaign hay project. - Hệ thống kiểm tra quyền truy cập và xác định phạm vi trạng thái cần hiển thị.

	3. Hệ thống hiển thị trạng thái hiện tại của các đối tượng liên quan như campaign, project, nguồn dữ liệu hoặc quá trình xử lý. 4. Khi có trạng thái mới, lỗi xử lý hoặc cảnh báo nghiệp vụ trong phạm vi user được phép theo dõi, hệ thống cập nhật nội dung cần thông báo. 5. Hệ thống phân loại thông báo theo loại sự kiện, mức độ nghiêm trọng và đối tượng liên quan. 6. Nếu user đang có phiên theo dõi hoặc nhận thông báo phù hợp, hệ thống hiển thị thông báo hoặc cảnh báo trên giao diện. 7. User xem nội dung tóm tắt gồm mức độ, thời điểm và campaign/project liên quan. 8. User mở chi tiết cảnh báo hoặc điều hướng tới campaign/project liên quan để xem thêm thông tin. 9. Hệ thống hiển thị chi tiết trạng thái hoặc cảnh báo để user quyết định thao tác tiếp theo ở use case phù hợp.
Alternative Flows	User chỉ xem trạng thái hiện tại Thay cho Bước 4 của luồng cơ bản: • S1. Không có sự kiện mới phát sinh trong thời điểm user truy cập. • S2. Hệ thống hiển thị trạng thái hiện tại và thời điểm cập nhật gần nhất. • Kết thúc use case. Không có phiên nhận thông báo hợp lệ Tại Bước 6 của luồng cơ bản: • R1. User không có phiên theo dõi hoặc nhận thông báo hợp lệ ở thời điểm thông báo được phát sinh. • R2. Hệ thống không hiển thị thông báo cho phiên đó. • R3. User vẫn có thể xem trạng thái hiện tại khi tải lại hoặc mở lại màn hình theo dõi. • Kết thúc use case. Lọc hoặc ưu tiên cảnh báo Tại Bước 7 của luồng cơ bản: • F1. User chọn bộ lọc theo campaign, project, mức độ nghiêm trọng hoặc loại cảnh báo. • F2. Hệ thống chỉ hiển thị các cảnh báo phù hợp với điều kiện lọc.

	• Kết thúc use case.
Exceptions	User không có quyền xem trạng thái hoặc cảnh báo Tại Bước 2 của luồng cơ bản, nếu user không có quyền với phạm vi được chọn: • 2.E.1. Hệ thống từ chối hiển thị thông tin trạng thái hoặc cảnh báo. • 2.E.2. Không có dữ liệu nào ngoài phạm vi quyền được trả về. • Kết thúc use case. Thông báo hoặc cảnh báo không thể hiển thị đầy đủ Tại Bước 5 của luồng cơ bản, nếu nội dung thông báo thiếu thông tin cần thiết để hiển thị đáng tin cậy: • 5.E.1. Hệ thống bỏ qua thông báo lỗi hoặc hiển thị theo dạng an toàn nếu phù hợp. • 5.E.2. User không nhận thông tin cảnh báo không đáng tin cậy. • Kết thúc use case. Không thể lấy trạng thái hiện tại Tại Bước 3 hoặc Bước 9 của luồng cơ bản, nếu hệ thống không truy xuất được trạng thái hiện tại: • 9.E.1. Hệ thống hiển thị thông báo không thể tải trạng thái. • 9.E.2. User có thể thử tải lại sau. • Kết thúc use case.
Phạm vi	Use case này bao phủ nhu cầu theo dõi và nhận biết sự kiện từ phía user. Các thao tác thay đổi trạng thái vận hành thuộc UC-02. Chi tiết kỹ thuật về kênh truyền thông báo thuộc phần thiết kế hệ thống ở Chương 5.

4.4.5 UC-05: Thiết lập và quản lý quy tắc cảnh báo khủng hoảng

Bảng 16: Use Case UC-05

Use Case name	UC-05: Thiết lập và quản lý quy tắc cảnh báo khủng hoảng
Actors	• Primary Actor: A-01 - Nhóm người dùng chuyên môn nội bộ • Secondary Actor: Không có
Description	Người dùng thiết lập, xem, cập nhật hoặc xóa các quy tắc cảnh báo khủng hoảng cho một project. Các quy tắc này xác định điều kiện cần theo dõi như keyword, volume, sentiment hoặc

	influencer để hệ thống có cơ sở phát hiện và cảnh báo ở các luồng vận hành sau đó.
Trigger	User mở chức năng cấu hình cảnh báo khủng hoảng của một project và yêu cầu xem, lưu, cập nhật hoặc xóa bộ quy tắc.
Preconditions	1. User đã đăng nhập và có quyền quản lý project tương ứng. 2. User đã chọn project cần thiết lập hoặc quản lý quy tắc cảnh báo khủng hoảng. 3. User có thông tin điều kiện cảnh báo muốn thiết lập hoặc có nhu cầu xem cấu hình hiện tại.
Postconditions	1. Bộ quy tắc cảnh báo khủng hoảng được tạo mới hoặc cập nhật nếu dữ liệu hợp lệ. 2. User có thể xem lại cấu hình đã lưu cho project. 3. Nếu user xóa cấu hình, project không còn bộ quy tắc cảnh báo khủng hoảng tương ứng. 4. Quy tắc hợp lệ trở thành đầu vào cho các luồng giám sát và phát cảnh báo sau này.
Normal Flows	1. User mở trang thiết lập cảnh báo khủng hoảng của một project. 2. Hệ thống kiểm tra quyền quản lý project và nạp cấu hình hiện có nếu đã được thiết lập. 3. Hệ thống hiển thị các nhóm quy tắc có thể cấu hình như keyword, volume, sentiment hoặc influencer. 4. User bật một hoặc nhiều nhóm quy tắc và nhập điều kiện cảnh báo tương ứng. 5. User gửi yêu cầu lưu cấu hình. 6. Hệ thống kiểm tra project, quyền thao tác và tính hợp lệ của từng nhóm quy tắc được bật. 7. Nếu dữ liệu hợp lệ, hệ thống lưu bộ quy tắc cảnh báo khủng hoảng cho project. 8. Hệ thống trả cấu hình đã lưu thành công để user xác nhận. 9. User có thể quay lại màn hình theo dõi trạng thái và cảnh báo để quan sát kết quả trong quá trình vận hành.
Alternative Flows	Xem cấu hình hiện có Thay cho Bước 4 của luồng cơ bản: • D1. User chỉ muốn xem cấu hình cảnh báo khủng hoảng đang áp dụng. • D2. Hệ thống hiển thị cấu hình hiện có hoặc thông báo chưa có cấu hình.

	• D3. Không phát sinh thao tác lưu mới. • Kết thúc use case. Chỉ bật một phần nhóm quy tắc Tại Bước 4 của luồng cơ bản: • P1. User chỉ bật một số nhóm quy tắc cần thiết thay vì cấu hình toàn bộ nhóm điều kiện. • P2. Hệ thống chỉ validate và lưu các nhóm quy tắc được bật. • Tiếp tục tại bước 5. Xóa cấu hình cảnh báo khủng hoảng Thay cho Bước 5 của luồng cơ bản: • X1. User chọn xóa cấu hình cảnh báo khủng hoảng của project. • X2. Hệ thống yêu cầu xác nhận thao tác xóa nếu giao diện có bước xác nhận. • X3. Hệ thống xóa cấu hình tương ứng khỏi project. • X4. Hệ thống trả kết quả xóa thành công. • Kết thúc use case.
Exceptions	Project không hợp lệ hoặc user không có quyền Tại Bước 2 hoặc Bước 6 của luồng cơ bản, nếu project không tồn tại hoặc user không có quyền quản lý: • 6.E.1. Hệ thống từ chối thao tác cấu hình. • 6.E.2. Không có quy tắc nào được tạo, cập nhật hoặc xóa. • Kết thúc use case. Dữ liệu quy tắc không hợp lệ Tại Bước 6 của luồng cơ bản, nếu không có nhóm quy tắc nào được bật hoặc một nhóm được bật nhưng thiếu điều kiện bắt buộc: • 6.E.3. Hệ thống trả lỗi validation cho nhóm quy tắc liên quan. • 6.E.4. User chỉnh sửa cấu hình và gửi lại. • Tiếp tục tại bước 4. Cấu hình cần xem hoặc xóa không tồn tại Trong luồng xem hoặc xóa cấu hình, nếu project chưa có bộ quy tắc cảnh báo khủng hoảng: • D.E.1. Hệ thống thông báo chưa có cấu hình tương ứng. • D.E.2. Với thao tác xóa, không có thay đổi nào được thực hiện. • Kết thúc use case.

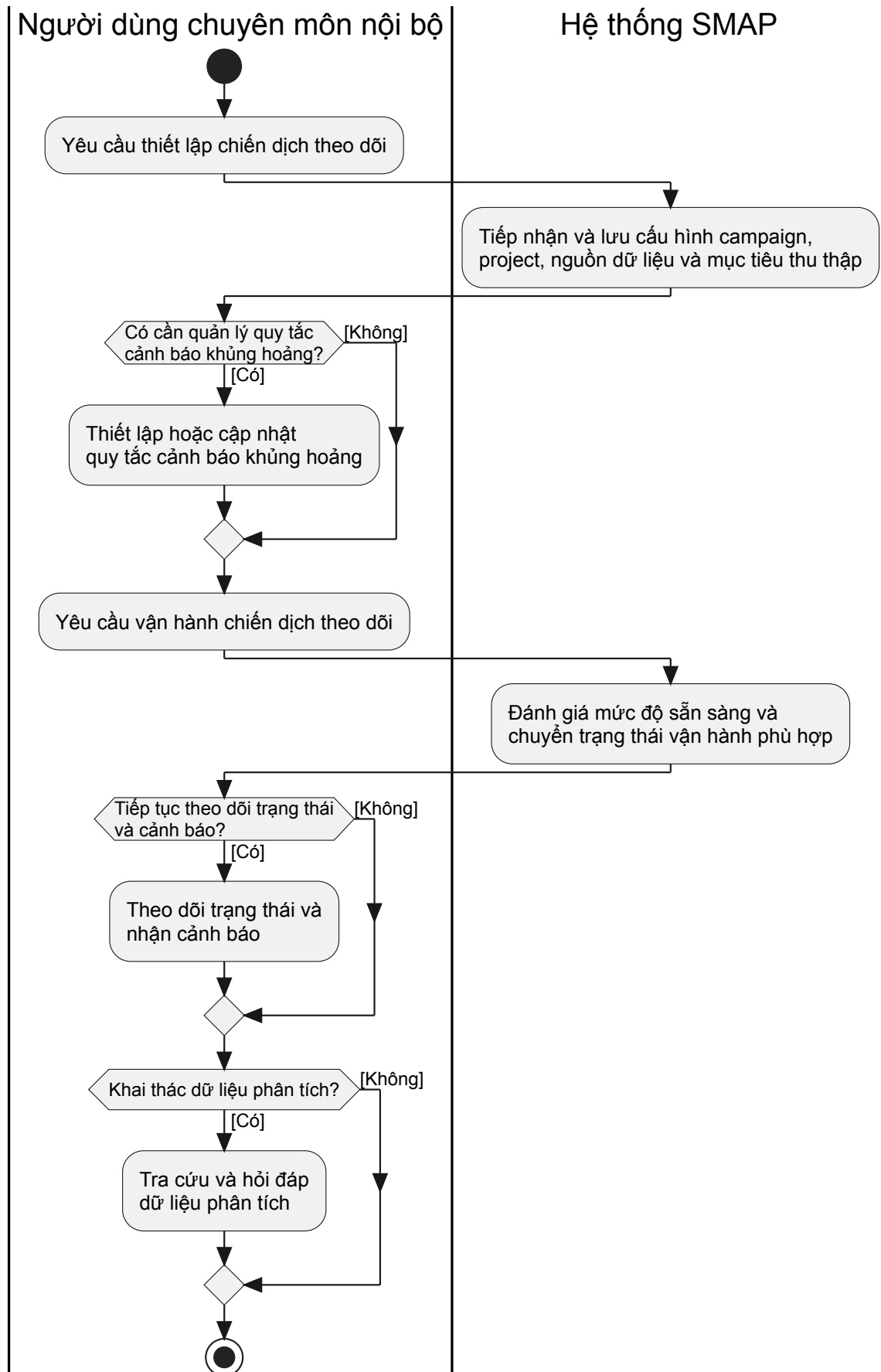
	Không thể lưu hoặc xóa cấu hình Tại Bước 7 của luồng cơ bản hoặc X3 của luồng xóa, nếu hệ thống không thể hoàn tất thao tác: • 7.E.1. Hệ thống thông báo thao tác cấu hình chưa thành công. • 7.E.2. User có thể thử lại sau. • Kết thúc use case.
Phạm vi	Use case này chỉ bao phủ việc người dùng thiết lập và quản lý quy tắc cảnh báo khủng hoảng. Việc phát hiện khủng hoảng trong quá trình vận hành và việc phân phối cảnh báo cho user thuộc các luồng xử lý hệ thống và được user quan sát thông qua UC-04.

4.5 Sơ đồ hoạt động

Phần này sử dụng sơ đồ hoạt động để mô tả dòng công việc nghiệp vụ tương ứng với các use case đã xác định ở mục 4.4. Các sơ đồ trong mục này tập trung vào thứ tự hoạt động, điểm quyết định và kết quả quan sát được ở mức người dùng; không mô tả trình tự gọi giữa các thành phần kỹ thuật. Các sơ đồ tuân tự chi tiết thuộc Chương 5.

4.5.1 Quy trình nghiệp vụ tổng quát

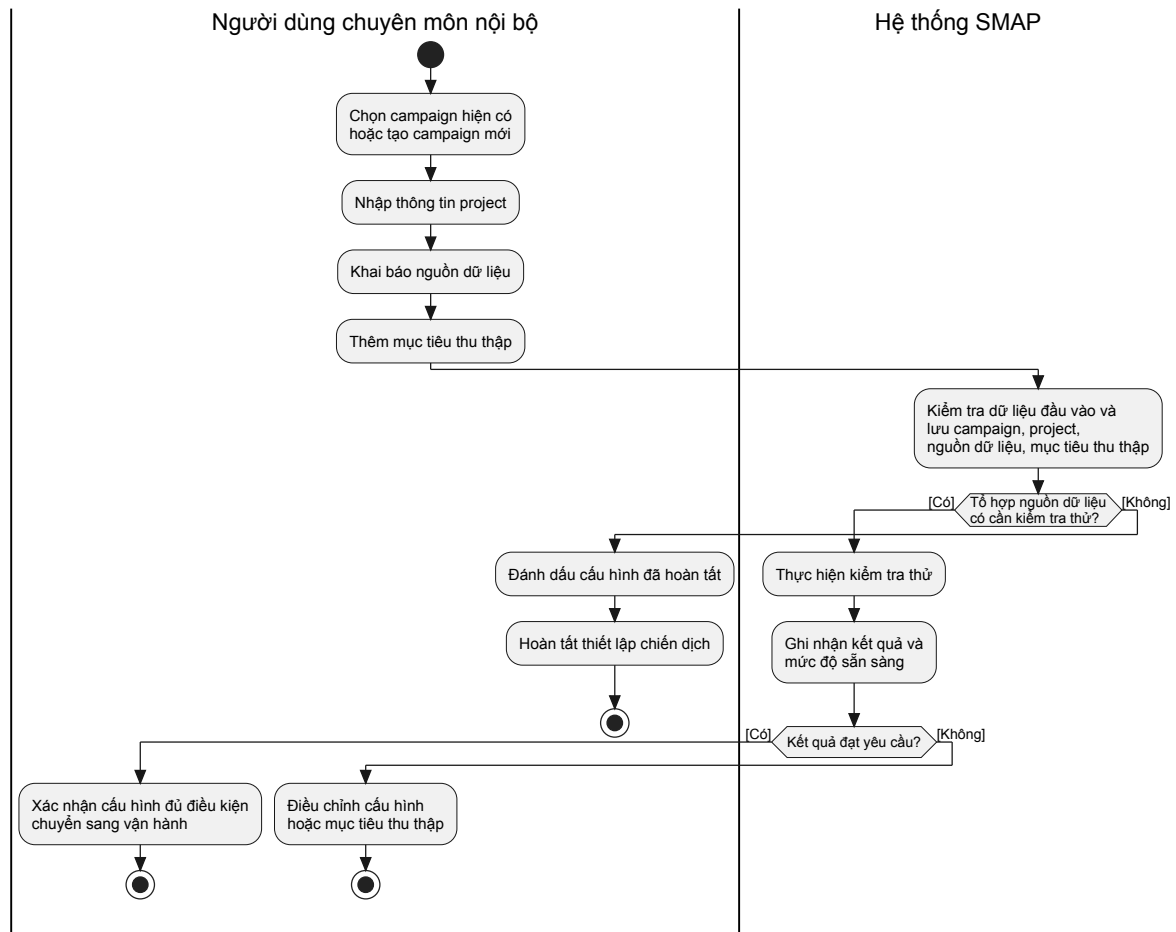
Sơ đồ dưới đây mô tả dòng nghiệp vụ tổng quát của SMAP từ thiết lập chiến dịch, cấu hình cảnh báo bổ sung khi cần, vận hành theo dõi, cho đến khai thác kết quả hoặc nhận cảnh báo.



Hình 2: Sơ đồ hoạt động tổng quát của hệ thống

4.5.2 UC-01: Thiết lập chiến dịch theo dõi

Sơ đồ hoạt động này mô tả cách người dùng tạo hoặc chọn campaign, tạo project, khai báo nguồn dữ liệu, cấu hình mục tiêu thu thập và thực hiện kiểm tra thử khi cần. Kết quả của luồng là chiến dịch có cấu hình đầu vào đủ điều kiện để chuyển sang vận hành hoặc danh sách điểm cần điều chỉnh.



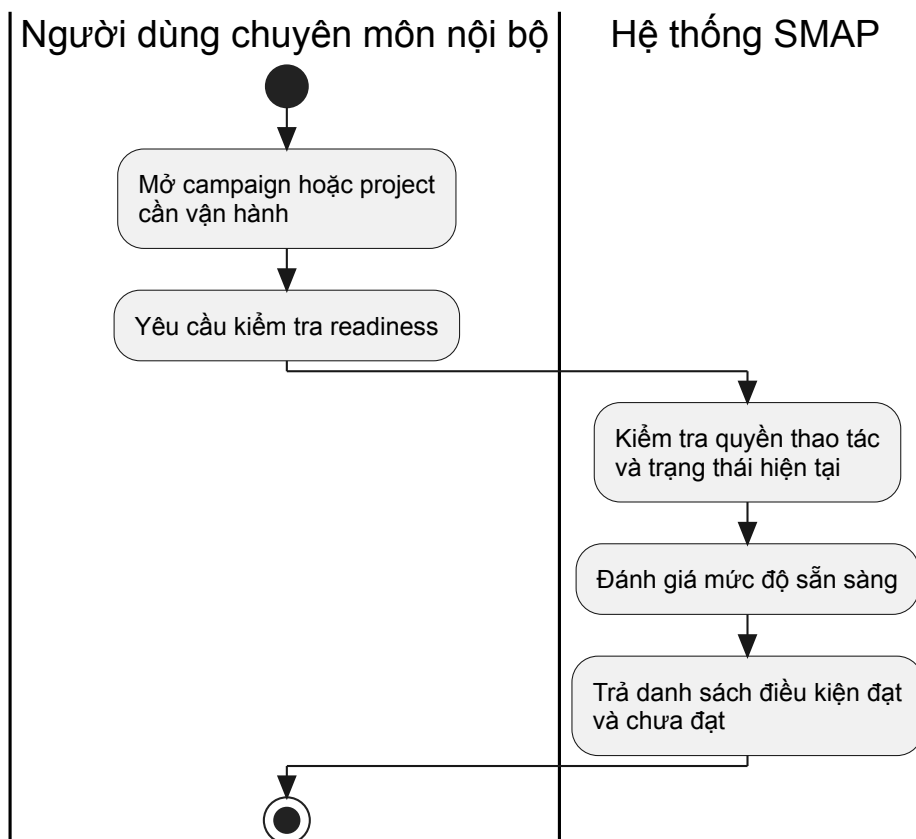
Hình 3: Sơ đồ hoạt động UC-01 - Thiết lập chiến dịch theo dõi

4.5.3 UC-02: Vận hành chiến dịch theo dõi

Use case này được tách thành ba sơ đồ hoạt động nhỏ hơn để phản ánh đúng ba nhóm thao tác nghiệp vụ chính: kiểm tra readiness, kích hoạt hoặc tiếp tục chiến dịch, và các thao tác thay đổi trạng thái còn lại như tạm dừng, lưu trữ hoặc mở lại.

a. Kiểm tra readiness

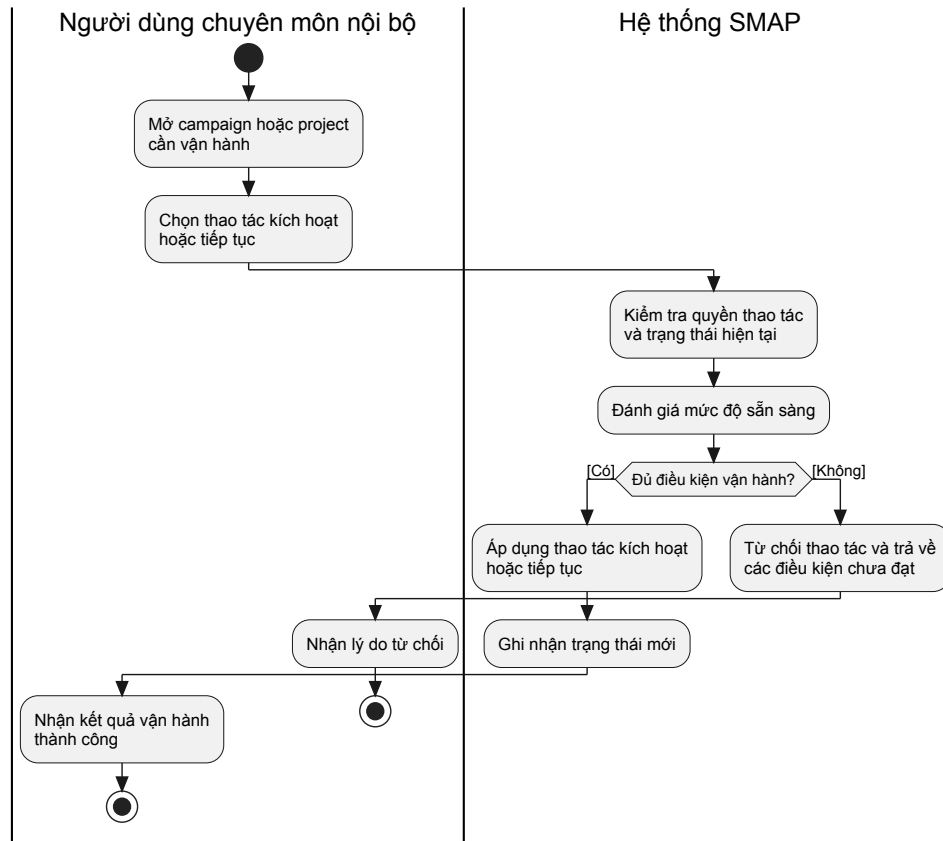
Sơ đồ này mô tả trường hợp user chỉ yêu cầu đánh giá mức độ sẵn sàng của chiến dịch mà chưa đổi trạng thái vận hành.



Hình 4: Sơ đồ hoạt động UC-02A - Kiểm tra readiness

b. Kích hoạt hoặc tiếp tục chiến dịch

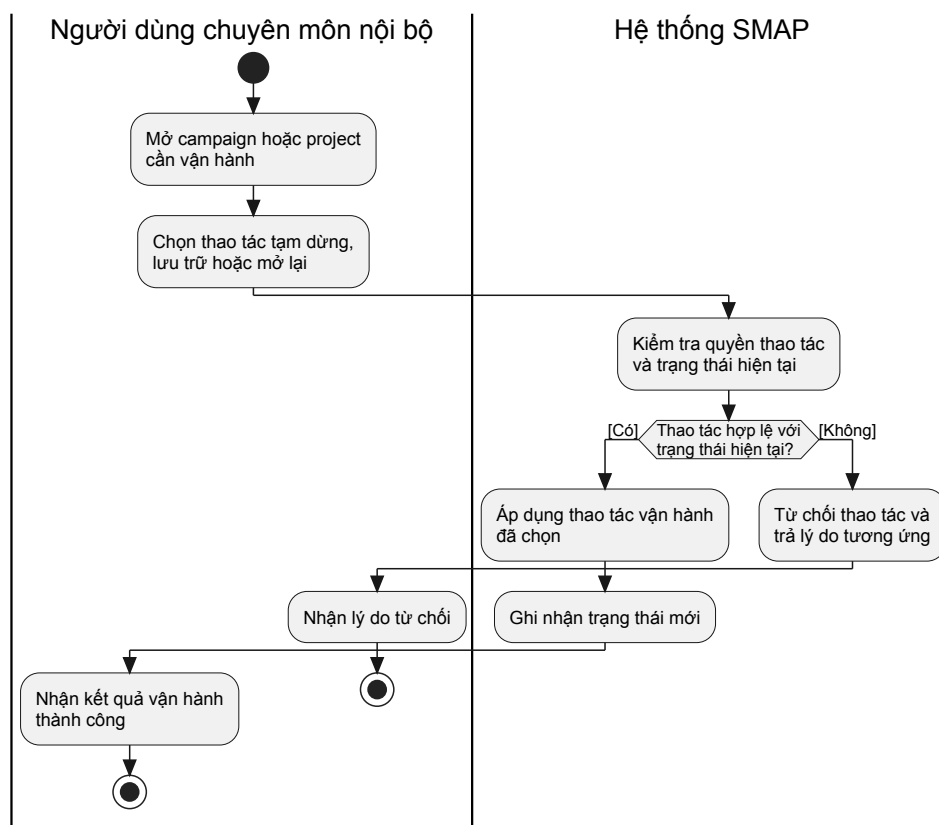
Sơ đồ này mô tả nhánh vận hành cần đi qua kiểm tra readiness trước khi hệ thống cho phép campaign hoặc project chuyển sang trạng thái hoạt động.



Hình 5: Sơ đồ hoạt động UC-02B - Kích hoạt hoặc tiếp tục chiến dịch

c. Tạm dừng, lưu trữ hoặc mở lại

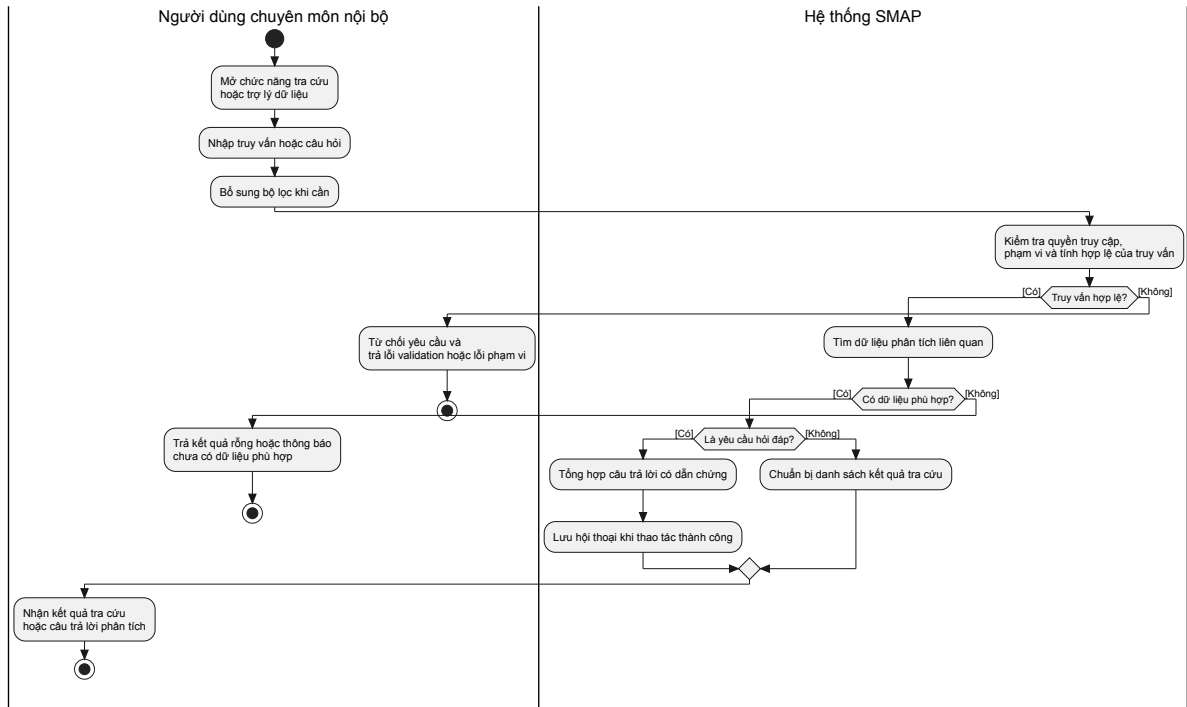
Sơ đồ này mô tả các thao tác thay đổi trạng thái không đi qua cùng một nhánh readiness như kích hoạt hoặc tiếp tục, mà được kiểm tra theo tính hợp lệ của trạng thái hiện tại.



Hình 6: Sơ đồ hoạt động UC-02C - Tạm dừng, lưu trữ hoặc mở lại

4.5.4 UC-03: Tra cứu và hỏi đáp dữ liệu phân tích

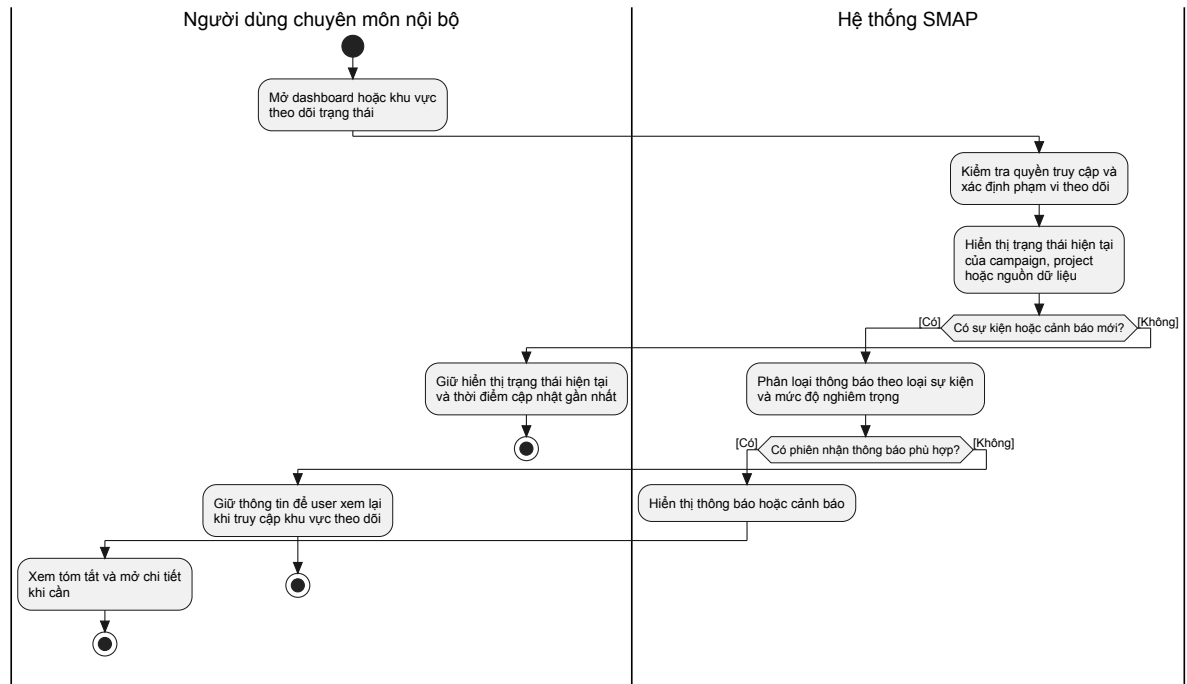
Sơ đồ hoạt động này mô tả cách người dùng gửi truy vấn hoặc câu hỏi, hệ thống kiểm tra phạm vi truy cập, tìm dữ liệu liên quan và trả kết quả tra cứu hoặc câu trả lời có dẫn chứng. Nếu không có dữ liệu phù hợp, hệ thống trả kết quả rỗng có kiểm soát thay vì suy diễn thiếu căn cứ.



Hình 7: Sơ đồ hoạt động UC-03 - Tra cứu và hỏi đáp dữ liệu phân tích

4.5.5 UC-04: Theo dõi trạng thái và nhận cảnh báo

Sơ đồ hoạt động này mô tả cách người dùng mở khu vực theo dõi, xem trạng thái hiện tại và nhận thông báo hoặc cảnh báo khi có sự kiện liên quan. Use case này chỉ giúp người dùng quan sát và phản ứng, không trực tiếp thay đổi trạng thái vận hành.



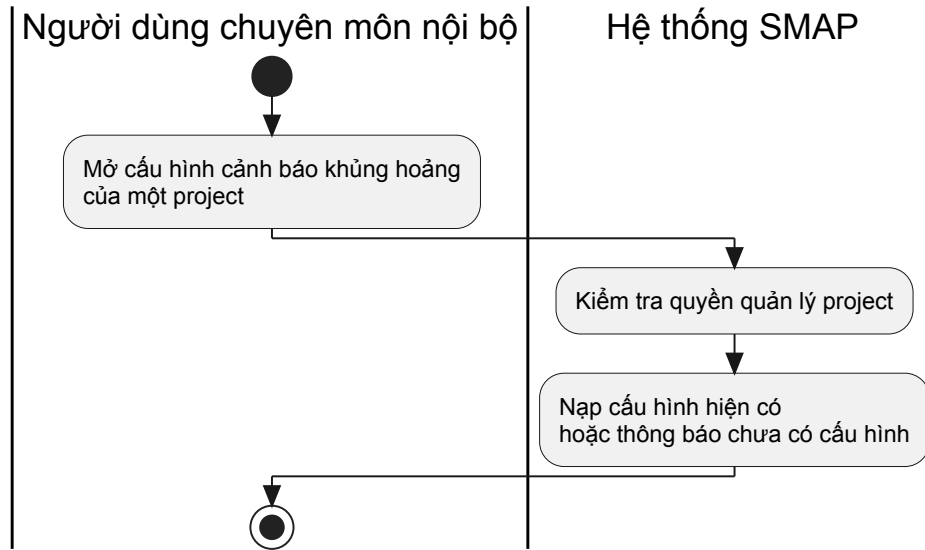
Hình 8: Sơ đồ hoạt động UC-04 - Theo dõi trạng thái và nhận cảnh báo

4.5.6 UC-05: Thiết lập và quản lý quy tắc cảnh báo khủng hoảng

Use case này được tách thành ba sơ đồ hoạt động nhỏ hơn để phản ánh rõ ba nhóm thao tác chính: xem cấu hình hiện có, lưu hoặc cập nhật quy tắc, và xóa cấu hình cảnh báo khủng hoảng.

a. Xem cấu hình hiện có

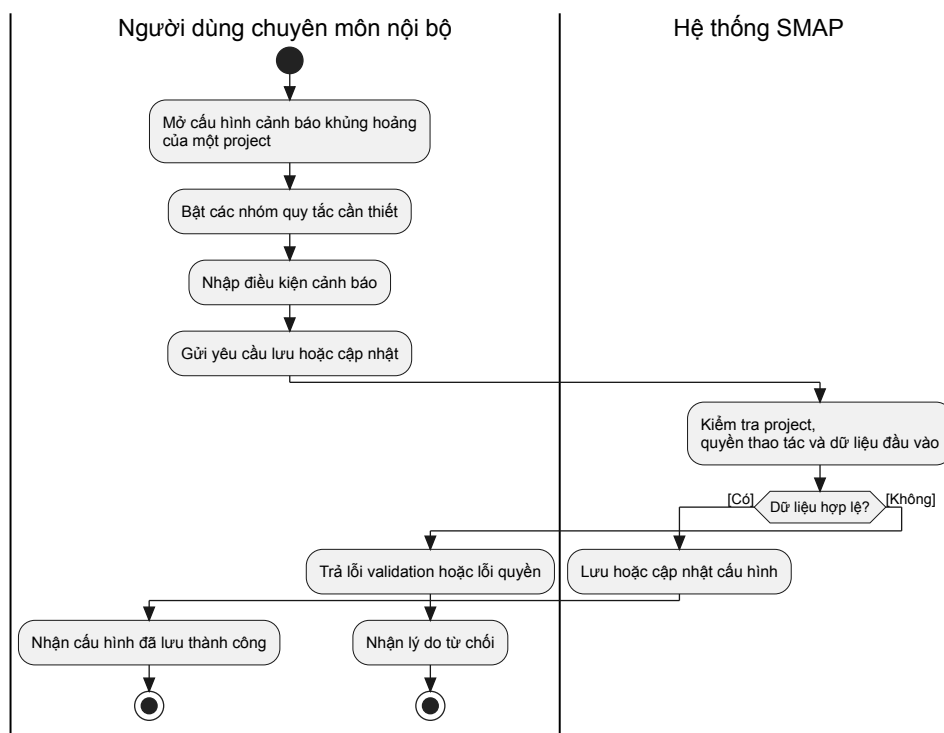
Sơ đồ này mô tả trường hợp user chỉ mở cấu hình cảnh báo khủng hoảng để xem trạng thái hiện có của một project.



Hình 9: Sơ đồ hoạt động UC-05A - Xem cấu hình cảnh báo khủng hoảng

b. Lưu hoặc cập nhật quy tắc

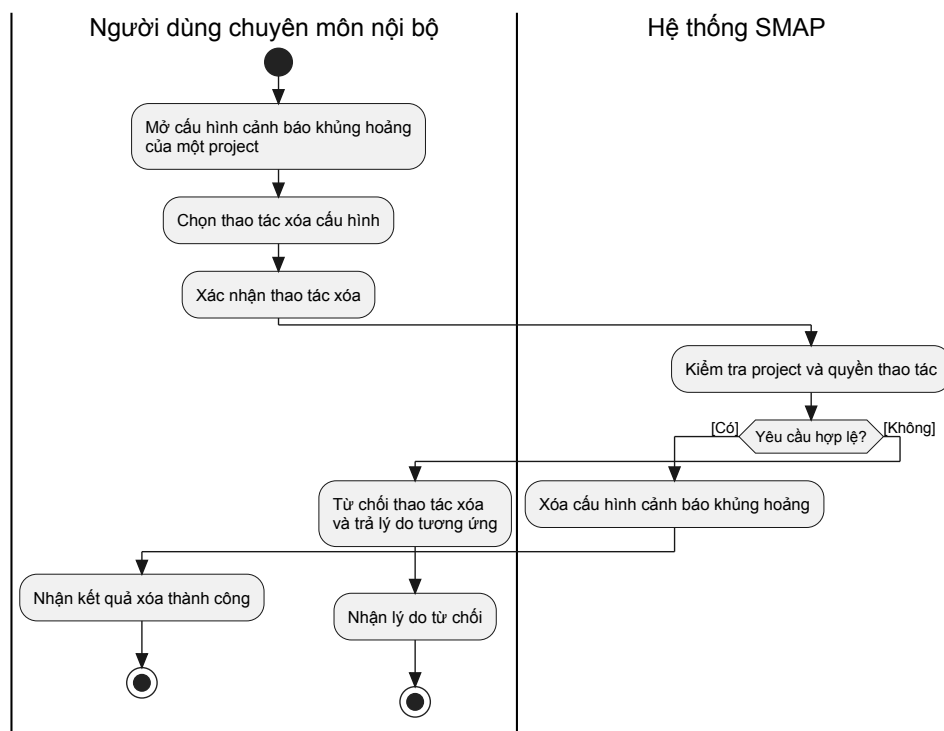
Sơ đồ này mô tả trường hợp user bật các nhóm quy tắc, nhập điều kiện cảnh báo và gửi yêu cầu lưu hoặc cập nhật cấu hình.



Hình 10: Sơ đồ hoạt động UC-05B - Lưu hoặc cập nhật cấu hình cảnh báo khủng hoảng

c. Xóa cấu hình

Sơ đồ này mô tả trường hợp user yêu cầu xóa cấu hình cảnh báo khủng hoảng của project sau bước xác nhận thao tác.



Hình 11: Sơ đồ hoạt động UC-05C - Xóa cấu hình cảnh báo khủng hoảng

CHƯƠNG 5: THIẾT KẾ HỆ THỐNG

5.1 Phương pháp tiếp cận

Chương này trình bày cách hệ thống SMAP được thiết kế dựa trên các yêu cầu và ràng buộc đã phân tích ở Chương 4. Thay vì xem thiết kế như một tập hợp sơ đồ độc lập, chương này tiếp cận kiến trúc theo hướng design-from-drivers, tức lấy các yêu cầu chức năng, yêu cầu phi chức năng và ranh giới service hiện tại làm cơ sở để lựa chọn tổ chức thành phần, cơ chế giao tiếp, mô hình lưu trữ và phương thức triển khai.

Trong bối cảnh của SMAP, thiết kế không chỉ nhằm mô tả các thành phần của hệ thống, mà còn nhằm trả lời ba câu hỏi cốt lõi. Thứ nhất, mỗi service sở hữu capability nào và không nên gánh trách nhiệm nào. Thứ hai, dữ liệu đi qua các lane xử lý bằng cơ chế nào cho phù hợp với từng loại workload. Thứ ba, các quyết định kiến trúc đó phản hồi ra sao đối với các yêu cầu về hiệu năng, bảo mật, tính sẵn sàng, tính toàn vẹn dữ liệu và khả năng quan sát hệ thống.

5.1.1 Architectural Drivers

Các architectural drivers chính của SMAP được kế thừa trực tiếp từ phần NFR ở Chương 4 và được xem như cơ sở định hướng cho các quyết định thiết kế ở chương này.

Bảng 17: Các architectural drivers chính của SMAP

ID	Driver	Ý nghĩa thiết kế
AC-01	Modularity	Hệ thống cần được chia thành các bounded context rõ ràng để tránh coupling không cần thiết giữa business context, execution plane, analytics, knowledge và notification.
AC-02	Scalability	Các lane có tải khác nhau như API, analytics consumer, worker runtime và notification delivery cần có khả năng mở rộng theo vai trò riêng.
AC-03	Performance	Các xử lý nặng không nên nằm trong request path đồng bộ; các lane thời gian thực cần có độ trễ phù hợp với trải nghiệm sử dụng.
AC-04	Security	Xác thực, quản lý phiên và internal validation cần được tổ chức thành security boundary rõ ràng.
AC-05	Availability	Các workload quan trọng cần cơ chế health check, tách lane xử lý và khả năng khôi phục phù hợp để giảm blast radius.

AC-06	Data Integrity	Các luồng bất đồng bộ phải duy trì liên kết nhất quán giữa task, completion metadata, raw artifacts và kết quả downstream.
AC-07	Observability	Thiết kế cần hỗ trợ logging, metrics hoặc các operational indicators đủ để theo dõi và chẩn đoán lỗi runtime.

5.1.2 Nguyên tắc thiết kế

Từ các architectural drivers trên, nhóm nguyên tắc thiết kế được sử dụng cho SMAP có thể tóm lược như sau:

1. Phân tách hệ thống theo bounded context và trách nhiệm nghiệp vụ rõ ràng.
2. Tổ chức service theo hướng có thể triển khai và mở rộng tương đối độc lập.
3. Chuyên biệt hóa transport theo workload thay vì dùng một cơ chế giao tiếp duy nhất cho toàn bộ hệ thống.
4. Tách business control plane khỏi execution plane và các lane xử lý nền.
5. Áp dụng polyglot persistence theo nhu cầu dữ liệu thực tế của từng lớp hệ thống.
6. Tách logic nghiệp vụ khỏi delivery và infrastructure layers để tăng khả năng bảo trì và kiểm thử.

Các nguyên tắc này không chỉ là lựa chọn mang tính lý thuyết, mà còn là khung dùng để đọc và giải thích các quyết định thiết kế ở những mục tiếp theo. Chúng giúp lý giải vì sao hệ thống không được tổ chức theo một khối thống nhất, vì sao nhiều loại storage cùng tồn tại, và vì sao mỗi lane xử lý lại gắn với một cơ chế giao tiếp khác nhau.

5.1.3 Liên hệ với SMAP

Trong architecture của SMAP, các nguyên tắc trên được phản ánh khá rõ. `project-srv` giữ business context, trong khi `ingest-srv` đảm nhiệm execution plane. `analysis-srv` xử lý analytics pipeline ở lane bất đồng bộ tách khỏi request path. `knowledge-srv` và `notification-srv` tạo thành các lớp khai thác kết quả và delivery. Ở mức storage, hệ thống đồng thời sử dụng PostgreSQL, Redis, Qdrant và MinIO theo vai trò chuyên biệt. Ở mức giao tiếp, internal HTTP, RabbitMQ, Kafka và Redis Pub/Sub cùng tồn tại như các transport được chọn theo tính chất workload.

Những đặc điểm này cho thấy thiết kế của SMAP không nên được đọc như một bản áp dụng máy móc của một pattern duy nhất, mà là kết quả của nhiều quyết định phối hợp để phản hồi trực tiếp với các yêu cầu và ràng buộc của hệ thống hiện tại. Đây cũng là cơ sở để các mục sau đi sâu vào kiến trúc tổng thể, dữ liệu, giao tiếp, triển khai và truy xuất nguồn gốc thiết kế.

5.2 Kiến trúc tổng thể

5.2.1 Lựa chọn phong cách kiến trúc

5.2.1.1 Bối cảnh quyết định

Việc lựa chọn kiến trúc cho SMAP không chỉ là quyết định công nghệ, mà là phản hồi trực tiếp đối với các architectural drivers đã xác định ở mục 5.1. Hệ thống cần đồng thời xử lý API/business context, điều phối runtime ingest, analytics pipeline, semantic retrieval, realtime delivery và nhiều loại storage khác nhau. Điều này làm cho việc tổ chức kiến trúc trở thành một bài toán về workload specialization, service ownership và transport suitability, chứ không chỉ là việc chia code thành nhiều thư mục.

Ba động lực chính chi phối quyết định kiến trúc của SMAP là:

- nhu cầu tách biệt rõ giữa business context và execution plane;
- nhu cầu mở rộng độc lập cho các lane có tải khác nhau;
- nhu cầu cô lập lỗi giữa các thành phần như authentication, ingest runtime, analytics và delivery.

5.2.1.2 Đánh giá các lựa chọn

Trong phạm vi của đồ án, ba phong cách kiến trúc thường được xem xét gồm Monolithic Architecture, Modular Monolith và Microservices Architecture. Mỗi phong cách có ưu điểm riêng, nhưng mức độ phù hợp với SMAP là khác nhau.

Bảng 18: So sánh các lựa chọn kiến trúc ở mức định tính

Tiêu chí	Monolithic	Modular Monolith	Microservices
Tách biệt bounded context	Thấp	Trung bình	Cao
Mở rộng độc lập theo workload	Thấp	Thấp đến trung bình	Cao
Phù hợp đa runtime	Thấp	Thấp	Cao
Cô lập lỗi giữa các lane	Thấp	Trung bình	Cao
Độ đơn giản triển khai ban đầu	Cao	Trung bình	Thấp
Phù hợp với SMAP	Thấp	Trung bình	Cao

Monolithic Architecture có lợi thế về độ đơn giản trong giai đoạn đầu, nhưng không phù hợp khi hệ thống cần tách rõ nhiều lane xử lý có tính chất khác nhau. Modular Monolith cải thiện tổ chức code tốt hơn, nhưng vẫn không giải quyết triệt để bài toán polyglot runtime, fault isolation và scale theo từng vai trò runtime. Trong khi đó, Microservices

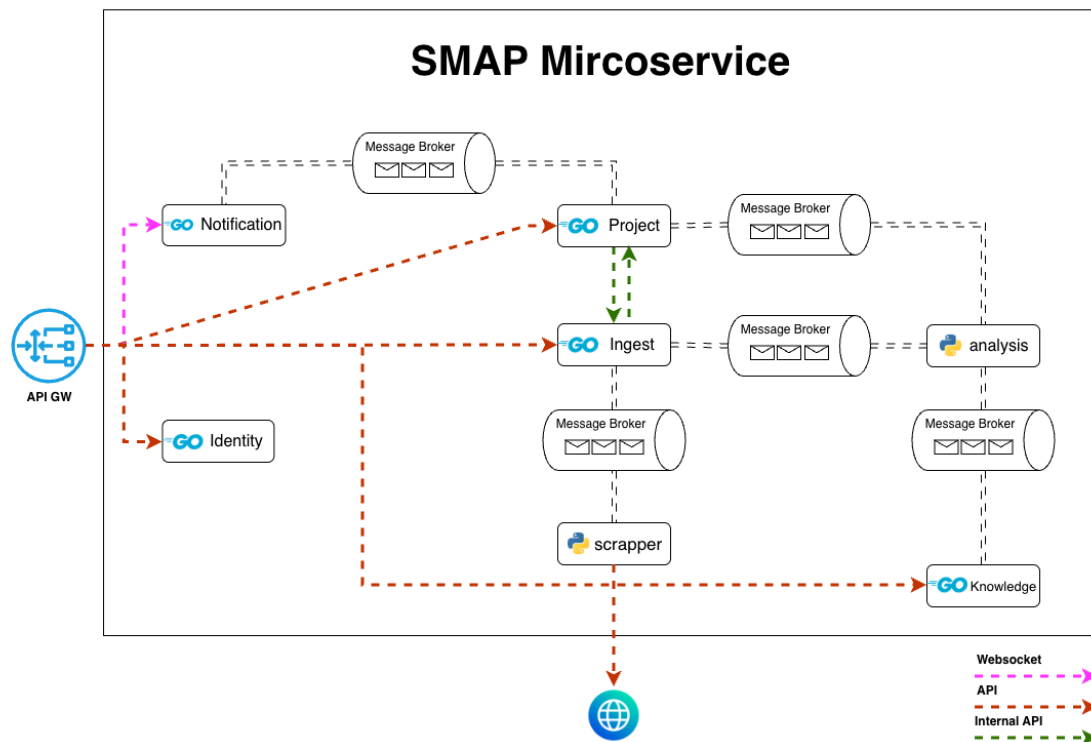
Architecture phù hợp hơn với SMAP vì cho phép phân tách business context, execution plane, analytics pipeline, retrieval và notification thành các thành phần có thể phát triển, triển khai và mở rộng tương đối độc lập.

5.2.1.3 Kết luận lựa chọn kiến trúc

Từ các so sánh trên, Microservices Architecture là lựa chọn phù hợp nhất cho SMAP. Lý do chính không nằm ở việc theo đuổi một mô hình kiến trúc phổ biến, mà ở chỗ cách tổ chức này phản hồi tốt nhất với các ràng buộc thật của hệ thống: nhiều bounded context, nhiều loại workload, nhiều công nghệ runtime và nhiều transport được chuyên biệt hóa theo từng lane xử lý.

5.2.2 Kiến trúc tổng thể

Hình dưới đây trình bày kiến trúc tổng thể của SMAP.



Hình 12: Kiến trúc tổng thể của SMAP

Kiến trúc này cho thấy SMAP được chia thành sáu nhóm thành phần chính. **identity-srv** tạo security boundary cho toàn hệ thống. **project-srv** giữ business context và lifecycle của campaign/project. **ingest-srv** cùng **scrapper-srv** hình thành execution plane để điều phối crawl và chuẩn hóa dữ liệu. **analysis-srv** xử lý pipeline AI/NLP. **knowledge-srv** xây dựng lớp semantic search và khai thác thông tin theo ngữ cảnh. **notification-srv** chịu trách nhiệm delivery theo thời gian thực. Ngoài ra, frontend

được tổ chức như một lớp giao diện đa vai trò, hỗ trợ web delivery, analytics dashboard integration và desktop packaging khi cần.

Một điểm quan trọng của kiến trúc này là không áp dụng một transport duy nhất cho toàn bộ hệ thống. Control plane giữa `project-srv` và `ingest-srv` nghiêng về internal HTTP. Crawl runtime dùng RabbitMQ. Analytics data plane dùng Kafka. Notification ingress dùng Redis Pub/Sub. Cách tổ chức này giúp mỗi lane xử lý sử dụng cơ chế giao tiếp phù hợp hơn với tính chất workload thay vì ép toàn bộ hệ thống chạy theo cùng một khuôn mẫu.

Bảng 19: Các quyết định kiến trúc chính

Quyết định	Lựa chọn	Lý do chính
Tách identity khỏi business services	<code>identity-srv</code> riêng	Cô lập xác thực, JWT, session và OAuth2 khỏi các domain nghiệp vụ khác.
Tách project context khỏi ingest runtime	<code>project-srv</code> và <code>ingest-srv</code>	Giữ ownership business context tách khỏi execution plane.
Dùng internal HTTP cho lifecycle control	Control plane đồng bộ	Readiness check và activate/pause/resume cần phản hồi đồng bộ trước khi đổi business status.
Dùng RabbitMQ cho crawl runtime	Task queue theo platform	Phù hợp cho work queue, dispatch task và completion correlation.
Dùng Kafka cho analytics data plane	Event streaming	Phù hợp cho batch consumer, downstream fanout và replay.
Dùng Redis cho notification ingress	Pub/Sub channels	Phù hợp cho fanout thời gian thực và routing nhẹ.
Dùng Qdrant cho knowledge layer	Vector store	Phục vụ semantic search, chat và retrieval theo ngữ cảnh.

5.2.3 Service Ownership và ranh giới trách nhiệm

Một thiết kế tốt không chỉ chỉ ra có bao nhiêu service, mà còn phải xác định rõ service nào sở hữu capability nào và service nào không nên gánh trách nhiệm nào. Điều này đặc biệt quan trọng trong các hệ thống đa service để tránh chồng chéo ownership và nguy cơ biến hệ thống thành distributed monolith.

Bảng 20: Service Ownership and Boundary Matrix

Service	Ownership chính	Không nên sở hữu	Ý nghĩa ranh giới
---------	-----------------	------------------	-------------------

identity-srv	OAuth2, JWT, session, token validation	business metadata, crawl runtime, analytics facts	security boundary tách khỏi domain logic
project-srv	campaign, project, crisis config, business metadata	raw batch, queue runtime state, vector index	owner của business context
ingest-srv	datasource, target, dry run, scheduled job, external task, raw batch, UAP publishing	OAuth/session, rich NLP logic, semantic retrieval	owner của execution plane và data ingress
analysis-srv	NLP enrichment, reporting bundle, crisis signals, analytics topics	project CRUD, datasource CRUD, websocket delivery	owner của analytics pipeline
knowledge-srv	semantic search, chat, indexed documents, conversations, knowledge-side report artifacts	crawl orchestration, project lifecycle control	owner của retrieval và knowledge consumption
notification-srv	WebSocket connection management, alert formatting và dispatch	project decision logic, analytics computation	owner của delivery channels
scraper-srv	platform-specific crawling, raw artifact storage và completion publish	project business logic, analytics, semantic retrieval	worker runtime tách khỏi business services

5.2.4 Technology Stack và các lựa chọn hạ tầng

5.2.4.1 Backend và frontend

Bảng 21: Technology stack cho các lớp chính

Lớp / Service	Công nghệ chính	Lý do chính
Go services	Go + Gin	Phù hợp cho API services, internal HTTP control plane và các lane cần concurrency tốt với memory footprint thấp.
Analytics runtime	Python + consumer runtime	Phù hợp với AI/NLP pipeline và hệ sinh thái xử lý dữ liệu.

Scraper runtime	Python + FastAPI/worker	Phù hợp cho browser automation, crawling runtime và worker lifecycle.
Frontend	Next.js + React + TypeScript + Tailwind CSS	Phù hợp cho giao diện người dùng hiện đại, hỗ trợ i18n, analytics API integration và mở rộng đa hình thức triển khai.
Desktop packaging	Electron	Cho phép tái sử dụng frontend stack hiện có cho desktop runtime khi cần.

5.2.4.2 Data storage

Bảng 22: Lựa chọn lớp lưu trữ dữ liệu

Storage	Vai trò chính	Lý do lựa chọn
PostgreSQL	Metadata và dữ liệu nghiệp vụ	Phù hợp cho dữ liệu quan hệ, trạng thái nghiệp vụ và transaction-oriented workloads.
Redis	Cache, session, pub/sub	Phù hợp cho độ trễ thấp, fanout realtime, token blacklist và coordination nhẹ.
Qdrant	Vector search	Phù hợp cho semantic retrieval, search/chat và khai thác thông tin theo ngữ cảnh.
MinIO	Object storage	Phù hợp cho raw artifact, report output và các object trung gian trong pipeline.

5.2.4.3 Transport và orchestration

Bảng 23: Lựa chọn transport và orchestration

Thành phần	Vai trò	Lý do lựa chọn
Internal HTTP	Control plane	Phù hợp cho readiness check và lifecycle control cần phản hồi đồng bộ.
RabbitMQ	Task queue	Phù hợp cho crawl runtime, dispatch task và completion lane.
Kafka	Event streaming	Phù hợp cho analytics data plane, fanout downstream và replay.
Redis Pub/Sub	Realtime ingress	Phù hợp cho notification fanout và lightweight channel routing.
Docker / Docker Compose	Containerization và local orchestration	Hỗ trợ đóng gói và dựng local/test stacks nhiều service.
Kubernetes	Workload orchestration	Phù hợp cho deployment, scale-by-role và health-managed workloads.

5.2.5 Tổng kết mục

Kiến trúc tổng thể của SMAP nên được hiểu như một hệ thống đa service được tổ chức theo capability và workload specialization, thay vì như một tập hợp service được tách ra thuần túy vì lý do kỹ thuật. Giá trị của thiết kế này nằm ở chỗ nó tách rõ business context khỏi execution plane, tách analytics khỏi request path, và tách knowledge/delivery thành các lớp tiêu thụ riêng. Cùng với đó, việc chuyên biệt hóa transport và storage giúp mỗi lane xử lý được tối ưu theo đặc điểm thực tế của nó.

Những lựa chọn này tạo nền cho các mục tiếp theo của Chương 5, nơi từng service, từng lớp dữ liệu và từng mẫu giao tiếp sẽ được phân tích sâu hơn ở mức thiết kế chi tiết.

5.3 Thiết kế chi tiết các dịch vụ

Mục 5.2 đã trình bày kiến trúc tổng thể của hệ thống SMAP ở cấp độ container và ranh giới trách nhiệm giữa các service. Mục này đi sâu hơn vào các thành phần cốt lõi ở mức thiết kế chi tiết, nhằm làm rõ cách các capability quan trọng của hệ thống được hiện thực bên trong từng service hoặc runtime lane tương ứng.

Mục đích của mục này là:

- Làm rõ cấu trúc nội bộ của từng service: Các components, layers, và cách tổ chức code theo Clean Architecture.
- Mô tả trách nhiệm của từng component: Input, output, và technology category.
- Giải thích các design patterns được áp dụng: Lý do chọn pattern và lợi ích mang lại.
- Liên hệ với các yêu cầu ở Chương 4: FRs, NFRs và các Use Case tương ứng.

Mục này được tổ chức theo các capability và runtime lane quan trọng của hệ thống. Các phần ưu tiên trước hết là xác thực và session management, business context persistence, lifecycle control, analytics pipeline, knowledge retrieval và realtime delivery.

5.3.1 Identity Service

Identity Service là dịch vụ chịu trách nhiệm cho security boundary của toàn hệ thống SMAP. Dịch vụ này tập trung vào xác thực người dùng, quản lý session, phát hành token, hỗ trợ internal token validation cho các service khác, và duy trì một audit processing lane chạy nền để xử lý audit events bất đồng bộ.

Vai trò của Identity Service trong kiến trúc tổng thể:

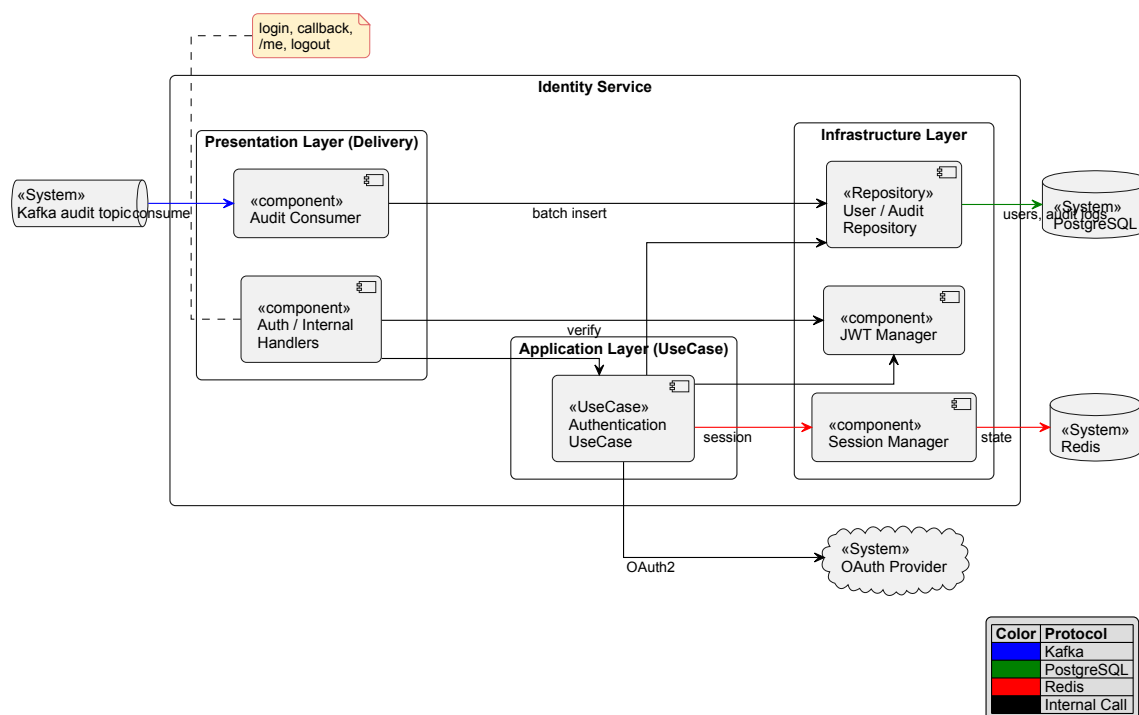
- Authentication Gateway: Xử lý đăng nhập và callback với external identity provider.
- Session Manager: Tạo và duy trì phiên truy cập sau khi xác thực thành công.
- Token Authority: Phát hành và xác thực JWT cho protected routes và internal validation.
- Security Boundary: Tách riêng xác thực khỏi business logic của project, ingest, analytics và knowledge.

- Audit Processing Lane: Tiêu thụ audit events và flush dữ liệu truy vết vào persistence layer theo batch.

Service này đáp ứng trực tiếp FR-01 về User Authentication như một supporting concern của toàn hệ thống. Ngoài capability auth-facing, service còn có một consumer lane phục vụ audit logging và truy vết vận hành.

5.3.1.1 Component Diagram - C4 Level 3

Identity Service được tổ chức theo hướng tách biệt delivery, usecase và infrastructure để giữ rõ ranh giới giữa protocol handling, auth logic và persistence.



Hình 13: Biểu đồ thành phần của Identity Service

5.3.1.2 Component Catalog

Bảng 24: Component Catalog - Identity Service

Component	Responsibility	Input / Output	Technology
Auth / Internal Handlers	Xử lý login, callback, logout, /me và các internal validation routes ở lớp HTTP delivery	HTTP request / JSON response hoặc redirect	Gin + HTTP handler

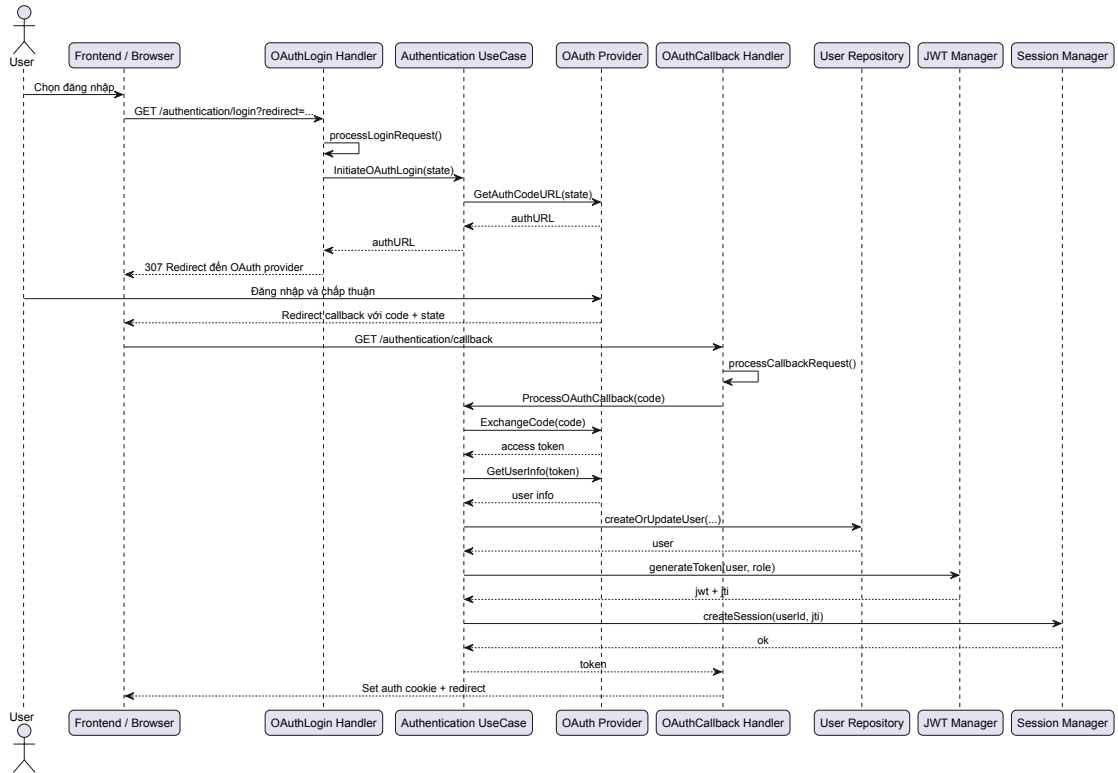
Authentication UseCase	Điều phối OAuth2 flow, xử lý callback, hoàn tất phiên truy cập	Login input / session output	Pure Go logic
JWT Manager	Phát hành và xác thực token dùng cho protected routes và internal validation	Claims / signed token	JWT library
Session Manager	Tạo, duy trì và thu hồi session hoặc blacklist state khi cần	Session key / session state	Redis-backed session layer
Audit Consumer	Tiêu thụ audit events từ Kafka, gom batch theo size hoặc timeout và flush vào persistence layer	Audit event / persisted audit rows	Kafka consumer runtime
User / Audit Repository	Lưu thông tin người dùng, audit log và dữ liệu identity liên quan	Persistence commands / query results	PostgreSQL repository

5.3.1.3 Data Flow

Luồng xử lý chính của Identity Service có thể nhìn qua ba flow quan trọng: OAuth2 login flow, protected-route validation flow và audit event processing flow.

a. OAuth2 Login Flow

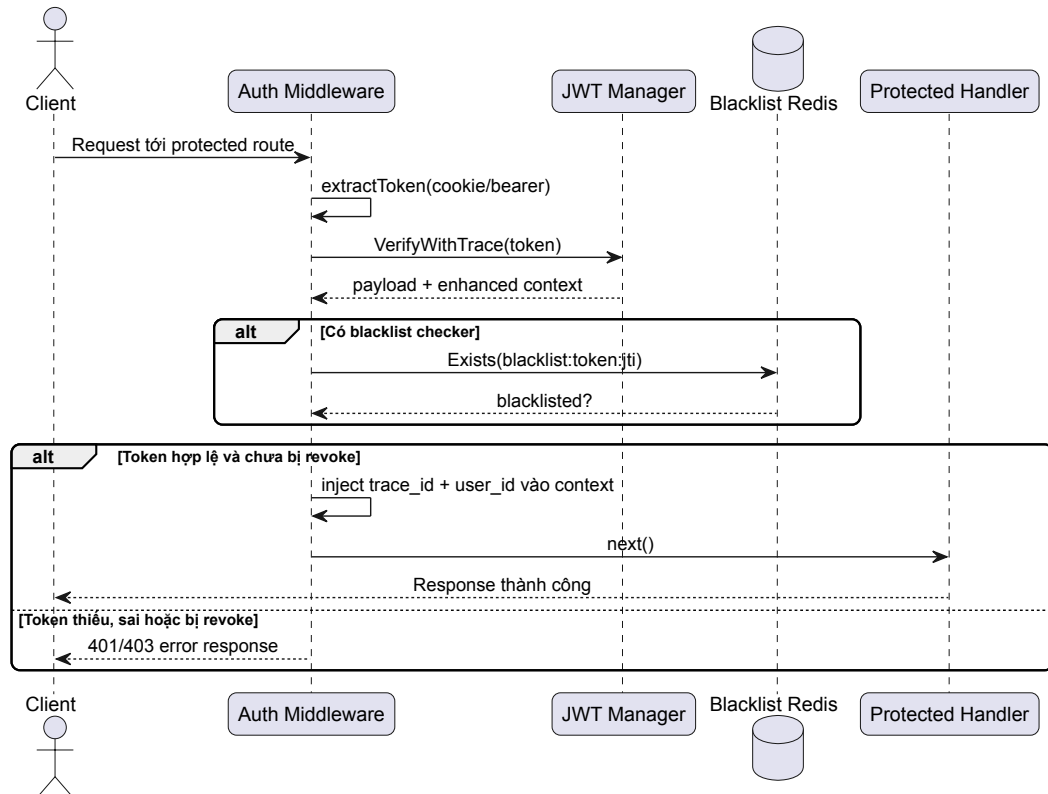
Luồng này mô tả quá trình người dùng bắt đầu đăng nhập, được chuyển sang identity provider và quay lại hệ thống với phiên truy cập hợp lệ:



Hình 14: Luồng OAuth2 login trong Identity Service

b. Protected Route Validation Flow

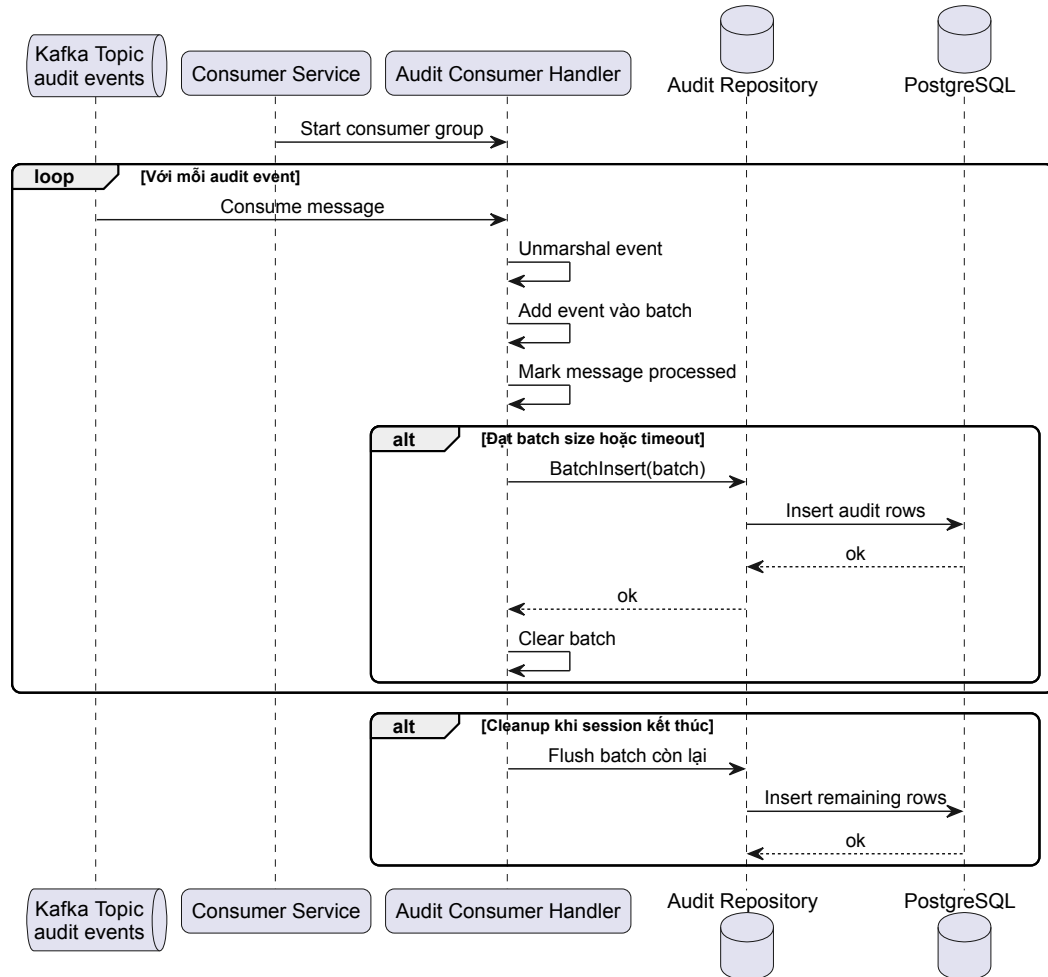
Luồng này mô tả cách token hoặc cookie được kiểm tra trước khi các route bảo vệ cho phép truy cập:



Hình 15: Luồng xác thực protected routes trong Identity Service

c. Audit Event Processing Flow

Luồng này phản ánh consumer lane chạy nền của service:



Hình 16: Luồng audit event processing trong Identity Service

5.3.1.4 Design Patterns áp dụng

Identity Service áp dụng các design patterns sau:

- OAuth2 Delegation Pattern: Tận dụng external identity provider cho bước xác thực ban đầu, giảm việc tự quản lý mật khẩu nội bộ.
- Token-Based Authentication: Dùng JWT làm lớp đại diện cho phiên truy cập sau đăng nhập.
- Session Management Pattern: Dùng session layer để kiểm soát vòng đời truy cập thay vì chỉ phụ thuộc hoàn toàn vào token stateless.
- Background Consumer Pattern: Audit processing được tách khỏi request path và flush theo batch ở runtime nền.
- Repository Pattern: Tách truy cập dữ liệu người dùng và audit log khỏi business logic xác thực.

- Clean Architecture: Giữ ranh giới rõ giữa handler, usecase và infrastructure để tăng tính bảo trì và khả năng kiểm thử.

5.3.1.5 Key Decisions

- Kết hợp OAuth2 với JWT và HttpOnly cookie để vừa tận dụng identity provider bên ngoài, vừa giữ được quyền kiểm soát phiên truy cập ở lớp ứng dụng nội bộ.
- Giữ session management ở usecase/infrastructure layer thay vì nhúng trực tiếp trong handler.
- Tách internal token validation thành một capability riêng để các service khác dùng chung security boundary của hệ thống.
- Giữ audit consumer trong cùng service boundary để auth và audit trail vẫn được quản lý gần nhau, nhưng tách execution của audit khỏi request path chính.

5.3.1.6 Dependencies

Internal Dependencies:

- Authentication UseCase: Điều phối login flow, callback và session completion.
- JWT Manager: Ký và xác thực token.
- Session Manager: Tạo và duy trì session hoặc blacklist state.
- Audit Consumer: tiêu thụ và flush audit events theo batch.
- User / Audit Repository: Lưu user data và audit trail.

External Dependencies:

- OAuth2 Provider: Nguồn xác thực ban đầu của người dùng.
- Redis: Quản lý session/blacklist runtime.
- PostgreSQL: Lưu trữ dữ liệu identity và audit log.
- Kafka: audit event ingress cho consumer lane.
- Backend Services khác: Tiêu thụ internal token validation khi cần.

5.3.2 Analytics Service

Analytics Service là dịch vụ chịu trách nhiệm cho analytics pipeline và analytics read API của hệ thống SMAP. Dịch vụ này được triển khai thành hai runtime chính: **analysis-consumer** tiếp nhận dữ liệu đã được chuẩn hóa từ analytics data plane để xử lý bất đồng bộ, còn **analysis-api** cung cấp các endpoint đọc dữ liệu phân tích cho dashboard và giao diện người dùng.

Vai trò của Analytics Service trong kiến trúc tổng thể:

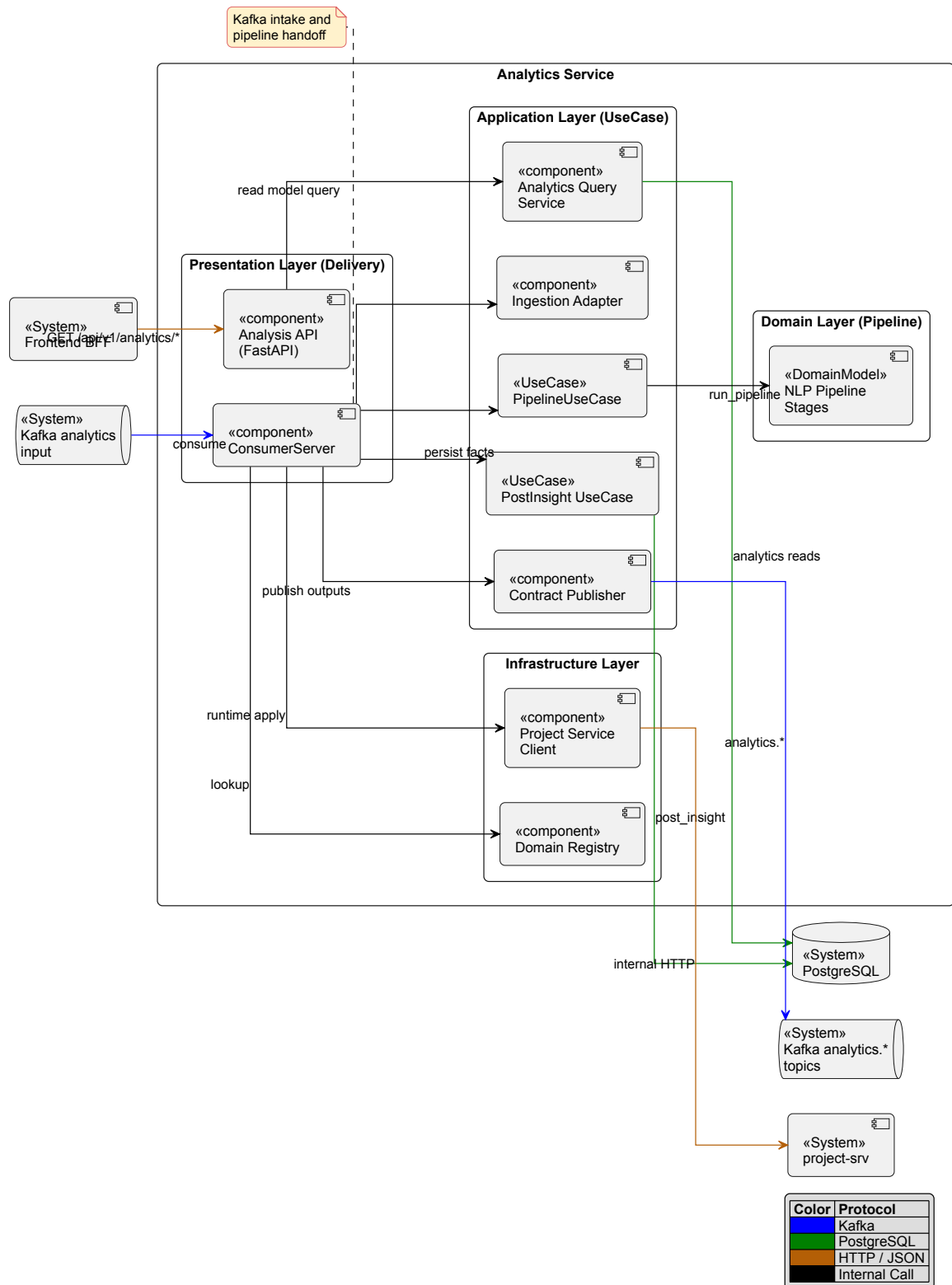
- Analytics API Runtime:Expose các endpoint `/api/v1/analytics/*` cho dashboard, KPI, sentiment, keyword, post và heap view.
- Kafka Consumer Runtime: Tiêu thụ dữ liệu đầu vào từ analytics data plane.
- Pipeline Orchestrator: Điều phối các bước normalization, enrichment và NLP processing.

- Result Persister: Lưu các thực thể phân tích có cấu trúc vào persistence layer.
- Downstream Publisher: Phát hành các topic analytics phục vụ knowledge indexing và các lớp tiêu thụ phía sau.

Service này đáp ứng trực tiếp FR-09 về Analytics Processing. Ở mức use case, service hỗ trợ UC-02 về Vận hành chiến dịch theo dõi ở lane xử lý downstream và cung cấp dữ liệu đầu vào cho UC-03 về Tra cứu và hỏi đáp dữ liệu phân tích.

5.3.2.1 Component Diagram - C4 Level 3

Analytics Service được tổ chức quanh hai bề mặt runtime tách biệt: **analysis-api** phục vụ read path đồng bộ cho giao diện, còn **analysis-consumer** xử lý message bất đồng bộ từ Kafka. Cách tách này giúp dashboard đọc dữ liệu đã persist mà không chặn pipeline NLP hoặc consumer runtime.



Hình 17: Biểu đồ thành phần của Analytics Service

5.3.2.2 Component Catalog

Bảng 25: Component Catalog - Analytics Service

Component	Responsibility	Input / Output	Technology
Analysis API	Expose FastAPI endpoint cho dashboard và các hook analytics của frontend	HTTP request / analytics JSON response	FastAPI + async PostgreSQL
Analytics Query Service	Tổng hợp KPI, platform, sentiment, keyword, post, project stats và heap view từ dữ liệu đã persist	Query params / normalized dashboard dataset	SQLAlchemy async + query layer
ConsumerServer	Tiếp nhận message từ Kafka, parse dữ liệu và điều phối intake flow	Kafka message / parsed input bundle	Python async + Kafka consumer
Ingestion Adapter	Chuyển UAP hoặc ingest flat payload sang <code>IngestedBatchBundle</code> dùng cho pipeline	Parsed record / pipeline-ready bundle	Adapter layer
PipelineUseCase	Wrapper điều phối <code>run_pipeline()</code> với <code>RunContext</code> và cấu hình runtime	Batch bundle / pipeline result	Python usecase layer
NLP Pipeline Stages	Thực thi các bước normalization, dedup, spam, thread topology và NLP enrichment	MentionRecord / NLPFact / analytics outputs	Pipeline modules
PostInsight UseCase	Lưu các kết quả phân tích có cấu trúc vào PostgreSQL	NLPFact / persisted insight rows	Persistence layer
Contract Publisher	Phát hành <code>analytics.*</code> topics cho downstream consumers	Insight outputs / Kafka topics	Kafka producer

5.3.2.3 Data Flow

Luồng xử lý chính của Analytics Service có thể chia thành ba flow: HTTP analytics query flow, Kafka intake flow và pipeline execution flow.

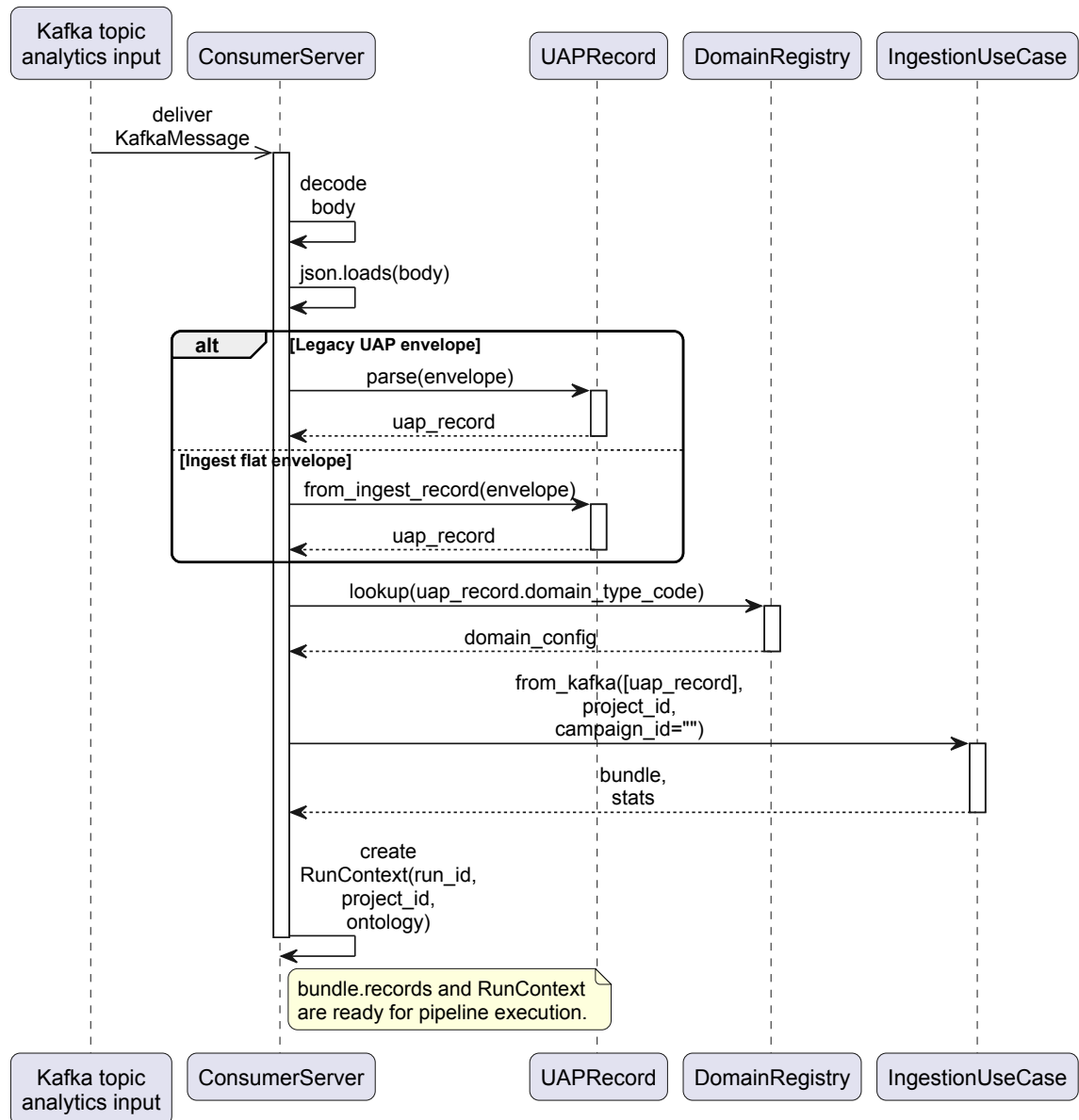
a. HTTP Analytics Query Flow

Luồng này phục vụ các màn hình dashboard và analytics trong frontend:

1. Frontend gọi route same-origin như `/api/analytics/kpis`, `/api/analytics/sentiment`, `/api/analytics/keywords`, `/api/analytics/posts` hoặc `/api/analytics/heap`.
2. Next.js route handler proxy request sang `analysis-api` qua service nội bộ `analysis-api.smap.svc.cluster.local`.
3. `analysis-api` kiểm tra tham số, gọi analytics query service và đọc dữ liệu đã persist trong PostgreSQL.
4. Kết quả được chuẩn hóa thành JSON phù hợp cho KPI card, chart, post table hoặc heap visualization.
5. Nếu truy vấn vượt thời gian cho phép, API trả lỗi timeout rõ ràng để frontend có thể hiển thị trạng thái retry hoặc giảm phạm vi truy vấn.

b. Kafka Intake Flow

Luồng này bắt đầu khi dữ liệu chuẩn hóa được publish vào analytics data plane:



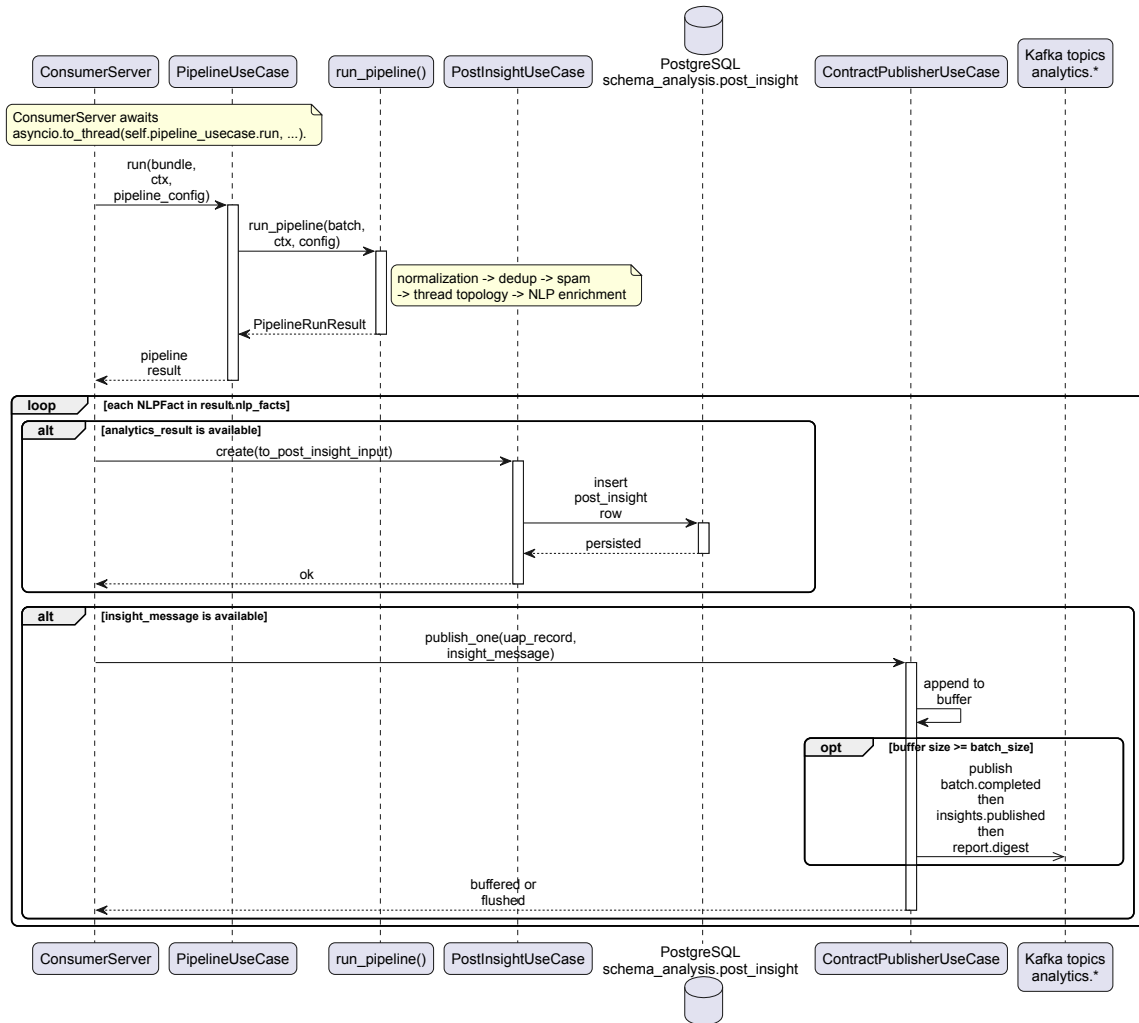
Hình 18: Luồng Kafka intake trong Analytics Service

Ở flow này, ConsumerServer thực hiện các bước chính sau:

1. nhận Kafka message từ topic đầu vào;
2. decode và parse dữ liệu theo định dạng UAP hoặc ingest flat payload;
3. route domain bằng domain_type_code và tạo RunContext;
4. chuyển dữ liệu sang IngestedBatchBundle để sẵn sàng cho pipeline.

c. Pipeline Execution Flow

Sau khi intake hoàn tất, service chuyển sang pipeline execution flow:



Hình 19: Luồng analytics pipeline trong Analytics Service

Ở flow này, pipeline được chạy theo kiểu consumer-based processing:

1. ConsumerServer chạy PipelineUseCase.run() bên trong asyncio.to_thread(...) để tránh block event loop;
2. run_pipeline() thực thi các stage normalization, dedup, spam, thread topology và NLP enrichment;
3. mỗi NLPFact có analytics_result được persist vào PostgreSQL thông qua post_insight layer;
4. mỗi NLPFact có insight_message được đưa vào publish_one(), tại đây message được buffer và auto-flush khi đạt batch_size;
5. sau khi flush, các topic analytics.* được downstream consumers như knowledge-srv tiếp tục tiêu thụ;
6. nếu crisis assessment tạo ra trạng thái mới và runtime auto-apply được bật, analysis-consumer có thể gọi internal API của project-srv để áp dụng crawl mode tương ứng. Đây là runtime bridge riêng, không phải publisher Redis của notification lane.

5.3.2.4 Design Patterns áp dụng

Analytics Service áp dụng các design patterns sau:

- Consumer-Based Processing: Dịch vụ vận hành như một consumer runtime tách khỏi request-response path.
- Query API Pattern: Dữ liệu analytics đã persist được expose qua API đọc riêng cho dashboard thay vì để frontend truy vấn trực tiếp database hoặc consumer runtime.
- Pipeline Pattern: Các bước xử lý được tổ chức thành một pipeline nhiều stage có thứ tự rõ ràng.
- Thread Offloading Pattern: Phần xử lý nặng được đẩy qua `asyncio.to_thread` để giảm nguy cơ block event loop.
- Port and Adapter Pattern: Intake, persistence và publishing được tách qua các adapter hoặc usecase tương ứng.
- Downstream Contract Publishing: Kết quả phân tích không kết thúc trong service mà được phát hành tiếp cho các lane downstream.

5.3.2.5 Key Decisions

- Chọn Kafka làm đầu vào chính cho analytics data plane thay vì RabbitMQ task queue.
- Tách pipeline processing khỏi HTTP path để phù hợp với workload NLP nặng.
- Tách `analysis-api` khỏi `analysis-consumer` để read path của dashboard có lifecycle, probe và scale độc lập với consumer runtime.
- Giữ `PipelineUseCase` như một wrapper mỏng quanh `run_pipeline()` để đơn giản hóa orchestration layer.
- Persist `post_insight` và publish `analytics.*` outputs như hai trách nhiệm liên tiếp nhưng tách biệt.

5.3.2.6 Dependencies

Internal Dependencies:

- ConsumerRegistry: wiring các dependency runtime cho consumer server.
- Analysis API runtime và analytics query service cho read path của dashboard.
- PipelineUseCase: điều phối chạy pipeline.
- IngestionUseCase: chuyển dữ liệu intake sang bundle xử lý.
- PostInsight UseCase: lưu kết quả phân tích.
- Contract Publisher: phát hành các topic downstream.

External Dependencies:

- Kafka: analytics data plane đầu vào và các topic downstream đầu ra.
- PostgreSQL: persistence cho các thực thể phân tích.
- Frontend Application: gọi `analysis-api` thông qua các route `/api/analytics/*` ở BFF layer.

- Project Service: nhận internal runtime apply khi crisis assessment cần đổi crawl mode.
- Knowledge Service: downstream consumer của các analytics topics.
- Runtime config và domain registry: định tuyến ontology theo `domain_type_code`.

5.3.3 Project Service

Project Service là dịch vụ giữ business context của hệ thống SMAP. Dịch vụ này sở hữu campaign, project và crisis configuration, đồng thời quản lý vòng đời nghiệp vụ của project trước khi các lane runtime phía sau được kích hoạt. Đây là service trung tâm của business control plane, nơi các thao tác của người dùng được chuyển thành trạng thái nghiệp vụ rõ ràng trước khi đi vào execution flow.

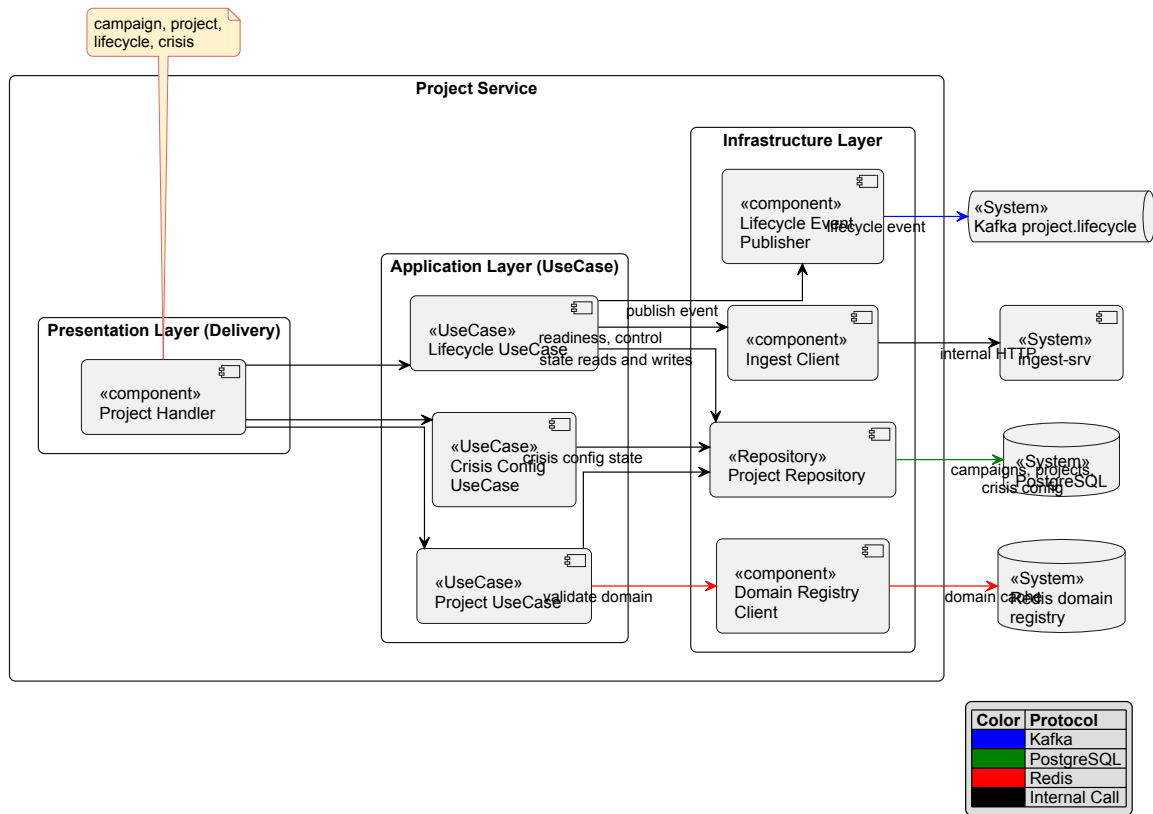
Vai trò của Project Service trong kiến trúc tổng thể:

- Business Context Owner: Quản lý campaign, project và các metadata nghiệp vụ gắn với đối tượng theo dõi.
- Lifecycle Control Layer: Điều khiển activate, pause, resume, archive và unarchive theo vòng đời nghiệp vụ của project.
- Crisis Configuration Owner: Lưu trữ và quản lý các cấu hình giám sát khủng hoảng theo project.
- Internal Control Client: Gọi internal HTTP sang `ingest-srv` để kiểm tra readiness hoặc điều khiển runtime tương ứng.

Service này đáp ứng trực tiếp FR-02 về Campaign and Project Management, FR-03 về Project Lifecycle Control và FR-04 về Crisis Configuration Management. Ở mức use case, nó liên quan trực tiếp đến UC-01 về Thiết lập chiến dịch theo dõi, UC-02 về Vận hành chiến dịch theo dõi và UC-05 về Thiết lập và quản lý quy tắc cảnh báo khủng hoảng.

5.3.3.1 Component Diagram - C4 Level 3

Project Service được tổ chức theo hướng tách biệt rõ delivery, usecase và persistence layer. Trọng tâm của service không nằm ở task runtime mà ở việc giữ trạng thái nghiệp vụ, điều phối lifecycle control và nối business context với các internal services liên quan.



Hình 20: Biểu đồ thành phần của Project Service

5.3.3.2 Component Catalog

Bảng 26: Component Catalog - Project Service

Component	Responsibility	Input / Output	Technology
Project Handler	Xử lý HTTP routes cho project CRUD, lifecycle control và internal detail lookup	HTTP request / JSON response	Gin + HTTP handler
Project UseCase	Điều phối business logic cho project, campaign relation và metadata handling	Create / update input, detail/list output	Pure Go logic
Lifecycle UseCase	Kiểm tra readiness, gọi internal HTTP tới ingest và cập nhật trạng thái project	Project state / lifecycle command	UseCase + internal client

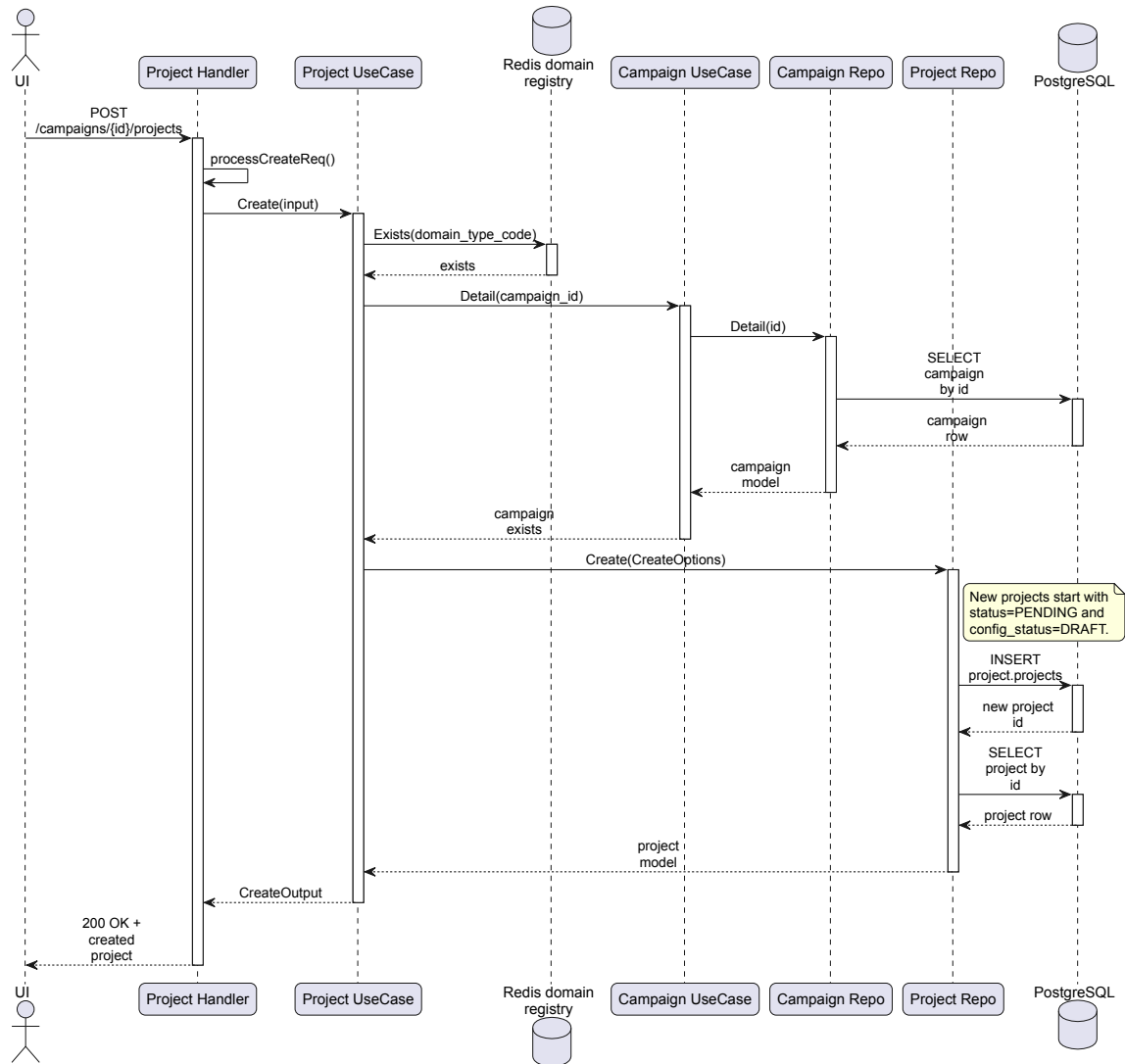
Crisis Config UseCase	Quản lý cấu hình giám sát khủng hoảng gắn với project	Project ID / config payload	Go domain logic
Project Repository	Lưu project, campaign relation và cập nhật trạng thái nghiệp vụ	SQL queries / project rows	PostgreSQL repository
Ingest Client	Giao tiếp internal HTTP với ingest-srv cho readiness và lifecycle control	Project ID / HTTP response	Internal HTTP client
Lifecycle Event Publisher	Phát hành lifecycle event sau khi transition nghiệp vụ thành công	Lifecycle payload / published event	Kafka producer

5.3.3.3 Data Flow

Luồng xử lý chính của Project Service có thể nhìn qua ba flow quan trọng: project creation flow, lifecycle control flow và crisis configuration flow.

a. Project Creation Flow

Luồng này được kích hoạt khi người dùng tạo project mới trong một campaign đã tồn tại:



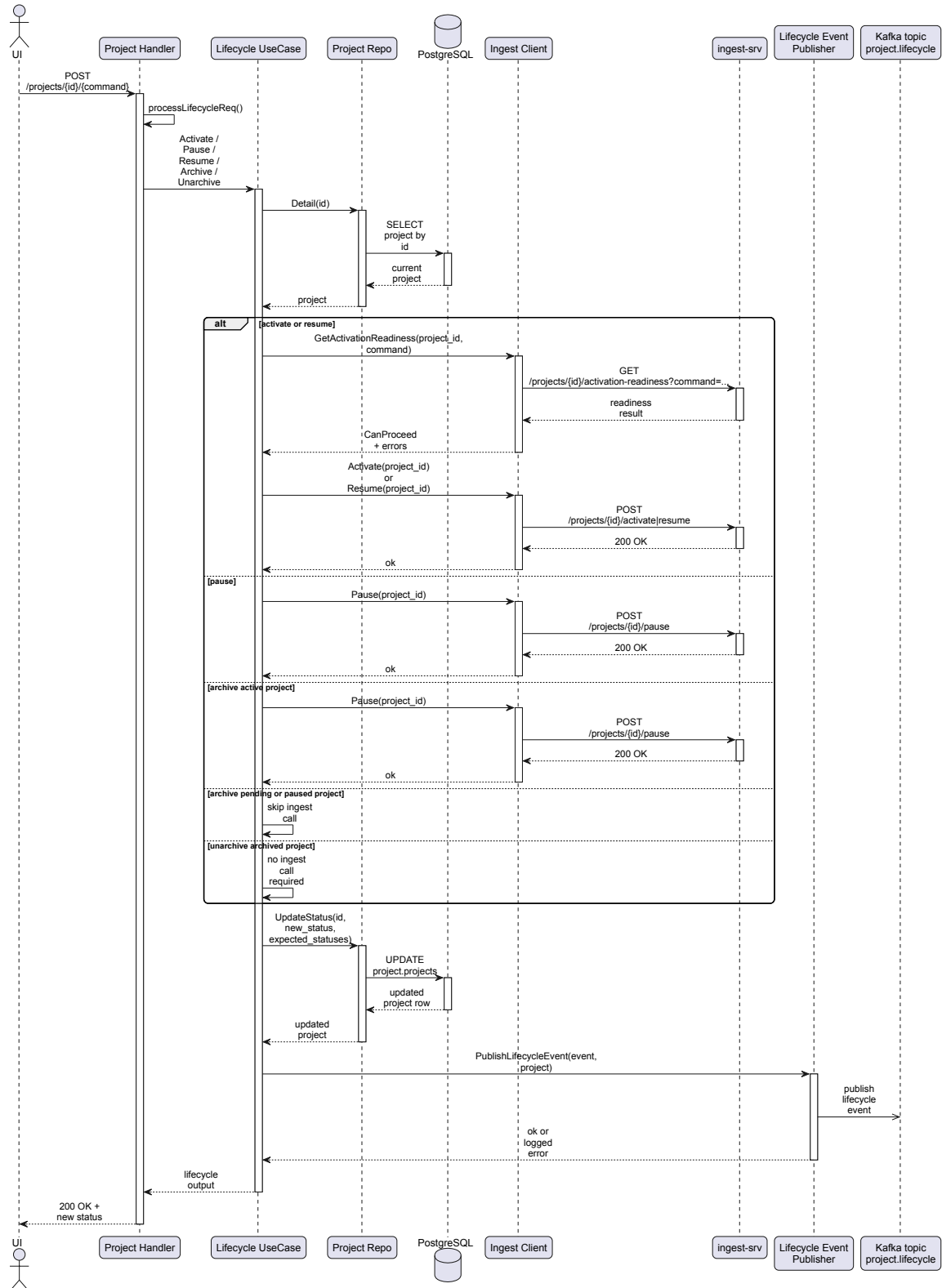
Hình 21: Luồng tạo project trong Project Service

Ở flow này, service thực hiện các bước chính sau:

1. nhận request tạo project trong campaign;
2. kiểm tra dữ liệu đầu vào, `domain_type_code` và campaign context tương ứng;
3. lưu project với business metadata cùng trạng thái khởi tạo **PENDING**;
4. trả về business context đã được tạo cho các bước cấu hình tiếp theo.

b. Project Lifecycle Control Flow

Luồng này được kích hoạt khi người dùng yêu cầu activate, pause, resume hoặc archive project:



Hình 22: Luồng điều phối vòng đời project trong Project Service

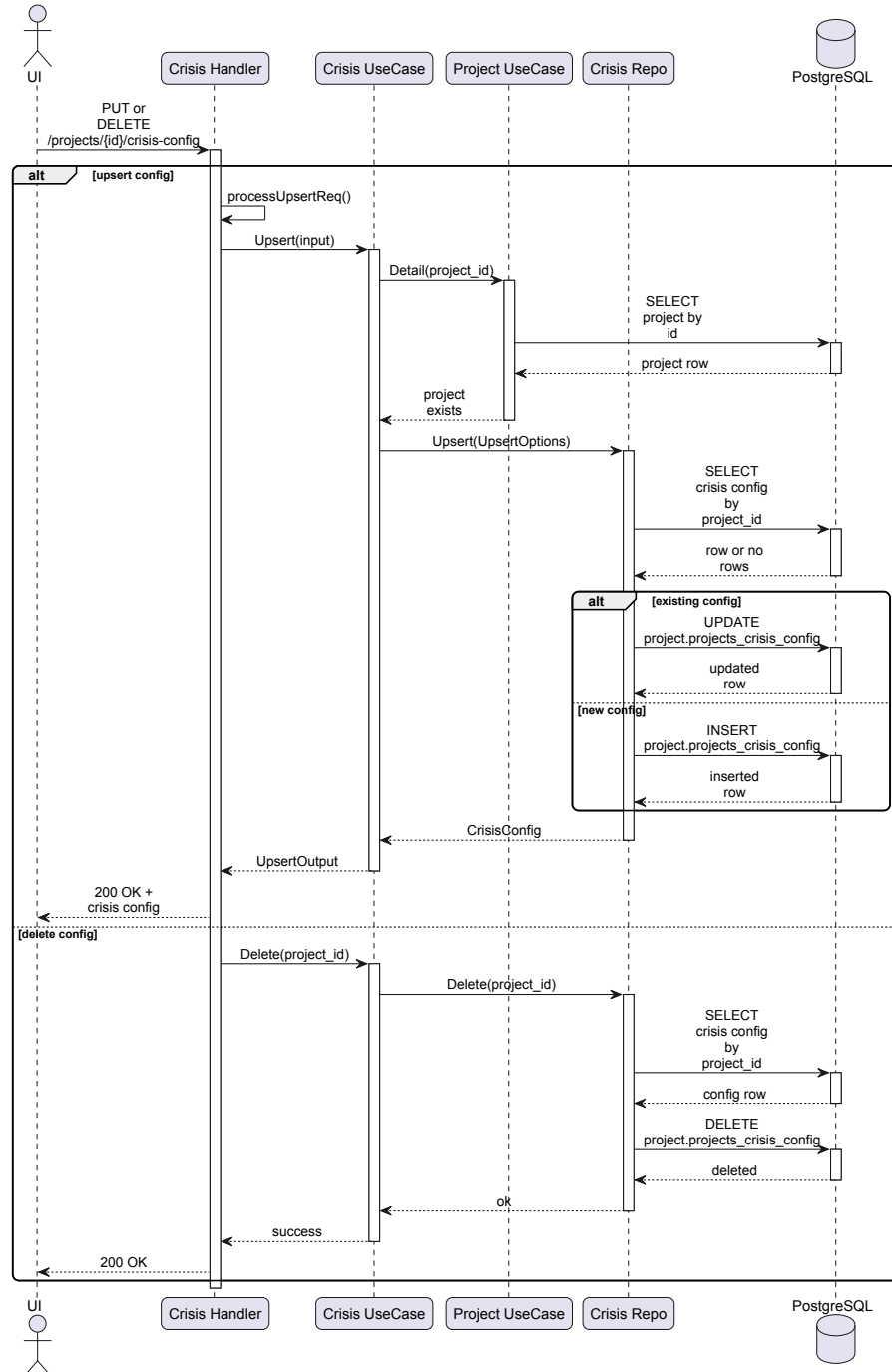
Ở flow này, project-srv không trực tiếp thực thi runtime mà:

1. kiểm tra trạng thái hiện tại của project;
2. gọi internal HTTP sang ingest-srv để lấy activation readiness hoặc gửi lệnh activate, pause, resume tương ứng;

3. cập nhật `project.status` cục bộ nếu transition hợp lệ;
4. phát hành lifecycle event như một lane lan truyền hậu chuyển trạng thái.

c. Crisis Configuration Flow

Luồng này xử lý việc tạo, cập nhật hoặc xóa cấu hình giám sát khủng hoảng gắn với từng project:



Hình 23: Luồng cấu hình giám sát khủng hoảng trong Project Service

- người dùng gửi cấu hình crisis liên quan đến project;
- service kiểm tra project context và tính hợp lệ của payload trước khi upsert;

- `project-srv` lưu, cập nhật hoặc xóa dữ liệu crisis configuration trong persistence layer;
- cấu hình này sau đó trở thành đầu vào cho các lane giám sát tương ứng.

5.3.3.4 Design Patterns áp dụng

Project Service áp dụng các design patterns sau:

- Clean Architecture: giữ ranh giới rõ giữa delivery, usecase và persistence layer.
- Repository Pattern: tách truy cập dữ liệu project và crisis configuration khỏi business logic.
- Control Plane Pattern: lifecycle control được giữ ở lớp nghiệp vụ thay vì bị đẩy sang execution runtime.
- Internal Service Client Pattern: dùng internal HTTP client để điều phối readiness và lifecycle giữa `project-srv` và `ingest-srv`.
- Event Publishing Pattern: phát lifecycle event sau khi business transition thành công để downstream consumers có thể phản ứng tiếp.

5.3.3.5 Key Decisions

- Xem project như business context owner, không phải runtime executor.
- Giữ lifecycle control ở `project-srv`, còn runtime execution state ở `ingest-srv`.
- Dùng internal HTTP cho lifecycle control plane để giữ phản hồi đồng bộ ở các thao tác quan trọng.
- Chỉ phát lifecycle event sau khi trạng thái nghiệp vụ đã được cập nhật thành công.

5.3.3.6 Dependencies

Internal Dependencies:

- Project UseCase: điều phối CRUD và metadata logic.
- Lifecycle UseCase: xử lý readiness và lifecycle transitions.
- Crisis Config UseCase: xử lý cấu hình giám sát khủng hoảng.
- Project Repository: lưu project metadata và trạng thái nghiệp vụ.

External Dependencies:

- PostgreSQL: lưu campaign, project và crisis configuration.
- Redis domain registry: hỗ trợ tra cứu domain context khi cần.
- `ingest-srv`: nhận internal HTTP call cho readiness và lifecycle control.
- Kafka producer: phát lifecycle event cho các consumer downstream.

5.3.4 Ingest Service

Ingest Service là dịch vụ giữ execution plane của hệ thống ở lớp ingest. Dịch vụ này chịu trách nhiệm quản lý datasource, crawl target, dry run, external task, raw batch và quá trình chuẩn hóa dữ liệu đầu vào sang UAP trước khi chuyển tiếp xuống analytics data plane.

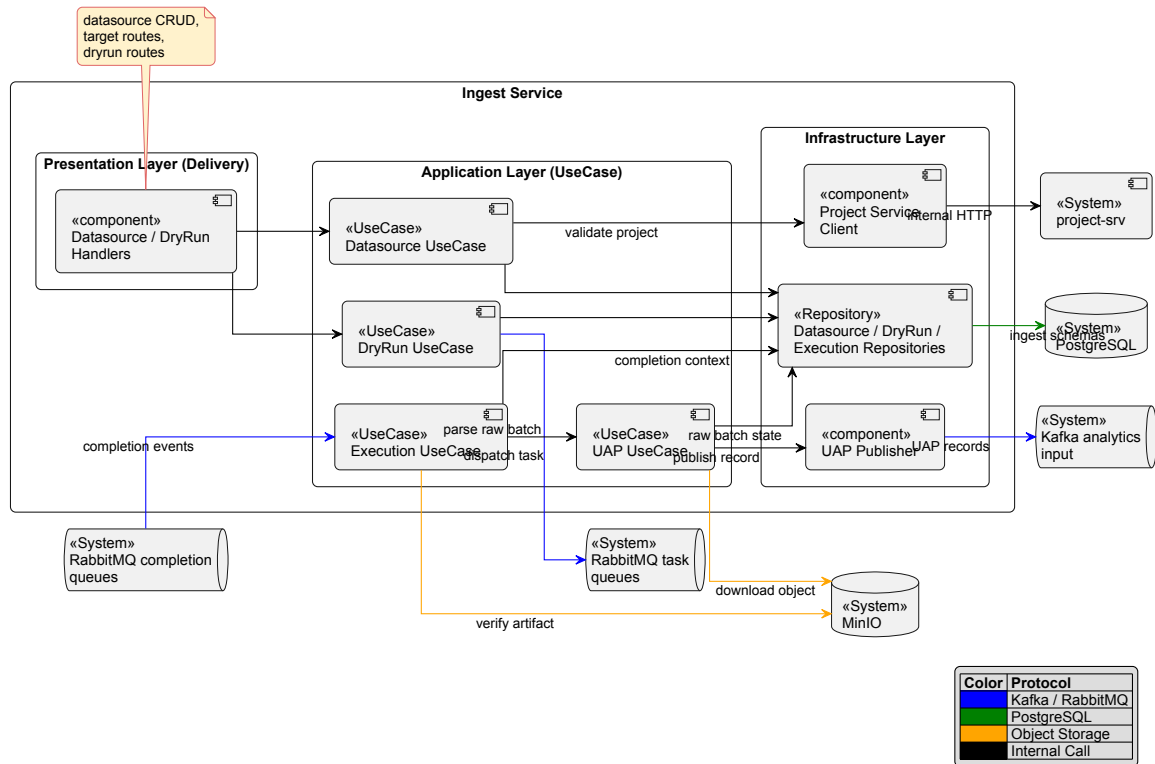
Vai trò của Ingest Service trong kiến trúc tổng thể:

- Datasource Owner: Quản lý metadata và vòng đời của datasource.
- Target Manager: Quản lý crawl target theo keyword, profile và post.
- Dry Run Runtime: Thực hiện kiểm tra readiness trước khi chạy chính thức.
- Execution Plane: Publish task, nhận completion và tạo raw batch lineage.
- UAP Publisher: Chuẩn hóa dữ liệu đầu vào và phát hành sang lane phân tích downstream.

Service này đáp ứng trực tiếp FR-05 về Datasource Management, FR-06 về Crawl Target Management, FR-07 về Dry Run Validation và FR-08 về Crawl Runtime Orchestration. Ở mức use case, nó liên quan trực tiếp đến UC-01 về Thiết lập chiến dịch theo dõi và UC-02 về Vận hành chiến dịch theo dõi.

5.3.4.1 Component Diagram - C4 Level 3

Ở mức thiết kế chi tiết, Ingest Service có thể được nhìn qua năm cụm thành phần chính.



Hình 24: Biểu đồ thành phần của Ingest Service

Bảng 27: Thành phần chính của Ingest Service

Thành phần	Trách nhiệm chính	Input / Output	Technology
------------	-------------------	----------------	------------

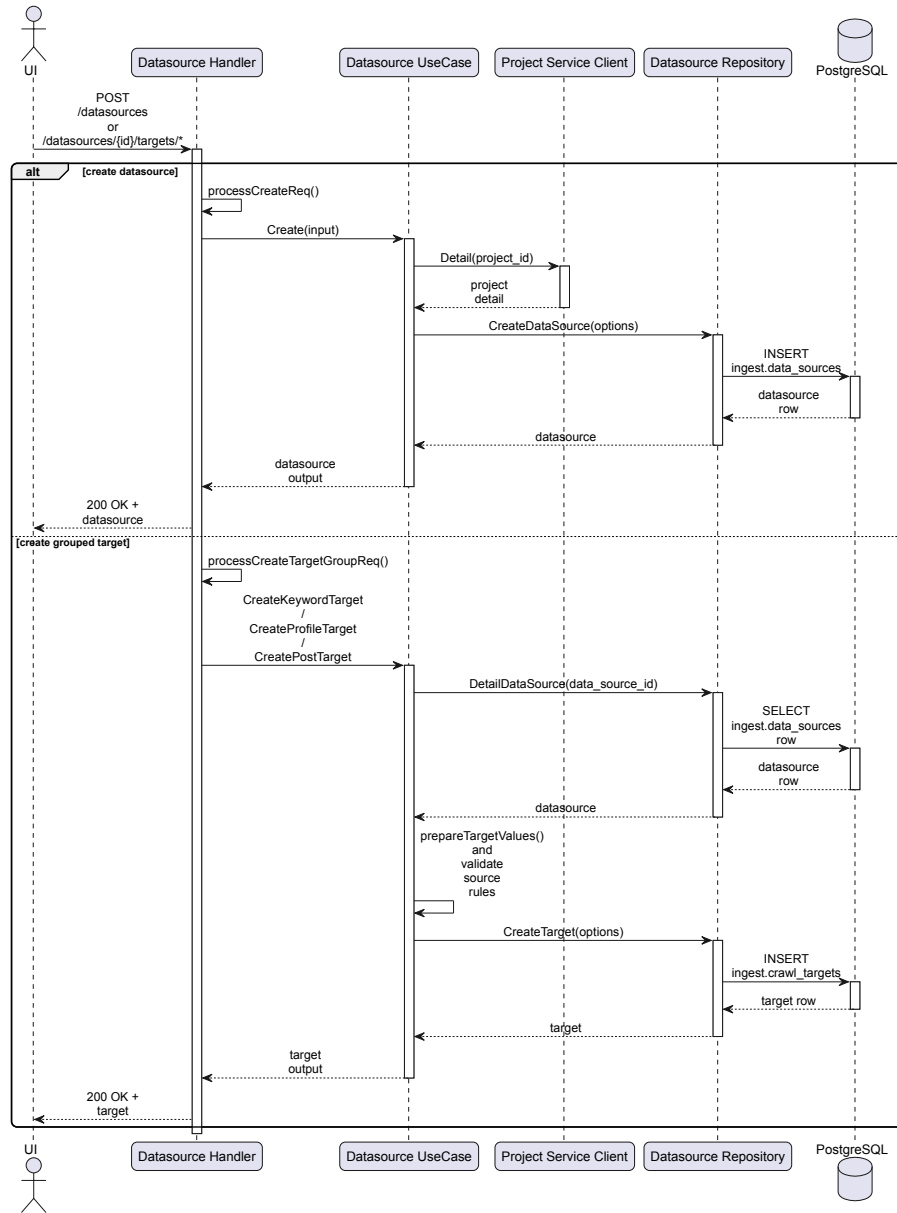
Datasource Handler	Xử lý HTTP routes cho datasource CRUD, target management và các internal lifecycle endpoints	HTTP request / JSON response	Gin + HTTP handler
Datasource UseCase	Điều phối business rules cho datasource, target và project lifecycle runtime	Datasource input / lifecycle output	Pure Go logic
DryRun UseCase	Khởi chạy dry run, ghi nhận kết quả và duy trì thông tin readiness	Dry run request / latest-history output	UseCase + RabbitMQ producer
Execution UseCase	Publish crawl task, nhận completion, kiểm tra object storage và tạo raw batch lineage	Task payload / completion metadata	RabbitMQ + MinIO client
UAP Publisher	Chuẩn hóa raw batch thành UAP và phát hành sang analytics data plane	Raw batch / UAP records	Kafka publishing layer

5.3.4.2 Data Flow

Luồng xử lý chính của Ingest Service có thể chia thành ba flow quan trọng: datasource & target management, dry run validation và completion handling + UAP publishing.

a. Datasource and Target Management Flow

Flow này bắt đầu khi người dùng tạo datasource, thêm crawl target hoặc chỉnh sửa các tham số vận hành liên quan.



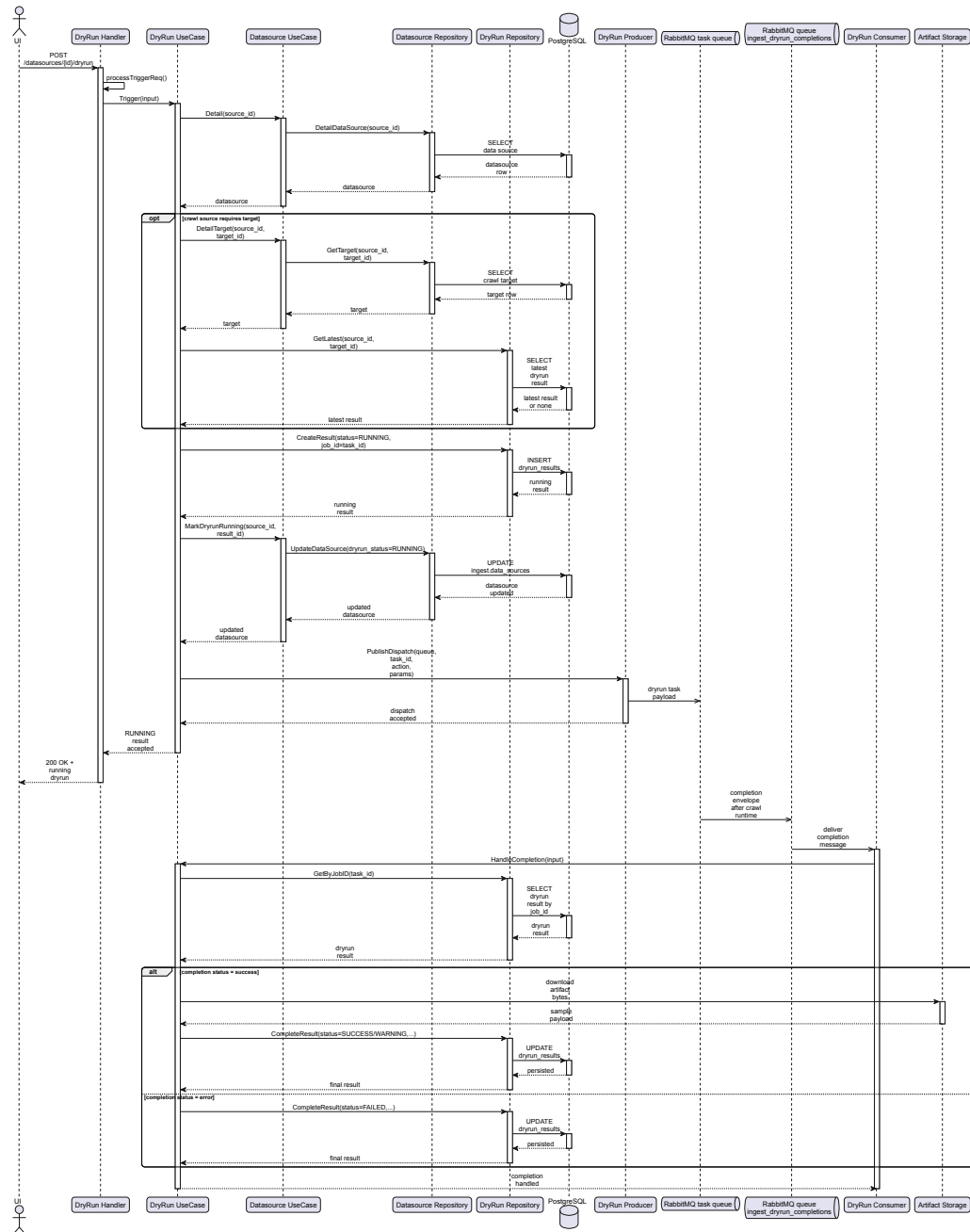
Hình 25: Luồng quản lý datasource và crawl target trong Ingest Service

Ở flow này, service thực hiện các bước chính sau:

1. nhận request tạo hoặc cập nhật datasource;
2. lưu metadata của datasource và target trong persistence layer;
3. áp dụng các business rules liên quan đến trạng thái và loại target;
4. chuẩn bị dữ liệu đầu vào cho dry run hoặc lifecycle control ở các bước tiếp theo.

b. Dry Run Validation Flow

Flow này được kích hoạt khi người dùng yêu cầu kiểm tra readiness trước khi chạy chính thức.



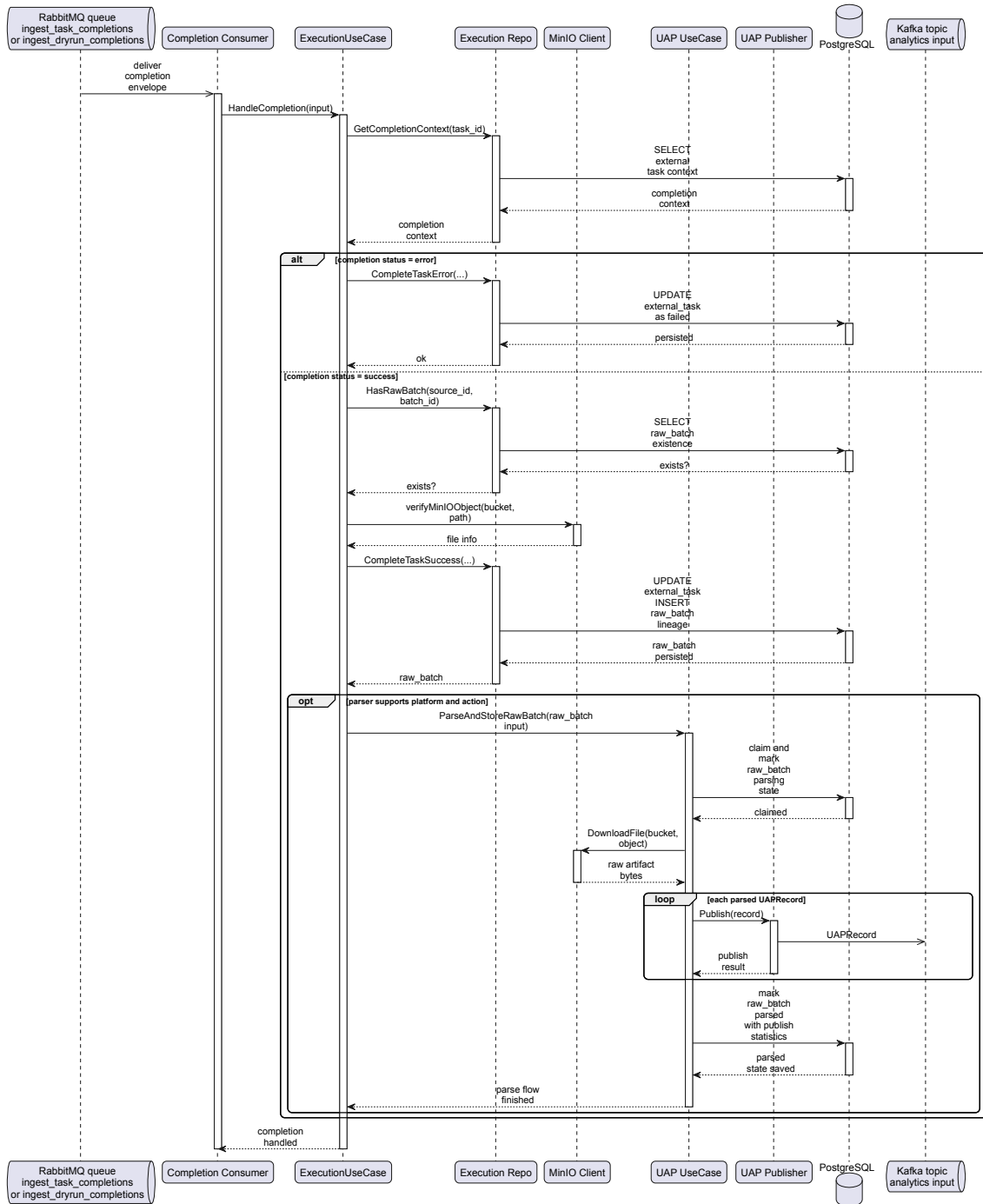
Hình 26: Luồng dry run validation trong Ingest Service

Ở flow này, Ingest Service:

1. tạo một dry run request cho datasource tương ứng;
2. publish task sang crawler runtime nếu cần lấy mẫu hoặc xác nhận dữ liệu;
3. nhận kết quả dry run và ghi vào dryrun_results;
4. duy trì kết quả readiness để dùng cho lifecycle control ở project-srv.

c. Completion Handling and UAP Publishing Flow

Flow này là phần quan trọng nhất của execution plane hiện tại.



Hình 27: Luồng completion handling và UAP publishing trong Ingest Service

1. Ingest Service publish crawl task sang RabbitMQ.
2. scrapper-srv thực thi task và publish completion metadata trở lại.
3. Ingest Service kiểm tra object storage, tạo **external_task** completion record và **raw_batch** lineage.
4. Nếu flow phù hợp, service parse và chuẩn hóa dữ liệu sang UAP.
5. Kết quả được publish sang analytics data plane để **analysis-srv** tiếp tục tiêu thụ.

5.3.4.3 Design Patterns áp dụng

Ingest Service áp dụng các design patterns sau:

- Repository Pattern: Tách truy cập datasource, dryrun, execution và raw batch persistence khỏi business logic.
- Control Plane / Execution Plane Separation: Lifecycle control được giữ rõ ở lớp business/runtime control, trong khi task execution được đẩy sang lane bất đồng bộ.
- Task Queue Pattern: RabbitMQ được dùng cho crawl task dispatch và completion handling.
- Idempotent Completion Handling: `task_id` và lineage metadata được dùng để chống xử lý trùng completion.
- Canonical Publishing Pattern: Sau khi dữ liệu được chuẩn hóa, service phát hành UAP để các lane downstream có thể tiêu thụ nhất quán.

5.3.4.4 Key Decisions

- Giữ ownership của datasource, target, dry run, external task và raw batch trong một service boundary duy nhất.
- Dùng internal HTTP từ `project-srv` sang `ingest-srv` cho readiness và lifecycle control.
- Dùng RabbitMQ cho crawl runtime thay vì ép toàn bộ flow vào synchronous control.
- Chỉ publish sang analytics data plane sau khi completion và raw batch lineage đã được xác nhận ở mức cần thiết.

5.3.4.5 Dependencies

Internal Dependencies:

- Datasource UseCase: quản lý datasource, target và lifecycle runtime.
- DryRun UseCase: quản lý flow dry run và kết quả readiness.
- Execution UseCase: quản lý task dispatch, completion handling và raw batch lineage.
- UAP layer: chuẩn hóa và phát hành dữ liệu downstream.

External Dependencies:

- PostgreSQL: lưu datasource, target, dryrun, scheduled job, external task và raw batch.
- RabbitMQ: task dispatch và completion lane.
- MinIO: object storage cho raw artifacts và completion verification.
- `project-srv`: lifecycle control plane và project context.
- `analysis-srv`: downstream consumer của UAP analytics input.

5.3.5 Notification Service

Notification Service là dịch vụ chịu trách nhiệm cho realtime delivery capability và alert dispatch của hệ thống SMAP. Dịch vụ này tiếp nhận message từ Redis Pub/Sub, định

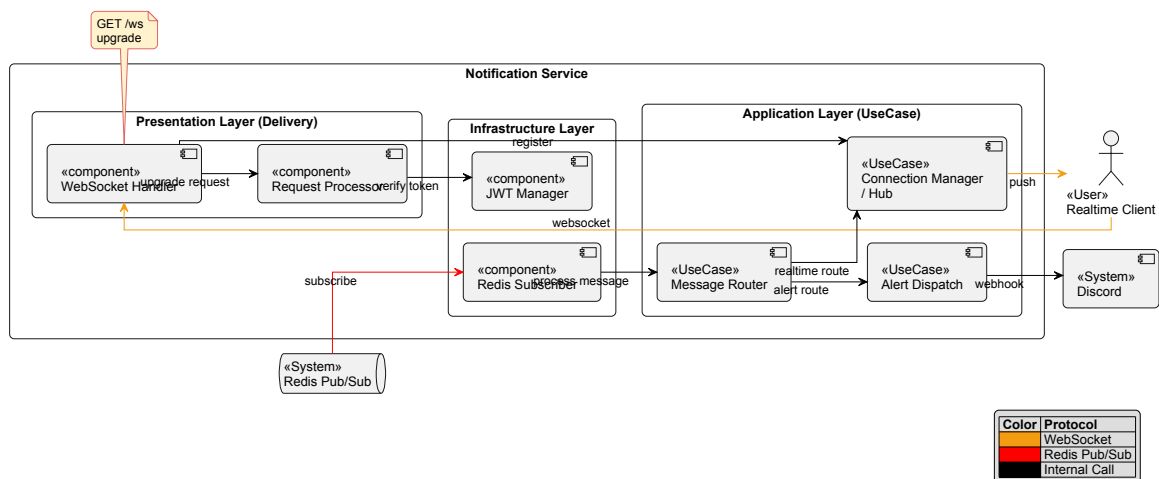
tuyến đến các realtime connections tương thích khi có, và trong các trường hợp phù hợp sẽ xây dựng alert payload để gửi sang các kênh bên ngoài như Discord.

Vai trò của Notification Service trong kiến trúc tổng thể:

- WebSocket Delivery Hub: Quản lý kết nối WebSocket và đẩy message đến realtime clients tương thích theo scope phù hợp.
- Redis Ingress Consumer: Tiếp nhận message từ các backend publishers qua channel patterns.
- Alert Dispatch Layer: Chuyển các message loại alert thành thông điệp giàu ngữ nghĩa cho kênh bên ngoài.
- Connection Lifecycle Manager: Kiểm soát upgrade, register và vòng đời kết nối realtime.

Service này đáp ứng trực tiếp FR-11 về Realtime Notification và liên quan trực tiếp đến UC-04 về Theo dõi trạng thái và nhận cảnh báo.

5.3.5.1 Component Diagram - C4 Level 3



Hình 28: Biểu đồ thành phần của Notification Service

Bảng 28: Thành phần chính của Notification Service

Thành phần	Trách nhiệm chính	Input / Output	Technology
WebSocket Handler	Xử lý HTTP upgrade request và thiết lập kết nối WebSocket	Upgrade request / WebSocket connection	Gin + Gorilla WebSocket

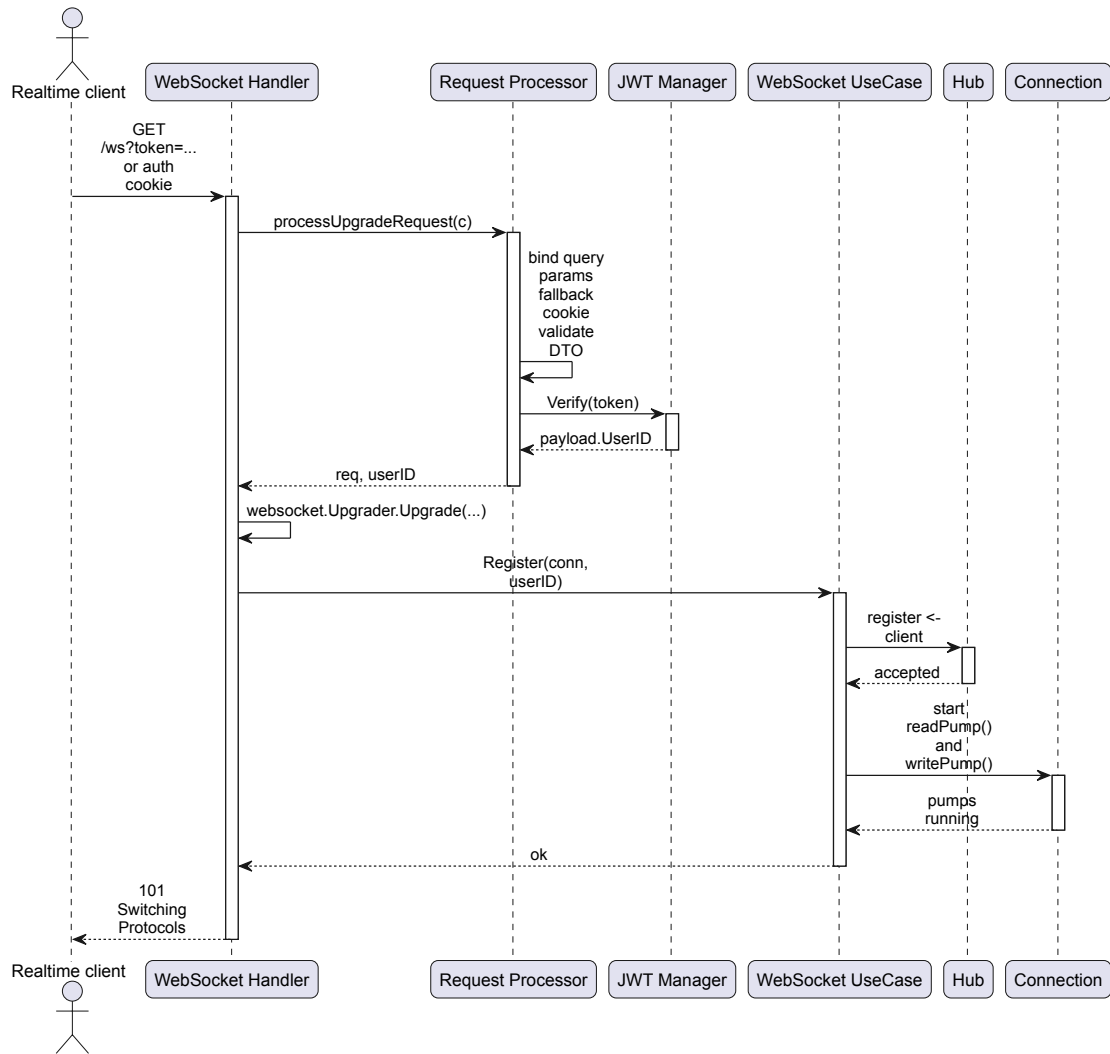
Request Processor	Extract token, fallback cookie, validate request DTO và xác minh JWT trước khi upgrade	Query params / user identity	JWT validation layer
Connection Manager / Hub	Đăng ký và quản lý vòng đời kết nối, route message đến client phù hợp	Connection / routed output	In-memory connection registry
Redis Subscriber	Subscribe các channel pattern và tiếp nhận message từ backend publishers	Channel / payload	Redis Pub/Sub
Message Router	Transform payload và quyết định đẩy về WebSocket hay alert lane tương ứng	Redis message / delivery output	Pure Go logic
Alert Dispatch UseCase	Xây dựng payload cảnh báo và dispatch ra Discord khi cần	Alert input / Discord message	Alert domain logic

5.3.5.2 Data Flow

Notification Service có hai luồng xử lý chính: connection establishment và realtime/alert delivery. Các luồng dưới đây mô tả delivery capability của service khi có client tương thích kết nối vào kênh realtime; chúng không mặc định đồng nghĩa rằng mọi frontend runtime hiện tại đều đã tiêu thụ trực tiếp luồng này.

a. WebSocket Connection Flow

Flow này bắt đầu khi một realtime-capable client mở kết nối GET /ws.



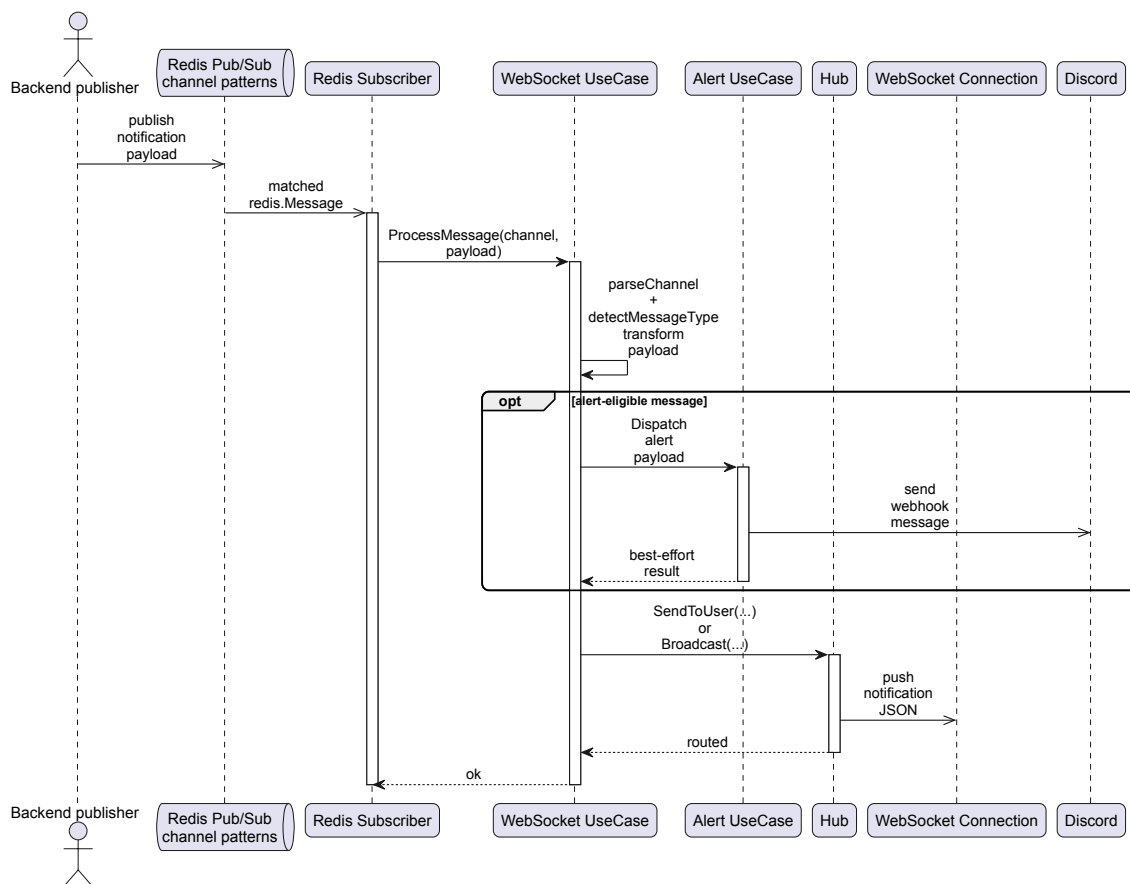
Hình 29: Luồng thiết lập kết nối WebSocket trong Notification Service

Ở flow này, service thực hiện các bước chính sau:

1. nhận request upgrade với token query hoặc auth cookie;
2. validate request DTO và xác minh JWT;
3. upgrade HTTP connection thành WebSocket;
4. đăng ký kết nối vào hub để sẵn sàng nhận message theo scope tương ứng.

b. Realtime and Alert Delivery Flow

Flow này bắt đầu khi backend publisher phát message vào Redis channels.



Hình 30: Luồng realtime và alert delivery trong Notification Service

Ở flow này, service thực hiện các bước chính sau:

1. Redis subscriber nhận message từ channel pattern phù hợp;
2. message được parse, phân loại và route theo loại sự kiện;
3. nếu là realtime update, message được đẩy đến các WebSocket connections phù hợp đang hoạt động;
4. nếu là alert, service có thể dựng payload và dispatch sang Discord hoặc kênh tương ứng.

5.3.5.3 Design Patterns áp dụng

Notification Service áp dụng các design patterns sau:

- Publish/Subscribe Ingress: nhận dữ liệu từ nhiều backend publishers qua Redis channel patterns.
- Hub / Connection Registry Pattern: quản lý các kết nối đang sống và route message theo scope.
- Delivery Routing Pattern: tách việc nhận message khỏi quyết định message đó đi về WebSocket hay alert lane khác.
- Alert Formatting Pattern: xây dựng message giàu ngữ nghĩa trước khi gửi ra kênh bên ngoài như Discord.

5.3.5.4 Key Decisions

- Dùng Redis Pub/Sub làm notification ingress thay vì để backend services nói chuyện trực tiếp với từng WebSocket connection.
- Xác thực ở bước upgrade để giữ security boundary rõ ràng cho realtime connection.
- Tách riêng alert dispatch khỏi connection management để service hỗ trợ được nhiều loại delivery semantics trong cùng một boundary.

5.3.5.5 Dependencies

Internal Dependencies:

- WebSocket handler và request processor cho connection flow.
- Connection manager / hub cho lifecycle của WebSocket connections.
- Message routing logic cho realtime updates và alerts.
- Alert usecase cho Discord dispatch.

External Dependencies:

- Redis Pub/Sub: nguồn message đầu vào từ backend publishers.
- Discord webhook/client: kênh nhận cảnh báo bên ngoài.
- Realtime-capable clients: bên tiêu thụ WebSocket messages khi có kết nối tương thích.
- Identity/JWT layer: nguồn xác minh token trước khi upgrade connection.

5.3.6 Knowledge Service

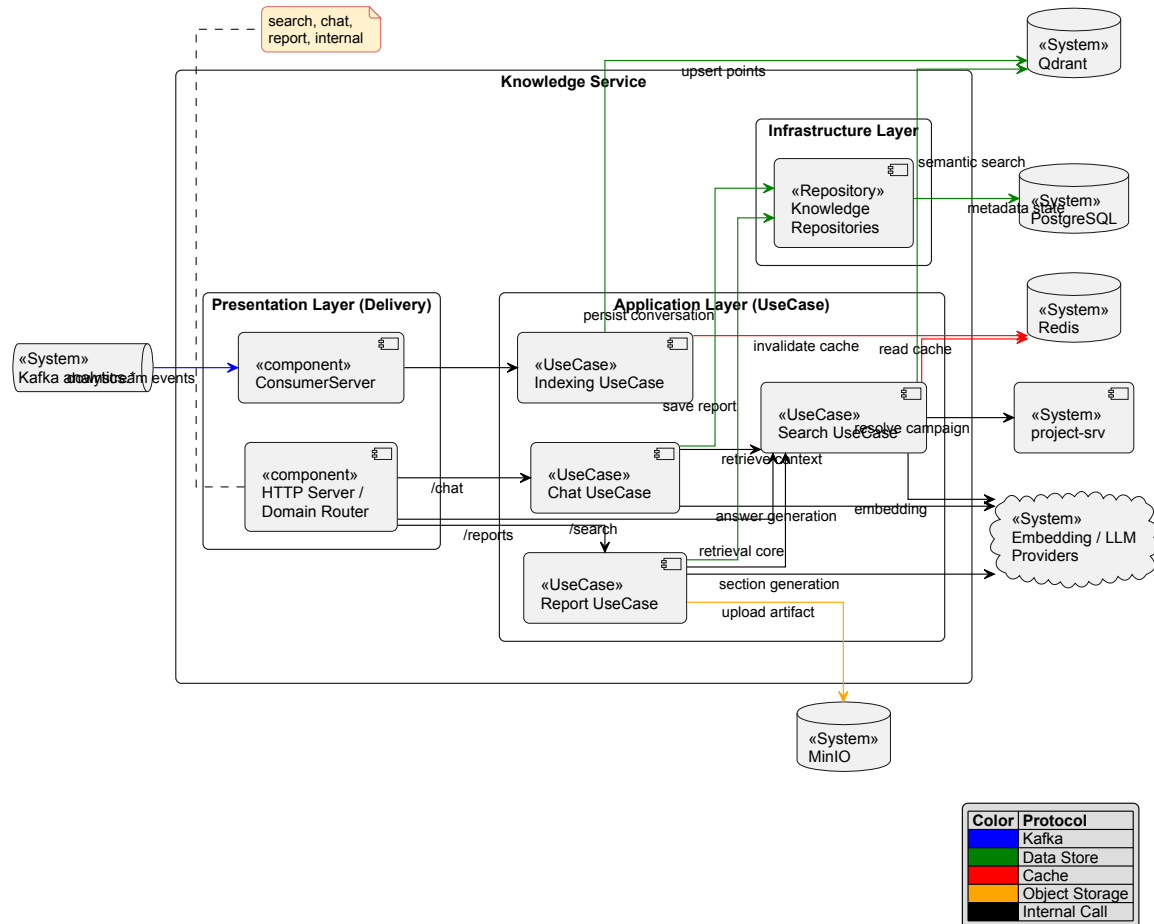
Knowledge Service là dịch vụ chịu trách nhiệm cho knowledge retrieval và knowledge materialization layer của hệ thống SMAP. Khác với các service chỉ thuần HTTP hoặc chỉ thuần consumer runtime, service này kết hợp cả lane indexing downstream lẫn lane consumption đồng bộ để biến analytics outputs thành tri thức có thể tìm kiếm, hỏi đáp và tái sử dụng. Ngoài bộ capability cốt lõi gắn với Chương 4, service còn có một nhánh report generation mở rộng ở phía knowledge layer.

Vai trò của Knowledge Service trong kiến trúc tổng thể:

- Downstream Indexing Runtime: Tiêu thụ analytics outputs và chuyển chúng thành vector points cùng tracking metadata.
- Semantic Retrieval Layer: Thực hiện semantic search trên các collection theo project với filtering và caching.
- RAG Chat Orchestrator: Kết hợp search results, LLM generation, citations và conversation history để tạo câu trả lời theo ngữ cảnh.
- Extended Report Materialization Layer: Tổng hợp kết quả truy hồi, sinh các section bằng LLM và materialize artifact báo cáo.
- Knowledge Metadata Manager: Lưu conversations, messages, reports và indexing status phục vụ truy vết và tái sử dụng.

Service này đáp ứng trực tiếp FR-10 về Knowledge Search and Chat, đồng thời liên quan trực tiếp đến UC-03 về Tra cứu và hỏi đáp dữ liệu phân tích. Capability report generation của service được xem như phần mở rộng ngoài bộ FR/UC cốt lõi của Chương 4.

5.3.6.1 Component Diagram - C4 Level 3



Hình 31: Biểu đồ thành phần của Knowledge Service

Bảng 29: Thành phần chính của Knowledge Service

Thành phần	Trách nhiệm chính	Input / Output	Technology
HTTP Server / Domain Router	Công bố các route đã xác thực cho search, chat, reports (capability mở rộng) và các route nội bộ cho indexing control	HTTP request / routed usecase	Gin + middleware

ConsumerServer	Điều phối Kafka consumer runtime để intake các topic analytics downstream	Kafka message / indexing input	Sarama consumer runtime
Indexing UseCase	Sinh embedding, ensure collection, upsert vào Qdrant, cập nhật tracking status và hỗ trợ retry hoặc reconcile	Analytics payload / indexed points + tracking result	Go usecase + Qdrant + PostgreSQL
Search UseCase	Resolve campaign sang project scope, cache lookup, embed query, search nhiều collection và aggregate kết quả	Search input / search results	Redis + Qdrant + embedding layer
Chat UseCase	Phân loại intent, kết hợp retrieval với LLM, sinh citations và persist conversation history	Chat request / contextual answer	LLM orchestration + conversation store
Report UseCase (Capability mở rộng)	Deduplicate yêu cầu, chạy background generation, tạo markdown artifact và upload sang object storage	Report request / report artifact + metadata	LLM + MinIO + PostgreSQL

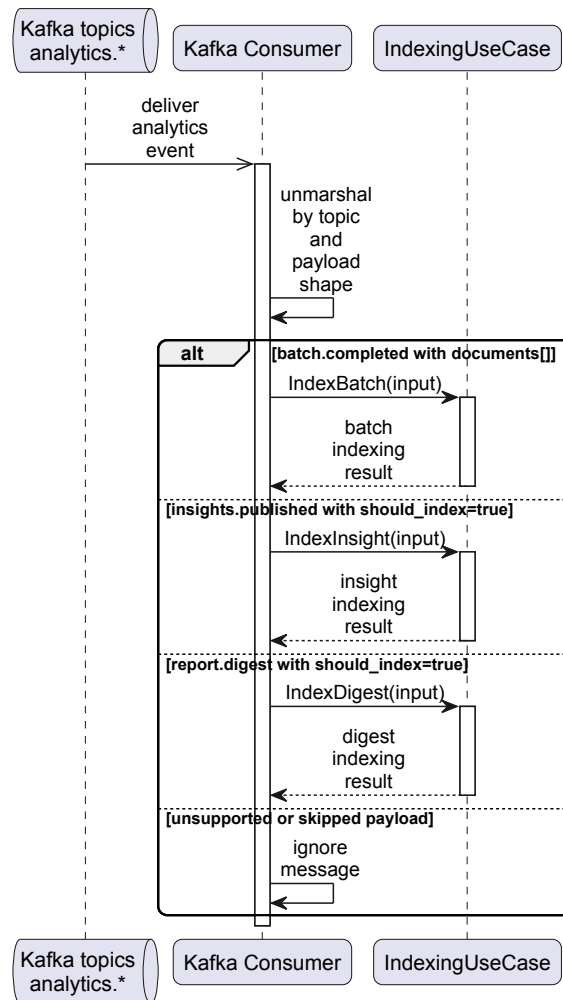
5.3.6.2 Data Flow

Knowledge Service có năm luồng xử lý đáng chú ý: Kafka downstream indexing flow, internal indexing control flow, contextual search flow, contextual chat flow, và report generation capability flow. Trong đó, hai luồng đầu tiên gắn với indexing runtime; search và chat gắn trực tiếp với phạm vi yêu cầu cốt lõi của Chương 4; còn luồng report phản ánh một capability mở rộng của knowledge layer nằm ngoài phạm vi này.

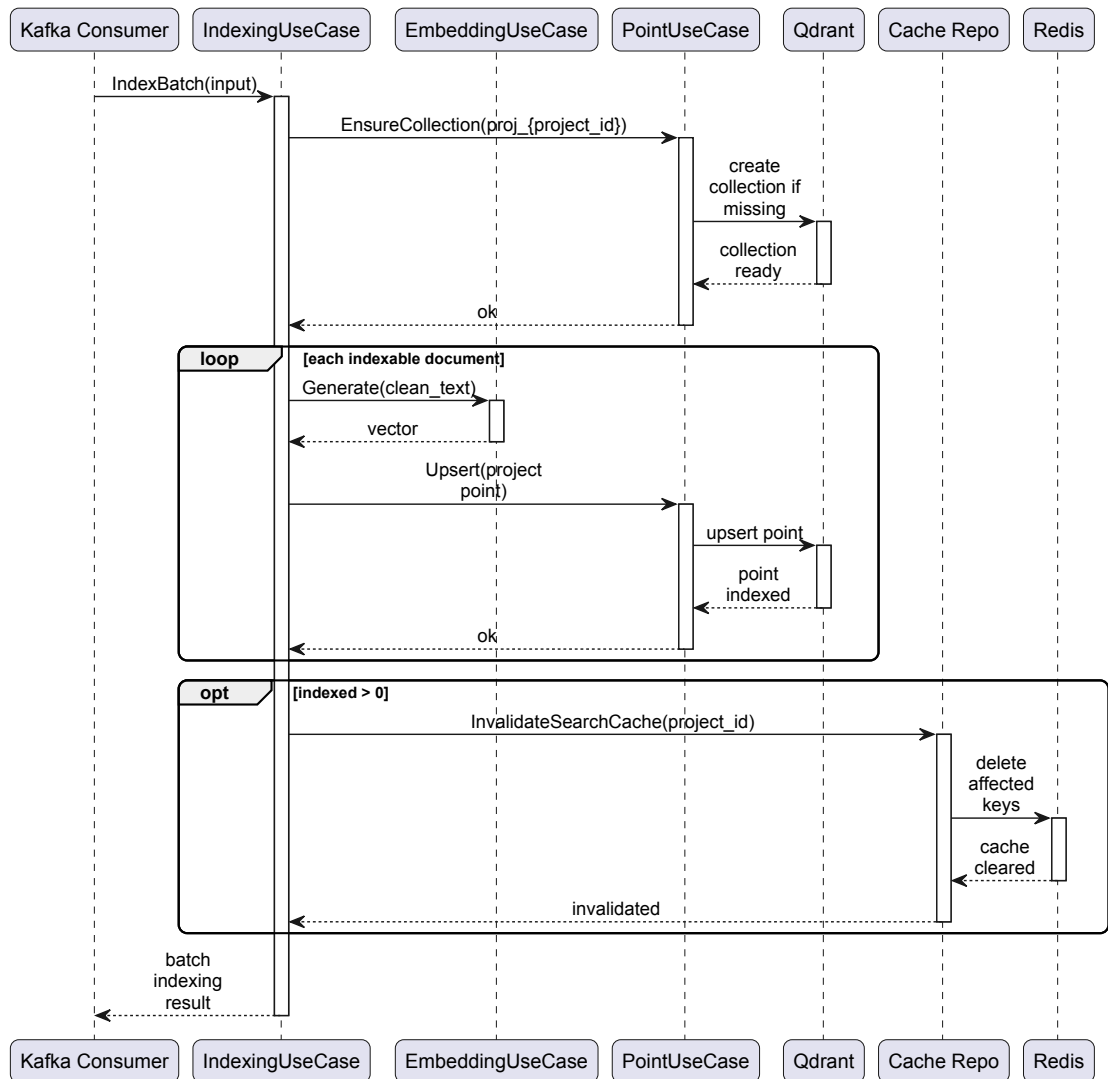
a. Kafka Downstream Indexing Flow

Luồng này bắt đầu khi dữ liệu analytics downstream được phát hành.

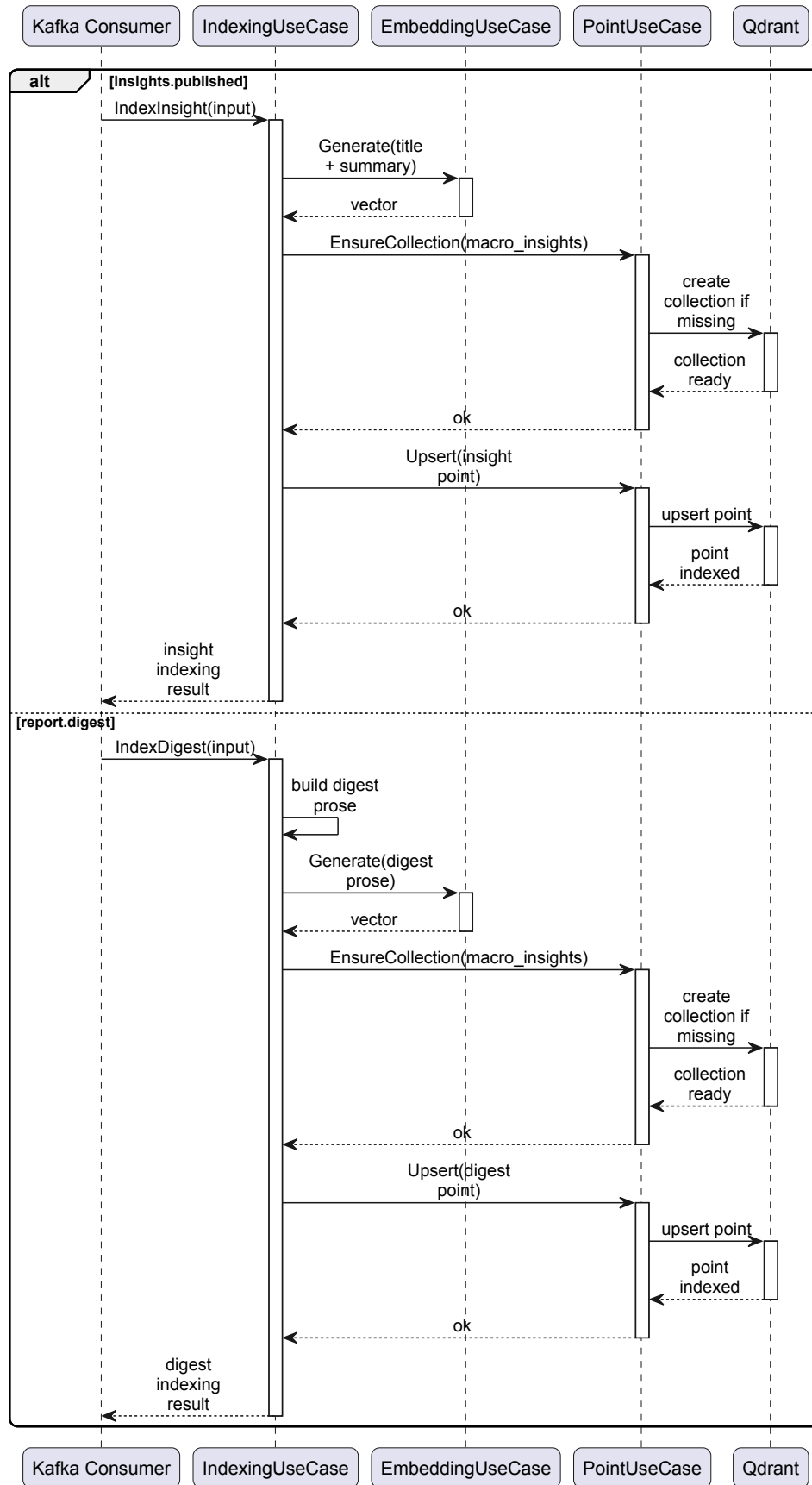
Để dễ đọc hơn, flow này được tách thành ba phase nhỏ.



Hình 32: Pha 1 - Intake và route analytics event vào indexing usecase



Hình 33: Pha 2 - Index batch documents theo project collection



Hình 34: Pha 3 - Index insight và digest vào macro collection

1. Consumer runtime nhận message từ các topic như `analytics.batch.completed`, `analytics.insights.published` hoặc `analytics.report.digest`.
2. Handler route message sang usecase indexing phù hợp như `IndexBatch`, `IndexInsight` hoặc `IndexDigest`.
3. Indexing layer sinh embedding, ensure collection như `proj_{project_id}` hoặc `macro_insights`, rồi upsert points vào Qdrant.
4. Tracking metadata và trạng thái **PENDING**, **INDEXED** hoặc **FAILED** được cập nhật trong PostgreSQL; các statistics hoặc retry path tiếp tục dựa trên lớp metadata này.
5. Search cache liên quan bị invalidate sau khi index thành công để các truy vấn sau nhìn thấy dữ liệu mới.

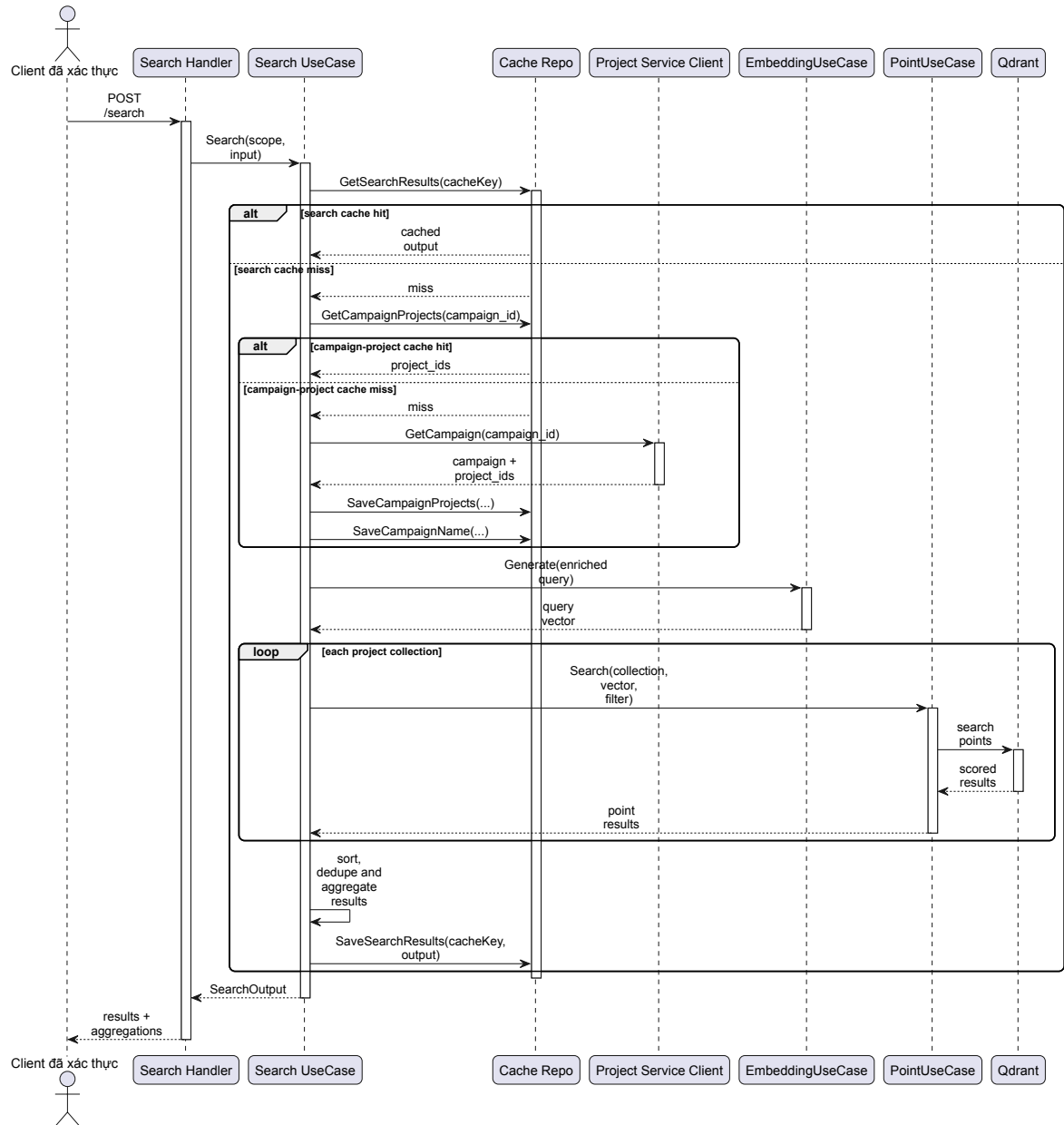
b. Internal Indexing Control Flow

Luồng này phản ánh lớp control routes nội bộ của service, tách biệt với Kafka indexing lane.

1. Internal caller gọi các route như `/internal/index`, `/internal/index/retry`, `/internal/index/reconcile` hoặc `/internal/index/statistics/:project_id`.
2. HTTP layer xác thực bằng internal auth middleware và bind request tương ứng.
3. Service thực thi usecase điều khiển như index từ MinIO file URL, retry failed records, reconcile pending records hoặc truy vấn thống kê indexing.
4. Kết quả được trả về như control hoặc monitoring output cho runtime caller, thay vì như search result hướng người dùng cuối.

c. Contextual Search Flow

Luồng này bắt đầu khi client đã xác thực gửi yêu cầu `search`.

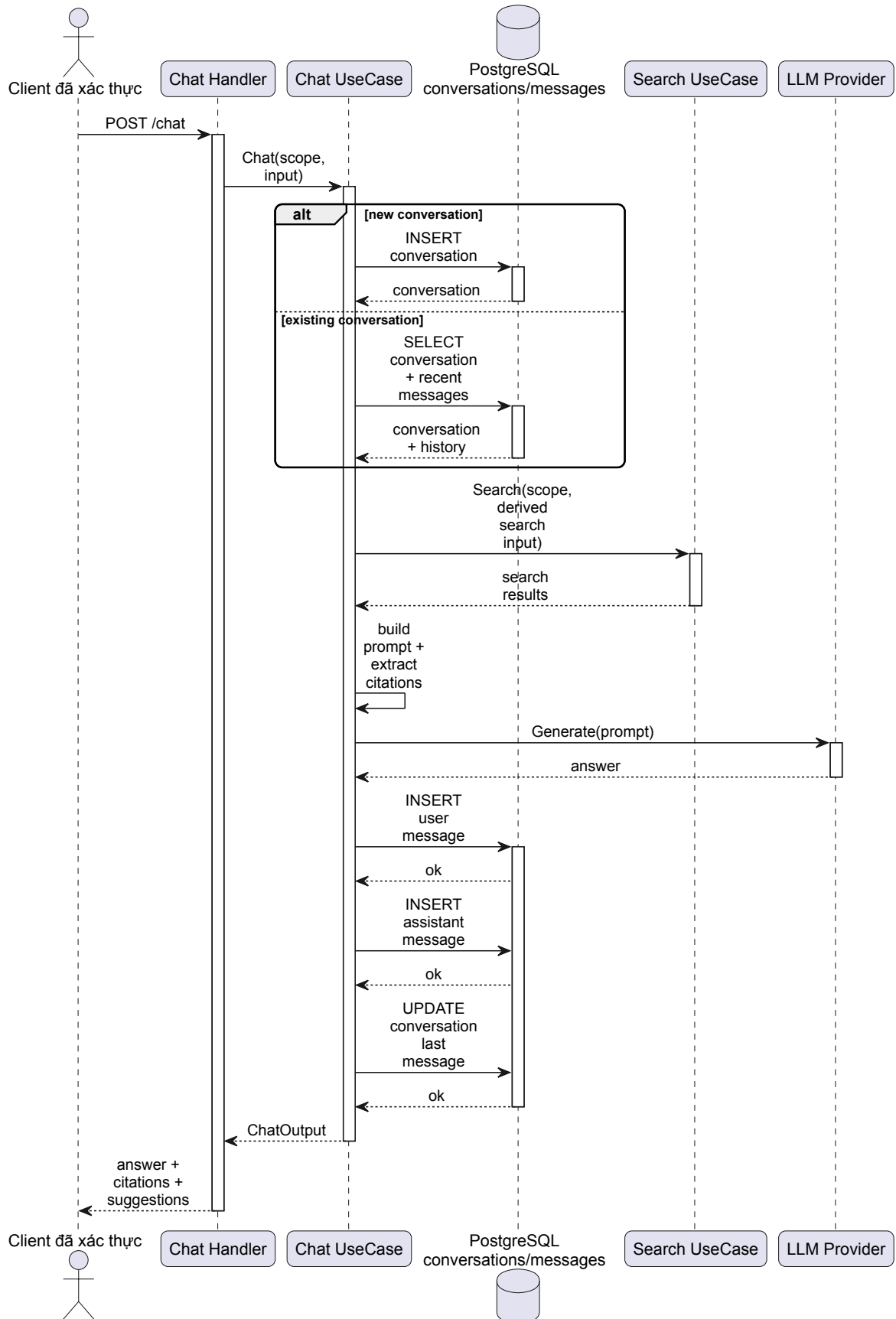


Hình 35: Luồng contextual search trong Knowledge Service

1. Client gọi route `/search` thông qua HTTP API đã được bảo vệ bởi auth middleware.
2. Search usecase kiểm tra search cache, rồi resolve `campaign_id` sang `project_ids` qua cache hoặc `project-srv`.
3. Truy vấn được enrich theo campaign context và chuyển sang embedding vector.
4. Service thực hiện parallel search trên các Qdrant collections phù hợp, lọc theo score, dedupe snapshot lặp và aggregate kết quả.
5. Kết quả được cache lại trước khi trả về cho client.

d. Contextual Chat Flow

Luồng này bắt đầu khi client đã xác thực gửi yêu cầu chat.



Hình 36: Luồng contextual chat trong Knowledge Service

1. Client gọi route `/chat` với `campaign_id`, `message` và `conversation_id` nếu đang tiếp tục hội thoại cũ.
2. Chat usecase tạo conversation mới hoặc nạp conversation cùng lịch sử message gần nhất.
3. Chat usecase gọi lại search layer để lấy context phù hợp cho câu hỏi hiện tại.
4. Search results, conversation history và input hiện tại được ghép thành prompt để gọi LLM.
5. Câu trả lời, citations, suggestions và conversation or message history được persist để phục vụ các lượt tương tác tiếp theo.

e. Report Generation Capability Flow

Luồng này phản ánh capability mở rộng của knowledge layer.

1. Client gọi route `/reports/generate` với campaign scope và các filter tương ứng.
2. Report usecase validate input, hash tham số để deduplicate, tạo report record và khởi động background generation.
3. Background pipeline gọi search usecase để aggregate tài liệu liên quan, chọn sample đại diện và sinh từng section bằng LLM với concurrency có kiểm soát.
4. Markdown artifact được compile và upload lên MinIO như một object báo cáo.
5. Report status, file metadata và thời gian sinh được cập nhật trong PostgreSQL; client có thể truy vấn trạng thái hoặc lấy presigned download URL sau đó.

5.3.6.3 Design Patterns áp dụng

Knowledge Service áp dụng các design patterns sau:

- Dual Runtime Pattern: cùng một bounded context chứa cả HTTP APIs cho consumption và Kafka consumer lane cho indexing downstream.
- Collection-per-Project Vector Indexing: dữ liệu post được index vào collection theo project, trong khi macro insights và digest dùng collection chia sẻ theo loại tri thức.
- Cache-Aside Retrieval Pattern: kết quả search và campaign-to-project resolution được cache để giảm latency cho các truy vấn lặp lại.
- RAG Orchestration Pattern: chat layer không gọi LLM trực tiếp trên dữ liệu thô mà luôn đi qua retrieval results, citations và conversation history.
- Async Materialization Pattern: report generation được tách khỏi request path và materialize artifact xuống object storage ở capability mở rộng của knowledge layer.
- Internal Control Route Pattern: các route nội bộ cho index, retry, reconcile và statistics được tách biệt khỏi public knowledge APIs và khỏi Kafka indexing lane.

5.3.6.4 Key Decisions

- Tách vector store Qdrant khỏi metadata store PostgreSQL để tối ưu riêng cho retrieval và operational tracking.

- Giữ Search UseCase làm capability trung tâm được tái sử dụng bởi chat và bởi capability report mở rộng thay vì để từng domain tự truy hỏi riêng.
- Scope truy vấn theo campaign -> project collections để tránh cross-campaign leakage và giữ knowledge boundary rõ ràng.
- Dùng background generation kết hợp MinIO artifact cho report để tránh block request path trong workload LLM.
- Giữ Kafka lane cho downstream indexing, còn retry, reconcile và statistics đi qua internal control routes để không trộn streaming semantics với operational control.
- Giữ indexing retry và reconcile trong cùng service boundary để việc vận hành knowledge lane không phải tách sang một runtime control riêng.

5.3.6.5 Dependencies

Internal Dependencies:

- HTTP server wiring cho các domain indexing, search, chat và report.
- Indexing usecase, point layer, embedding layer và tracking repositories.
- Search usecase được chat và report usecases tái sử dụng như retrieval core.
- Conversation, message, report và indexing repositories trên PostgreSQL.

External Dependencies:

- Kafka topics downstream từ analysis-srv.
- Qdrant cho vector indexing và semantic retrieval.
- Redis cho caching.
- PostgreSQL cho conversations, reports và indexed document tracking.
- MinIO cho report artifacts và một số indexing inputs theo file URL.
- project-srv cho campaign/project resolution.
- Embedding và LLM providers cho vector generation, answer synthesis và report generation.

5.3.7 Scrapper Worker Service

Scrapper Worker Service là runtime lane chịu trách nhiệm thực thi các crawl task bắt đồng bộ trong execution plane của hệ thống SMAP. Service này tiếp nhận platform-specific task từ RabbitMQ, gọi TinLikeSub SDK để thực hiện các action crawl tương ứng, materialize raw artifact theo storage contract của runtime, rồi phát hành completion envelope trở lại ingest runtime để lane ingest tiếp tục xử lý downstream.

Khác với một worker thuần túy, service còn cung cấp một lớp FastAPI mỏng để submit task, kiểm tra kết quả và giám sát health của runtime khi cần cho nhu cầu vận hành cục bộ hoặc kiểm tra môi trường phát triển.

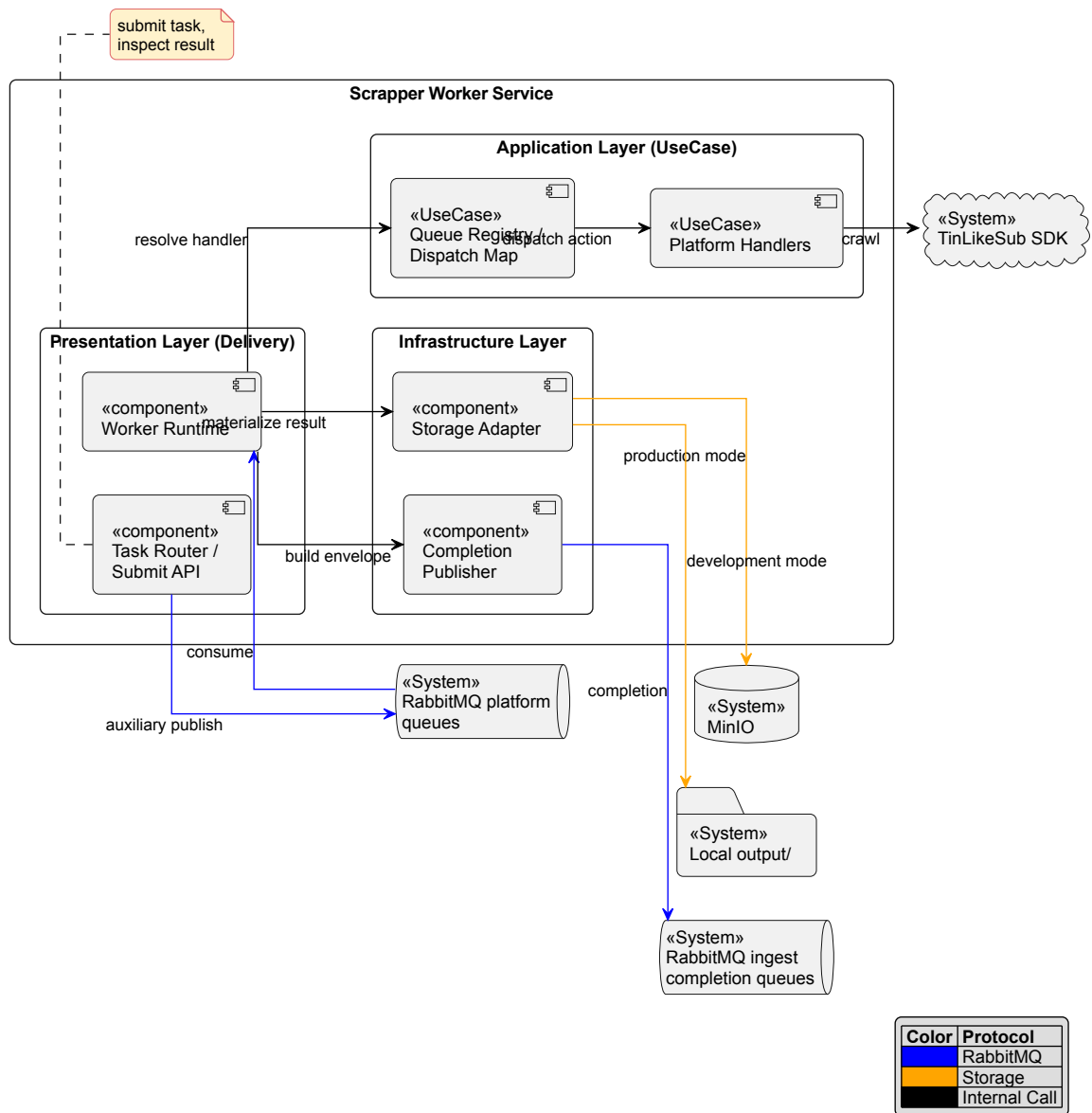
Vai trò của Scrapper Worker Service trong kiến trúc tổng thể:

- RabbitMQ Crawl Consumer: Tiêu thụ các queue tiktok_tasks, facebook_tasks và youtube_tasks.

- Platform Action Dispatcher: Định tuyến **action** sang handler phù hợp theo queue và platform.
- SDK-Based Crawl Executor: Thực thi các action đơn lẻ hoặc composite flows như `full_flow` thông qua TinLikeSub SDK.
- Raw Artifact Materializer: Ghi raw output xuống local storage hoặc MinIO theo runtime mode và storage contract của runtime.
- Completion Publisher: Publish completion envelope về ingest completion queue tương ứng để giữ correlation giữa crawl lane và ingest lane.
- Auxiliary Task API Surface: Cung cấp các route submit hoặc kiểm tra kết quả để hỗ trợ thao tác vận hành cục bộ và kiểm tra runtime.

Service này đáp ứng trực tiếp FR-08 về Crawl Runtime Orchestration và liên quan gián tiếp đến UC-01 về Thiết lập chiến dịch theo dõi và UC-02 về Vận hành chiến dịch theo dõi thông qua các lane dry run và crawl execution.

5.3.7.1 Component Diagram - C4 Level 3



Hình 37: Biểu đồ thành phần của Scrapper Worker Service

Bảng 30: Thành phần chính của Scrapper Worker Service

Thành phần	Trách nhiệm chính	Input / Output	Technology
FastAPI App / Lifespan Manager	Khởi động worker trong app lifespan, công bố router và health surface của runtime	Startup or HTTP request / running runtime	FastAPI + lifespan

Task Router / Submit API	Map platform sang queue, validate action và publish task payload cho các trigger cục bộ, hoặc trả kết quả kiểm tra khi được yêu cầu	HTTP payload / RabbitMQ message or inspection response	FastAPI route layer
Worker Runtime	Giữ kết nối RabbitMQ, cấp một channel cho mỗi queue, consume message và điều phối vòng đời xử lý	RabbitMQ message / task result	aio-pika + async Python
Queue Registry / Dispatch Map	Duy trì mapping queue -> platform và action -> handler để route đúng crawl flow	Queue name + action / handler	In-process registry
Platform Handlers	Thực thi các action như <code>search</code> , <code>post_detail</code> , <code>comments</code> , <code>video_detail</code> hoặc <code>full_flow</code> qua TinLikeSub SDK	Task params / crawl result	TinLikeSub SDK adapters
Storage Adapter	Materialize raw output xuống <code>output/</code> ở dev mode hoặc MinIO ở production mode, đồng thời sinh <code>batch_id</code> và <code>checksum</code>	Task result / storage metadata	Filesystem or MinIO
Completion Publisher	Đóng gói completion envelope và route sang queue completion phù hợp cho execution hoặc dryrun lane	Storage metadata / completion message	RabbitMQ publisher

5.3.7.2 Data Flow

Scraper Worker Service có ba luồng đáng chú ý: auxiliary task submission flow, crawl execution and completion flow chính, và result inspection flow ở lớp hỗ trợ.

a. Auxiliary Task Submission Flow

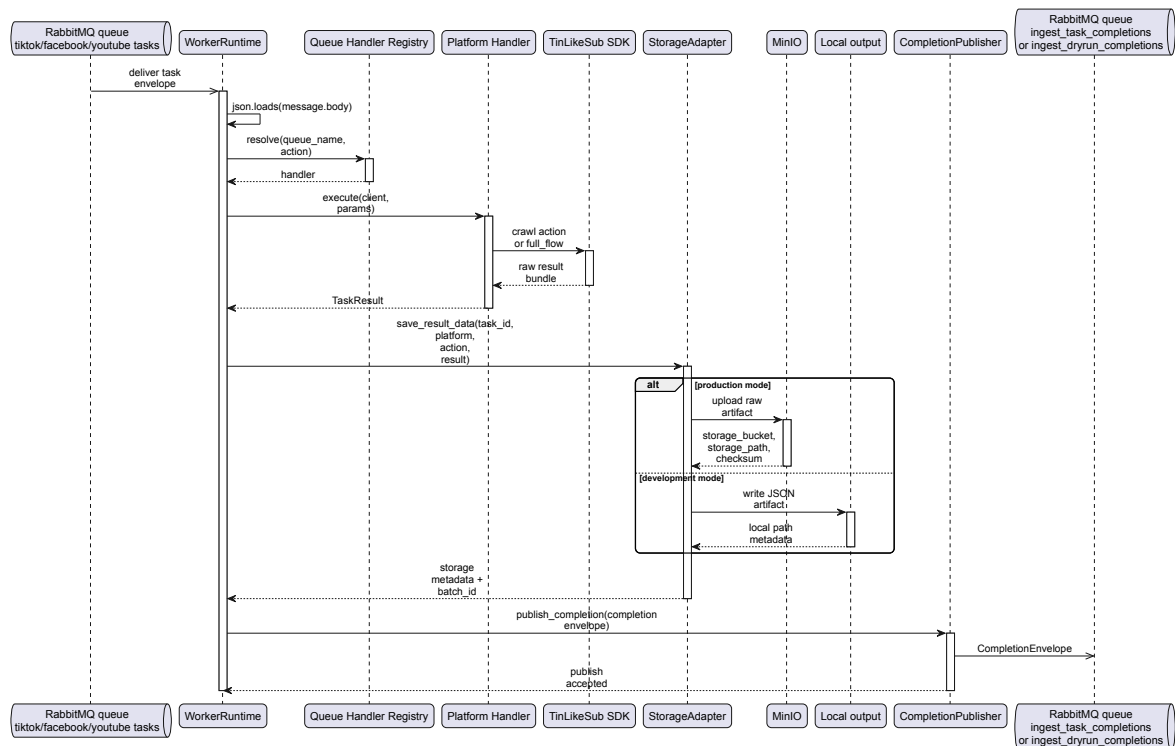
Luồng này phản ánh bề mặt submit phụ trợ của service, hữu ích cho nhu cầu vận hành cục bộ, kiểm tra môi trường phát triển và một số trigger ngoài worker runtime. Đây không phải cơ chế handoff chính giữa ingest-srv và scraper-srv, vì luồng vận hành chính dùng request envelope với `task_id` do ingest-srv cấp.

1. Client gọi `POST /api/v1/tasks/{platform}` với action và params tương ứng.

- Router map platform sang queue name, validate action hợp lệ theo registry của queue đó.
- Service tạo TaskPayload với task_id và created_at, rồi publish message sang RabbitMQ queue tương ứng như một luồng gọi phụ trợ phục vụ kiểm tra hoặc vận hành cục bộ.
- Message trở thành đầu vào cho worker runtime hoặc cho một worker instance khác đang consume cùng queue.

b. Crawl Execution and Completion Flow

Đây là luồng cốt lõi của service trong execution plane.



Hình 38: Luồng crawl execution và completion trong Scraper Worker Service

- Worker runtime consume message từ queue platform-specific như `tiktok_tasks`, `facebook_tasks` hoặc `youtube_tasks`; trong luồng vận hành chính, message này tương ứng với request envelope chuẩn từ `ingest-srv`.
- Registry xác định handler phù hợp theo queue và action; handler sau đó gọi TinLikeSub SDK để thực thi crawl action tương ứng.
- Nếu action là composite như `full_flow`, handler sẽ tự điều phối nhiều bước crawl liên tiếp và gom kết quả về một raw result bundle.
- Sau khi crawl xong, service materialize raw result xuống local output hoặc MinIO theo runtime mode, đồng thời sinh `storage_path`, `batch_id` và `checksum` theo contract.
- Service đóng gói `CompletionEnvelope` và publish sang `ingest_task_completions` hoặc `ingest_dryrun_completions` dựa trên `runtime_kind`, để ingest lane tiếp tục correlation và downstream processing.

c. Local Result Inspection Flow

Luồng này chỉ đóng vai trò phụ trợ cho kiểm tra hoặc xác minh cục bộ.

1. Client gọi GET `/api/v1/tasks/{task_id}/result` hoặc GET `/api/v1/tasks`.
2. Router đọc các JSON artifacts trong `output/` và trả về payload hoặc danh sách recent tasks tìm thấy.
3. Kết quả này hữu ích cho kiểm tra cục bộ, nhưng không thay thế cơ chế handoff chính thức của lane ingest vốn dựa trên raw artifact storage và completion envelope.

5.3.7.3 Design Patterns áp dụng

Scraper Worker Service áp dụng các design patterns sau:

- Hybrid API and Worker Runtime Pattern: cùng một service cung cấp cả HTTP surface mỏng và RabbitMQ worker runtime.
- Queue-per-Platform Dispatch Pattern: mỗi platform được gán với queue và action registry riêng để tránh lẫn semantics giữa các nguồn crawl.
- Handler Registry Pattern: việc map `action` sang handler cụ thể được giữ trong một dispatch map tập trung thay vì rải khắp worker code.
- Materialize-then-Notify Pattern: raw artifact phải được lưu trước khi completion envelope được phát hành cho ingest lane.
- Mode-Switched Storage Pattern: dev mode ưu tiên local file output, còn production mode dùng MinIO theo storage contract chung.
- Completion Routing Pattern: completion queue được chọn theo `runtime_kind` để tách execution flow và dryrun flow.

5.3.7.4 Key Decisions

- Dùng RabbitMQ queue tách theo platform để đơn giản hóa dispatch và cho phép tuning concurrency theo từng lane.
- Giữ một channel riêng cho mỗi queue để `prefetch_count` hoạt động độc lập giữa các platform runtimes.
- Giữ `task_id` làm correlation key xuyên suốt request, storage artifact và completion envelope.
- Chỉ publish completion sau khi storage metadata đã sẵn sàng để ingest lane không phải tiêu thụ raw payload trực tiếp từ message body.
- Duy trì submit API, local result API và `output/` artifacts như một bề mặt hỗ trợ kiểm tra, nhưng không xem đó là cơ chế handoff chính thức giữa ingest và worker runtime.

5.3.7.5 Dependencies

Internal Dependencies:

- FastAPI app, router và lifespan wiring.
- Worker runtime và dispatch registry theo queue hoặc platform.

- Platform handlers cho TikTok, Facebook và YouTube.
- Storage adapter cho local filesystem và MinIO.
- Completion publisher cho execution hoặc dryrun completion queues.

External Dependencies:

- RabbitMQ cho request queues và completion queues.
- TinLikeSub SDK hoặc upstream crawl API runtime.
- MinIO trong production mode để lưu raw artifact.
- Local filesystem `output/` trong dev mode.
- `ingest-srv` như producer của crawl tasks và consumer của completion envelopes.

5.3.8 Frontend Application

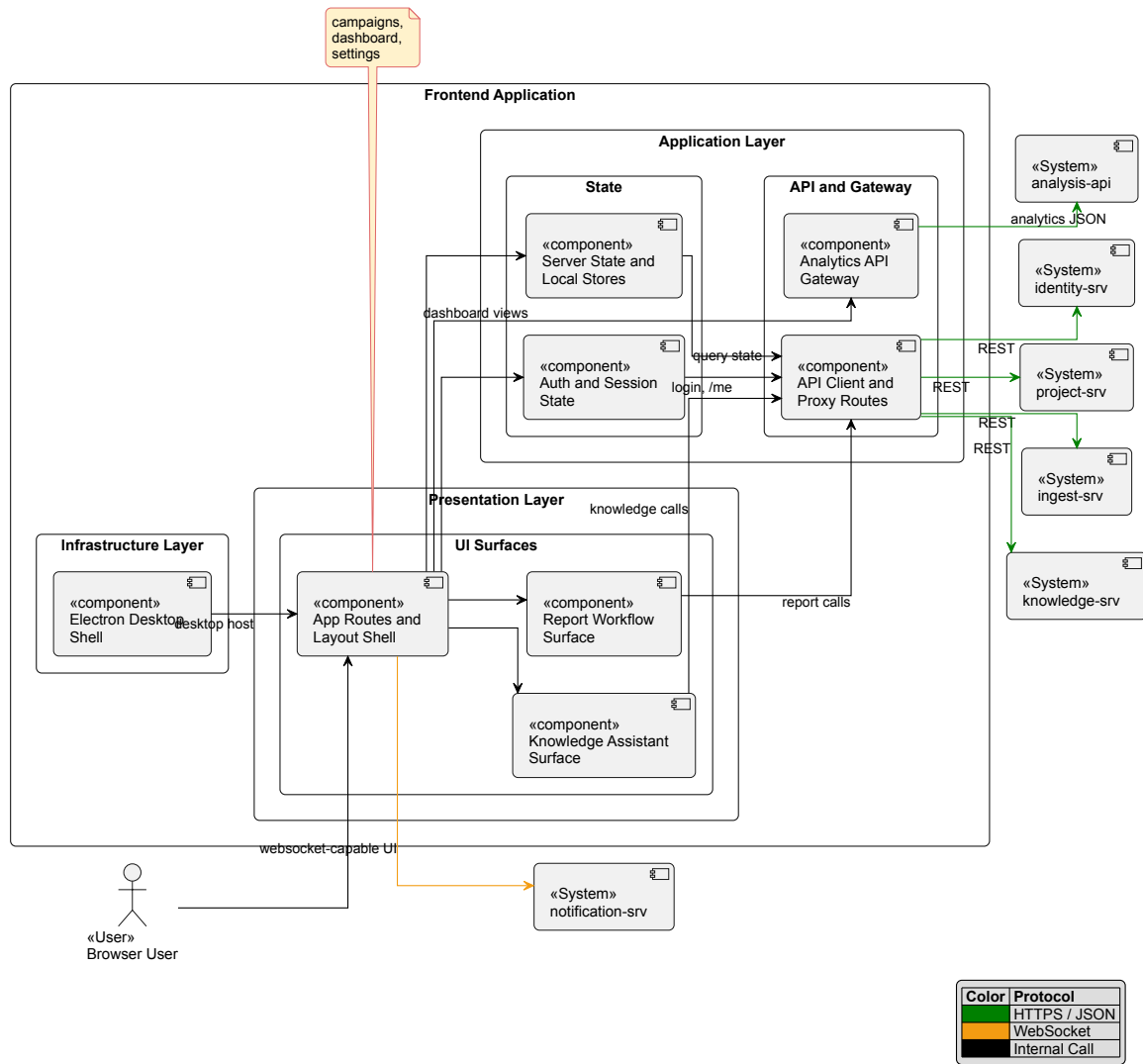
Frontend Application là lớp giao diện và web delivery boundary của hệ thống SMAP. Thành phần này được xây dựng bằng Next.js và React, cung cấp các màn hình thao tác cho người dùng nội bộ, đồng thời giữ vai trò trung gian same-origin cho nhiều lời gọi đến backend services. Thay vì để browser gọi trực tiếp từng service, frontend gom phần lớn request qua các Next.js route handlers để xử lý proxy, cookie, analytics API access và một số workflow hướng UI.

Vai trò của Frontend Application trong kiến trúc tổng thể:

- Web UI Shell: Cung cấp layout, navigation, dashboard, campaign workspace, reports, settings và assistant panel.
- API Gateway for Browser Calls: Định tuyến request từ browser qua `/api/proxy/*` trước khi chuyển tiếp đến backend gateway.
- Auth Session Bridge: Điều phối OAuth redirect, kiểm tra session và xử lý cookie trên frontend domain.
- Analytics API Gateway: Gọi `analysis-api` từ server-side route handlers để lấy dữ liệu analytics phục vụ dashboard, insight cards và chart.
- Knowledge Assistant Client: Gửi chat request và suggestion request đến `knowledge-srv`, đồng thời hiển thị câu trả lời, citation và gợi ý truy vấn.
- Auxiliary Report Workflow Surface: Cung cấp UI, một nhóm route proxy riêng và polling state cho report list, report detail, process tracking, post review và lazy comment loading ở mức frontend workflow phụ trợ.
- Desktop Shell: Đóng gói Next.js standalone server trong Electron để phân phối cùng một web UI dưới dạng desktop runtime.

Frontend Application hỗ trợ trực tiếp các use case có tương tác người dùng như cấu hình campaign/project, quản lý datasource, xem analytics dashboard, khai thác knowledge assistant và nhận notification trong giao diện. Bên cạnh các use case cốt lõi gắn với Chương 4, frontend còn có một workflow report phụ trợ ở mức giao diện.

5.3.8.1 Component Diagram - C4 Level 3



Hình 39: Biểu đồ thành phần của Frontend Application

Bảng 31: Thành phần chính của Frontend Application

Thành phần	Trách nhiệm chính	Input / Output	Technology
App Routes and Layout Shell	Tổ chức route, layout, provider tree, navigation, dashboard workspace, report pages, settings và các vùng UI chính	URL route + scope / rendered UI	Next.js App Router + React
Auth and Session State	Khởi tạo OAuth login, kiểm tra user hiện hành, lưu trạng thái auth và chuyển hướng khi session không hợp lệ	OAuth/cookie/session / user state	Zustand + Next.js route handler

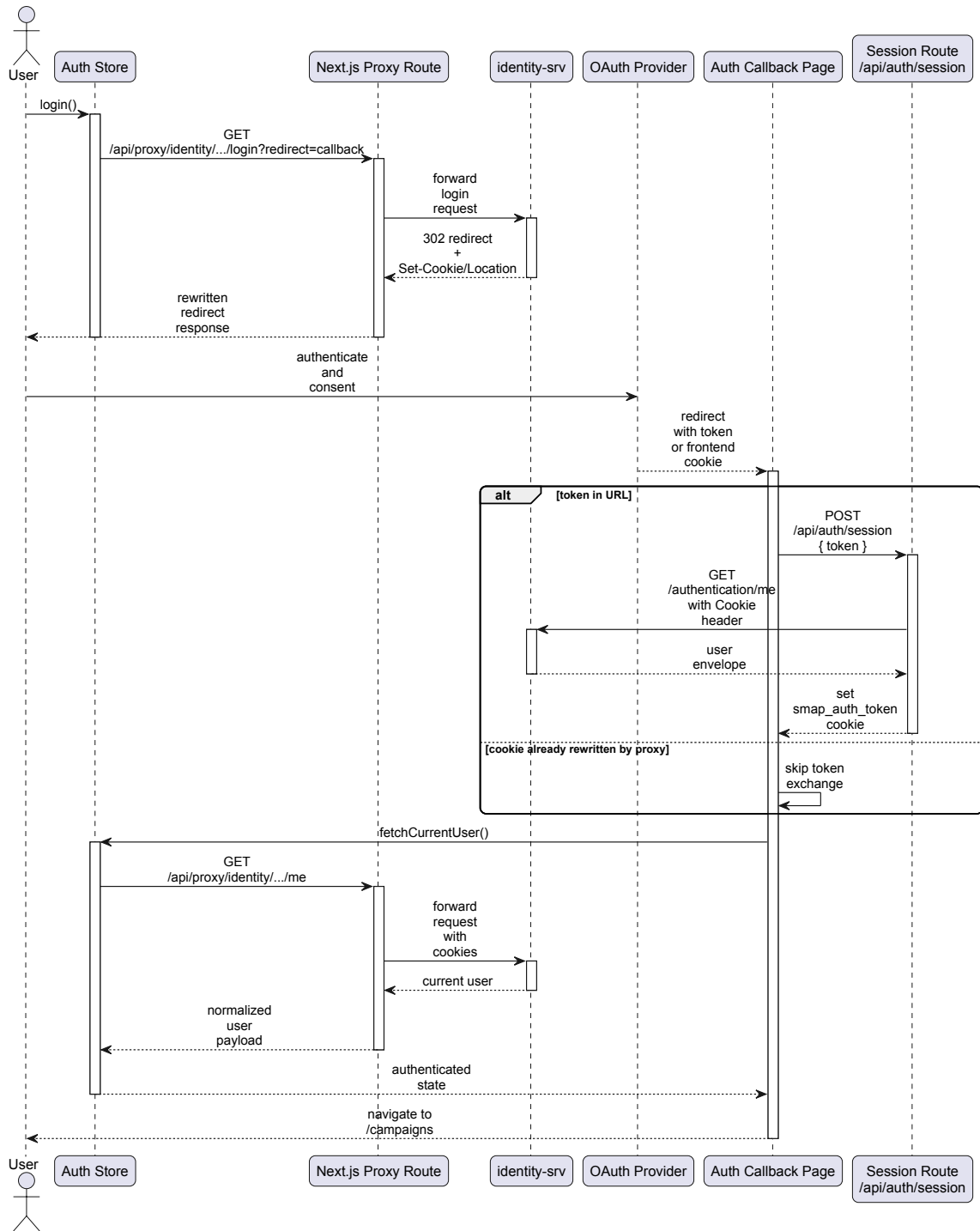
API Client and Proxy Routes	Chuẩn hóa endpoint, gửi request kèm cookie, unwrap response envelope và proxy browser request sang backend gateway	HTTP request / normalized response	Fetch API + Next.js route handlers
Server State and Local Stores	Quản lý cache dữ liệu server, auth state, notification state, selected campaign scope và long-running report jobs	Query params + UI events / cached state	TanStack Query + Zustand
Analytics API Gateway	Thực thi các analytics route server-side, proxy request sang analysis-api và chuyển JSON response thành dữ liệu phù hợp cho UI	Analytics query / dashboard dataset	Next.js API routes + analysis-api
Knowledge Assistant Surface	Gửi chat và suggestion request theo campaign scope, hiển thị markdown answer, citations và follow-up suggestions	User question / assistant response	React component + Knowledge API client
Report Workflow Surface	Điều phối list/detail/process/posts/comments của report workflow, lưu job state và polling tiến trình xử lý	Report action / report state	Next.js route handlers + React Query
Electron Desktop Shell	Khởi động Next.js standalone server, mở BrowserWindow và đóng gói web UI thành desktop application	Standalone web bundle / desktop runtime	Electron + electron-builder

5.3.8.2 Data Flow

Frontend Application có sáu luồng xử lý chính: auth and session flow, service API consumption flow, analytics API flow, knowledge assistant flow, report workflow flow và notification presentation flow.

a. Auth and Session Flow

Luồng này bắt đầu khi người dùng đăng nhập hoặc khi ứng dụng cần xác nhận session đang tồn tại.



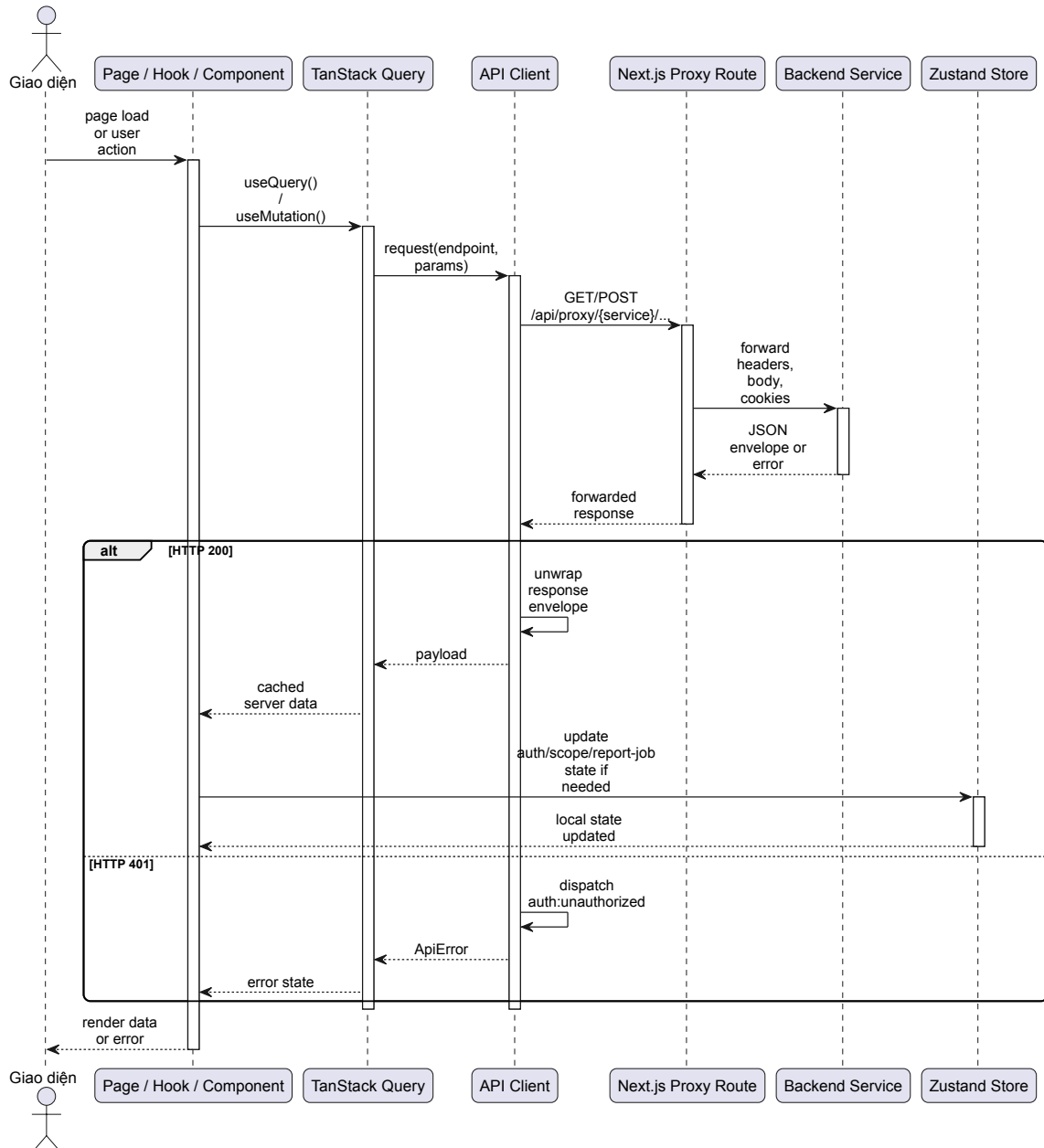
Hình 40: Luồng xác thực và thiết lập session trong Frontend Application

1. Người dùng chọn login provider trong giao diện authentication.
2. Auth store tạo login URL qua `/api/proxy` để đi đến endpoint login của `identity-srv` và gán redirect URL về frontend callback.
3. Proxy route chuyển tiếp request đến backend gateway và rewrite `Set-Cookie` để cookie hoạt động trên frontend domain.
4. Khi callback có token, route `/api/auth/session` xác thực token bằng endpoint `/me` của `identity-srv` trước khi set cookie `smap_auth_token` trên frontend domain.

5. AuthProvider gọi `fetchCurrentUser()` khi app mount để nạp user state cho các component phía dưới.
6. API client phát sự kiện `auth:unauthorized` khi gặp HTTP 401; auth store reset state và đưa người dùng về login route nếu đang ở protected area.

b. Service API Consumption Flow

Luồng này bao phủ các thao tác UI gọi đến `identity-srv`, `project-srv`, `ingest-srv` và `knowledge-srv`.



Hình 41: Luồng gọi API same-origin qua proxy trong Frontend Application

1. Page, component hoặc hook gọi API module tương ứng như `projectApi`, `datasourceApi`, `knowledgeApi` hoặc `reportsApi`.

2. `API_CONFIG` ánh xạ endpoint theo service prefix và chọn base URL phù hợp với môi trường thực thi.
3. Ở browser, request đi qua same-origin path `/api/proxy/*` để tránh CORS và giữ cookie flow thống nhất.
4. Catch-all proxy route loại bỏ hop-by-hop headers, chuyển tiếp cookie, forward method/body đến backend gateway và rewrite cookie ở response khi cần.
5. API client unwrap response envelope của Go services, chuẩn hóa lỗi và trả dữ liệu cho React Query hoặc component caller.
6. TanStack Query cache server state theo query key; Zustand giữ các state thuần UI như auth, notification, campaign scope và report jobs.

c. Analytics API Flow

Luồng này phục vụ dashboard, chart builder và các hook analytics.

1. UI gọi các route như `/api/analytics/kpis`, `/api/analytics/heap`, `/api/analytics/posts`, `/api/analytics/project-stats` hoặc `/api/analytics/platforms`.
2. Route handler chạy server-side và dùng analysis client để gọi `analysis-api` qua service nội bộ hoặc URL cấu hình bằng `ANALYSIS_API_INTERNAL_URL`.
3. `analysis-api` trả dữ liệu JSON đã chuẩn hóa cho từng view như KPI, platform distribution, sentiment, keyword, post list, project stats hoặc heap space.
4. Các hook analytics nhận JSON đã chuẩn hóa và bind vào card, chart, heap space hoặc bảng dữ liệu.
5. Campaign filtering được truyền qua query parameter để dashboard chỉ lấy dữ liệu thuộc scope đang chọn.

d. Knowledge Assistant Flow

Luồng này bắt đầu khi người dùng mở assistant panel trong campaign workspace.

1. Assistant đọc `activeCampaignId` từ scope provider và lấy campaign name từ campaign hook.
2. Khi campaign thay đổi, assistant gọi suggestion endpoint của `knowledge-srv`; nếu response không có suggestion thì UI dùng bộ gợi ý mặc định.
3. Khi người dùng gửi câu hỏi, assistant gọi `knowledgeApi.chat()` với `campaign_id`, `message` và `conversation_id` nếu đã có hội thoại trước đó.
4. Response được chuyển thành message blocks để hiển thị markdown, citations, suggestions và trạng thái lỗi nếu request thất bại.
5. Conversation state được lưu theo campaign trong `localStorage` để người dùng tiếp tục phiên hỏi đáp trong cùng frontend runtime.

e. Report Workflow Flow

Luồng này bao phủ report list, generate action, process tracking và post review cho một workflow report phụ trợ ở phía frontend.

1. Reports tab gọi `useReports()` theo `activeCampaignId` để lấy danh sách report.
2. Generate Report modal gửi request tạo report qua `reportsApi.generateCompetitor()` với `campaign`, `competitor URLs`, `platforms`, `sections` và giới hạn số post.
3. Khi request thành công, report job được ghi vào `useReportJobsStore`, đồng thời list query bị invalidate để UI cập nhật.
4. `useReportProcess()` polling process endpoint mỗi 3 giây khi job đang ở trạng thái non-terminal và dừng khi job hoàn tất, failed hoặc cancelled.
5. Report detail và review modal lazy-load posts/comments theo trang để tránh tải toàn bộ dữ liệu review trong một request.
6. Cancel và retry action cập nhật report job store, invalidate query liên quan và làm mới UI state.

Luồng này khác với knowledge-side report endpoints của `knowledge-srv`. Ở lớp frontend, workflow report trên đi qua một nhóm route proxy riêng dưới `/api/proxy/reports/*` và được xem như workflow phụ trợ ở lớp giao diện, không phải một chức năng trung tâm trong phạm vi yêu cầu cốt lõi của Chương 4.

f. Notification Presentation Flow

Luồng notification trong frontend tập trung ở lớp presentation state.

1. Component có thể push notification vào `useNotificationStore` với severity, title và content.
2. Store giới hạn số notification lưu trữ, đánh dấu read/unread và quản lý trạng thái dismiss cho toast hoặc banner.
3. `NotificationBanner`, `NotificationToasts` và `NotificationBell` cùng đọc từ store để render các biến thể hiển thị khác nhau.
4. Critical notification không auto-dismiss; các severity còn lại có thời gian dismiss mặc định để giảm nhiễu giao diện.

5.3.8.3 Design Patterns áp dụng

Frontend Application áp dụng các design patterns sau:

- Backend-for-Frontend / Proxy Route Pattern: Browser gọi same-origin route handlers, còn frontend server chịu trách nhiệm forward đến backend gateway và xử lý cookie.
- Server State and Client State Separation: TanStack Query quản lý dữ liệu server, trong khi Zustand quản lý auth state, scope, notification và report job state.
- Route-Based UI Composition: App Router tổ chức UI theo route, layout và feature area để giữ ranh giới màn hình rõ ràng.

- Server-Side Analytics Proxy: Browser gọi same-origin route `/api/analytics/*`, còn frontend server forward request tới `analysis-api` để giữ network boundary và timeout handling tập trung.
- Polling-Based Job Tracking: Report process dùng polling có điều kiện cho workload dài thay vì block request path hoặc dựa vào state cục bộ thuần túy.
- Assistant Client Orchestration: Assistant panel giữ conversation state phía client, còn knowledge generation được delegated sang `knowledge-srv`.
- Desktop Shell Pattern: Electron tái sử dụng Next.js standalone output và mở web UI trong BrowserWindow.

5.3.8.4 Key Decisions

- Chọn Next.js App Router để kết hợp page routing, server-side route handlers và web UI trong cùng một frontend boundary.
- Đưa browser request qua `/api/proxy/*` để giảm CORS complexity và giữ cookie/session behavior nhất quán giữa local, web và desktop runtime.
- Giữ analytics service access ở server-side route handlers nhằm không buộc browser gọi trực tiếp service nội bộ.
- Dùng TanStack Query cho server state để có cache, invalidation, polling và pagination thống nhất giữa dashboard, workflow report phụ trợ và detail views.
- Dùng Zustand cho state tương tác cục bộ như auth persistence, selected scope, notifications và report job tracking.
- Theo dõi report process bằng polling có điều kiện, phù hợp với trạng thái long-running job mà không yêu cầu persistent push connection trong frontend.
- Đóng gói desktop bằng Electron trên cùng Next.js standalone server để tránh duy trì một lớp giao diện riêng.

5.3.8.5 Dependencies

Internal Dependencies:

- App routes, layouts, providers và feature components trong Next.js App Router.
- API configuration, API client và catch-all proxy route `/api/proxy/[...path]`.
- Auth store, notification store, scope provider, assistant provider và report jobs store.
- TanStack Query hooks cho campaigns, projects, datasources, analytics, knowledge và workflow report phụ trợ.
- Analytics route handlers và analysis client server-side.
- Electron main process, preload script và standalone server packaging scripts.

External Dependencies:

- Backend gateway hoặc `API_BASE_URL` chứa các service prefix `identity`, `project`, `ingest`, `knowledge` và các API route liên quan.
- `identity-srv` cho OAuth login, logout và current-user validation.

- project-srv cho campaign/project management và lifecycle actions.
- ingest-srv cho datasource, target và dry run operations.
- knowledge-srv cho chat, suggestions, knowledge-side report endpoints và insight-facing endpoints.
- analysis-api cho các endpoint analytics dashboard dưới `/api/v1/analytics/*`.
- Nhóm route proxy của Next.js dưới `/api/proxy/reports/*` phục vụ workflow report phụ trợ ở lớp giao diện.
- Electron runtime cho desktop packaging.

5.4 Thiết kế cơ sở dữ liệu

Section này mô tả thiết kế dữ liệu của hệ thống SMAP, bao gồm chiến lược lựa chọn storage, schema dữ liệu theo từng service và các pattern quản lý dữ liệu phân tán.

5.4.1 Chiến lược lựa chọn Storage

Thiết kế dữ liệu của SMAP không thể dựa trên một database duy nhất vì hệ thống có nhiều loại workload khác nhau: dữ liệu nghiệp vụ cần transaction, raw crawl artifact có kích thước thay đổi, metadata bán cấu trúc cần linh hoạt, dữ liệu truy hồi ngữ nghĩa cần vector index, và notification/session/cache cần latency thấp. Vì vậy, lựa chọn storage được đánh giá theo từng nhóm nhu cầu trước khi chốt kiến trúc lưu trữ cuối cùng.

5.4.1.1 Tiêu chí đánh giá

Các phương án lưu trữ được so sánh theo sáu tiêu chí chính:

- Transactional consistency: mức độ phù hợp với dữ liệu cần ACID, constraint, index và transaction.
- Semi-structured flexibility: khả năng lưu metadata hoặc payload có cấu trúc thay đổi theo platform/domain.
- Large artifact handling: khả năng lưu raw crawl output hoặc report artifact mà không làm phình message queue hoặc bảng quan hệ.
- Retrieval model: mức độ phù hợp với relational query, cache lookup, graph traversal hoặc semantic vector search.
- Operational complexity: số lượng hệ thống phải vận hành, backup, monitor và migrate.
- Service ownership: khả năng giữ ranh giới dữ liệu theo service boundary và tránh coupling trực tiếp giữa các bounded context.

5.4.1.2 So sánh các phương án lưu trữ

Bảng 32: So sánh các phương án lưu trữ cho SMAP

Phương án	Workload phù hợp	Điểm mạnh	Trade-off	Vai trò trong SMAP
PostgreSQL	Metadata nghiệp vụ, lineage, analytics rows, conversation/report tracking	ACID, schema rõ ràng, FK nội bộ schema, index mạnh, JSONB cho metadata linh hoạt	Không tối ưu cho raw artifact lớn hoặc vector similarity search; cross-service FK vẫn phải tránh	Storage chính cho dữ liệu có cấu trúc
MongoDB	Document payload linh hoạt, dữ liệu bán cấu trúc thay đổi thường xuyên	Schema linh hoạt, dễ lưu document theo platform	Tăng thêm một database vận hành; raw crawl artifact vẫn phù hợp hơn với object storage; metadata quan hệ vẫn cần constraint	Không chọn làm storage chính
Neo4j	Graph traversal, relationship discovery, entity graph nhiều bậc	Mạnh khi quan hệ giữa actor, topic, campaign và entity là trung tâm của truy vấn	Các quan hệ chính của SMAP vẫn được xử lý tốt bằng relational model và vector retrieval; thêm Neo4j làm tăng chi phí vận hành	Không chọn cho phạm vi hiện tại
Redis	Session, cache, registry nhỏ, Pub/Sub, state ngắn hạn	Latency thấp, TTL, atomic operations, phù hợp dữ liệu tạm	Không thay thế durable database; dữ liệu cần có nguồn bền vững để tái tạo khi cache mất	Cache/session/PubSub layer

MinIO / S3-compatible object storage	Raw crawl artifacts, report artifacts, file output	Lưu object lớn tốt, tách payload khỏi queue, API S3-compatible	Không phù hợp cho query nghiệp vụ trực tiếp; cần metadata lineage trong database quan hệ	Object storage chính
Qdrant	Embedding, semantic search, RAG retrieval	Vector similarity search, payload filter, collection theo project	Không thay thế metadata store; cần đồng bộ với PostgreSQL tracking và indexing status	Vector store cho Knowledge Service
Lưu raw payload trực tiếp trong message queue	Task nhỏ, event metadata ngắn	Đơn giản vì consumer nhận đủ dữ liệu từ message	Làm message lớn, khó retry/replay artifact, tăng áp lực broker và memory	Không dùng cho raw artifact lớn

Từ bảng so sánh trên, SMAP chọn chiến lược polyglot persistence thay vì ép toàn bộ dữ liệu vào một database duy nhất. PostgreSQL giữ phần dữ liệu bền vững có cấu trúc; Redis giữ cache/session/state ngắn hạn; MinIO giữ artifact; Qdrant giữ vector index. MongoDB và Neo4j không được chọn làm thành phần lõi vì workload hiện tại chưa cần document database hoặc graph database riêng vượt quá khả năng kết hợp PostgreSQL JSONB, object storage và vector retrieval.

5.4.1.3 Tổng quan Storage Stack

Kiến trúc lưu trữ của SMAP gồm các nhóm thành phần chính sau:

- PostgreSQL được sử dụng làm relational database chính cho metadata và dữ liệu nghiệp vụ có cấu trúc. Các schema chính gồm **identity**, **project**, **ingest**, **schema_analysis** và **knowledge**, tương ứng với dữ liệu người dùng, campaign/project, datasource/runtime lineage, analytics insights và knowledge metadata.
- Redis được sử dụng cho các state ngắn hạn và truy cập nhanh như session/auth state, token/session mapping, domain registry, search cache, embedding/search result cache và Redis Pub/Sub cho notification delivery.

- MinIO đóng vai trò object storage cho raw crawl artifacts, storage metadata của ingest runtime và các artifact báo cáo. Service xử lý chỉ trao đổi metadata hoặc reference qua message/event thay vì truyền toàn bộ raw payload qua queue.
- Qdrant được sử dụng làm vector store cho Knowledge Service. Các analytics outputs sau khi được index sẽ trở thành vector points để phục vụ semantic search, RAG chat và các truy vấn tri thức theo campaign/project scope.
- Local filesystem output/ được sử dụng trong scraper runtime ở development mode như một biến thể triển khai của raw artifact storage, trong khi production mode dùng MinIO.

5.4.1.4 Lý do lựa chọn

PostgreSQL phù hợp với các dữ liệu cần tính nhất quán và truy vấn quan hệ như user, JWT key, audit log, campaign, project, datasource, crawl target, external task, raw batch lineage, analytics insight, indexing status, conversation và report metadata. Khả năng hỗ trợ transaction, index, foreign key, enum và JSONB giúp mỗi service vừa giữ được cấu trúc dữ liệu rõ ràng, vừa có đủ độ linh hoạt cho các trường metadata bán cấu trúc.

Redis được lựa chọn cho các dữ liệu có vòng đời ngắn hoặc cần latency thấp. Trong SMAP, Redis không thay thế database quan hệ mà bổ sung cho các nhu cầu như session lookup, blacklist/session mapping, cache kết quả search, cache mapping campaign-project, cache domain registry và Pub/Sub notification. TTL và atomic operations giúp giảm tải database chính cho các dữ liệu tạm thời hoặc được tính lại từ nguồn dữ liệu bền vững.

MinIO được sử dụng cho các artifact có kích thước hoặc vòng đời không phù hợp để lưu trực tiếp trong message body hoặc bảng quan hệ. Scraper runtime ghi raw result thành artifact, Ingest Service nhận completion metadata, kiểm tra object metadata và tạo raw_batches lineage trước khi parse hoặc publish dữ liệu chuẩn hóa. Với S3-compatible API, MinIO cũng phù hợp cho các artifact báo cáo hoặc file output cần được truy xuất lại.

Qdrant được lựa chọn vì workload của Knowledge Service là semantic retrieval thay vì relational lookup. Vector embeddings, payload filters và collection theo project cho phép hệ thống tìm kiếm theo ngữ nghĩa trên kết quả analytics, trong khi PostgreSQL vẫn giữ phần tracking metadata như indexed documents, conversations, messages và reports.

5.4.1.5 Redis TTL và data retention

Redis trong SMAP được dùng cho dữ liệu tạm thời hoặc có thể tái tạo từ nguồn bền vững, nên mỗi nhóm key có retention khác nhau theo vai trò vận hành:

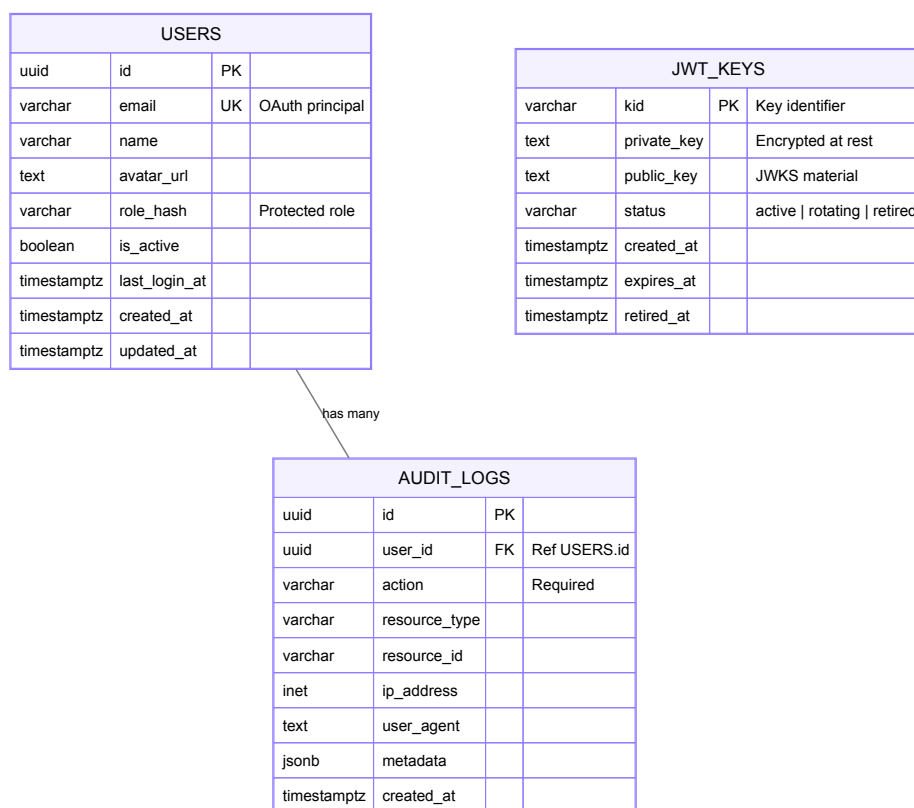
- Session runtime của identity-srv dùng key `session:{jti}` với TTL mặc định theo `session.ttl` là 8 giờ; trường hợp remember-me dùng TTL 7 ngày. Mapping `user_sessions:{user_id}` được giữ 7 ngày để hỗ trợ logout-all và tự lọc các JTI đã hết hạn.
- Token blacklist dùng key `blacklist:{jti}` với TTL bằng thời gian sống còn lại của JWT, nhờ đó token đã bị revoke tự biến mất sau khi token gốc hết hạn.
- Domain registry dùng key `smap:domains` không đặt TTL vì đây là registry dùng chung do analysis-srv publish lại khi khởi động; nếu Redis mất dữ liệu, registry được tái tạo từ cấu hình domain khi service publish lại.
- Knowledge cache dùng TTL ngắn cho dữ liệu phụ thuộc truy vấn: campaign-name và campaign-project cache khoảng 10 phút, search result cache khoảng 5 phút. Embedding cache dùng TTL dài hơn, mặc định 7 ngày, vì vector theo nội dung đầu vào có khả năng tái sử dụng cao.
- Notification Redis Pub/Sub không dùng Redis làm durable message store; message chỉ được fanout tới WebSocket runtime tại thời điểm publish. Riêng key chống trùng crisis alert dùng TTL theo `repeat_cooldown`, mặc định 1 giờ nếu payload không cung cấp giá trị hợp lệ.

Vì vậy, mất dữ liệu Redis không được xem là mất dữ liệu nghiệp vụ bền vững. Các dữ liệu cần audit, lineage, report tracking hoặc search index vẫn được giữ ở PostgreSQL, MinIO hoặc Qdrant; Redis chỉ giữ session/cache/registry/pub-sub state có thể hết hạn hoặc tái tạo.

5.4.2 Identity Service

Identity Service quản lý security data cho cơ chế xác thực OAuth2/JWT của hệ thống SMAP. Ở lớp lưu trữ bền vững, service này sử dụng ba bảng chính là `users`, `jwt_keys` và `audit_logs`. Các state runtime có vòng đời ngắn như session hoặc token blacklist được giữ ở Redis theo chiến lược lưu trữ đã nêu ở mục 5.4.1, nên không xuất hiện trong relational schema của service.

5.4.2.1 Database Schema



Hình 42: Database Schema - Identity Service

Schema `identity` được tổ chức quanh ba nhóm dữ liệu. Nhóm thứ nhất là hồ sơ người dùng đã được xác thực qua identity provider, lưu trong `users`. Nhóm thứ hai là vòng đời khóa ký JWT, lưu trong `jwt_keys` để hỗ trợ phát hành token và key rotation. Nhóm thứ ba là dữ liệu truy vết xác thực và ủy quyền, lưu trong `audit_logs` như một audit trail cục bộ của security boundary này.

Thiết kế này cho thấy Identity Service không theo mô hình username/password truyền thống, cũng không nhúng session runtime vào PostgreSQL như nguồn dữ liệu chính. Thay vào đó, PostgreSQL chỉ giữ các thực thể cần tính bền vững và truy vết lâu dài, còn session hoặc blacklist state được xử lý ở Redis layer.

5.4.2.2 Table Catalog

Bảng 33: Identity Service Tables

Table	Mục đích	Key	Indexed fields	Constraints
users	Thông tin tài khoản người dùng đã đăng nhập qua OAuth2 SSO	id (PK), email (UK)	email, is_active	email unique; email, role_hash not null

jwt_keys	Lưu cặp khóa ký JWT và trạng thái vòng đời của từng key	kid (PK)	status, created_at	private_key, public_key, status not null
audit_logs	Audit trail cho các sự kiện authentication và authorization	id (PK), user_id (FK nội bộ)	user_id, created_at, action	action not null; user_id tham chiếu users.id

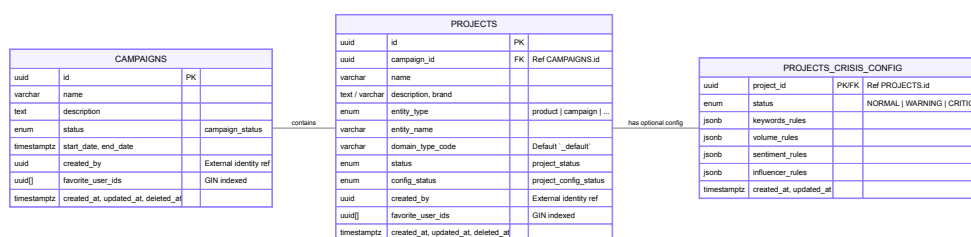
5.4.2.3 Design Decisions

- Email as Principal Identifier: `users.email` là định danh duy nhất của người dùng vì service bám cơ chế OAuth2 SSO thay vì username/password truyền thống.
- Protected Role Storage: thông tin vai trò không được lưu dưới dạng role text đơn giản mà đi qua trường `role_hash`, giúp tách representation nghiệp vụ khỏi raw identity payload từ provider.
- JWT Key Lifecycle Table: `jwt_keys` được tách riêng để service có thể quản lý active, rotating và retired keys mà không trộn với bảng người dùng.
- Append-Only Audit Trail: `audit_logs` giữ dữ liệu truy vết authentication/authorization ở PostgreSQL để phục vụ audit và phân tích vận hành lâu dài.
- Redis as Runtime Complement: session lookup và token blacklist là state ngắn hạn ở Redis, còn PostgreSQL chỉ giữ các thực thể bảo mật cần tính bền vững.

5.4.3 Project Service

Project Service giữ business context của campaign/project và cấu hình crisis ở schema `project`. Ở lớp dữ liệu bền vững, service này được tổ chức quanh ba thực thể chính là `campaigns`, `projects` và `projects_crisis_config`. Cấu trúc này cho phép tách rõ aggregate marketing cấp campaign, thực thể vận hành cấp project và phần cấu hình giám sát crisis theo từng project.

5.4.3.1 Database Schema



Hình 43: Database Schema - Project Service

5.4.3.2 Table Catalog

Bảng 34: Project Service Tables

Table	Mục đích	Key	Indexes	Constraints
campaigns	Aggregate cấp campaign cho hoạt động theo dõi và grouping project	id (PK)	status, created_by, date_range, favorite_user_ids (GIN)	status enum; soft delete qua deleted_at
projects	Thực thể project gắn campaign, entity context và trạng thái vận hành nghiệp vụ	id (PK), campaign_id (FK)	campaign_id, status, config_status, domain_type_code, favorite_user_ids (GIN)	FK nội bộ sang campaigns; status và config_status là enum
projects crisis config	Cấu hình trigger crisis theo từng project	project_id (PK/FK)	status	One-to-one với projects; xóa cascade khi project bị xóa

5.4.3.3 Relationship và Design Decisions

Thiết kế dữ liệu của Project Service bám sát business ownership. `campaigns.id` -> `projects.campaign_id` là quan hệ nội bộ một-nhiều bên trong cùng schema. `projects.id` -> `projects_crisis_config.project_id` là quan hệ một-một, trong đó `project_id` đồng thời là primary key và foreign key để đảm bảo mỗi project có tối đa một crisis configuration đang hiệu lực ở lớp dữ liệu chính.

Các trường như `created_by` ở `campaigns` và `projects` chỉ giữ external reference tới identity boundary, không có FK xuyên service. Điều này giữ nguyên nguyên tắc database-per-service trong khi vẫn cho phép application layer kiểm tra quyền hoặc user context qua JWT và internal validation.

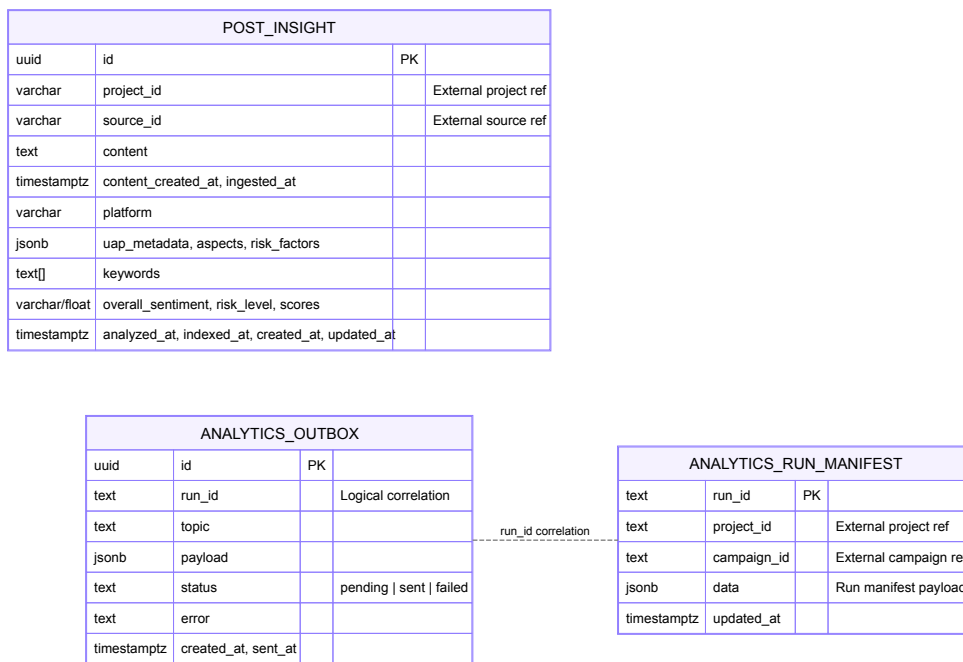
Một quyết định quan trọng khác là tách `status` khỏi `config_status` ở `projects`. `status` phản ánh vòng đời nghiệp vụ ở mức activate/pause/archive, còn `config_status` phản ánh mức hoàn tất của các bước cấu hình như draft, onboarding hoặc dry run. Cách tách này giúp service không phải nhồi nhiều semantics khác nhau vào một cột trạng thái duy nhất.

Phần crisis configuration được giữ dưới dạng JSONB theo từng nhóm rule như keywords, volume, sentiment và influencer. Cách làm này phù hợp với tính bán cấu trúc của rule payload, nhưng ownership vẫn được neo chặt vào `project_id` để tránh biến cấu hình crisis thành một bounded context tách rời khỏi project.

5.4.4 Analytics Service

Analytics Service lưu kết quả phân tích và dữ liệu điều phối lane bất đồng bộ trong schema `schema_analysis`. Ở lớp relational, ba đối tượng chính là `post_insight`, `analytics_outbox` và `analytics_run_manifest`. Chúng phản ánh ba nhu cầu khác nhau: lưu insight đã enrich, đảm bảo publish downstream đáng tin cậy, và giữ run-level manifest cho audit hoặc replay context.

5.4.4.1 Database Schema



Hình 44: Database Schema - Analytics Service

5.4.4.2 Table Catalog

Bảng 35: Analytics Service Tables

Table	Mục đích	Key	Indexed fields
post_insight	Insight đã enrich cho từng nội dung hoặc mention đã qua pipeline phân tích	id (PK)	project_id, platform, sentiment, aspects (GIN), uap_metadata (GIN)
analytics_outbox	Transactional outbox cho Kafka delivery của các message downstream	id (PK), run_id	pending status
analytics_run_manifest	Manifest theo từng run để lưu context và hỗ trợ audit hoặc replay reasoning	run_id (PK)	project_id

5.4.4.3 Data Shape và Design Decisions

`post_insight` là bảng trung tâm của analytics schema. So với phương án tách `post_analytics` và `post_comments` thành các bảng chi tiết riêng, service dùng một cấu trúc insight giàu thông tin hơn, chứa nội dung, sentiment, aspects, keywords, risk, engagement, quality và processing metadata trong cùng một row. Điều này phù hợp với cách pipeline phân tích phát sinh NLPFact và persist thành `post_insight`.

`analytics_outbox` và `analytics_run_manifest` không đại diện cho dữ liệu người dùng cuối mà cho nhu cầu điều phối lane bất đồng bộ. `analytics_outbox` giữ các payload chờ gửi hoặc đã gửi xuống Kafka, trong khi `analytics_run_manifest` lưu context ở mức `run_id` để service có thể truy lại batch reasoning, project scope hoặc campaign scope mà không cần đọc lại toàn bộ raw input.

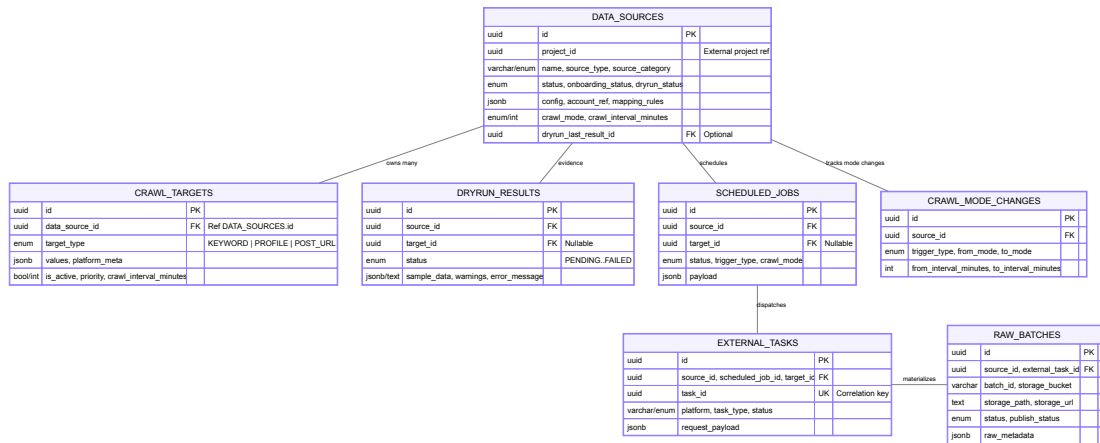
Các trường bán cấu trúc như `uap_metadata`, `aspects` hoặc `risk_factors` được giữ ở JSONB thay vì bể thành hàng loạt bảng con. Đây là trade-off hợp lý vì dữ liệu enrichment mang tính đa dạng theo domain, model version và runtime stage, nhưng vẫn cần được query/filter ở mức vừa phải bằng GIN index.

- Direct Insight Persistence: kết quả enrichment được materialize trực tiếp thành `post_insight` thay vì chia nhỏ thành nhiều bảng chi tiết phụ.
- Transactional Outbox: `analytics_outbox` là lớp đệm bền vững giữa persistence và Kafka publish để giảm rủi ro mất message downstream.
- Run-Level Audit Context: `analytics_run_manifest` giữ metadata ở cấp pipeline run thay vì trộn với từng row insight.
- External References over Cross-Service FK: `project_id`, `campaign_id` hay `source_id` được giữ như external identifiers, không tạo FK xuyên service boundary.

5.4.5 Ingest Service

Ingest Service sở hữu schema `ingest`, nơi lưu cả metadata cấu hình nguồn dữ liệu lẫn lineage của runtime bất đồng bộ. Đây là phần dữ liệu phức tạp nhất trong hệ thống vì phải nối được business context của project với task dispatch, completion metadata, raw artifact storage và trạng thái publish sang analytics data plane.

5.4.5.1 Database Schema



Hình 45: Database Schema - Ingest Service

5.4.5.2 Table Catalog

Bảng 36: Ingest Service Tables

Table	Mục đích	Key	Indexed fields	Constraints
data_sources	Root entity cho datasource theo project và loại source	id (PK), dryrun_last_result_id (FK)	project_id + deleted_at, status + category, config (GIN), mapping_rules (GIN)	Enum cho type/ category/status; nhiều check constraint theo source semantics
crawl_targets	Per-target schedule và input cụ thể cho keyword/profile/post URL	id (PK), data_source_id (FK)	data_source_id, next_crawl_at + active, target_type	crawl_interval_minutes > 0
dryrun_results	Bảng chứng readiness và sample output cho dry run	id (PK), source_id (FK), target_id (FK nullable)	project_id + created_at, source_id + created_at, job_id	Non-negative checks cho sample/total counts
scheduled_jobs	Job điều phối lịch chạy crawl hoặc trigger runtime	id (PK), source_id (FK), target_id (FK nullable)	status + scheduled_for, project_id + scheduled_for, target_id	Retry count non- negative; enum trigger/crawl mode
external_tasks	Correlation layer giữa ingest runtime và worker task bên ngoài	id (PK), task_id (UK), scheduled_job_id (FK)	status + created_at, scheduled_job_id, target_id	task_id unique để chống duplicate completion

raw_batches	Lineage của raw artifact và trạng thái publish sang UAP/ analytics	id (PK), external_task_id (FK)	project_id + received_at, publish_status + received_at, checksum	Unique (source_id, batch_id); bucket/path thay raw payload
crawl_mode_changes	Audit trail cho việc đổi crawl mode và interval	id (PK), source_id (FK)	source_id + triggered_at, project_id + triggered_at	Interval non-negative checks

5.4.5.3 Lineage Model và Design Decisions

`data_sources` là root entity của ingest domain. `crawl_targets` mở rộng root này theo mô hình một datasource có nhiều target, và đây cũng là nơi per-target scheduling được giữ thay vì gói hết vào datasource row. `dryrun_results` là lớp bằng chứng readiness độc lập, cho phép service lưu lịch sử dry run và tái dùng kết quả gần nhất trong lifecycle control.

Chuỗi runtime bất đồng bộ được biểu diễn qua `scheduled_jobs` -> `external_tasks` -> `raw_batches`. `scheduled_jobs` phản ánh ý định chạy ở mức ingest runtime, `external_tasks` phản ánh task đã dispatch sang worker, còn `raw_batches` phản ánh artifact đã materialize và trạng thái publish sang analytics. Chuỗi này quan trọng vì nó cho phép trace từ project/source/target đến task ngoài, rồi đến object storage và cuối cùng đến UAP publishing.

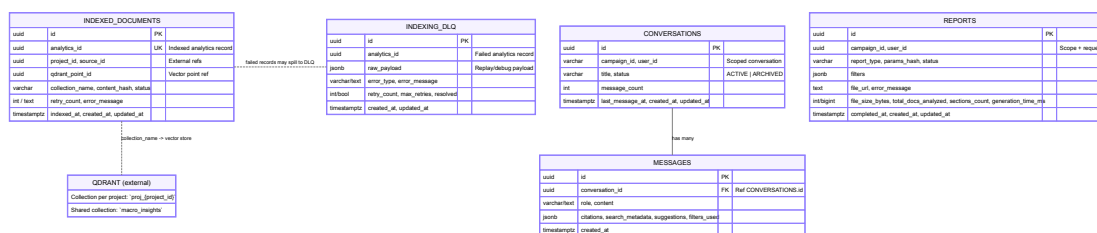
Thiết kế dữ liệu ở đây ưu tiên reference metadata thay vì nhúng raw payload lớn vào relational rows hoặc message queue. Các cột như `storage_bucket`, `storage_path`, `checksum` và `publish_status` trong `raw_batches` cho thấy service xem object storage là nơi giữ artifact thực, còn database chỉ giữ lineage và operational state.

Các enum như `source_status`, `dryrun_status`, `job_status`, `batch_status` và `publish_status` cũng là quyết định thiết kế quan trọng. Chúng làm cho runtime lane có thể được điều phối bằng trạng thái rõ ràng ở cấp dữ liệu, thay vì chỉ dựa vào log hoặc queue state bên ngoài.

5.4.6 Knowledge Service

Knowledge Service dùng mô hình polyglot persistence rõ ràng nhất trong hệ thống: PostgreSQL giữ metadata bền vững cho indexing, chat và report; Qdrant giữ vector index; Redis giữ search cache; MinIO giữ report artifact. Trong phạm vi mục 5.4 này, phần relational schema tập trung vào các bảng chính của schema `knowledge` và vai trò của chúng trong việc phối hợp với vector store.

5.4.6.1 Database Schema



Hình 46: Database Schema - Knowledge Service

5.4.6.2 Table Catalog

Bảng 37: Knowledge Service Tables

Table / View	Mục đích	Key	Indexed fields / Derived Object	Ghi chú
indexed_documents	Tracking metadata cho record đã hoặc đang được index vào Qdrant	id (PK), analytics_id (UK)	project_id, batch_id + status, content_hash, created_at	Chứa collection_name và qdrant_point_id như vector references
indexing_dlq	Dead-letter queue cho các record indexing thất bại	id (PK)	analytics_id, batch_id, unresolved errors, retry eligibility	Giữ raw_payload để debug hoặc replay
conversations	Session hỏi đáp đa lượt theo campaign và user	id (PK)	campaign_id + user_id, status, last_message_at, created_at	Scoped theo campaign thay vì project trực tiếp
messages	Lưu từng message user hoặc assistant cùng metadata truy hồi	id (PK), conversation_id (FK)	conversation_id + created_at	Cascade delete theo conversation
reports	Tracking metadata cho capability report generation ở knowledge layer	id (PK)	campaign_id, user_id + status, params_hash	Capability mở rộng ngoài FR/UC cốt lõi
stats_by_project, stats_by_batch, error_summary, health_check	Derived views cho monitoring và vận hành indexing lane	View	Tạo từ indexed_documents và indexing_dlq	Không phải nguồn dữ liệu gốc

5.4.6.3 Relational Metadata, Vector Store và Design Decisions

`indexed_documents` là cầu nối giữa relational metadata và vector store. Bảng này không lưu embedding vector, mà lưu `qdrant_point_id`, `collection_name`, `content_hash`, trạng thái index và metrics xử lý. Điều đó cho phép Knowledge Service dùng Qdrant cho retrieval nhưng vẫn giữ được khả năng query operational state bằng SQL.

`indexing_dlq` đóng vai trò vùng cách ly cho các record indexing thất bại. Thay vì ghi đè lỗi lên record chính một cách mơ hồ, service giữ riêng raw payload, error type, `retry_count` và `resolved flag` để vận hành có thể retry hoặc xử lý ngoại lệ có kiểm soát.

Ở nhánh consumption đồng bộ, `conversations` và `messages` tạo thành mô hình normalized cho chat history. `conversations` giữ session-level metadata, còn `messages` giữ user message, assistant response, citations, `filters_used` và search metadata. Thiết kế này phù hợp với multi-turn RAG interaction hơn là nhúng toàn bộ exchange vào một blob duy nhất.

`reports` phản ánh capability mở rộng của knowledge layer. Bảng này lưu request scope, report type, params hash, artifact URL, status và metrics generation; trong khi bản thân report artifact được materialize ở object storage chứ không nhúng vào database row. Trong cấu trúc relational này, notebook/maestro cũng không được xem là capability trung tâm của service.

5.4.7 Data Management Patterns

5.4.7.1 Database per Service và External Reference IDs

SMAP giữ database-per-service ở mức logical schema ownership. `identity`, `project`, `ingest`, `schema_analysis` và `knowledge` đều sở hữu dữ liệu cục bộ của mình, trong khi các liên kết xuyên service như `created_by`, `project_id`, `campaign_id` hay `source_id` chỉ được giữ như external identifiers. Cách làm này tránh FK xuyên boundary nhưng vẫn cho phép trace ở application layer.

5.4.7.2 Artifact Reference Pattern

Các payload lớn không được coi là relational first-class data. Scraper runtime materialize raw output vào local filesystem ở development mode hoặc MinIO ở production mode; `ingest.raw_batches` chỉ giữ bucket/path/checksum/size và lineage metadata. Tương tự, knowledge-side reports lưu artifact URL thay vì nhúng toàn bộ báo cáo vào PostgreSQL row.

5.4.7.3 Vector and Metadata Split

Knowledge retrieval được tách thành hai lớp. Qdrant giữ vector points và payload phục vụ semantic search; PostgreSQL giữ `indexed_documents`, `indexing_dlq`, `conversations`,

messages và reports. Thiết kế này cho phép tối ưu retrieval model mà không đánh đổi khả năng query operational tracking bằng SQL.

5.4.7.4 Async Operational Tracking

Các lane bất đồng bộ quan trọng đều có bảng tracking riêng. Ở analytics có analytics_outbox và analytics_run_manifest. Ở ingest có dryrun_results, scheduled_jobs, external_tasks, raw_batches, crawl_mode_changes. Ở knowledge có indexed_documents, indexing_dlq và các statistics views. Pattern chung ở đây là không để queue broker hoặc object storage trở thành nguồn trạng thái duy nhất; database giữ đủ metadata để audit, retry và reconciliation.

5.4.8 Tổng kết

Thiết kế dữ liệu của SMAP là một ví dụ điển hình của polyglot persistence theo service boundary và workload. PostgreSQL giữ metadata có cấu trúc và operational tracking; Redis giữ cache, session và state ngắn hạn; MinIO giữ artifact lớn; Qdrant giữ vector index. Mỗi loại storage được chọn theo retrieval model và vòng đời dữ liệu thay vì vì một khuôn mẫu duy nhất áp cho toàn hệ thống.

Ở mức schema chi tiết, Identity Service giữ users, jwt_keys, audit_logs; Project Service giữ campaigns, projects, projects_crisis_config; Analytics Service giữ post_insight, analytics_outbox, analytics_run_manifest; Ingest Service giữ execution lineage quanh data_sources, crawl_targets, dryrun_results, scheduled_jobs, external_tasks, raw_batches, crawl_mode_changes; Knowledge Service giữ tracking, chat và report metadata quanh indexed_documents, indexing_dlq, conversations, messages, reports cùng các derived views vận hành.

Những lựa chọn này giúp hệ thống giữ được ranh giới ownership rõ ràng, hỗ trợ tracing cho các lane bất đồng bộ, và tránh nhét raw artifact hoặc vector retrieval vào những storage model không phù hợp. Đây cũng là nền dữ liệu để các mục giao tiếp, triển khai và liên hệ yêu cầu ở các phần sau được trình bày nhất quán với cấu trúc hệ thống đã mô tả trong chương này.

5.5 Sơ đồ tuần tự

Sơ đồ tuần tự trong mục này minh họa các tương tác động giữa giao diện, các dịch vụ nghiệp vụ và các runtime lane của SMAP theo trình tự thời gian. Thay vì được đọc như một ánh xạ một-một tuyệt đối với từng bảng use case ở mục 4.4, các sơ đồ ở đây đóng vai trò lớp minh họa cho những interaction patterns quan trọng giữa business control plane, execution plane, analytics pipeline, knowledge retrieval và notification delivery.

Trong kiến trúc của SMAP, các sơ đồ tập trung làm rõ cách các thành phần trao đổi qua internal HTTP, RabbitMQ, Kafka và Redis Pub/Sub, cũng như cách metadata hoặc artifacts đi qua PostgreSQL, Redis, MinIO và Qdrant theo từng luồng xử lý. Các

nhân UC trong các tiểu mục bên dưới được dùng như mã tham chiếu của từng sơ đồ trong phần trình bày này.

Một số sơ đồ dưới đây là supporting technical sequence cho một use case người dùng. Các sơ đồ này không biến xử lý nền thành use case mới, mà chỉ làm rõ chuỗi phối hợp giữa các runtime lane để tạo ra kết quả mà người dùng quan sát trong use case tương ứng.

Các sơ đồ tuần tự trong mục này được tổ chức theo năm nhóm mục tiêu nghiệp vụ tương ứng với Chương 4:

- Thiết lập chiến dịch theo dõi.
- Vận hành chiến dịch theo dõi.
- Tra cứu và hỏi đáp dữ liệu phân tích.
- Theo dõi trạng thái và nhận cảnh báo.
- Thiết lập và quản lý quy tắc cảnh báo khủng hoảng.

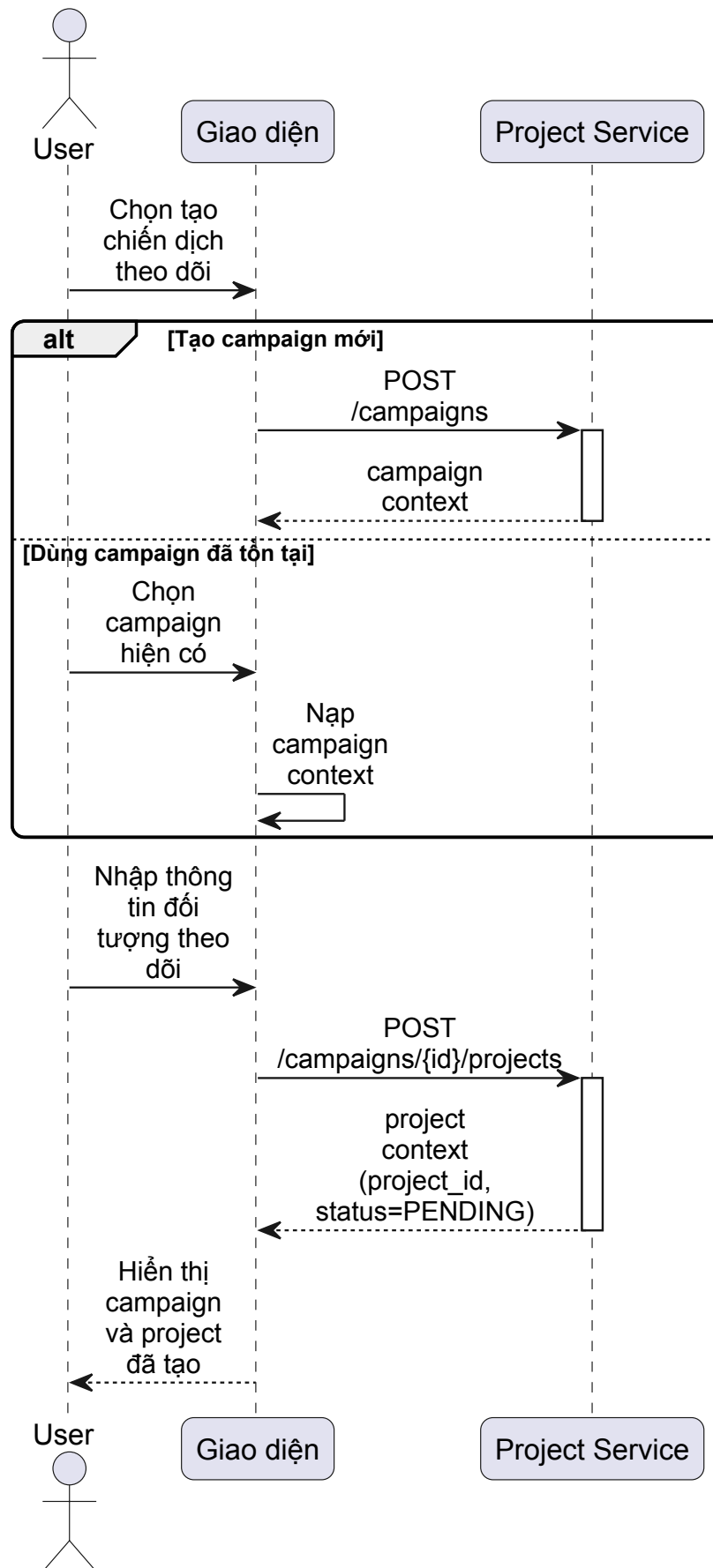
5.5.1 UC-01: Thiết lập chiến dịch theo dõi

Trong bộ use case hiện tại ở Chương 4, UC-01 được hiểu là mục tiêu thiết lập một chiến dịch theo dõi đủ điều kiện vận hành. Ở mức sequence, mục tiêu này được cụ thể hóa qua ba interaction flows chính: tạo campaign và project, khai báo nguồn dữ liệu cùng mục tiêu thu thập, và thực hiện kiểm tra thử khi cần.

Điểm chung của các flow trong mục này là chúng chỉ hoàn tất phần cấu hình đầu vào. Hệ thống chưa bước sang giai đoạn vận hành chính thức cho đến khi người dùng chủ động chuyển sang UC-02.

5.5.1.1 Tạo campaign và project

Luồng này mô tả phần đầu của mục tiêu thiết lập chiến dịch, nơi người dùng tạo campaign mới hoặc gắn project vào campaign đã tồn tại.



Hình 47: Sequence Diagram UC-01 Part 1: Tạo campaign và project

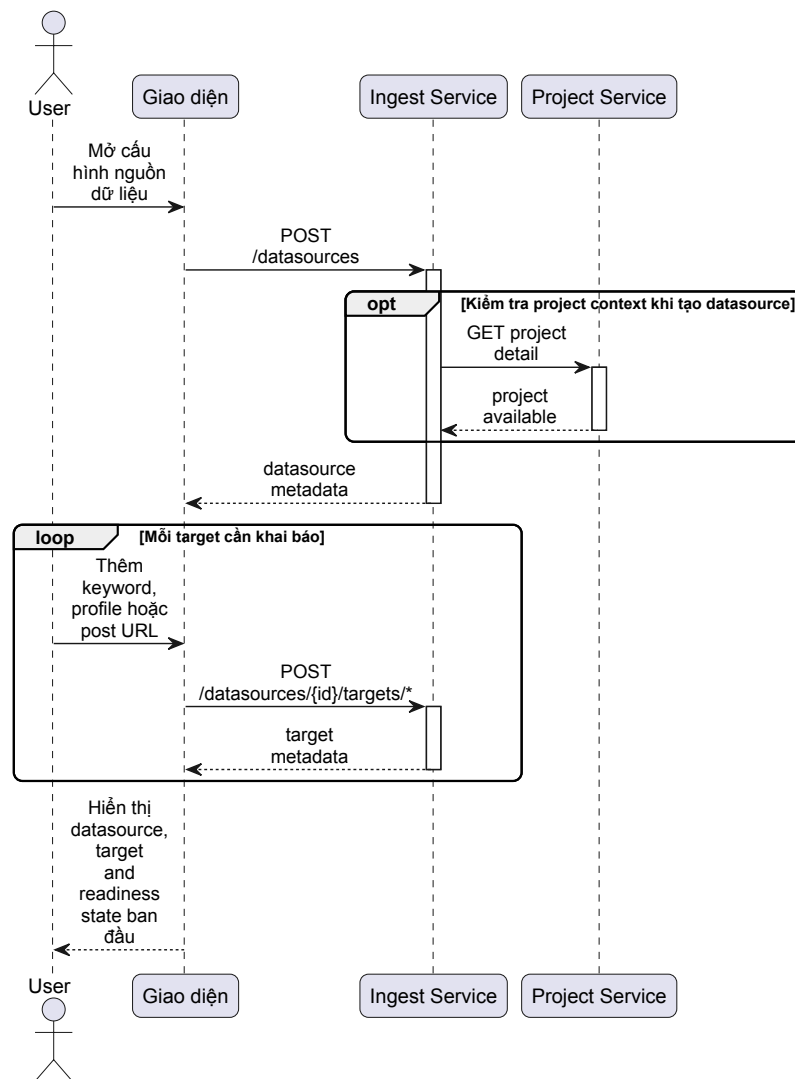
Luồng xử lý:

- User bắt đầu một chiến dịch theo dõi mới hoặc chọn campaign đã tồn tại làm ngữ cảnh nghiệp vụ.
- Nếu cần, giao diện gửi request tạo campaign mới đến Project Service; sau đó gửi request tạo project trong campaign đã chọn.
- Project Service kiểm tra dữ liệu đầu vào và tạo project gắn với campaign tương ứng.
- Hệ thống trả lại campaign_id, project_id và trạng thái cấu hình ban đầu để người dùng tiếp tục thiết lập các thành phần còn lại.

Điểm quan trọng: Việc tạo campaign và project chưa kích hoạt runtime thu thập hay xử lý nền. Đây mới là bước thiết lập business context cho chiến dịch.

5.5.1.2 Khai báo nguồn dữ liệu và mục tiêu thu thập

Luồng này tiếp nối ngay sau khi project đã tồn tại, tập trung vào việc khai báo datasource và target cần thu thập cho chiến dịch.



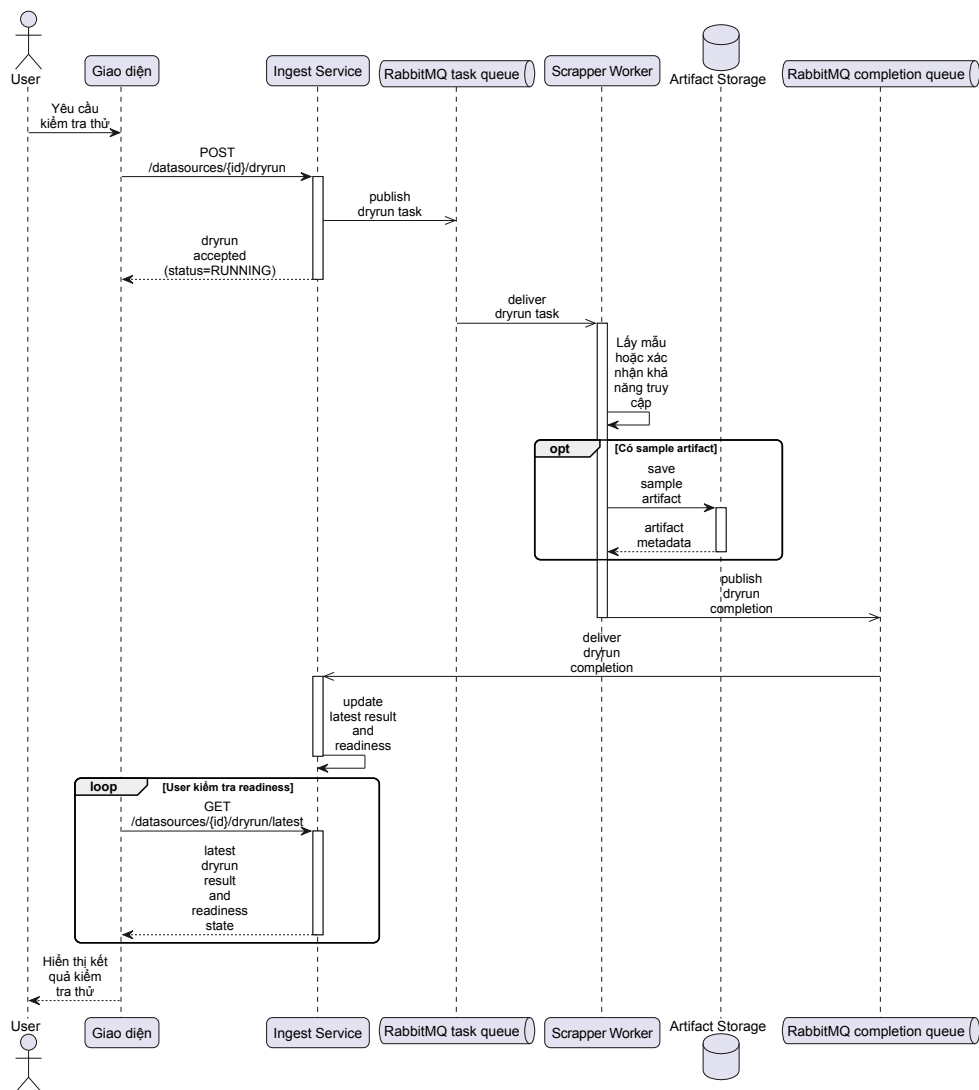
Hình 48: Sequence Diagram UC-01 Part 2: Khai báo nguồn dữ liệu và mục tiêu thu thập

Luồng xử lý:

- User mở phần cấu hình nguồn dữ liệu của project vừa tạo.
- Giao diện gửi request đến Ingest Service để tạo datasource theo loại nền tảng hoặc nguồn dữ liệu phù hợp.
- User tiếp tục khai báo một hoặc nhiều mục tiêu thu thập như keyword, profile hoặc post URL.
- Ingest Service lưu datasource, tiếp nhận các target được khai báo và cập nhật trạng thái cấu hình phục vụ bước readiness ở phía sau.
- Giao diện nhận lại metadata cần thiết để người dùng quyết định có cần thực hiện kiểm tra thử hay không.

5.5.1.3 Kiểm tra thử và đánh giá mức sẵn sàng

Luồng này phản ánh nhánh kiểm tra thử của UC-01, chỉ được thực hiện khi tổ hợp nguồn dữ liệu và mục tiêu thu thập yêu cầu bước xác nhận trước khi vận hành.



Hình 49: Sequence Diagram UC-01 Part 3: Kiểm tra thử và đánh giá mức sẵn sàng

Luồng xử lý:

- User yêu cầu chạy kiểm tra thử cho một tổ hợp datasource-target đã khai báo.
- Ingest Service tạo yêu cầu dry run, ghi nhận trạng thái kiểm tra và chuyển tác vụ phù hợp xuống runtime thu thập nếu cần lấy mẫu.
- Runtime thu thập truy xuất dữ liệu mẫu hoặc xác nhận khả năng truy cập nguồn dữ liệu, sau đó trả kết quả về ingest lane.
- Ingest Service nhận completion từ runtime thu thập, lưu kết quả kiểm tra thử gần nhất và readiness liên quan.
- Hệ thống cập nhật chỉ báo readiness của datasource hoặc target để người dùng đánh giá liệu chiến dịch đã đủ điều kiện bước sang UC-02 hay chưa.

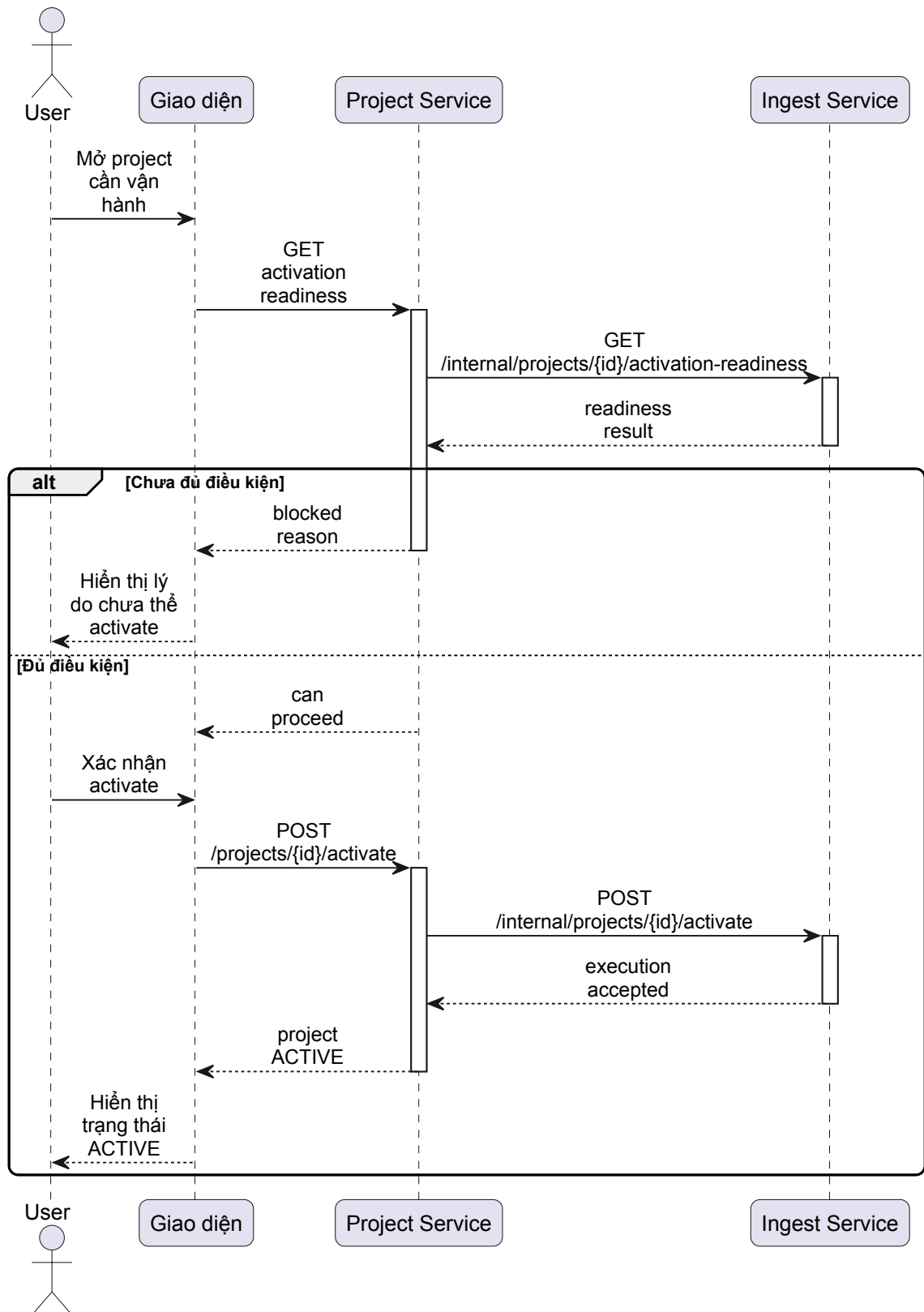
5.5.2 UC-02: Vận hành chiến dịch theo dõi

Trong bộ use case hiện tại ở Chương 4, UC-02 được hiểu là mục tiêu đưa một chiến dịch đã được thiết lập sang trạng thái vận hành có kiểm soát. Ở mức sequence, mục tiêu này được thể hiện qua hai interaction flows chính: kiểm tra mức sẵn sàng trước khi kích hoạt và điều khiển các thao tác vận hành như tạm dừng, tiếp tục, lưu trữ hoặc mở lại chiến dịch.

Điểm chung của các flow trong mục này là Project Service giữ vai trò business control plane, còn Ingest Service giữ execution plane. Vì vậy, giao diện không điều khiển trực tiếp runtime thu thập dữ liệu mà luôn đi qua lớp lifecycle control của project.

5.5.2.1 Kiểm tra mức sẵn sàng và kích hoạt chiến dịch

Luồng này mô tả cách người dùng kiểm tra readiness và kích hoạt một chiến dịch khi cấu hình đầu vào đã được thiết lập.



Hình 50: Sequence Diagram UC-02 Part 1: Kiểm tra mức sẵn sàng và kích hoạt chiến dịch

Luồng xử lý:

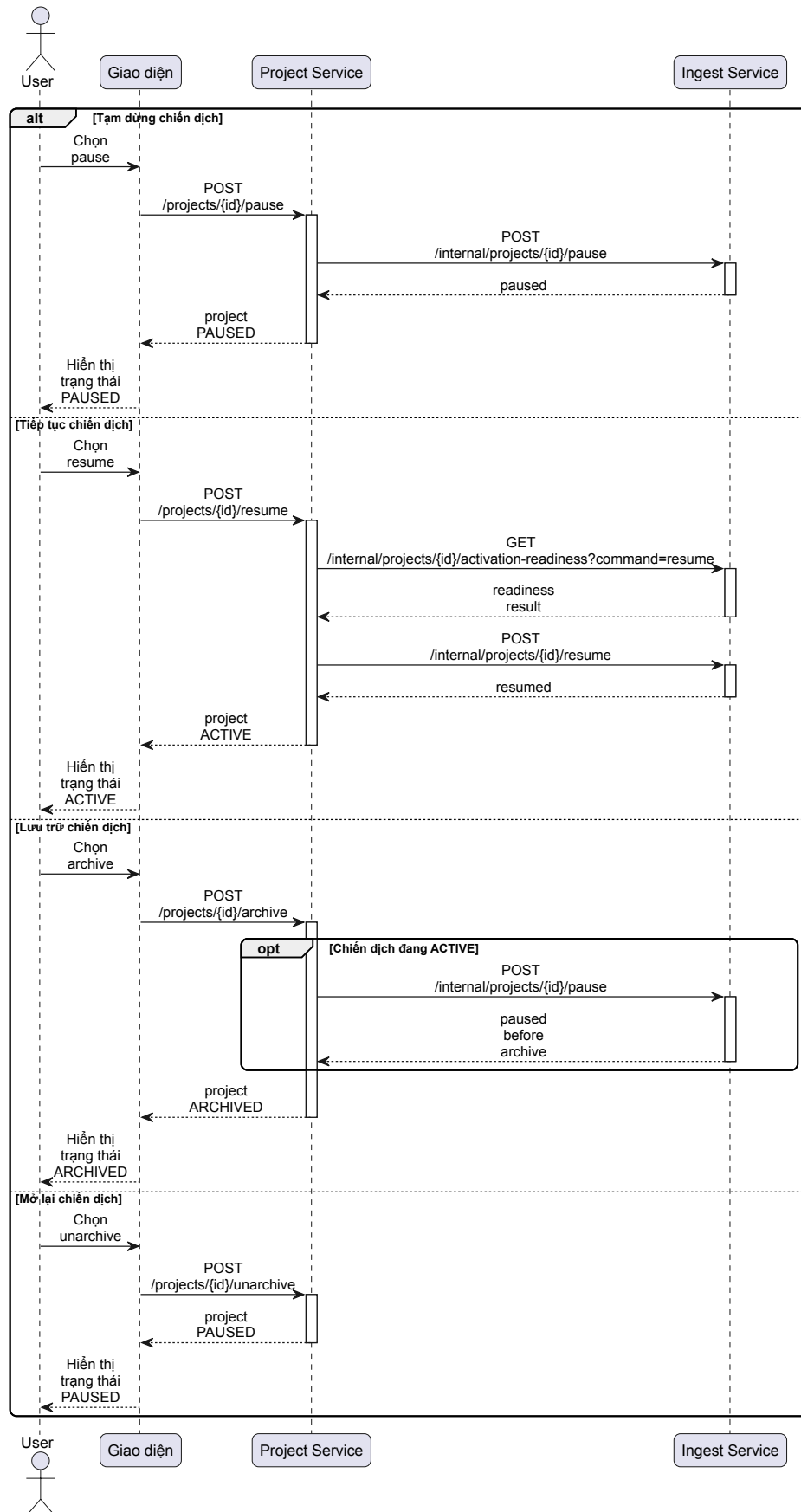
- User mở project hoặc campaign cần vận hành và yêu cầu kiểm tra mức sẵn sàng.

- Giao diện gửi request đến Project Service để lấy kết quả readiness của chiến dịch.
- Project Service kiểm tra trạng thái hiện tại của project và gọi internal HTTP sang Ingest Service để đánh giá readiness dựa trên nguồn dữ liệu, mục tiêu thu thập và kết quả kiểm tra thử cần thiết.
- Nếu readiness đạt yêu cầu, user xác nhận thao tác kích hoạt.
- Project Service gửi lệnh activate sang Ingest Service để chuyển các nguồn dữ liệu đủ điều kiện sang trạng thái vận hành.
- Sau khi điều phối thành công, Project Service cập nhật trạng thái project sang ACTIVE và trả kết quả cho giao diện.

Điểm quan trọng: Kích hoạt chiến dịch là thao tác điều khiển nghiệp vụ ở `project-srv`, không phải việc giao diện trực tiếp publish task thu thập hay tự gọi runtime xử lý nền.

5.5.2.2 Tạm dừng, tiếp tục, lưu trữ và mở lại chiến dịch

Luồng này mô tả các thao tác vận hành còn lại sau khi chiến dịch đã được tạo và có thể đã hoạt động trước đó.



Hình 51: Sequence Diagram UC-02 Part 2: Tạm dừng, tiếp tục, lưu trữ và mở lại chiến dịch

Luồng xử lý:

- Khi user chọn tạm dừng, giao diện gửi request pause đến Project Service.
- Project Service kiểm tra transition hợp lệ, rồi gọi Ingest Service để dừng các nguồn dữ liệu đang hoạt động trước khi cập nhật project sang PAUSED.
- Khi user chọn tiếp tục, Project Service kiểm tra lại mức sẵn sàng trước khi gọi Ingest Service resume runtime và chuyển project về ACTIVE.
- Khi user chọn lưu trữ, Project Service chuyển project sang ARCHIVED; nếu chiến dịch đang hoạt động thì phần execution liên quan được dừng trước.
- Khi user mở lại một chiến dịch đã lưu trữ, Project Service đưa chiến dịch về trạng thái PAUSED để user có thể kiểm tra lại trước khi vận hành tiếp.

Điểm quan trọng: Các thao tác pause, resume, archive và unarchive đều được áp dụng như các business transitions có kiểm soát. Trạng thái project không được thay đổi chỉ dựa trên giao diện; mỗi thao tác đều phải đi qua validation và lifecycle control tương ứng.

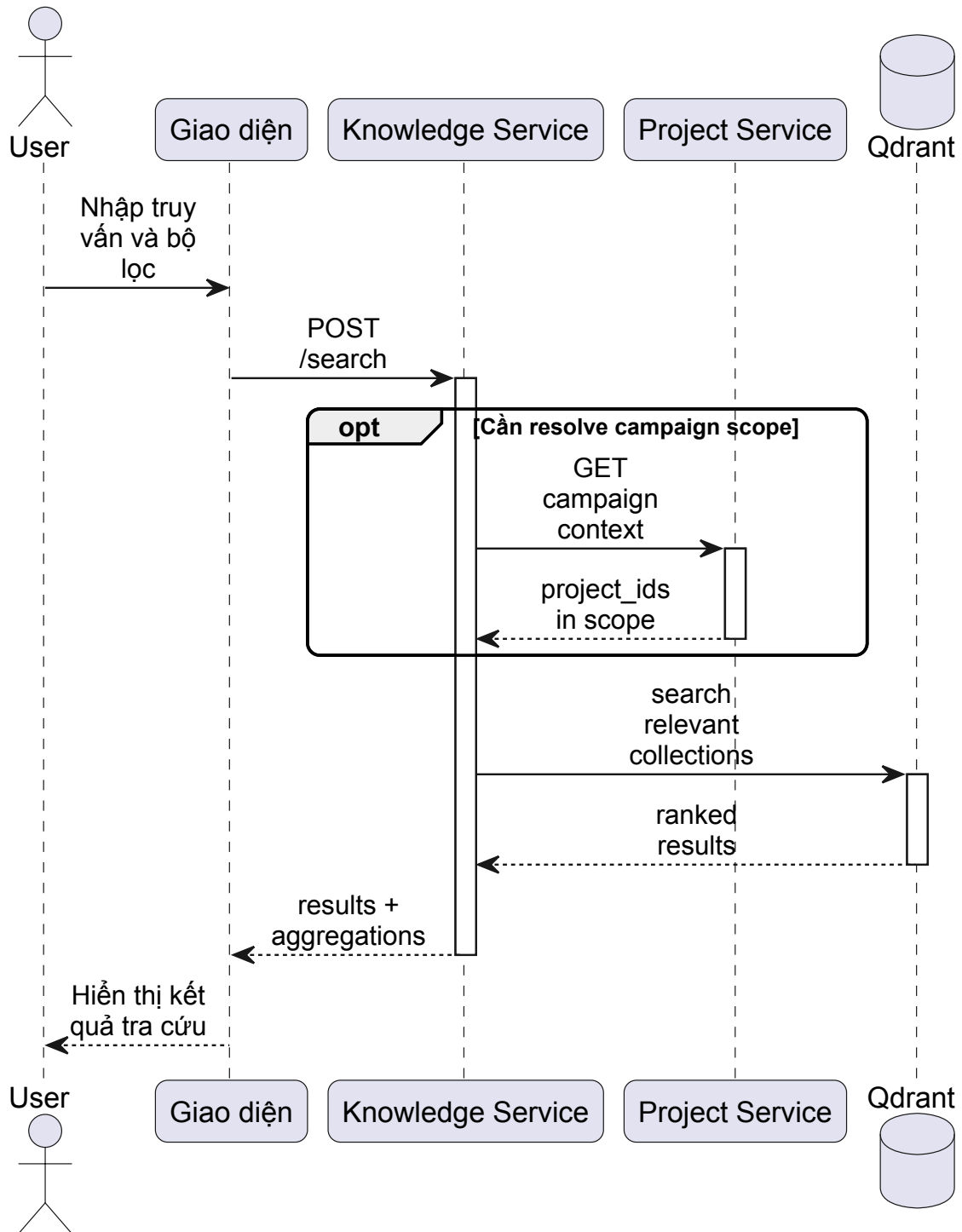
5.5.3 UC-03: Tra cứu và hỏi đáp dữ liệu phân tích

Trong bộ use case hiện tại ở Chương 4, UC-03 được hiểu là mục tiêu khai thác dữ liệu phân tích đã sẵn sàng thông qua tra cứu hoặc hỏi đáp theo ngữ cảnh chiến dịch. Ở mức sequence, mục tiêu này được cụ thể hóa qua hai interaction flows chính: truy hồi dữ liệu theo truy vấn có ngữ cảnh và hỏi đáp có dẫn chứng trên nền retrieval.

Điểm chung của các flow trong mục này là giao diện không truy cập trực tiếp vector store hay dữ liệu phân tích gốc. Mọi truy vấn đều đi qua Knowledge Service để kiểm tra phạm vi, truy hồi ngữ cảnh và tổng hợp kết quả trước khi trả về cho người dùng.

5.5.3.1 Tra cứu theo ngữ cảnh chiến dịch

Luồng này mô tả cách người dùng gửi truy vấn tìm kiếm trong phạm vi một campaign hoặc project và nhận về danh sách kết quả phù hợp.



Hình 52: Sequence Diagram UC-03 Part 1: Tra cứu theo ngữ cảnh chiến dịch

Luồng xử lý:

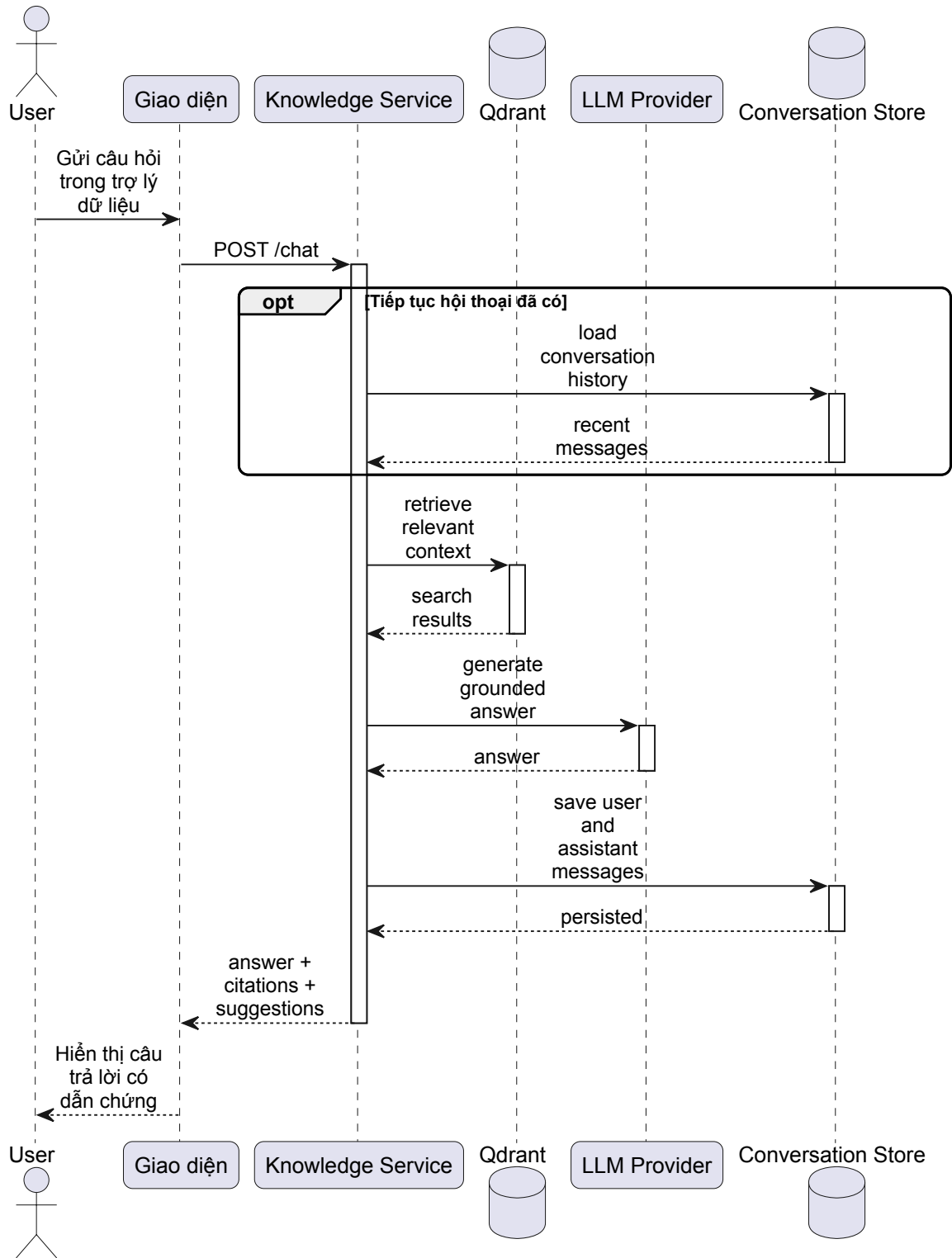
- User mở giao diện tra cứu và nhập truy vấn cùng các bộ lọc cần thiết.
- Giao diện gửi request search đến Knowledge Service kèm campaign, nội dung truy vấn và phạm vi lọc.
- Knowledge Service kiểm tra tính hợp lệ của truy vấn và tìm kết quả cache nếu phù hợp.

- Nếu cần, service resolve campaign sang tập project liên quan, sinh biểu diễn truy vấn và thực hiện truy hồi trên các collection tri thức tương ứng.
- Hệ thống lọc, sắp xếp, loại bỏ các kết quả lặp và tổng hợp metadata cần hiển thị.
- Kết quả tra cứu được trả về cho giao diện để người dùng xem nội dung liên quan, điểm phù hợp và các thông tin bổ trợ.

Điểm quan trọng: Flow này tập trung vào retrieval có ngữ cảnh. Dữ liệu phân tích chỉ được khai thác thông qua lớp tri thức đã được chuẩn bị từ trước, không truy vấn ad-hoc trực tiếp từ các lane xử lý nền.

5.5.3.2 Hỏi đáp có dẫn chứng và lưu hội thoại

Luồng này mô tả cách người dùng gửi câu hỏi trong trợ lý dữ liệu và nhận câu trả lời có dẫn chứng trong cùng ngữ cảnh chiến dịch.



Hình 53: Sequence Diagram UC-03 Part 2: Hỏi đáp có dẫn chứng và lưu hội thoại

Luồng xử lý:

- User gửi câu hỏi mới hoặc tiếp tục một hội thoại đã có trong giao diện trợ lý dữ liệu.
- Giao diện gửi request chat đến Knowledge Service, kèm campaign và conversation_id nếu đang tiếp tục hội thoại.

- Knowledge Service kiểm tra input, nạp hoặc tạo conversation phù hợp và lấy lịch sử trao đổi cần thiết.
- Service thực hiện truy hồi ngữ cảnh liên quan cho câu hỏi hiện tại, sau đó kết hợp context với lịch sử hội thoại để xây prompt.
- Hệ thống gọi mô hình sinh câu trả lời dựa trên prompt đã được grounding bởi kết quả truy hồi.
- Sau khi có câu trả lời, service trích citations, sinh gợi ý tiếp theo và lưu message của user cùng message phản hồi vào persistence layer.
- Giao diện nhận câu trả lời, citations và suggestions để người dùng tiếp tục khai thác dữ liệu phân tích trong cùng ngữ cảnh.

Điểm quan trọng: Flow này không xem mô hình sinh là nguồn dữ liệu chính. Câu trả lời chỉ có giá trị khi được neo vào kết quả truy hồi và được trả lại cùng phần dẫn chứng liên quan.

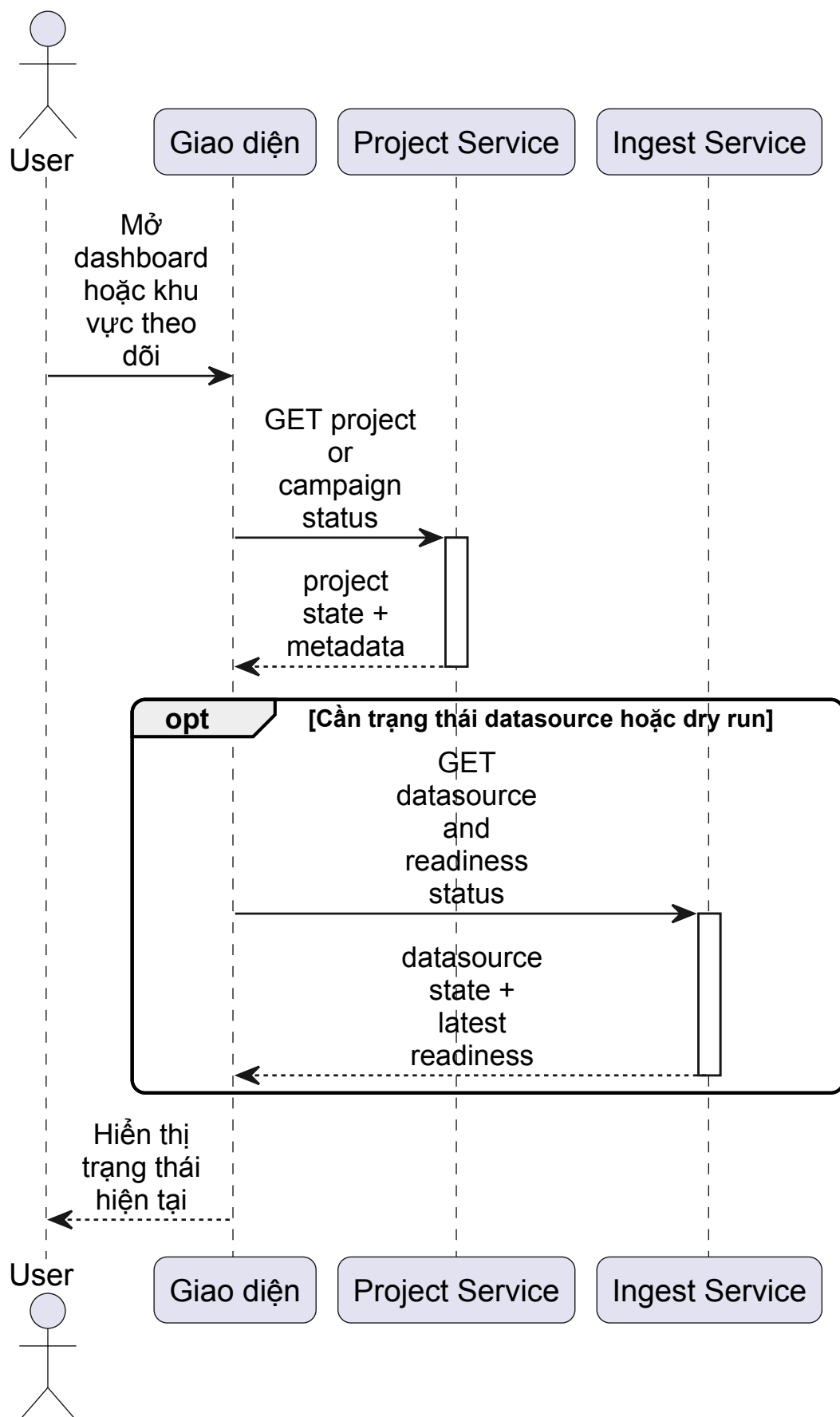
5.5.4 UC-04: Theo dõi trạng thái và nhận cảnh báo

Trong bộ use case hiện tại ở Chương 4, UC-04 được hiểu là mục tiêu giúp người dùng theo dõi trạng thái vận hành của campaign, project hoặc nguồn dữ liệu và kịp thời nhận biết khi có sự kiện hoặc cảnh báo quan trọng. Ở mức sequence, mục tiêu này được cụ thể hóa qua hai interaction flows chính là quan sát trạng thái hiện tại và nhận thông báo hoặc cảnh báo khi hệ thống phát sinh sự kiện liên quan. Bên cạnh đó, mục này bổ sung một supporting technical sequence để làm rõ cách cảnh báo khủng hoảng có thể được tạo ra từ chuỗi crawl và analytics trước khi đi vào notification lane.

Điểm chung của các flow trong mục này là người dùng chỉ quan sát và phản ứng; giao diện không trực tiếp thay đổi trạng thái vận hành. Các thao tác thay đổi vòng đời vẫn thuộc UC-02.

5.5.4.1 Quan sát trạng thái hiện tại

Luồng này mô tả cách người dùng mở khu vực theo dõi và xem trạng thái hiện tại của campaign, project, nguồn dữ liệu hoặc các quá trình xử lý liên quan.



Hình 54: Sequence Diagram UC-04 Part 1: Quan sát trạng thái hiện tại

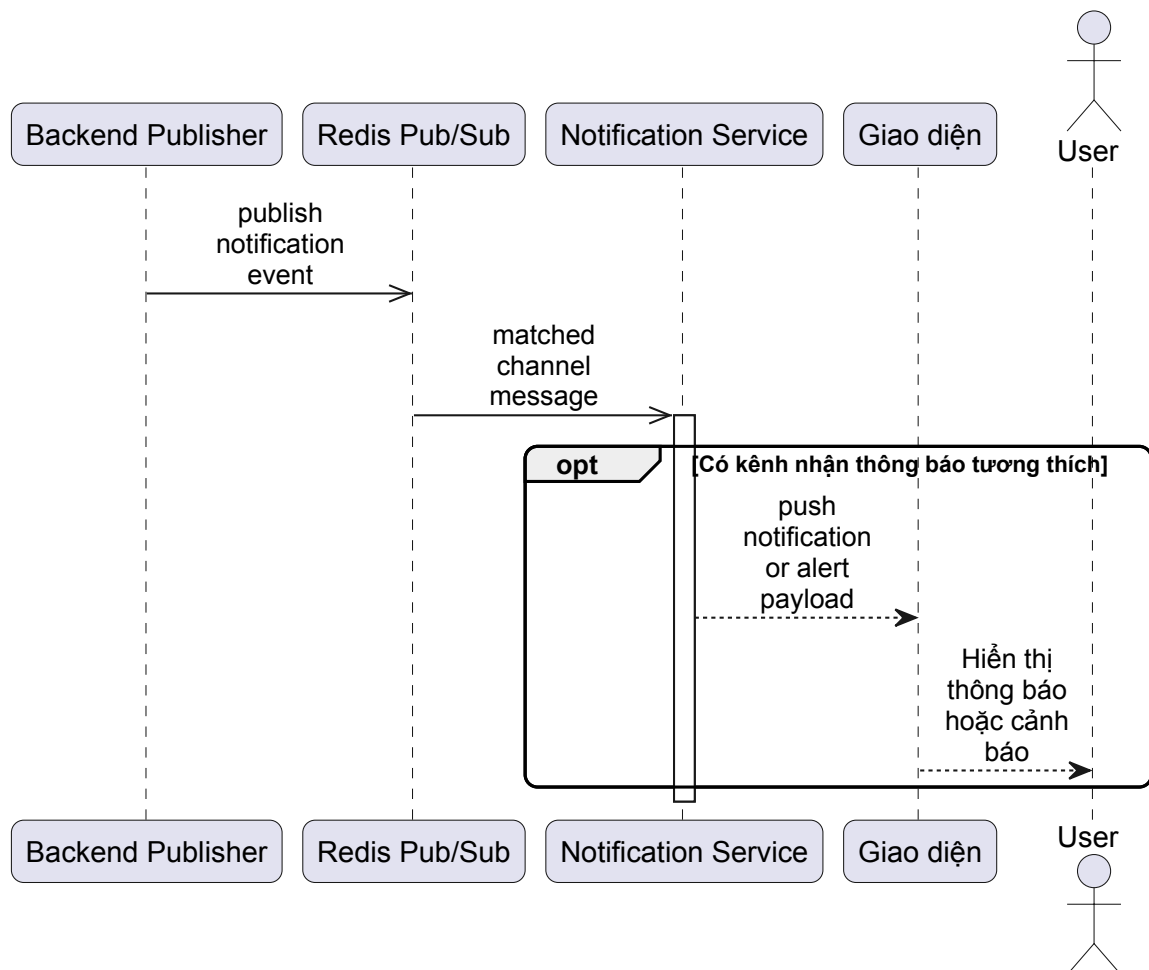
Luồng xử lý:

- User mở dashboard hoặc khu vực theo dõi của một campaign hay project.
- Giao diện gửi request lấy trạng thái hiện tại trong phạm vi user được phép theo dõi.
- Hệ thống kiểm tra quyền truy cập và xác định campaign, project hoặc nguồn dữ liệu nào được phép hiển thị.
- Dịch vụ tương ứng trả về trạng thái hiện tại, thời điểm cập nhật gần nhất và các metadata cần thiết để giao diện biểu diễn ngữ cảnh vận hành.
- Giao diện hiển thị trạng thái hiện tại để user đánh giá và quyết định có cần mở sâu hơn hay thực hiện thao tác ở use case khác hay không.

Điểm quan trọng: Flow này tập trung vào khả năng quan sát trạng thái, không đồng nghĩa rằng hệ thống đang phát sinh cảnh báo mới ở thời điểm user truy cập.

5.5.4.2 Nhận thông báo và cảnh báo

Luồng này mô tả cách hệ thống phân loại sự kiện, route thông báo và hiển thị nội dung cảnh báo cho user trong phạm vi được phép theo dõi.



Hình 55: Sequence Diagram UC-04 Part 2: Nhận thông báo và cảnh báo

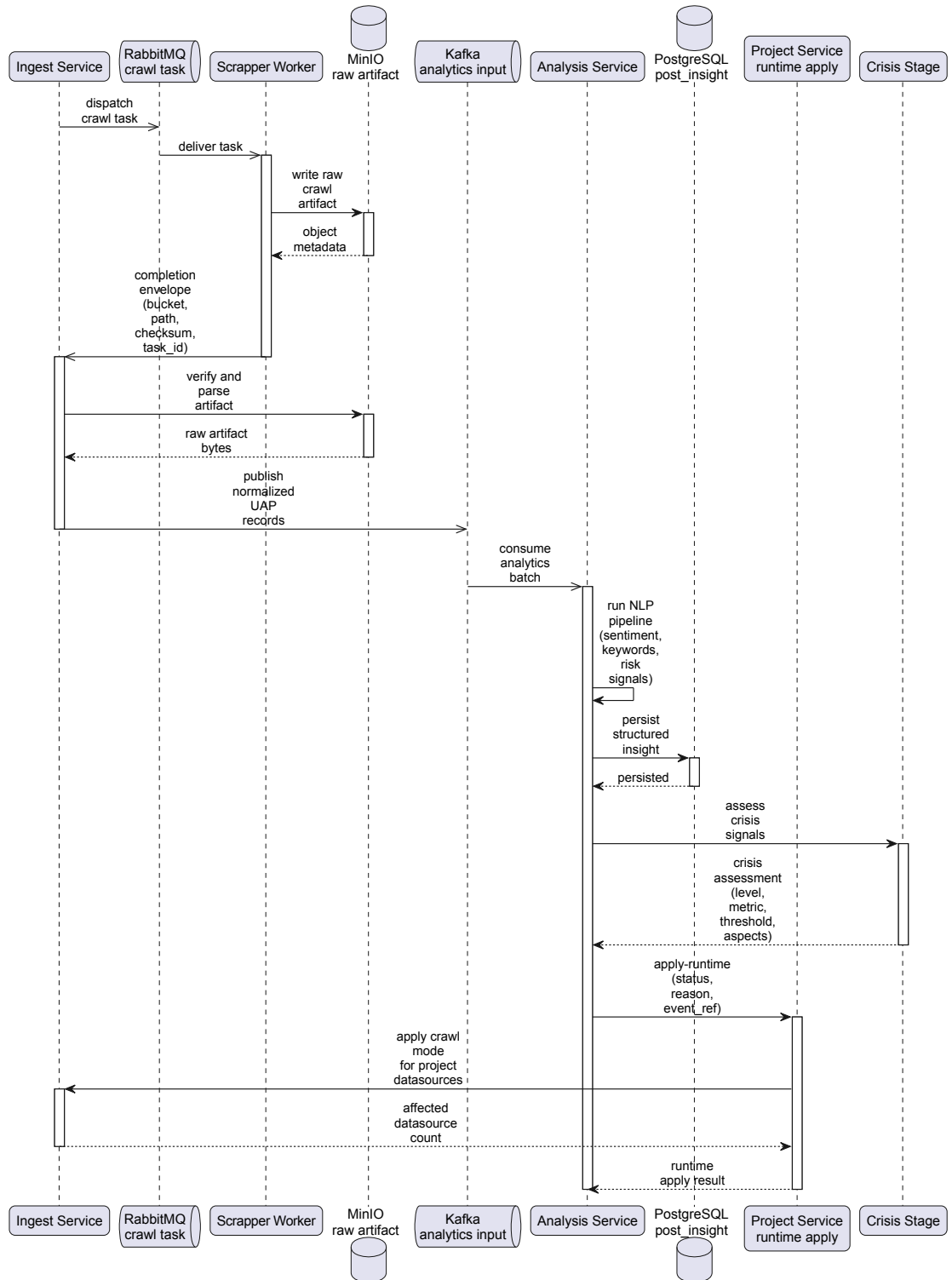
Luồng xử lý:

- Khi có trạng thái mới, lỗi xử lý hoặc cảnh báo nghiệp vụ trong phạm vi theo dõi, backend publisher phát sinh notification event tương ứng.
- Notification Service nhận message đầu vào, parse loại sự kiện và xác định recipient hoặc scope liên quan.
- Hệ thống chuẩn hóa payload cảnh báo, route về connection hub hoặc lane delivery phù hợp nếu có active compatible connection.
- Nếu user đang có phiên nhận thông báo hợp lệ, hệ thống hiển thị thông báo trên giao diện hoặc cập nhật lớp presentation tương ứng.
- User xem nội dung tóm tắt, mở chi tiết cảnh báo hoặc điều hướng đến campaign/project liên quan để tiếp tục đánh giá tình hình.

Điểm quan trọng: Flow này mô tả delivery capability của hệ thống khi có sự kiện và khi có phiên nhận thông báo phù hợp. Nó không mặc định khẳng định mọi loại giao diện trong mọi thời điểm đều đang tiêu thụ trực tiếp cùng một kênh realtime.

5.5.4.3 Supporting flow: phát hiện khủng hoảng từ dữ liệu crawl

Luồng này nói rõ phần xử lý nền đứng phía sau UC-04: dữ liệu được thu thập bởi execution plane, chuẩn hóa và đưa vào analytics pipeline; kết quả phân tích được đánh giá thành crisis assessment; khi runtime auto-apply được bật, analysis lane gọi Project Service để áp dụng trạng thái khủng hoảng xuống crawl mode. Notification delivery vẫn được mô tả riêng ở mục 5.5.4.2 và boundary giao tiếp liên quan ở mục 5.6.3.4.



Hình 56: Sequence Diagram UC-04 Supporting Flow: Crawl, phân tích và đánh giá khủng hoảng

Luồng xử lý:

- Cấu hình cảnh báo khủng hoảng được Project Service quản lý theo project; service này đồng thời cung cấp internal endpoint `apply-runtime` để áp dụng trạng thái khủng hoảng vào vận hành crawl.

- Ingest Service dispatch task thu thập sang RabbitMQ; Scraper Worker thực thi crawl, materialize raw artifact vào MinIO và trả completion envelope về ingest lane.
- Ingest Service verify artifact, parse dữ liệu và publish các bản ghi chuẩn hóa vào Kafka analytics input để analytics pipeline tiêu thụ.
- Analysis Service chạy pipeline NLP, persist insight có cấu trúc và đánh giá các tín hiệu như sentiment, keyword, risk hoặc áp lực volume thành crisis assessment.
- Nếu crisis assessment tạo ra trạng thái cần áp dụng và runtime auto-apply được bật, analysis-consumer gọi Project Service với trạng thái **NORMAL**, **WARNING** hoặc **CRITICAL** cùng reason/correlation metadata.
- Project Service ánh xạ trạng thái đó sang crawl mode phù hợp và cập nhật các datasource liên quan qua ingest lane; notification lane chỉ xử lý delivery khi đã có notification event tương ứng.
- Phần delivery cụ thể của notification không lặp lại trong supporting flow này; nó được mô tả ở flow notification delivery tại mục 5.5.4.2 và communication pattern tại mục 5.6.3.4.

Điểm quan trọng: Flow này mô tả quan hệ nhân quả giữa crawl, phân tích và cảnh báo, nhưng vẫn giữ đúng ranh giới use case. Người dùng không trực tiếp điều khiển analytics pipeline trong UC-04; họ quan sát kết quả cảnh báo được hệ thống phát sinh từ các lane xử lý nền.

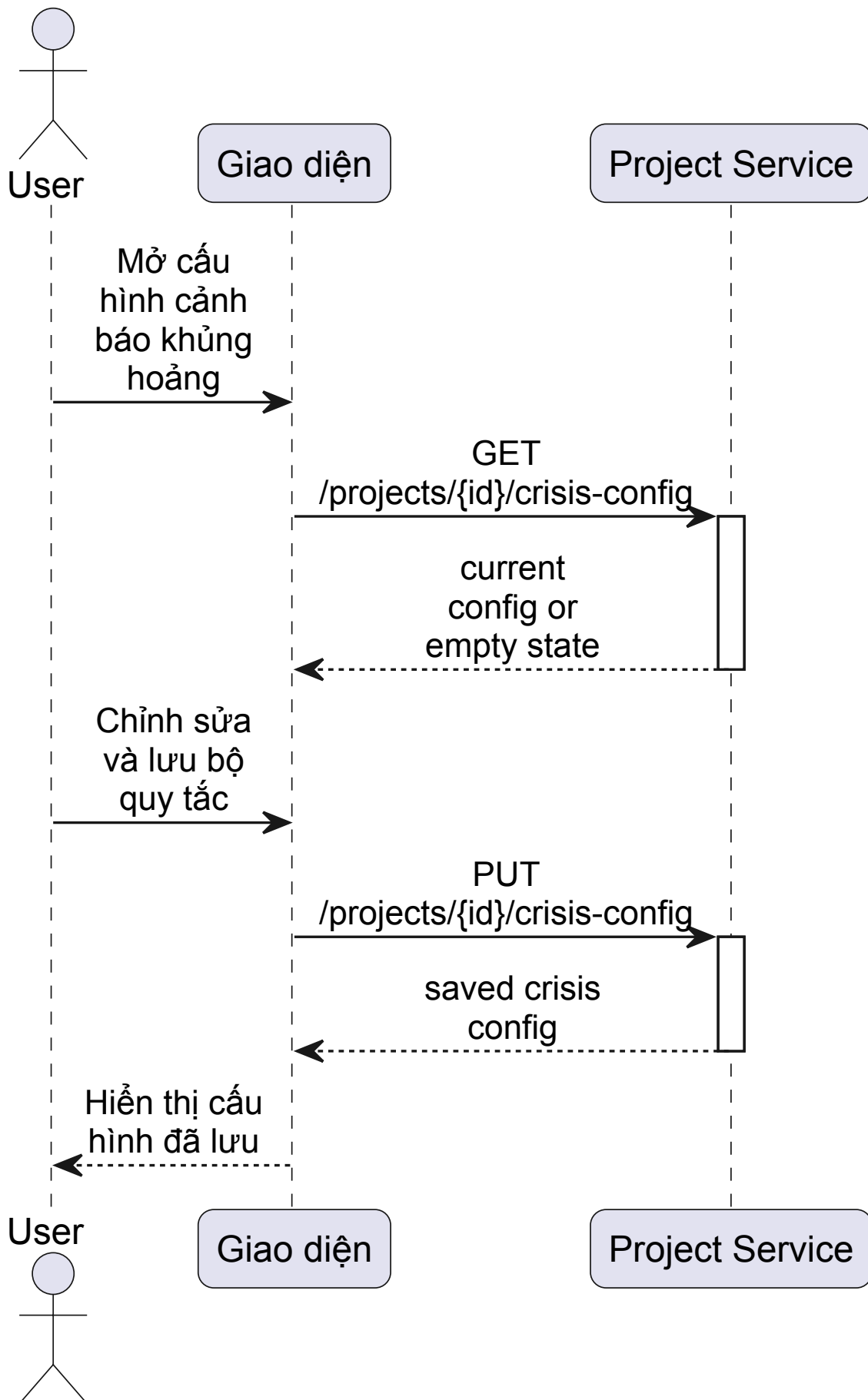
5.5.5 UC-05: Thiết lập và quản lý quy tắc cảnh báo khủng hoảng

Trong bộ use case hiện tại ở Chương 4, UC-05 được hiểu là mục tiêu cho phép người dùng thiết lập, xem, cập nhật hoặc xóa bộ quy tắc cảnh báo khủng hoảng của một project. Ở mức sequence, mục tiêu này được cụ thể hóa qua hai interaction flows chính: xem hoặc lưu bộ quy tắc cảnh báo và xóa cấu hình cảnh báo khi không còn cần sử dụng.

Điểm chung của các flow trong mục này là Project Service giữ ownership của crisis configuration như một phần của business context. Cấu hình được quản lý theo project, còn việc hệ thống sử dụng các quy tắc đó để phát hiện hoặc phát cảnh báo ở các luồng xử lý phía sau, và phần người dùng quan sát các cảnh báo phát sinh thuộc UC-04.

5.5.5.1 Xem và lưu bộ quy tắc cảnh báo khủng hoảng

Luồng này mô tả cách người dùng mở cấu hình crisis của một project, xem trạng thái hiện có và lưu bộ quy tắc mới hoặc cập nhật bộ quy tắc đang áp dụng.



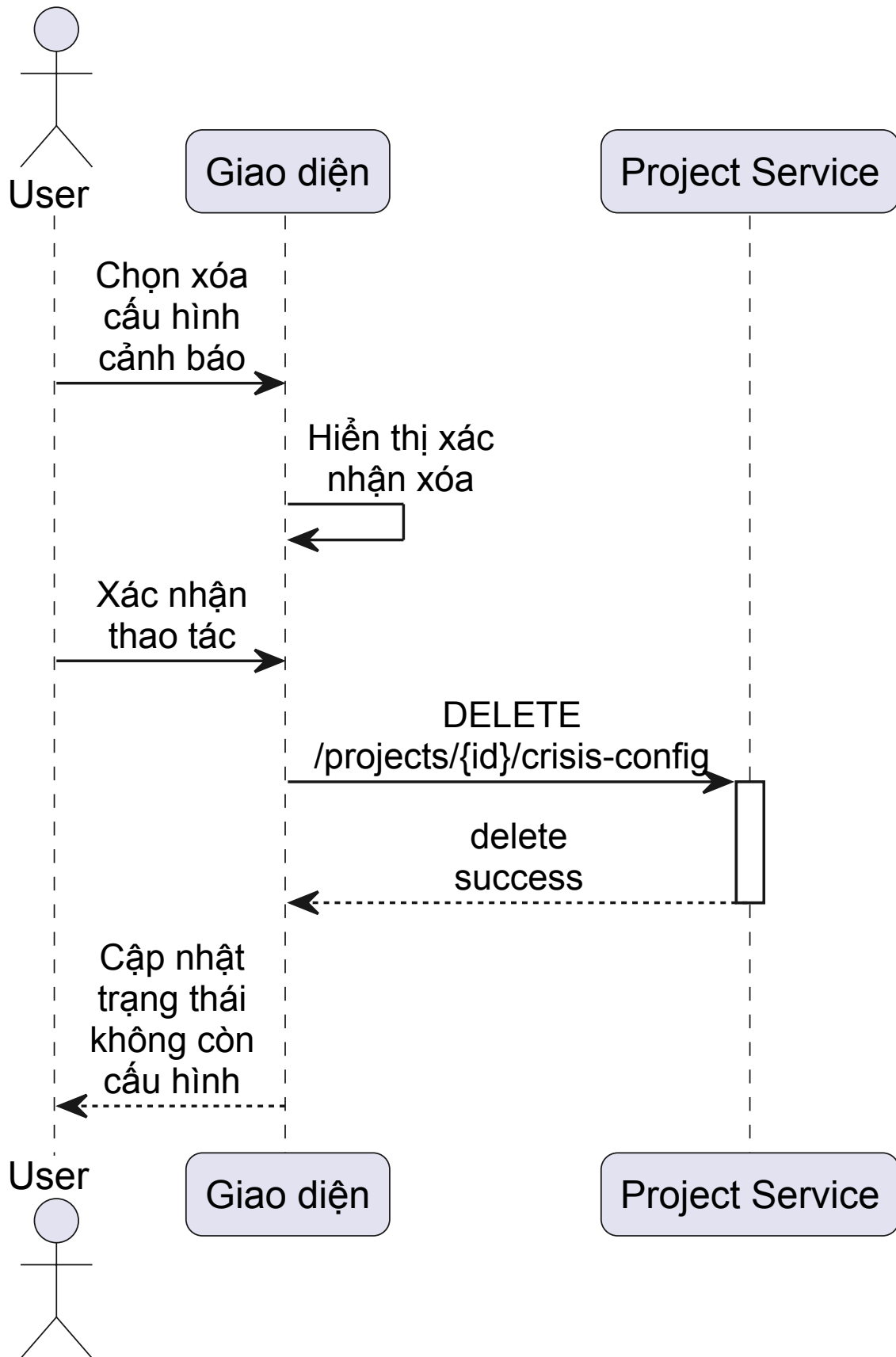
Hình 57: Sequence Diagram UC-05 Part 1: Xem và lưu bộ quy tắc cảnh báo khủng hoảng
Luồng xử lý:

- User mở chức năng cấu hình cảnh báo khủng hoảng của một project.
- Giao diện gửi request lấy cấu hình hiện có để nạp trạng thái ban đầu nếu project đã được thiết lập trước đó.
- Project Service kiểm tra quyền truy cập, project context và trả về bộ cấu hình hiện có hoặc trạng thái chưa có cấu hình.
- User bật một hoặc nhiều nhóm quy tắc như keyword, volume, sentiment hoặc influencer và nhập các điều kiện cảnh báo tương ứng.
- Giao diện gửi request lưu cấu hình mới đến Project Service.
- Service kiểm tra project, quyền thao tác và tính hợp lệ của từng nhóm quy tắc được bật.
- Nếu dữ liệu hợp lệ, service tạo mới hoặc cập nhật crisis configuration gắn với project trong persistence layer.
- Cấu hình đã lưu được trả lại cho giao diện để user xác nhận và tiếp tục theo dõi chiến dịch nếu cần.

Điểm quan trọng: Flow này chỉ quản lý dữ liệu cấu hình cảnh báo. Việc phát hiện khủng hoảng trong runtime không nằm trong sequence của UC-05.

5.5.5.2 Xóa cấu hình cảnh báo khủng hoảng

Luồng này mô tả cách người dùng loại bỏ bộ quy tắc cảnh báo khủng hoảng đang gắn với một project.



Hình 58: Sequence Diagram UC-05 Part 2: Xóa cấu hình cảnh báo khủng hoảng

Luồng xử lý:

- User chọn thao tác xóa cấu hình cảnh báo khủng hoảng trong phạm vi một project.
- Giao diện có thể yêu cầu xác nhận trước khi gửi request xóa.
- Project Service kiểm tra quyền quản lý project và xác nhận rằng project hoặc cấu hình tương ứng tồn tại.
- Nếu request hợp lệ, service xóa bản ghi crisis configuration gắn với project khỏi persistence layer.
- Hệ thống trả kết quả xóa thành công cho giao diện.
- Giao diện cập nhật lại trạng thái hiển thị để phản ánh rằng project không còn bộ quy tắc cảnh báo khủng hoảng đang áp dụng.

Điểm quan trọng: Việc xóa cấu hình chỉ thay đổi dữ liệu cấu hình của project. Nó không tự kích hoạt hành động vận hành khác ngoài việc loại bỏ đầu vào mà các luồng cảnh báo phía sau có thể sử dụng.

5.5.6 Tổng kết

Section này mô tả các sequence diagrams dùng để minh họa cách SMAP hiện thực hóa các mục tiêu nghiệp vụ đã xác định ở Chương 4. Ở mức kỹ thuật, các sơ đồ làm rõ những tương tác quan trọng giữa giao diện, project context, ingest runtime, analytics pipeline, knowledge retrieval và notification delivery.

Bên trong mỗi nhóm mục tiêu nghiệp vụ, một số sơ đồ có thể đi sâu vào các subflow kỹ thuật như readiness validation, dispatch thu thập, xử lý nền, truy hồi tri thức hoặc phát thông báo. Mục tiêu của phần này không phải lặp lại logic use case ở Chương 4, mà là cho thấy các thành phần hệ thống phối hợp với nhau như thế nào để tạo ra kết quả quan sát được đối với người dùng.

5.6 Mẫu giao tiếp và tích hợp

Mục 5.3 đã trình bày chi tiết cấu trúc nội bộ của từng service, còn mục 5.5 đã mô tả các tương tác giữa services theo thời gian qua Sequence Diagrams. Mục này tập trung vào các mẫu giao tiếp và cơ chế tích hợp được sử dụng trong hệ thống SMAP, giải thích tại sao chọn từng pattern và cách chúng hoạt động.

5.6.1 Mẫu giao tiếp

Hệ thống SMAP không sử dụng một cơ chế giao tiếp duy nhất cho toàn bộ kiến trúc. Thay vào đó, các thành phần được kết nối bằng nhiều communication patterns khác nhau tùy theo tính chất của từng lane xử lý: tương tác người dùng cần phản hồi nhanh, control plane nội bộ cần đồng bộ, execution plane cần dispatch bất đồng bộ, analytics downstream cần fanout theo stream, còn notification cần khả năng phát thông báo kịp thời tới các phiên theo dõi phù hợp.

5.6.1.1 Tổng quan các mẫu giao tiếp

Bảng 38: Tổng quan các mẫu giao tiếp trong hệ thống SMAP

Pattern	Loại tương tác	Công nghệ chính	Khi sử dụng	Ví dụ trong hệ thống
Request-response	Giao diện hoặc service gọi trực tiếp và chờ kết quả	HTTP, route handlers, internal HTTP	CRUD, readiness check, search hoặc chat	Giao diện gọi project API; project-srv gọi ingest-srv
Async task dispatch	Execution plane publish task và nhận completion bất đồng bộ	RabbitMQ	Crawl runtime, dry run runtime, completion correlation	ingest-srv → scrapper-srv → ingest completion queues
Event streaming	Downstream fanout và stream processing giữa analytics với knowledge	Kafka	Analytics intake, insights published, batch completed, downstream knowledge materialization	analysis-srv → knowledge-srv
Realtime notification delivery	Notification ingress và route message tới phiên theo dõi phù hợp	Redis Pub/Sub + WebSocket hub	Thông báo trạng thái, campaign event, crisis alert	backend publisher → notification-srv
Artifact reference	Payload lớn được materialize ra object storage, chỉ trao đổi metadata hoặc reference	MinIO + metadata or event reference	Raw crawl artifacts, knowledge-side report artifacts,	raw batch lineage, knowledge report file_url

			batch file inputs	
--	--	--	----------------------	--

Lưu ý: Cột “Công nghệ chính” phản ánh transport hoặc integration mechanism giữ vai trò trung tâm trong kiến trúc của SMAP. Một pattern có thể đi kèm thêm cache, object storage hoặc route handler hỗ trợ, nhưng bảng chỉ liệt kê lớp giao tiếp chính.

5.6.1.2 Synchronous Request-Response Pattern

Synchronous request-response được sử dụng ở những nơi hệ thống cần trả kết quả ngay sau khi kiểm tra quyền truy cập, tính hợp lệ của input hoặc trạng thái hiện tại của đối tượng nghiệp vụ. Pattern này xuất hiện ở cả hai cấp: giữa giao diện với backend và giữa các service trong control plane nội bộ.

Các interaction tiêu biểu:

Bảng 39: Các interaction đồng bộ tiêu biểu

Pattern con	Bên tham gia chính	Transport	Vai trò	Ví dụ
User-facing API call	Giao diện → identity, project, ingest, knowledge	HTTP qua same-origin proxy hoặc route handlers	CRUD, truy vấn, search và chat	Tạo project, tạo datasource, gửi câu hỏi chat
Internal control call	project-srv ↔ ingest-srv	Internal HTTP	Readiness check và lifecycle control	activation-readiness, activate, pause, resume
Knowledge retrieval request	Giao diện → knowledge-srv	HTTP API đã xác thực	Tra cứu và hỏi đáp theo ngữ cảnh	search, chat, conversation detail

Ở phía người dùng, giao diện chủ yếu gọi backend thông qua lớp proxy hoặc route handlers để giữ same-origin behavior cho cookie và giảm CORS complexity. Ở phía nội bộ, internal HTTP được dùng khi business control plane cần phản hồi đồng bộ trước khi thay đổi trạng thái nghiệp vụ, chẳng hạn readiness check và activate hoặc resume ở project-srv.

5.6.1.3 Asynchronous Messaging Pattern

Asynchronous messaging được sử dụng ở các lane mà xử lý không nên nằm trên synchronous request path. Trong kiến trúc của SMAP, hai nhóm bất đồng bộ quan trọng nhất không dùng cùng một transport: RabbitMQ phục vụ execution plane của crawl runtime, còn Kafka phục vụ analytics data plane và downstream fanout sang knowledge lane.

Ở execution plane, RabbitMQ phù hợp với task dispatch và completion correlation:

- ingest-srv publish crawl hoặc dry run task theo queue platform-specific;
- scraper-srv consume task, thực thi runtime và publish completion envelope trở lại;
- ingest-srv correlate completion bằng task_id, object metadata và raw batch lineage.

Ở analytics downstream, Kafka phù hợp với stream fanout và retrieval preparation:

- analysis-srv consume dữ liệu đầu vào đã chuẩn hóa từ analytics data plane;
- analysis-srv publish các outputs downstream như batch completed, insight hoặc digest;
- knowledge-srv consume các topic này để tiếp tục indexing, retrieval và materialization liên quan.

Điểm cốt lõi của pattern bất đồng bộ ở đây là tách control plane khỏi processing plane. Người dùng không chờ trực tiếp quá trình crawl, analytics hay indexing kết thúc trong một HTTP request duy nhất; thay vào đó, trạng thái và kết quả được materialize dần qua các lane runtime tương ứng.

5.6.1.4 Realtime Notification Delivery Pattern

Realtime notification delivery được sử dụng khi hệ thống cần phát thông báo hoặc cảnh báo kịp thời cho user trong phạm vi được phép theo dõi. Trong SMAP, pattern này được tổ chức thành hai lớp: Redis Pub/Sub làm notification ingress và Notification Service làm delivery boundary.

Luồng delivery chính:

- backend publisher phát message vào Redis channel theo user scope, project scope, campaign scope hoặc system scope;
- notification-srv subscribe các channel phù hợp, parse message type và route theo recipient hoặc broadcast scope;
- nếu có active compatible connection, message được đẩy vào WebSocket hub để giao diện hoặc client tương thích nhận và hiển thị.

Vì connection là stateful, Redis Pub/Sub giúp nhiều notification instances nhìn thấy cùng một notification ingress stream. Tuy nhiên, pattern này chỉ mô tả delivery capability ở phía hệ thống; nó không mặc định khẳng định mọi loại giao diện đều tiêu thụ trực tiếp cùng một kênh realtime ở mọi tình huống.

Ở lớp giao diện, notification presentation vẫn có lớp state riêng. Vì vậy, pattern này phản ánh delivery semantics của notification boundary nhiều hơn là một ràng buộc bắt buộc cho mọi client.

5.6.1.5 Artifact Reference Pattern

Artifact reference pattern được dùng khi payload hoặc output không phù hợp để truyền trực tiếp qua message body hoặc lưu hoàn toàn trong relational row. Thay vì truyền nguyên dữ liệu lớn qua queue, hệ thống materialize artifact ra object storage và chỉ trao đổi metadata hoặc reference cần thiết giữa các lane xử lý.

Trong SMAP, pattern này xuất hiện ở hai nơi chính:

- scrapper runtime materialize raw output vào local filesystem ở development mode hoặc MinIO ở production mode; ingest-srv chỉ nhận storage bucket, storage path, checksum, batch_id và completion metadata để tiếp tục xử lý;
- knowledge-side report generation, như một capability mở rộng của knowledge layer ngoài bộ use case cốt lõi ở Chương 4, lưu artifact báo cáo ở object storage, còn PostgreSQL chỉ giữ report metadata, status và file URL.

Lợi ích của cách làm này là giữ queue message và relational metadata ở kích thước hợp lý, đồng thời vẫn bảo toàn được lineage giữa task, artifact và kết quả downstream. Pattern này đồng bộ với cách mục 5.4 mô tả storage stack và artifact lineage, nên không cần giả định benchmark cứng hay contract dữ liệu nên khi chưa có cơ sở thực nghiệm hoặc mô tả thiết kế tương ứng.

5.6.2 RabbitMQ cho execution plane

Trong kiến trúc của SMAP, RabbitMQ không đóng vai trò event bus trung tâm cho toàn bộ hệ thống. Phạm vi của nó hẹp hơn và rõ ràng hơn: làm lớp tích hợp bất đồng bộ giữa ingest-srv và scrapper-srv cho hai nhu cầu chính là dispatch crawl task theo nền tảng và nhận completion envelope để correlate kết quả xử lý.

5.6.2.1 Topology dispatch và completion

Topology RabbitMQ của SMAP được tổ chức theo từng lane nghiệp vụ cụ thể thay vì một exchange tổng quát cho mọi business event. Task dispatch đi qua direct exchange riêng cho từng nền tảng, còn completion được publish trở lại các durable queue để ingest-srv consume theo lane execution hoặc dry run.

Bảng 40: Topology RabbitMQ trong execution plane

Tên	Loại	Publisher chính	Consumer chính	Vai trò
-----	------	--------------------	-------------------	---------

ingest_tiktok_tasks_exc	Direct exchange	ingest-srv	scraper-srv qua tiktok_tasks	Route task TikTok vào queue xử lý tương ứng
ingest_facebook_tasks_exc	Direct exchange	ingest-srv	scraper-srv qua facebook_tasks	Route task Facebook vào queue xử lý tương ứng
ingest_youtube_tasks_exc	Direct exchange	ingest-srv	scraper-srv qua youtube_tasks	Route task YouTube vào queue xử lý tương ứng
tiktok_tasks	Durable queue	ingest-srv	scraper-srv	Hàng đợi task TikTok dùng chung cho execution và dry run
facebook_tasks	Durable queue	ingest-srv	scraper-srv	Hàng đợi task Facebook dùng chung cho execution và dry run
youtube_tasks	Durable queue	ingest-srv	scraper-srv	Hàng đợi task YouTube dùng chung cho execution và dry run
ingest_task_completions	Durable queue	scraper-srv	ingest-srv execution consumer	Nhận completion cho execution lane và correlate theo task_id
ingest_dryrun_completions	Durable queue	scraper-srv	ingest-srv dryrun consumer	Nhận completion cho dry run lane để cập nhật readiness và kết quả kiểm thử

Một số điểm quan trọng của topology này:

- Mỗi nền tảng có một direct exchange riêng, và routing key trùng với tên queue đích tương ứng.
- Execution lane và dry run lane dùng chung các task queue theo nền tảng; việc tách hai lane xảy ra ở completion path dựa trên metadata runtime_kind.
- Completion envelope được scraper-srv publish trực tiếp vào queue đích bằng queue name, thay vì đi qua một exchange tổng quát cho mọi completion event.

- Cả queue lẫn message đều được cấu hình theo hướng durable và persistent để hỗ trợ xử lý bất đồng bộ ổn định hơn qua các lần restart thông thường.

5.6.2.2 Delivery semantics và xử lý lỗi

Trong hiện thực của SMAP, delivery semantics ở RabbitMQ được giữ theo hướng đơn giản nhưng rõ ràng ở phía consumer thay vì dựa vào một broker-side retry fabric phức tạp. ingest-srv consume completion queues với manual acknowledgment và phân biệt rõ giữa payload không hợp lệ về mặt nghiệp vụ với lỗi xử lý tạm thời.

Bảng 41: Hành vi xử lý completion message ở ingest-srv

Tình huống	Hành vi consumer	Ý nghĩa
Completion hợp lệ và xử lý thành công	Ack	Kết thúc vòng đời message sau khi ingest-srv đã correlate và cập nhật trạng thái tương ứng
Payload không parse được hoặc JSON không hợp lệ	Ack và discard	Tránh để poison message bị redelivery lặp lại khi bản thân payload đã hỏng
task_id không tồn tại hoặc completion input không hợp lệ	Ack và discard	Không retry các message sai ngữ nghĩa nghiệp vụ vì khả năng thành công lại gần như không tăng
Lỗi xử lý tạm thời ở use case hoặc hạ tầng phụ trợ	Nack với requeue	Cho phép RabbitMQ giao lại message để completion được thử xử lý lại ở lượt sau

Thiết kế này cho thấy fault handling ở lane này chủ yếu nằm ở application consumer logic: message hợp lệ sẽ được correlate theo task_id cùng storage metadata, message sai ngữ nghĩa sẽ bị loại bỏ có chủ đích, còn lỗi tạm thời sẽ được trả lại queue để redelivery. Vì vậy, mục này không giả định sẵn một DLX, DLQ riêng hay exponential backoff cố định nếu thiết kế và hiện thực của SMAP chưa thể hiện rõ các cơ chế đó.

5.6.3 Giao tiếp thời gian thực

Trong SMAP, giao tiếp thời gian thực được tổ chức quanh notification-srv như một delivery boundary chuyên biệt. Backend publishers không gửi trực tiếp message tới từng phiên người dùng; thay vào đó, chúng publish notification vào Redis Pub/Sub, còn notification-srv subscribe ingress stream này, chuẩn hóa payload và đẩy kết quả ra WebSocket cho các kết nối đang hoạt động.

5.6.3.1 Redis ingress và routing theo scope

Vì WebSocket connection là stateful và gắn với từng instance cụ thể, notification-srv cần một ingress layer dùng chung để mọi instance đều quan sát cùng một notification stream. Redis Pub/Sub đáp ứng vai trò này bằng pattern subscription trên các channel scope hóa theo user, project, campaign hoặc system.

Bảng 42: Các channel pattern của notification ingress

Channel pattern	Scope	Loại message tiêu biểu	Ý nghĩa delivery
project:*.user:*	Project + user	DATA_ONBOARDING, ANALYTICS_PIPELINE	Thông báo tiến độ onboarding hoặc analytics gắn với một project nhưng nhắm đến user cụ thể
campaign:*.user:*	Campaign + user	CAMPAIGN_EVENT	Thông báo lifecycle hoặc artifact event của campaign cho user liên quan
alert:*.user:*	Alert + user	CRISIS_ALERT	Cảnh báo khủng hoảng hoặc alert chuyên biệt gửi đến user mục tiêu
system:*	System-wide	SYSTEM	Thông báo broadcast không ràng buộc vào một user cụ thể

Luồng xử lý ở notification-srv gồm các bước sau:

- mọi instance của notification-srv pattern-subscribe các channel trên;
- khi nhận message, service parse channel để xác định scope, entity_id, subtype và user_id đích nếu có;
- payload được nhận diện loại message dựa trên cấu trúc dữ liệu, sau đó transform thành WebSocket envelope thống nhất;
- hub trong bộ nhớ gửi message đến tất cả active connections của user tương ứng, hoặc broadcast nếu channel thuộc system scope.

Điểm quan trọng ở đây là routing lõi của notification lane mang tính user-centric. Dù channel name mang ngữ nghĩa project hoặc campaign, hub chủ yếu index connection

theo `user_id`. Vì vậy, mục này mô tả delivery semantics theo user-targeted notification và system broadcast, không giả định cơ chế subscription động theo từng project ở mọi client.

5.6.3.2 Thiết lập kết nối và vòng đời phiên

Trong hiện thực của SMAP, WebSocket được xác thực ngay tại HTTP upgrade boundary thay vì dùng một auth frame sau khi kết nối đã mở. Client mở `GET /ws` và cung cấp JWT theo một trong hai cách:

- cookie `smap_auth_token`, là cách dùng chính khi giao diện chạy cùng hệ xác thực;
- query parameter `token`, chủ yếu phù hợp cho debug hoặc các client tích hợp đơn giản.

Notification Service verify token trước khi nâng cấp kết nối và trích xuất `user_id` từ JWT payload. Sau khi upgrade thành công, connection được đăng ký vào hub trong bộ nhớ theo `user_id` và bắt đầu hai vòng lặp nội bộ:

- `readPump` chủ yếu giữ connection sống, nhận close hoặc control frames và hiện chưa định nghĩa business message riêng từ client lên server;
- `writePump` chịu trách nhiệm đẩy notification ra ngoài và phát WebSocket ping định kỳ;
- `pong` được dùng để gia hạn read deadline, qua đó phát hiện stale connection và cleanup khi peer không còn phản hồi.

Thiết kế này cho thấy realtime lane thiên về server-to-client push. Các message nghiệp vụ được đẩy từ backend qua Redis ingress rồi ra WebSocket, còn phía client không cần thực hiện thêm bước subscribe hay gửi auth envelope sau khi connection đã được chấp nhận.

5.6.3.3 Notification envelope và message families

Tất cả WebSocket frames gửi tới client đều dùng cùng một envelope gồm ba thành phần: `type`, `timestamp` và `payload`. Cách đóng gói thống nhất này giúp frontend hoặc client tương thích xử lý nhiều nhóm thông báo khác nhau mà không cần thay đổi transport contract.

Bảng 43: Các loại WebSocket messages

Loại	Hướng	Mô tả
DATA_ONBOARDING	Server → Client	Thông báo trạng thái và kết quả onboarding dữ liệu theo source hoặc project context
ANALYTICS_PIPELINE	Server → Client	Cập nhật tiến độ pipeline analytics với phase, progress và các bộ đếm xử lý

CRISIS_ALERT	Server → Client	Cảnh báo khủng hoảng với severity, metric, threshold và các affected aspects
CAMPAIGN_EVENT	Server → Client	Thông báo các sự kiện liên quan đến campaign như hoàn tất xử lý hoặc artifact đã sẵn sàng
SYSTEM	Server → Client	Thông báo broadcast hoặc maintenance event ở phạm vi toàn hệ thống

Ngoài WebSocket delivery, một số message families như `CRISIS_ALERT`, `DATA_ONBOARDING` và `CAMPAIGN_EVENT` còn có thể được chuyển tiếp sang alert dispatch layer của notification-srv cho các kênh ngoài trình duyệt như Discord. Tuy vậy, realtime contract ở đây vẫn được giữ thống nhất dưới cùng một WebSocket envelope, giúp notification boundary tách rời khỏi logic hiển thị cụ thể ở từng client.

5.6.3.4 Crisis runtime path và notification boundary

Trong luồng khủng hoảng, notification lane không tự phát hiện khủng hoảng mà chỉ đóng vai trò delivery boundary khi có notification event. Việc đánh giá bắt đầu từ analytics lane: dữ liệu crawl đã được ingest-srv chuẩn hóa và publish vào Kafka analytics input, sau đó `analysis-consumer` xử lý NLP, persist insight và tạo crisis assessment từ các tín hiệu phân tích.

Trong hiện thực runtime đã kiểm chứng, khi crisis assessment tạo ra trạng thái cần áp dụng và runtime auto-apply được bật, `analysis-consumer` ánh xạ level sang status `NORMAL`, `WARNING` hoặc `CRITICAL`, rồi gọi internal API `project-srv` tại `/api/v1/internal/projects/{project_id}/crisis-config/apply-runtime`. Project Service giữ ownership của crisis configuration/runtime policy, ánh xạ status sang crawl mode và cập nhật ingest lane cho các datasource liên quan. Đây là runtime adaptation path đã được đối chiếu bằng response API, datasource state, audit table và log consumer.

Notification-srv vẫn hỗ trợ delivery family `CRISIS_ALERT` qua Redis Pub/Sub và WebSocket. Tuy nhiên, đoạn này chỉ nên được hiểu là contract delivery khi một publisher phát sinh alert event phù hợp, không khẳng định `analysis-srv` luôn publish trực tiếp `CRISIS_ALERT` vào Redis trong runtime hiện tại. Payload của một alert event cần giữ đủ thông tin để notification-srv không phải truy vấn ngược vào analytics lane trên request path, bao gồm các nhóm trường như:

- `project_id` hoặc `campaign_id` để gắn cảnh báo với ngữ cảnh nghiệp vụ;
- `severity` để biểu diễn mức độ nghiêm trọng;
- `metric` và `threshold` để chỉ ra điều kiện bị vượt ngưỡng;

- `affected_aspects` hoặc nhóm keyword/sentiment/influencer liên quan để user hiểu nguyên nhân cảnh báo;
- `triggered_at` và correlation metadata để hỗ trợ truy vết giữa analytics output và notification event.

Sau khi nhận message từ Redis, notification-srv chỉ thực hiện các bước thuộc delivery lane: parse channel, nhận diện `CRISIS_ALERT`, chuẩn hóa WebSocket envelope, route theo `user_id` hoặc broadcast scope phù hợp, và chuyển tiếp sang alert dispatch layer nếu message thuộc nhóm cần gửi qua kênh ngoài trình duyệt như Discord. Cách tách này giúp analytics/project lane giữ trách nhiệm đánh giá và áp dụng runtime policy, còn notification-srv giữ trách nhiệm phân phối cảnh báo tới phiên người dùng.

Điểm cần lưu ý là realtime ở đây chỉ áp dụng cho đoạn delivery sau khi alert event đã được phát sinh. Độ trễ end-to-end của cảnh báo vẫn phụ thuộc vào nhịp crawl, thời điểm completion được ingest-srv xử lý, cadence của analytics consumer và tình trạng kết nối của user tại notification-srv.

5.6.4 Giám sát hệ thống

Trong thiết kế và hiện thực của SMAP đã có các khối observability cốt lõi, nhưng mức độ hoàn thiện chưa đồng đều giữa mọi service. Phần rõ ràng nhất hiện nay gồm logging có ngữ cảnh, health hoặc readiness probes ở các API services chính, `trace_id` propagation xuyên qua HTTP và RabbitMQ, cùng một số metric domain-specific ở analysis-srv. Vì vậy, mục này mô tả các cơ chế observability đang được tổ chức trong hệ thống thay vì giả định một monitoring surface hoàn toàn đồng nhất cho toàn bộ hệ thống.

5.6.4.1 Logging và ngữ cảnh chẩn đoán

Logging trong SMAP được tổ chức theo shared abstractions thay vì để từng service tự định nghĩa hoàn toàn độc lập. Ở Go services, lớp logger dùng Zap và có thể xuất console hoặc JSON tùy cấu hình. Ở Python runtime như scrapper-srv, Loguru được dùng với patcher để chèn `trace_id` vào record và cũng tách riêng giữa chế độ production với development.

Bảng 44: Cơ chế logging hiện có theo nhóm runtime

Nhóm service	Library	Hành vi output	Đặc điểm chẩn đoán
Go API services	Zap	Console hoặc JSON tùy cấu hình	Có thể enrich theo context với <code>trace_id</code> , <code>user_id</code> ; middleware HTTP phân mức log theo status code và bỏ qua health endpoints

Python runtime services	Loguru	JSON trong production, dễ đọc hơn trong chế độ chẩn đoán	trace_id được gắn vào record; scrapper-srv còn intercept logging chuẩn của framework để gom về cùng logger
-------------------------	--------	--	--

Một số quy ước logging hiện có đáng chú ý:

- HTTP middleware ở Go services log 2xx/3xx ở mức **INFO**, 4xx ở mức **WARN**, và 5xx ở mức **ERROR**.
- Các probe như `/health`, `/ready`, `/live` được bỏ khỏi access logging để tránh làm nhiều operational logs.
- Trong production mode, logger của scrapper-srv chuẩn hóa record thành các trường như timestamp, trace_id, level, caller, message và service.

Thiết kế này cho phép truy vết theo yêu cầu xử lý và theo ngữ cảnh người dùng tốt hơn so với logging thuần văn bản, nhưng điều đó không có nghĩa mọi service đều luôn chạy ở cùng một encoding hoặc cùng một backend log collector.

5.6.4.2 Health và readiness probes

Phần lớn Go API services trong SMAP đều expose bộ probe `health`, `ready` và `live`, nhưng nội dung kiểm tra cụ thể khác nhau theo dependency footprint của từng service. Ngược lại, Python runtime như scrapper-srv hiện chỉ expose một endpoint `health` đơn giản hơn.

Bảng 45: Probe surface hiện có theo service

Service	Endpoints	Readiness hoặc health chính	Ghi chú
identity-srv	<code>/health</code> , <code>/ready</code> , <code>/live</code>	<code>/ready</code> ping PostgreSQL	Phù hợp cho API auth và session lane
project-srv	<code>/health</code> , <code>/ready</code> , <code>/live</code>	<code>/ready</code> ping PostgreSQL	Probe đơn giản, tập trung vào dependency quan trọng nhất của control plane
knowledge-srv	<code>/health</code> , <code>/ready</code> , <code>/live</code>	<code>/ready</code> ping PostgreSQL và Redis	Phản ánh retrieval lane phụ thuộc cả relational store lẫn cache

ingest-srv	/health, /ready, /live	/ready yêu cầu PostgreSQL và Redis; đồng thời report trạng thái MinIO, Kafka, RabbitMQ	Probe giàu thông tin nhất trong nhóm Go services của hệ thống
notification-srv	/health, /ready, /live	/health và /ready ping Redis; /health trả thêm hub stats	Cho thấy trạng thái delivery boundary và số connection đang hoạt động
scraper-srv	/health	/health trả worker_active cùng một số config hints	Chưa có bộ ready hoặc live tách riêng như các Go API services

Các probe này đủ để phục vụ triển khai có orchestrator hoặc gateway health monitoring, đồng thời phản ánh mức độ phụ thuộc và độ sâu kiểm tra khác nhau giữa các service hơn là một contract hạ tầng hoàn toàn chuẩn hóa.

5.6.4.3 Metric instrumentation hiện có

Surface `/metrics` trong hệ thống chưa được chuẩn hóa đồng nhất giữa mọi service. Tuy nhiên, `analysis-srv` đã có lớp metric instrumentation dùng Prometheus primitives với graceful fallback sang no-op khi thư viện metrics chưa được cài hoặc chưa được bật trong môi trường chạy.

Bảng 46: Metric instrumentation tiêu biểu trong hệ thống

Service	Metric	Loại	Nhãn
analysis-srv	analysis_pipeline_runs_total	Counter	status
analysis-srv	analysis_stage_duration_seconds	Histogram	stage
analysis-srv	analysis_kafka_publish_total	Counter	topic, status
analysis-srv	analysis_crisis_level	Gauge	project_id

Ngoài các metric object này, `notification-srv` trả thêm `active_connections` và `total_unique_users` qua endpoint `health`, cho thấy một số operational stats đang được surfacing qua probe response thay vì qua metric endpoint chuyên dụng. Điều này củng cố nhận định rằng lớp observability của hệ thống vẫn đang phát triển theo từng lane cụ thể, chưa phải một platform metrics đồng nhất hoàn toàn.

5.6.4.4 Trace ID propagation

SMAP hiện implement mức tracing cơ bản dựa trên X-Trace-Id thay vì một distributed tracing stack đầy đủ theo span. Dù vậy, cơ chế này đã đủ để nối một số hop quan trọng trong control plane và execution plane:

- Go API services dùng middleware tracing để nhận hoặc tạo X-Trace-Id, gắn nó vào request context và echo lại trên response header.
- Shared HTTP client trong Go có thể inject X-Trace-Id vào outbound requests, giúp duy trì correlation khi một service gọi sang service khác.
- Shared RabbitMQ layer chèn X-Trace-Id vào message headers khi publish nếu context mang trace_id.
- scrapper-srv trích xuất X-Trace-Id từ inbound HTTP requests và từ RabbitMQ message headers, sau đó tiếp tục dùng cùng trace_id khi publish completion envelope trở lại queue.

Với mức propagation này, việc chẩn đoán liên service chủ yếu vẫn dựa trên log correlation hơn là trên trace visualization. Hệ thống chưa thể hiện đầy đủ span model hoặc backend như Jaeger hay OpenTelemetry collector trong hiện thực của SMAP.

5.6.5 Tổng kết

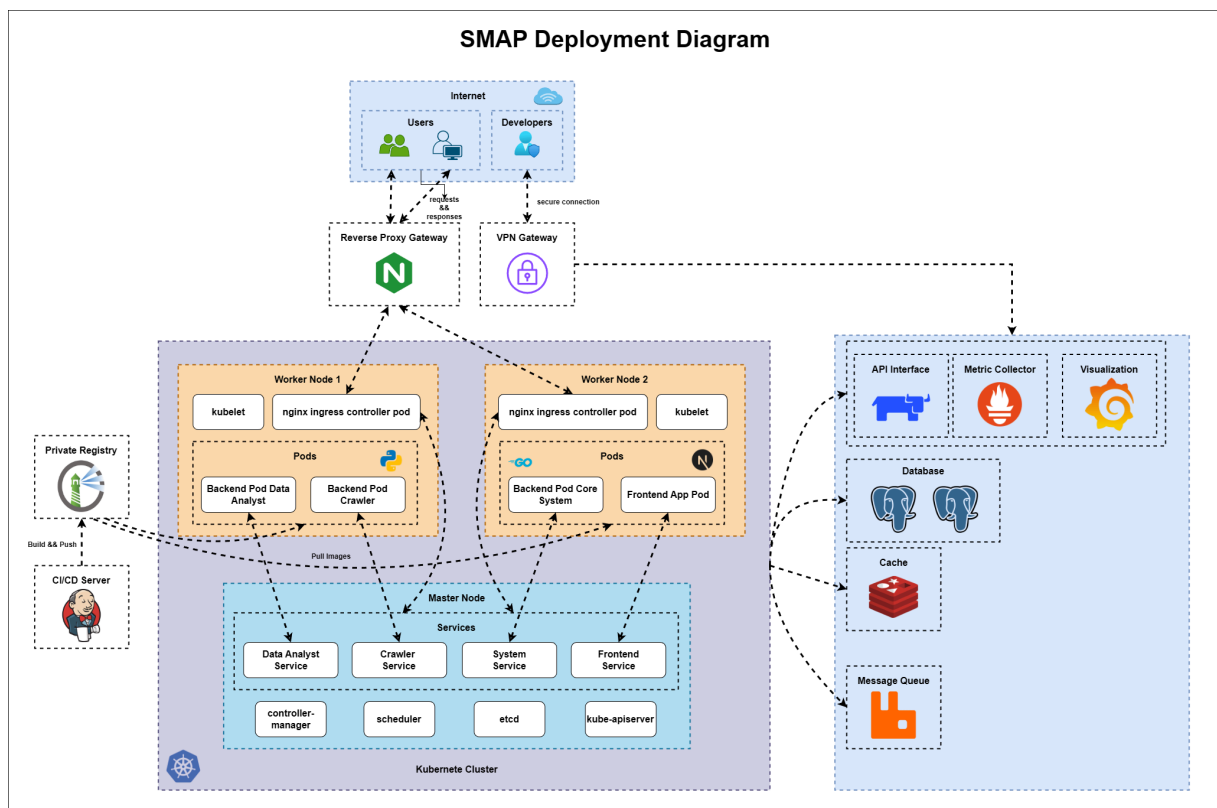
- Hệ thống sử dụng nhiều communication patterns theo từng lane xử lý: request-response cho CRUD, control và retrieval; RabbitMQ cho execution task dispatch; Kafka cho analytics downstream; Redis Pub/Sub kết hợp WebSocket cho notification delivery; và artifact reference cho raw crawl artifacts cùng knowledge-side report artifacts, trong đó nhóm report thuộc capability mở rộng của knowledge layer ngoài bộ use case cốt lõi ở Chương 4.
- RabbitMQ trong kiến trúc của SMAP chỉ đóng vai trò execution-plane broker giữa ingest-srv và scrapper-srv. Task được route theo exchange hoặc queue platform-specific, còn completion được correlate lại ở ingest lane bằng task_id và metadata kèm theo.
- Realtime notification được tổ chức theo user-targeted delivery boundary. notification-srv xác thực kết nối ngay tại HTTP upgrade, subscribe Redis ingress theo các channel pattern định trước, rồi đẩy các WebSocket envelope thống nhất ra các phiên đang hoạt động.
- Lớp observability hiện có đủ cho vận hành cơ bản: logging có ngữ cảnh, probe endpoints ở các API services chính, trace_id propagation qua HTTP và RabbitMQ, cùng metric instrumentation bước đầu ở analysis-srv. Tuy nhiên, monitoring surface vẫn chưa hoàn toàn đồng nhất giữa mọi runtime và vẫn còn dư địa để mở rộng trong các vòng hoàn thiện tiếp theo.

5.7 Thiết kế triển khai hệ thống

Phần này trình bày thiết kế triển khai hệ thống SMAP lên môi trường production, bao gồm kiến trúc hạ tầng, mô hình container hóa, quy trình CI/CD và các chiến lược hỗ trợ tăng cường tính sẵn sàng của hệ thống.

5.7.1 Mô hình triển khai

Hệ thống SMAP được triển khai theo mô hình Kubernetes-based Microservices Architecture, trong đó các services được đóng gói thành containers và orchestrate bởi Kubernetes cluster. Ở góc nhìn triển khai, các trách nhiệm API, scheduler, consumer và worker được tách thành các pod riêng khi cần, giúp mỗi workload có vòng đời scale, restart và rollout độc lập hơn.



Hình 59: Sơ đồ triển khai hệ thống SMAP

Kiến trúc triển khai được tổ chức thành các tầng sau:

- Tầng Ingress: Traefik Gateway đóng vai trò load balancer và reverse proxy, xử lý TLS termination và routing requests đến các services tương ứng.
- Tầng Frontend: web-ui được triển khai như một pod giao diện và backend-for-frontend, phục vụ giao diện người dùng và gọi API đến backend services.
- Tầng API Pods: Các pod phục vụ HTTP hoặc WebSocket bao gồm identity-api, project-api, ingest-api, analysis-api, knowledge-api, notification-delivery và scrapper-api.

- Tầng Runtime Pods: Các pod nền bao gồm identity-audit-consumer, project-consumer, ingest-scheduler, ingest-completion-consumer, analysis-consumer, knowledge-consumer và scrapper-worker.
- Tầng Infrastructure: Các dịch vụ hạ tầng bao gồm PostgreSQL cho lưu trữ dữ liệu quan hệ, Redis cho caching và pub/sub, RabbitMQ cho task queue, Kafka cho analytics data plane, MinIO cho object storage và Qdrant cho vector retrieval.

5.7.2 Cấu hình phần cứng và phần mềm

Kubernetes cluster được thiết kế với cấu hình phần cứng phù hợp cho workload của hệ thống SMAP.

Bảng 47: Cấu hình phần cứng Kubernetes Cluster

Loại Node	Số lượng	CPU	RAM	Storage
Master Nodes	3	4 cores	8GB	100GB SSD
Worker Nodes	5	8 cores	16GB	200GB SSD
AI/ML Nodes	2	16 cores	32GB	500GB SSD

Phần mềm được sử dụng trong môi trường triển khai bao gồm:

- Hệ điều hành: Ubuntu 20.04 LTS
- Container Runtime: Docker 24.0 trở lên
- Kubernetes: phiên bản 1.28 trở lên
- Ingress Gateway: Traefik
- Container Registry: Harbor Registry tại registry.tantai.dev

5.7.3 Cấu hình mạng và bảo mật

Hệ thống được cấu hình với domain smap-api.tantai.dev cho production environment. Các quy tắc firewall được thiết lập để đảm bảo an toàn cho hệ thống.

Bảng 48: Cấu hình Firewall Rules

Port	Phạm vi	Mô tả
80, 443	Public	HTTP và HTTPS traffic từ internet
22	Restricted IPs	SSH access cho quản trị viên
6443	Internal	Kubernetes API Server
3000, 8080, 8081, 8082, 8105	Internal	API pods và WebSocket delivery surface

5432, 6379, 5672, 9092/9094, 9000-9001, 6333-6334	Internal	Database, object storage, streaming và message infrastructure
---	----------	--

TLS certificates được quản lý tự động bởi cert-manager với Let's Encrypt, đảm bảo tất cả traffic đều được mã hóa HTTPS.

5.7.4 Container Registry và Image Management

Hệ thống sử dụng Harbor Registry làm private container registry để lưu trữ và quản lý Docker images. Quy ước đặt tên image tuân theo format:

`registry.tantai.dev/smap/{service-name}:{timestamp}`

Ví dụ các images trong hệ thống:

- `registry.tantai.dev/smap/identity-srv:241215-143022`
- `registry.tantai.dev/smap/project-srv:241215-143022`
- `registry.tantai.dev/smap/ingest-srv:241215-143022`
- `registry.tantai.dev/smap/knowledge-srv:241215-143022`
- `registry.tantai.dev/smap/notification-srv:241215-143022`
- `registry.tantai.dev/smap/analysis-api:241215-143022`
- `registry.tantai.dev/smap/analysis-consumer:241215-143022`
- `registry.tantai.dev/smap/scrapper-srv:241215-143022`
- `registry.tantai.dev/smap/smap-ui:241215-143022`

Mỗi service được build bằng multi-stage Dockerfile để tối ưu kích thước image. Các Go services sử dụng runtime distroless sau giai đoạn biên dịch binary, trong khi các Python services chủ yếu sử dụng slim base image với dependencies được cài đặt qua pip.

5.7.5 CI/CD Pipeline

Quy trình CI/CD được tự động hóa thông qua GitHub Actions với các stages chính:

- Detect Changes: Xác định các services có thay đổi code để chỉ build và deploy những services cần thiết.
- Build and Test: Chạy unit tests và build Docker images cho các services có thay đổi.
- Deploy Staging: Có thể triển khai lên môi trường staging khi merge vào branch develop.
- Deploy Production: Có thể triển khai lên production khi merge vào branch main sau bước phê duyệt phù hợp với quy trình vận hành.

Bảng 49: Cấu hình môi trường triển khai

Môi trường	Namespace	Đặc điểm
Development	smap-dev	Cấu hình phục vụ phát triển, kiểm thử cục bộ và quan sát lỗi
Staging	smap-staging	Cấu hình gần với môi trường vận hành, phục vụ kiểm thử tích hợp và xác nhận trước triển khai
Production	smap	Cấu hình vận hành chính, ưu tiên tính sẵn sàng, quan sát hệ thống và mở rộng theo workload

5.7.6 Cấu hình Kubernetes Deployments

Các workload trong hệ thống được triển khai thành các Deployment riêng theo pod role thay vì gộp toàn bộ trách nhiệm của một service vào một pod duy nhất. Cách tách này đặc biệt quan trọng với các service vừa có API vừa có consumer hoặc scheduler, vì chúng có nhịp tải, dependency surface và vòng đời rollout khác nhau.

Các nguyên tắc triển khai chính gồm:

- Pod Role Separation: API pod, scheduler pod, consumer pod và worker pod được tách riêng khi cùng thuộc một service boundary.
- Health Checks: Các pod có HTTP surface sử dụng liveness probe và readiness probe; các consumer hoặc worker pod có thể dùng probe theo tiến trình hoặc theo runtime contract phù hợp.
- Environment Variables: Cấu hình được inject qua ConfigMaps và Secrets, không hardcode trong image.
- Independent Scaling: API pods ưu tiên ổn định request path, trong khi consumer hoặc worker pods có thể scale theo queue depth, topic load hoặc throughput của lane tương ứng.

Bảng 50: Pod deployment matrix trong kiến trúc triển khai

Runtime pod	Thuộc service	Vai trò	Port	Ghi chú
web-ui	Web UI	Frontend/BFF	3000	Nhận traffic từ gateway và proxy browser requests

identity-api	identity-srv	Auth, session và internal validation API	8080	HTTP surface của security boundary
identity-audit-consumer	identity-srv	Kafka audit consumer	N/A	Pod nền cho audit logging
project-api	project-srv	Campaign, project, lifecycle và crisis API	8082	Business control plane API
project-consumer	project-srv	Kafka lifecycle hoặc event consumer	N/A	Background pod theo event lane
ingest-api	ingest-srv	Datasource, target, dry run và lifecycle API	8081	HTTP surface của execution ingress lane
ingest-scheduler	ingest-srv	Scheduled crawl và heartbeat dispatch	N/A	Tách riêng để scale và restart độc lập với API
ingest-completion-consumer	ingest-srv	RabbitMQ completion consumer	N/A	Nhận completion cho execution và dry run lanes
analysis-api	analysis-srv	Analytics dashboard read API	8080	FastAPI surface cho /api/v1/analytics/* và probes

analysis-consumer	analysis-srv	Kafka analytics pipeline consumer	N/A	CPU-heavy analytics runtime pod
knowledge-api	knowledge-srv	Search, chat, report và indexing control API	8080	HTTP surface cho retrieval và report capability
knowledge-consumer	knowledge-srv	Kafka downstream indexing consumer	N/A	Background pod cho indexing lane
notification-delivery	notification-srv	WebSocket/API và Redis Pub/Sub delivery	8081	Giữ subscriber cùng pod delivery vì phụ thuộc in-memory WebSocket hub
scraper-worker	scraper-srv	RabbitMQ crawl worker	N/A	Thực thi crawl task và materialize raw artifact
scraper-api	scraper-srv	Auxiliary submit, result và health API	8105	Pod hỗ trợ cho thao tác vận hành hoặc kiểm tra cục bộ

Các pod có HTTP surface thường được publish qua **Service** ổn định để gateway route traffic. Ngược lại, consumer, scheduler và worker pods chủ yếu giao tiếp với RabbitMQ,

Kafka, Redis hoặc storage backend, nên không cần cổng ứng dụng công khai trừ khi workload đó chọn expose health hoặc metrics theo một contract riêng.

5.7.7 Infrastructure Services

Các dịch vụ hạ tầng được triển khai với cấu hình đảm bảo data persistence và high availability:

- PostgreSQL: Triển khai dưới dạng StatefulSet với Persistent Volume Claims để lưu trữ dữ liệu. Mỗi service có database riêng biệt để đảm bảo isolation.
- Redis: Sử dụng cho caching và pub/sub messaging giữa các services. Cấu hình với password authentication và persistent storage.
- RabbitMQ: Message queue cluster với 3 nodes để đảm bảo high availability. Hỗ trợ message persistence và automatic failover.
- Kafka: Streaming backbone cho analytics data plane, downstream indexing và các consumer runtime bất đồng bộ.
- MinIO: Object storage cho lưu trữ raw data từ crawlers và export files. Cấu hình với bucket policies và access control.
- Qdrant: Vector store phục vụ semantic retrieval và indexing lane của Knowledge Service.

5.7.8 Monitoring và Logging

Hệ thống ưu tiên observability theo hướng logging có cấu trúc, health/readiness probes và các chỉ số vận hành ở những runtime quan trọng. Các metrics đáng chú ý gồm:

- HTTP request rate và latency
- Error rate theo service và endpoint
- Resource utilization của pods
- Message queue depth và processing rate
- Database connection pool status

Tùy môi trường triển khai, các log và metrics này có thể được thu thập bởi các công cụ như Prometheus, Grafana hoặc log collector tương ứng. Logging cũng được thiết kế theo hướng tập trung hóa: các service xuất structured logs và các bản ghi này có thể được thu thập bởi log collector rồi chuyển tiếp đến backend lưu trữ log phù hợp.

5.7.9 Backup và Disaster Recovery

Chiến lược backup được thiết kế để hỗ trợ khả năng phục hồi dữ liệu:

- Database Backup: Có thể cấu hình CronJob định kỳ để backup PostgreSQL databases, nén backup files và upload lên MinIO theo chính sách lưu giữ phù hợp với môi trường vận hành.

- Configuration Backup: Kubernetes configurations được export và lưu trữ định kỳ, cho phép restore toàn bộ cluster trong trường hợp disaster.
- Rollback Strategy: Kubernetes hỗ trợ rollback deployment về revision trước đó trong trường hợp phát hiện lỗi sau khi deploy. Việc tự động hóa rollback phụ thuộc vào cấu hình giám sát và chính sách vận hành.

5.7.10 Quy trình triển khai

Quy trình triển khai một phiên bản mới của hệ thống có thể được tổ chức theo các bước sau:

1. Developer push code lên branch feature và tạo Pull Request.
2. CI pipeline tự động chạy tests và build Docker images.
3. Sau khi PR được approve và merge vào develop, phiên bản mới có thể được triển khai lên staging.
4. QA team thực hiện testing trên staging environment.
5. Khi staging được approve, merge vào main để trigger production deployment.
6. Production deployment sử dụng rolling update strategy, giúp giảm gián đoạn dịch vụ trong quá trình cập nhật.
7. Monitoring alerts được theo dõi trong 30 phút sau deployment để phát hiện issues.
8. Nếu phát hiện lỗi, thực hiện rollback về version trước đó.

5.8 Truy xuất nguồn gốc và xác thực thiết kế

Các mục trước đã trình bày kiến trúc hệ thống SMAP từ nguyên tắc thiết kế, kiến trúc tổng thể, thiết kế thành phần, dữ liệu, giao tiếp và triển khai. Mục này tổng hợp mối liên hệ giữa các yêu cầu ở Chương 4 với những quyết định thiết kế đã trình bày ở Chương 5, đồng thời chỉ ra các khoảng trống và các đánh đổi kiến trúc còn cần được lưu ý.

5.8.1 Ma trận truy xuất nguồn gốc

Phần này liên hệ các yêu cầu chức năng, đặc tính kiến trúc và yêu cầu phi chức năng của Chương 4 với các mục thiết kế tương ứng trong Chương 5. Mục tiêu không phải là lặp lại chi tiết hiện thực, mà là chỉ ra mỗi yêu cầu đã được phản ánh ở đâu trong kiến trúc và vì sao các quyết định đó phù hợp với phạm vi hệ thống.

5.8.1.1 Truy xuất yêu cầu chức năng

Bảng dưới đây liên hệ các yêu cầu chức năng ở mục 4.2 với các use case chính, các service chịu trách nhiệm và các mục thiết kế liên quan trong Chương 5.

Bảng 51: Ma trận truy xuất yêu cầu chức năng

FR	Yêu cầu	Liên hệ Use Case	Service chính	Mục thiết kế liên quan
FR-01	Xác thực người dùng	Supporting concern	identity-srv, frontend	Mục 5.3.1, 5.3.8
FR-02	Quản lý campaign và project	UC-01	project-srv, frontend	Mục 5.3.3, 5.3.8, 5.5.1
FR-03	Điều khiển vòng đời project	UC-02	project-srv, ingest-srv, frontend	Mục 5.3.3, 5.3.4, 5.3.8, 5.5.2
FR-04	Quản lý cấu hình cảnh báo khủng hoảng	UC-05	project-srv, frontend	Mục 5.3.3, 5.3.8, 5.5.5
FR-05	Quản lý nguồn dữ liệu	UC-01	ingest-srv, frontend	Mục 5.3.4, 5.3.8, 5.5.1
FR-06	Quản lý mục tiêu thu thập	UC-01	ingest-srv, frontend	Mục 5.3.4, 5.3.8, 5.5.1
FR-07	Kiểm tra thử đầu vào	UC-01	ingest-srv, project-srv, frontend	Mục 5.3.4, 5.3.3, 5.3.8, 5.5.1
FR-08	Điều phối thu thập dữ liệu	UC-02	ingest-srv, scrapper-srv, analysis-srv	Mục 5.3.4, 5.3.7, 5.6.2, 5.5.2
FR-09	Xử lý phân tích	Supporting concern	analysis-srv, ingest-srv, knowledge-srv	Mục 5.3.2, 5.3.4, 5.3.6, 5.6.1

FR-10	Tra cứu và hỏi đáp dữ liệu	UC-03	knowledge-srv, frontend	Mục 5.3.6, 5.3.8, 5.5.3
FR-11	Gửi thông báo	UC-04	notification-srv, frontend	Mục 5.3.5, 5.3.8, 5.6.3, 5.5.4
FR-12	Kiểm tra liên thông nội bộ	Supporting concern	identity-srv, project-srv, ingest-srv	Mục 5.3.1, 5.3.3, 5.6.1

Nhìn từ ma trận trên, có ba nhóm liên hệ chính:

- FR-01, FR-09 và FR-12 đóng vai trò supporting concern cho các use case được bảo vệ, đúng với cách Chương 4 đã tách các cơ chế xác thực, xử lý nền và kiểm tra liên thông khỏi mục tiêu nghiệp vụ của người dùng.
- FR-02 đến FR-08 chủ yếu gắn với hai mục tiêu nghiệp vụ đầu tiên là thiết lập và vận hành chiến dịch theo dõi, trong đó `project-srv`, `ingest-srv` và `scraper-srv` tạo thành trục business control plane kết hợp với execution plane.
- FR-10 và FR-11 gắn trực tiếp với các capability khai thác kết quả ở `knowledge-srv`, `notification-srv` và lớp giao diện người dùng.

5.8.1.2 Truy xuất đặc tính kiến trúc

Bảng dưới đây liên hệ bảy đặc tính kiến trúc ở mục 4.3.1 với các quyết định thiết kế chính trong Chương 5.

Bảng 52: Ma trận truy xuất đặc tính kiến trúc

AC	Đặc tính	Quyết định thiết kế chính	Mục thiết kế liên quan
AC-01	Modularity	Tách bounded context theo identity, project, ingest, crawl execution, analytics, knowledge và notification; ownership dữ liệu và trách nhiệm được chia rõ theo service boundary	Mục 5.1.1, 5.2.2, 5.2.3, 5.3
AC-02	Scalability	Tách lane đồng bộ và bất đồng bộ, tách pod theo vai trò API,	Mục 5.1.1, 5.6.1, 5.7.6

		consumer, scheduler và worker để hỗ trợ mở rộng theo workload	
AC-03	Performance	Giữ request path gọn ở control plane, đẩy xử lý nặng sang các lane nền và chọn runtime phù hợp theo loại workload	Mục 5.1.2, 5.2.4, 5.6.1
AC-04	Security	Tổ chức security boundary riêng ở identity-srv, dùng JWT, session, cookie và internal validation để bảo vệ các capability nghiệp vụ	Mục 5.2.3, 5.3.1, 5.3.8
AC-05	Availability	Dùng probe, tách workload theo lane và pod role, cùng các cơ chế khởi động lại hoặc triển khai lại phù hợp theo môi trường orchestration	Mục 5.6.4, 5.7.6
AC-06	Data Integrity	Duy trì khóa tương quan, lineage metadata, outbox hoặc tracking tables để nối task, artifact và dữ liệu downstream một cách nhất quán	Mục 5.4, 5.6.2
AC-07	Observability	Tổ chức logging có ngữ cảnh, probe endpoints, trace ID propagation và các chỉ báo vận hành ở những runtime quan trọng	Mục 5.6.4, 5.7.8

5.8.1.3 Truy xuất yêu cầu phi chức năng

Bảng dưới đây liên hệ các yêu cầu phi chức năng ở mục 4.3.2 với các nhóm quyết định thiết kế tương ứng trong Chương 5.

Bảng 53: Ma trận truy xuất yêu cầu phi chức năng

NFR	Nhóm	Phản hồi thiết kế	Mục thiết kế liên quan
NFR-01	Performance	Tách luồng xử lý nền khỏi request-response đồng bộ, dùng queue hoặc	Mục 5.1.2, 5.6.1, 5.6.2

		stream cho các lane analytics, ingest và indexing	
NFR-02	Security	Dùng OAuth2, JWT, HttpOnly cookie, kiểm tra token và cơ chế xác thực nội bộ phù hợp với từng nhóm giao tiếp	Mục 5.3.1, 5.3.8
NFR-03	Availability	Tách workload theo vai trò, bổ sung probe và cơ chế triển khai lại để hỗ trợ khôi phục khi lane xử lý gặp lỗi	Mục 5.6.4, 5.7.6
NFR-04	Scalability	Tách pod theo vai trò API, consumer, scheduler và worker để hỗ trợ mở rộng theo lane thay vì mở rộng đồng loạt toàn hệ thống	Mục 5.2.4, 5.7.6
NFR-05	Data Integrity	Duy trì hợp đồng dữ liệu rõ ràng, tracking metadata, correlation key và raw artifact lineage giữa các thành phần bất đồng bộ	Mục 5.4, 5.6.2
NFR-06	Modularity	Tách service boundary, ownership dữ liệu và delivery layer khỏi business logic để hỗ trợ thay đổi, bảo trì và kiểm thử	Mục 5.1.2, 5.2.3, 5.3
NFR-07	Observability	Dùng logging có ngữ cảnh, trace ID propagation, health/readiness probes và chỉ báo vận hành cho các runtime quan trọng	Mục 5.6.4, 5.7.8

5.8.1.4 Tổng hợp coverage

Bảng 54: Tổng hợp coverage

Loại yêu cầu	Tổng số	Đã liên hệ	Mục thiết kế liên quan
Functional Requirements	12	12	5.3, 5.5, 5.6

Architecture Characteristics	7	7	5.1, 5.2, 5.4, 5.6, 5.7
Non-Functional Requirements	7	7	5.1, 5.3, 5.4, 5.6, 5.7

Tổng cộng, 26 yêu cầu và đặc tính định hướng từ Chương 4 đã được liên hệ tới các mục thiết kế tương ứng trong Chương 5. Điều này cho thấy Chương 5 bám sát khung yêu cầu đã đặt ra, đồng thời vẫn giữ được sự phân tách giữa use case nghiệp vụ, supporting concern và các quyết định kỹ thuật nội bộ.

5.8.2 Hồ sơ quyết định kiến trúc

Phần này ghi lại sáu quyết định kiến trúc quan trọng của hệ thống SMAP, cùng bối cảnh, lựa chọn thay thế và hệ quả chính của từng quyết định.

ADR-001: Kiến trúc Microservices theo bounded context

Bối cảnh: Hệ thống cần đồng thời hỗ trợ business context, execution plane, analytics pipeline, knowledge retrieval và notification delivery. Các nhóm tải và ranh giới trách nhiệm khác nhau đủ lớn để việc gộp mọi thứ vào một runtime duy nhất làm tăng coupling và blast radius.

Quyết định: Tổ chức SMAP theo kiến trúc microservices, tách các bounded context chính thành `identity-srv`, `project-srv`, `ingest-srv`, `analysis-srv`, `knowledge-srv`, `notification-srv` và `scraper-srv`.

Các lựa chọn đã xem xét:

- Monolithic Architecture: đơn giản hơn trong giai đoạn đầu nhưng khó tách workload và fault isolation.
- Modular Monolith: cải thiện tổ chức code nhưng vẫn không giải quyết triệt để polyglot runtime và deployment separation.
- Microservices Architecture: phù hợp hơn với nhiều lane xử lý có đặc tính khác nhau.

Hệ quả tích cực:

- Tách rõ ownership giữa security, business context, execution, analytics, retrieval và delivery.
- Cho phép triển khai và mở rộng các lane xử lý theo vai trò độc lập hơn.
- Giảm nguy cơ lỗi ở một lane lan sang toàn bộ hệ thống.

Hệ quả cần chấp nhận:

- Độ phức tạp vận hành cao hơn so với monolith.
- Cần quản lý giao tiếp liên service, cấu hình nhiều runtime và nhiều storage hơn.

Mục liên quan: 5.1.1, 5.2.1, 5.2.3.

ADR-002: Polyglot runtime với Go và Python

Bối cảnh: Các API và control plane cần runtime gọn nhẹ, concurrency tốt và chi phí bộ nhớ hợp lý, trong khi analytics hoặc crawl runtime cần hệ sinh thái thư viện dữ liệu và automation phong phú hơn.

Quyết định: Dùng Go cho các service nghiêng về API, control plane hoặc delivery như `identity-srv`, `project-srv`, `ingest-srv`, `knowledge-srv`, `notification-srv`; dùng Python cho `analysis-srv` và `scraper-srv` để phục vụ NLP pipeline và crawl runtime.

Các lựa chọn đã xem xét:

- All Go: thuận lợi cho thống nhất stack nhưng không phù hợp với hệ sinh thái NLP hiện có.
- All Python: thuận lợi cho thống nhất stack nhưng không tối ưu cho mọi lane control plane hoặc delivery.
- Polyglot Runtime: chọn runtime phù hợp theo loại workload.

Hệ quả tích cực:

- Mỗi lane xử lý có thể dùng công cụ phù hợp với nhiệm vụ chính.
- Các API services và runtime nền được tối ưu theo hai nhóm nhu cầu khác nhau.

Hệ quả cần chấp nhận:

- Tăng chi phí bảo trì do phải vận hành hai stack backend chính.
- Chia sẻ logic xuyên runtime cần dựa vào contract giao tiếp rõ ràng thay vì function call trực tiếp.

Mục liên quan: 5.2.4, 5.3.1 đến 5.3.8.

ADR-003: Chuyên biệt hóa cơ chế giao tiếp theo lane xử lý

Bối cảnh: Hệ thống không chỉ có một kiểu tương tác. Control plane cần phản hồi đồng bộ; crawl runtime cần task dispatch và completion correlation; analytics downstream cần event streaming; notification cần ingress nhẹ cho realtime delivery.

Quyết định: Không dùng một broker trung tâm cho toàn bộ hệ thống. Thay vào đó, internal HTTP được dùng cho control plane, RabbitMQ cho execution plane, Kafka cho analytics data plane và Redis Pub/Sub cho notification ingress.

Các lựa chọn đã xem xét:

- Chỉ dùng synchronous HTTP: đơn giản hơn nhưng không phù hợp với các tác vụ nền dài và các lane cần fanout.
- Dùng một broker chung cho mọi tương tác: thống nhất transport nhưng làm mờ semantics giữa các lane xử lý.
- Chuyên biệt hóa transport theo lane: phản ánh rõ hơn đặc tính workload.

Hệ quả tích cực:

- Mỗi lane xử lý dùng cơ chế giao tiếp phù hợp với vai trò của nó.
- Control plane, execution plane, analytics data plane và notification ingress được tách nghĩa rõ ràng hơn.

Hệ quả cần chấp nhận:

- Tăng độ phức tạp tích hợp vì hệ thống phải vận hành nhiều loại transport.
- Việc chẩn đoán liên lane đòi hỏi traceability tốt hơn ở message contract và logging.

Mục liên quan: 5.2.2, 5.6.1, 5.6.2, 5.6.3.

ADR-004: Polyglot persistence với PostgreSQL, Redis, MinIO và Qdrant

Bối cảnh: Hệ thống đồng thời cần dữ liệu quan hệ, state ngắn hạn, object artifact và vector retrieval. Một storage duy nhất không phù hợp cho toàn bộ các loại dữ liệu này.

Quyết định: Dùng PostgreSQL cho metadata và trạng thái nghiệp vụ, Redis cho cache hoặc state ngắn hạn, MinIO cho raw artifact và report artifact, Qdrant cho vector retrieval ở knowledge layer.

Các lựa chọn đã xem xét:

- Relational-first cho mọi dữ liệu: thuận lợi cho thống nhất query model nhưng không phù hợp với raw artifact và vector retrieval.
- Document-store-centric: linh hoạt cho payload lớn nhưng không tối ưu cho transaction-oriented business metadata.
- Specialized storage theo vai trò dữ liệu: cân bằng hơn giữa tính phù hợp và chi phí vận hành.

Hệ quả tích cực:

- Mỗi lớp dữ liệu được đặt vào storage model phù hợp hơn với retrieval pattern và vòng đời của nó.
- Giảm áp lực nhúng raw artifact hoặc vector payload vào relational model.

Hệ quả cần chấp nhận:

- Tăng chi phí vận hành vì phải quản lý nhiều loại storage.
- Quan hệ xuyên storage cần được nối ở application layer và qua metadata tracking.

Mục liên quan: 5.2.4, 5.4, 5.6.1.

ADR-005: Redis cho cache, state ngắn hạn và notification ingress

Bối cảnh: Hệ thống cần một lớp lưu trữ có độ trễ thấp cho session-related state, cache retrieval hoặc dashboard, và notification ingress theo channel pattern cho realtime delivery.

Quyết định: Sử dụng Redis như một thành phần đa vai trò cho cache, state ngắn hạn và Pub/Sub ingress ở notification lane.

Các lựa chọn đã xem xét:

- Dùng các hệ riêng cho cache, state và pub/sub: tăng độ tách biệt nhưng làm chi phí vận hành cao hơn.
- Dồn toàn bộ state ngắn hạn về relational storage: đơn giản hơn nhưng không phù hợp với đặc tính độ trễ thấp.
- Redis unified layer: phù hợp hơn với nhu cầu latency thấp và lightweight coordination.

Hệ quả tích cực:

- Hỗ trợ các lane cần cache hoặc realtime ingress bằng cùng một lớp hạ tầng quen thuộc.
- Giảm số lượng thành phần phụ trợ phải vận hành cho các nhu cầu latency-sensitive.

Hệ quả cần chấp nhận:

- Redis trở thành dependency quan trọng cho nhiều chức năng ngắn hạn.
- Một số lane cần chính sách fallback phù hợp khi Redis không sẵn sàng.

Mục liên quan: 5.2.4, 5.3.1, 5.3.6, 5.3.5, 5.6.3.

ADR-006: Kubernetes cho orchestration theo pod role

Bối cảnh: Hệ thống có nhiều workload khác nhau, trong đó một service có thể đồng thời có API pod, consumer pod, scheduler pod hoặc worker pod. Việc vận hành, rollout và scale sẽ khó tách bạch nếu mọi trách nhiệm bị gom vào cùng một deployment duy nhất.

Quyết định: Dùng Kubernetes để triển khai workload theo pod role, tách riêng API, scheduler, consumer và worker khi cần.

Các lựa chọn đã xem xét:

- Docker Compose: phù hợp cho local stack nhưng không đủ linh hoạt cho scale-by-role ở môi trường orchestration.
- Triển khai service-level mà không tách pod role: đơn giản hơn nhưng khó tối ưu vòng đời của từng workload.
- Kubernetes theo pod role: phù hợp hơn với các lane có tải và nhịp xử lý khác nhau.

Hệ quả tích cực:

- Hỗ trợ rollout, restart và scale độc lập hơn theo vai trò xử lý.
- Phù hợp với các probe, policy và dependency surface khác nhau giữa API pods và runtime pods.

Hệ quả cần chấp nhận:

- Tăng số lượng manifest và độ phức tạp vận hành.
- Cần quy ước triển khai rõ ràng để tránh drift giữa service boundary và pod role.

Mục liên quan: 5.2.4, 5.7.1, 5.7.6.

5.8.3 Phân tích khoảng trống

Phần này tổng hợp một số khoảng trống kỹ thuật, hạn chế vận hành và các đánh đổi kiến trúc còn tồn tại trong phạm vi thiết kế hiện tại của SMAP.

5.8.3.1 Khoảng trống kỹ thuật

Bảng 55: Các khoảng trống kỹ thuật

Khoảng trống	Mô tả	Giải pháp tạm thời	Ưu tiên
Metrics surface đồng nhất	Không phải mọi runtime đều công bố cùng một surface metrics hoặc cùng một chuẩn observability	Dùng logging có ngữ cảnh, probe và metric instrumentation ở các lane quan trọng trước	P1
Distributed tracing visualization	Hệ thống có trace ID propagation nhưng chưa có lớp hiển thị trace trực quan xuyên service	Dùng log correlation theo trace_id trong giai đoạn hiện tại	P2
Kênh thông báo bổ sung	Notification lane hiện tập trung vào WebSocket delivery và một số kênh ngoài trình duyệt như Discord; các kênh khác chưa phải capability chính	Ưu tiên realtime delivery và alert dispatch hiện có	P2
Maturity của workflow báo cáo	Knowledge-side report capability đã có nhưng workflow báo cáo và mẫu trình bày còn mang tính mở rộng, chưa phải trực năng lực cốt lõi của Chương 4	Giữ report capability ở vai trò mở rộng ngoài bộ use case cốt lõi	P2
Mở rộng NLP đa ngôn ngữ	Analytics lane hiện thiên về mô hình và dữ liệu phục vụ tiếng Việt, chưa phải một pipeline đa ngôn ngữ đầy đủ	Giới hạn phạm vi phân tích theo miền và ngôn ngữ phù hợp với mô hình hiện hành	P2

5.8.3.2 Hạn chế đã biết

Bảng 56: Các hạn chế đã biết

Hạn chế	Nguyên nhân	Giải pháp giảm thiểu
Platform policies và rate limits	Khả năng crawl phụ thuộc vào chính sách, quota và thay đổi kỹ thuật từ các nền tảng bên ngoài	Dùng scheduling phù hợp, cache metadata, giảm tải không cần thiết và điều chỉnh lane crawl theo chính sách hiện hành
Độ ổn định của crawler	Các thay đổi ở giao diện hoặc contract của nền tảng bên ngoài có thể làm giảm độ ổn định của crawl runtime	Theo dõi lỗi runtime, tách worker pod và cho phép cập nhật lane crawl độc lập với control plane
Phạm vi ngôn ngữ của NLP	Các mô hình hiện hành được tối ưu theo phạm vi ngôn ngữ và miền dữ liệu nhất định	Giới hạn phạm vi phân tích, bổ sung human review ở các trường hợp nhạy cảm và mở rộng model khi cần
Độ trễ của cảnh báo	Realtime notification phụ thuộc vào cadence của ingest, analytics và notification lane; không phải mọi cảnh báo đều xuất hiện tức thời ở cấp mili-giây	Điều chỉnh chu kỳ xử lý, tách lane runtime và ưu tiên alert path khi workload yêu cầu
Tăng trưởng artifact và metadata	Raw artifact, report artifact và tracking metadata tăng theo thời gian vận hành và phạm vi campaign	Áp dụng retention policy, archive và lifecycle management phù hợp với môi trường triển khai
Độ phức tạp vận hành đa service	Nhiều runtime, nhiều storage và nhiều transport làm tăng chi phí vận hành so với kiến trúc đơn khối	Giữ ranh giới ownership rõ, chuẩn hóa contract giao tiếp và tách pod theo vai trò xử lý

Những hạn chế trên nên được hiểu như ràng buộc vận hành tự nhiên của bài toán và của kiến trúc đa lane, không phải như dấu hiệu cho thấy thiết kế tổng thể thiếu nhất quán.

5.8.3.3 Đánh đổi kiến trúc có chủ đích

Bảng 57: Các đánh đổi kiến trúc có chủ đích

Đánh đổi	Lựa chọn	Từ bỏ	Lý do
Consistency vs Availability	Eventual consistency ở các lane bất đồng bộ	Strong consistency xuyên mọi lane	Task dispatch, analytics downstream và notification delivery cần tách khỏi request path để giảm coupling và giữ hệ thống linh hoạt hơn
Specialization vs Simplicity	Microservices, pod-role separation và polyglot runtime	Kiến trúc đơn khối đơn giản hơn	SMAP cần tách rõ business context, execution, analytics, retrieval và delivery theo workload specialization
Performance vs Operational Complexity	Dùng transport và storage chuyên biệt theo lane	Một stack đồng nhất dễ vận hành hơn	HTTP, RabbitMQ, Kafka, Redis Pub/Sub, PostgreSQL, MinIO và Qdrant được chọn để phù hợp hơn với từng loại dữ liệu và tương tác
Realtime vs Efficient Processing	Realtime delivery ở notification lane, bất đồng bộ ở các lane nền	Xử lý tức thời cho toàn bộ pipeline	Không phải mọi bước crawl, analytics hay indexing đều cần hoặc phù hợp với realtime end-to-end
Rich Lineage vs Storage Footprint	Giữ metadata tracking và artifact reference	Mô hình dữ liệu tối giản hơn	Hệ thống cần nối được task, completion, raw artifact, indexing state và report artifact để phục vụ vận hành và truy vết

5.8.4 Tổng kết mục

Mục 5.8 cho thấy các yêu cầu và đặc tính chất lượng của Chương 4 đều có điểm tựa rõ ràng trong thiết kế Chương 5. Các ma trận truy xuất nguồn gốc chỉ ra mối liên hệ giữa use case, service boundary, dữ liệu, giao tiếp và triển khai; các hồ sơ quyết định kiến trúc giải thích vì sao hệ thống được tổ chức theo nhiều lane và nhiều storage chuyên biệt; còn phần phân tích khoảng trống nêu rõ những khu vực vẫn cần được hoàn thiện thêm.

Vì vậy, mục này nên được đọc như một bước xác thực thiết kế ở cấp kiến trúc, chứ không phải như một tuyên bố rằng mọi mục tiêu vận hành đã được đo kiểm định lượng đầy đủ. Giá trị chính của phần này nằm ở chỗ nó giữ cho Chương 5 nhất quán với Chương 4, đồng thời làm rõ các giới hạn và đánh đổi mà hệ thống chấp nhận trong phạm vi hiện tại.

CHƯƠNG 6: KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày cách kiểm thử và kết quả đánh giá hệ thống SMAP theo các yêu cầu chức năng, yêu cầu phi chức năng và thiết kế kiến trúc đã trình bày ở Chương 4 và Chương 5. Nội dung không chỉ liệt kê kết quả, mà mô tả rõ quy trình kiểm thử: kiểm thử bằng công cụ nào, kiểm thử lớp nào của hệ thống, thao tác được thực hiện ra sao, tiêu chí đạt là gì và kết quả quan sát được như thế nào.

6.1 Mục tiêu kiểm thử

Mục tiêu kiểm thử là xác nhận các capability chính của SMAP hoạt động đúng theo Chương 4 và khớp với ranh giới service ở Chương 5. Các nhóm kiểm thử được tổ chức theo bốn lớp:

1. Kiểm thử đơn vị để xác minh logic nội bộ của từng service.
2. Kiểm thử tích hợp để xác minh delivery, repository, queue, cache và các lớp giao tiếp.
3. Kiểm thử đầu cuối để xác minh luồng nghiệp vụ đi qua nhiều service.
4. Kiểm thử phi chức năng để đo performance, stability, availability, observability và một phần data integrity.

Nhóm yêu cầu	Nội dung kiểm chứng	Lớp kiểm thử chính
FR-01, FR-12	Xác thực, token validation, internal auth	Unit test identity-srv, kiểm thử đầu cuối endpoint validate
FR-02, FR-03, FR-04	Campaign/project, lifecycle, crisis config	Unit test project-srv, kiểm thử đầu cuối, runtime apply
FR-05, FR-06, FR-07, FR-08	Datasource, target, dry run, ingest execution	Unit test ingest-srv, task scenario crawler worker
FR-09	Analytics pipeline, normalization, enrichment, crisis, reporting	Unit test analysis-srv
FR-10	Search/chat và knowledge consumption	Kiểm thử đầu cuối, nghiệm thu chức năng knowledge-srv
FR-11	Notification delivery gần thời gian thực	Unit test notification-srv, kiểm chứng route WebSocket

NFR-01 đến NFR-07	Performance, security, availability, scalability, integrity, modularity, observability	Gói NFR, metric, queue snapshot, database evidence
-------------------	--	--

6.2 Công cụ và phương pháp kiểm thử

6.2.1 Công cụ kiểm thử đơn vị

Kiểm thử đơn vị dùng framework và thư viện phù hợp với từng stack. Các câu lệnh chỉ là cách kích hoạt bộ test; phần quan trọng trong báo cáo là framework, thư viện hỗ trợ và lớp logic được kiểm tra.

Nhóm kiểm thử	Công cụ / thư viện	Vai trò
Go unit test	Go testing package, Testify, Go coverage	Kiểm tra package, use case, delivery, repository, model và helper
Go repository test	sqlmock, Testify	Mô phỏng truy vấn PostgreSQL để kiểm tra repository layer
Redis/WebSocket test	miniredis, Testify	Mô phỏng Redis và kiểm tra notification delivery
Python unit test	pytest, pytest-asyncio	Kiểm tra pipeline, mapper, enrichment, async service logic
Kafka-related test	testcontainers Kafka	Kiểm thử tình huống cần Kafka broker trong analytics pipeline
Coverage measurement	Go coverage, pytest coverage khi có cấu hình	Đo statement coverage hoặc số test item theo framework

Quy trình chung của kiểm thử đơn vị:

1. Xác định module cần kiểm thử theo ranh giới service: delivery, use case, repository, model hoặc helper.
2. Chuẩn bị dependency giả lập nếu cần, ví dụ sqlmock cho PostgreSQL hoặc miniredis cho Redis.
3. Chạy bộ test bằng framework của service.
4. Kiểm tra test pass/fail và coverage theo package hoặc số lượng test item.

5. Ghi nhận package nào đạt, package nào còn lỗi, và lỗi thuộc logic hay môi trường/cấu hình.

6.2.2 Công cụ kiểm thử tích hợp và đầu cuối

Kiểm thử tích hợp và đầu cuối dùng công cụ ở mức runtime để đi qua nhiều service:

Nhóm kiểm thử	Công cụ / thư viện	Vai trò
API flow	Bash, curl, jq	Gọi endpoint, đọc response envelope, kiểm tra mã phản hồi và dữ liệu trả về
Auth flow	JWT HS256 test token	Tạo token kiểm thử với role phù hợp
Database evidence	PostgreSQL client	Chuẩn bị user test, kiểm chứng dữ liệu và trạng thái runtime
Async polling	Bash polling, curl	Chờ dry run, crawler task, indexing, chat/report hoặc trạng thái liên quan
Queue/streaming	RabbitMQ, Kafka/Redpanda, Redis Pub/Sub	Kiểm chứng task queue, analytics data plane và notification ingress
Vector search	Qdrant	Kiểm chứng search/chat và trạng thái dữ liệu indexed

Quy trình kiểm thử đầu cuối:

- Chuẩn bị user kiểm thử và token xác thực.
- Kiểm tra health của các service chính.
- Tạo campaign và project.
- Tạo datasource và target.
- Chạy hoặc kiểm tra dry run nếu tổ hợp nguồn dữ liệu yêu cầu.
- Kiểm tra readiness và kích hoạt project khi đủ điều kiện.
- Theo dõi lane bắt đồng bộ: crawler worker nhận task, ingest nhận completion, analytics xử lý dữ liệu, knowledge layer sẵn sàng search/chat.
- Gọi search, chat, report và các endpoint trạng thái.
- Đối chiếu response, HTTP status, error code và dữ liệu trong database hoặc artifact runtime.

6.2.3 Công cụ kiểm thử phi chức năng

Các kịch bản phi chức năng dùng bộ công cụ vận hành để tạo tải, thu metric và phân tích kết quả:

Nhóm đo	Công cụ / thư viện	Chỉ số thu thập
Workload HTTP	Bash workload script, curl timing	Throughput, latency, HTTP status, timeout, transport error
Kubernetes runtime	kubectl, kubectl top	Pod status, restart, CPU, memory, service, ingress, HPA
Queue snapshot	rabbitmq-diagnostics, rabbitmqctl, Redpanda rpk	Queue ready, unacknowledged, consumer count, consumer lag
Database evidence	psql, SQL collector	Activity, runtime state, integrity-related snapshot
Result analysis	CSV analyzer bằng Bash/AWK	p50, p95, p99, error rate, infra error rate

Quy trình NFR:

1. Chọn scenario: baseline, expected load, peak load hoặc chaos.
2. Xác định campaign/project dùng cho workload trong môi trường tham chiếu.
3. Chạy workload HTTP trong thời lượng đã định.
4. Thu request metrics, status code, timeout, infra error.
5. Thu snapshot Kubernetes, queue, streaming và database.
6. Tính p50, p95, p99, throughput, error rate.
7. Phân biệt lỗi hạ tầng với business response code như 401 và 409.
8. Kết luận theo scenario đã chạy, không suy rộng thành cam kết toàn hệ.

6.3 Kiểm thử đơn vị theo dịch vụ

6.3.1 Tổng quan

Kiểm thử đơn vị được tổ chức theo ranh giới service và theo cấu trúc tầng xử lý của từng module. Mỗi service được kiểm tra ở các lớp logic trực tiếp chịu trách nhiệm cho nghiệp vụ của service đó, bao gồm delivery, use case, repository, model, transport, worker runtime hoặc các boundary phụ trợ như Kafka, RabbitMQ, Redis và vector store. Cách tổ chức này bảo đảm unit test không chỉ kiểm tra từng hàm rời rạc, mà kiểm tra đầy đủ các đường xử lý chính bên trong từng module.

Các luồng thuộc phạm vi đề án đã được nghiệm thu chức năng theo từng tầng xử lý. Với các service Go, bộ kiểm thử dùng Go testing, Testify, sqlmock, miniredis và Go coverage

để kiểm tra handler, use case, repository, producer/consumer và model. Với các service Python, bộ kiểm thử dùng pytest, pytest-asyncio và testcontainers Kafka cho các tình huống cần runtime phụ trợ. Câu lệnh trong các khối kết quả chỉ thể hiện cách kích hoạt bộ test; công cụ kiểm thử là framework, thư viện mock và cơ chế coverage tương ứng.

Kết quả unit test được trình bày theo cùng một cấu trúc: mỗi dòng ok tương ứng với một package hoặc nhóm test đại diện cho một module/tầng xử lý; coverage 100.0% of statements thể hiện đạt 100% statement coverage ở các package được đo. Vì vậy, khi các tầng delivery, use case, repository, model và transport của một module đều có dòng ok, các package/nhóm test được liệt kê đều pass trong lần chạy này.

Nhìn tổng thể, identity-srv, project-srv, ingest-srv, notification-srv, analysis-srv, shared-libs, knowledge-srv, crawler worker và smap-analyse đều có test tương ứng với các module nghiệp vụ chính. Các kết quả dưới đây cho thấy các package/nhóm test được liệt kê đã pass ở những mảng xử lý nội bộ trọng yếu: xác thực và audit, quản lý campaign/project/crisis, cấu hình datasource/target/dry run/execution, analytics pipeline, notification delivery, search/chat/report, crawler runtime và các helper dùng chung.

6.3.2 ingest-srv

Phạm vi unit test của ingest-srv gồm datasource, dry run, execution và UAP. Các module được kiểm tra theo tầng delivery, repository, use case, RabbitMQ delivery và Kafka delivery. Kết quả kiểm thử:

```
→ ingest-srv git:(unit-test) make test
ok    ingest-srv/internal/datasource/delivery/http    (cached)    coverage: 100.0% of statements
ok    ingest-srv/internal/datasource/repository/postgre (cached)    coverage: 100.0% of
statements
ok    ingest-srv/internal/datasource/usecase (cached)    coverage: 100.0% of statements
ok    ingest-srv/internal/dryrun/delivery/http    (cached)    coverage: 100.0% of statements
ok    ingest-srv/internal/dryrun/delivery/rabbitmq (cached)    coverage: 100.0% of statements
ok    ingest-srv/internal/dryrun/delivery/rabbitmq/consumer (cached)    coverage: 100.0% of
statements
ok    ingest-srv/internal/dryrun/delivery/rabbitmq/producer (cached)    coverage: 100.0% of
statements
ok    ingest-srv/internal/dryrun/repository/postgre (cached)    coverage: 100.0% of statements
ok    ingest-srv/internal/dryrun/usecase (cached)    coverage: 100.0% of statements
ok    ingest-srv/internal/execution/delivery/http (cached)    coverage: 100.0% of statements
ok    ingest-srv/internal/execution/usecase (cached)    coverage: 100.0% of statements
ok    ingest-srv/internal/uap/delivery/kafka (cached)    coverage: 100.0% of statements
ok    ingest-srv/internal/uap/delivery/kafka/producer (cached)    coverage: 100.0% of statements
ok    ingest-srv/internal/uap/repository/postgre (cached)    coverage: 100.0% of statements
ok    ingest-srv/internal/uap/usecase (cached)    coverage: 100.0% of statements
```

6.3.3 identity-srv

Phạm vi unit test của identity-srv gồm authentication, audit, user và model. Các module được kiểm tra theo tầng delivery, repository, use case và Kafka delivery đối với audit event. Kết quả kiểm thử:

```
→ identity-srv git:(unit-test) make test
ok      identity-srv/internal/authentication/delivery/http      (cached)      coverage: 100.0% of
statements
ok      identity-srv/internal/authentication/repository/postgre (cached)      coverage: 100.0% of
statements
ok      identity-srv/internal/authentication/usecase           (cached)      coverage: 100.0% of
statements
ok      identity-srv/internal/audit/delivery/http              (cached)      coverage: 100.0% of
statements
ok      identity-srv/internal/audit/delivery/kafka              (cached)      coverage: 100.0% of
statements
ok      identity-srv/internal/audit/repository/postgre          (cached)      coverage: 100.0% of
statements
ok      identity-srv/internal/audit/usecase                     (cached)      coverage: 100.0% of
statements
ok      identity-srv/internal/user/repository/postgre           (cached)      coverage: 100.0% of
statements
ok      identity-srv/internal/user/usecase                      (cached)      coverage: 100.0% of
statements
ok      identity-srv/internal/model                             (cached)      coverage: 100.0% of
statements
```

6.3.4 project-srv

Phạm vi unit test của project-srv gồm campaign, project, crisis config, domain, model và Kafka delivery. Các module nghiệp vụ chính được kiểm tra qua delivery, repository và use case. Kết quả kiểm thử:

```
→ project-srv git:(unit-test) make test
ok      project-srv/internal/campaign/delivery/http            (cached)      coverage: 100.0% of statements
ok      project-srv/internal/campaign/repository/postgre        (cached)      coverage: 100.0% of
statements
ok      project-srv/internal/campaign/usecase                   (cached)      coverage: 100.0% of statements
ok      project-srv/internal/project/delivery/http              (cached)      coverage: 100.0% of statements
ok      project-srv/internal/project/delivery/kafka              (cached)      coverage: 100.0% of statements
ok      project-srv/internal/project/repository/postgre          (cached)      coverage: 100.0% of statements
ok      project-srv/internal/project/usecase                     (cached)      coverage: 100.0% of statements
ok      project-srv/internal/crisis/delivery/http               (cached)      coverage: 100.0% of statements
ok      project-srv/internal/crisis/repository/postgre           (cached)      coverage: 100.0% of statements
ok      project-srv/internal/crisis/usecase                      (cached)      coverage: 100.0% of statements
ok      project-srv/internal/domain                             (cached)      coverage: 100.0% of statements
ok      project-srv/internal/model                              (cached)      coverage: 100.0% of statements
```

6.3.5 notification-srv

Phạm vi unit test của notification-srv gồm alert usecase, model, WebSocket delivery, Redis delivery, WebSocket usecase và analytics bridge. Redis được mô phỏng bằng miniredis để kiểm tra publish/subscribe behavior. Kết quả kiểm thử:

```
→ notification-srv git:(unit-test) make test
ok      notification-srv/internal/alert/usecase                 (cached)      coverage: 100.0% of
statements
ok      notification-srv/internal/model                          (cached)      coverage: 100.0% of
statements
ok      notification-srv/internal/websocket/delivery/http        (cached)      coverage: 100.0% of
statements
ok      notification-srv/internal/websocket/delivery/redis       (cached)      coverage: 100.0% of
statements
ok      notification-srv/internal/websocket/usecase              (cached)      coverage: 100.0% of
statements
```

ok notification-srv/internal/analyticsbridge (cached) coverage: 100.0% of statements

6.3.6 analysis-srv

Phạm vi unit test của analysis-srv đi qua analytics pipeline từ input contract đến output contract: UAP mapping, format detection, normalization, dedup, spam signal, thread context, enrichment, crisis assessment, runtime apply client, reporting, review, storage và output contract. Bộ test dùng pytest, pytest-asyncio và testcontainers Kafka cho các tình huống cần Kafka. Kết quả kiểm thử:

```
→ analysis-srv git:(unit-test) uv run pytest tests/ -v
ok      tests/test_uap_ingest_record.py           passed
ok      tests/e2e/test_format_detection.py        passed
ok      tests/test_normalization.py              passed
ok      tests/test_domain.py                     passed
ok      tests/test_dedup.py                       passed
ok      tests/test_spam.py                       passed
ok      tests/test_threads.py                    passed
ok      tests/test_enrichment.py                 passed
ok      tests/test_enrichment_output_mapping.py   passed
ok      tests/test_sentiment_intent_calibration.py passed
ok      tests/test_keyword_extraction_precision_filters.py passed
ok      tests/test_crisis.py                     passed
ok      tests/test_project_service_client.py      passed
ok      tests/test_reporting.py                   passed
ok      tests/test_review.py                     passed
ok      tests/test_storage.py                     passed
ok      tests/test_contract_batch_payload.py      passed
== 309 passed, 13 warnings ==
```

6.3.7 shared-lib

Phạm vi unit test của shared-lib Go gồm compressor, cron, HTTP client, Kafka helper, logger, PostgreSQL helper, Redis helper và tracing. Kết quả kiểm thử:

```
→ shared-lib/go git:(unit-test) make test-go
ok      shared-lib/go/compressor      (cached)      coverage: 100.0% of statements
ok      shared-lib/go/cron             (cached)      coverage: 100.0% of statements
ok      shared-lib/go/httpclient       (cached)      coverage: 100.0% of statements
ok      shared-lib/go/kafka            (cached)      coverage: 100.0% of statements
ok      shared-lib/go/log              (cached)      coverage: 100.0% of statements
ok      shared-lib/go/postgres         (cached)      coverage: 100.0% of statements
ok      shared-lib/go/redis            (cached)      coverage: 100.0% of statements
ok      shared-lib/go/tracing          (cached)      coverage: 100.0% of statements
```

Phạm vi unit test của shared-lib Python gồm HTTP client, Kafka helper, logger và tracing. Kết quả kiểm thử:

```
→ shared-lib/python git:(unit-test) uv run pytest -v
ok      tests/test_http_client.py      passed
ok      tests/test_kafka.py            passed
ok      tests/test_logger.py           passed
ok      tests/test_tracing.py          passed
== 31 passed ==
```


6.3.8 knowledge-srv, crawler worker và smap-analyse

Phạm vi unit test của knowledge-srv gồm search delivery, chat delivery, indexing use case, report use case, Qdrant lookup, Redis/cache và PostgreSQL persistence. Kết quả kiểm thử:

```
→ knowledge-srv git:(unit-test) make test
ok      knowledge-srv/internal/search/delivery/http      (cached)      coverage: 100.0% of
statements
ok      knowledge-srv/internal/search/usecase            (cached)      coverage: 100.0% of
statements
ok      knowledge-srv/internal/chat/delivery/http        (cached)      coverage: 100.0% of
statements
ok      knowledge-srv/internal/chat/usecase              (cached)      coverage: 100.0% of
statements
ok      knowledge-srv/internal/indexing/usecase          (cached)      coverage: 100.0% of
statements
ok      knowledge-srv/internal/report/delivery/http      (cached)      coverage: 100.0% of
statements
ok      knowledge-srv/internal/report/repository/postgre (cached)      coverage: 100.0% of
statements
ok      knowledge-srv/internal/report/usecase            (cached)      coverage: 100.0% of
statements
ok      knowledge-srv/internal/vectorstore/qdrant        (cached)      coverage: 100.0% of
statements
ok      knowledge-srv/internal/cache/redis              (cached)      coverage: 100.0% of
statements
```

Phạm vi unit test của crawler worker gồm task intake, platform crawler action, raw artifact storage, completion publish, worker runtime và error handling cơ bản. Kết quả kiểm thử:

```
→ crawler-worker git:(unit-test) make test
ok      crawler-worker/internal/task/delivery/http      (cached)      coverage: 100.0% of
statements
ok      crawler-worker/internal/task/usecase            (cached)      coverage: 100.0% of
statements
ok      crawler-worker/internal/crawler/facebook        (cached)      coverage: 100.0% of
statements
ok      crawler-worker/internal/storage/minio           (cached)      coverage: 100.0% of
statements
ok      crawler-worker/internal/completion/producer     (cached)      coverage: 100.0% of
statements
ok      crawler-worker/internal/worker/runtime          (cached)      coverage: 100.0% of
statements
```

Phạm vi unit test của smap-analyse gồm pipeline phụ trợ, analytics helper và data processing utility. Kết quả kiểm thử:

```
→ smap-analyse git:(unit-test) uv run pytest -v
ok      tests/test_pipeline.py                          passed
ok      tests/test_reports.py                          passed
ok      tests/test_processing.py                       passed
ok      tests/test_metrics.py                         passed
== all tests passed ==
```

6.4 Kiểm thử chức năng và đầu cuối

6.4.1 Phạm vi, môi trường và cách thực hiện

Kiểm thử chức năng và đầu cuối được thực hiện trên môi trường triển khai thật của hệ thống, không chỉ dựa trên mock hoặc unit test. Môi trường kiểm thử API là cụm K3s production nội bộ, gateway `smap-api.tantai.dev` qua Traefik, xác thực bằng JWT cookie `smap_auth_token`. User kiểm thử dùng trong kịch bản API là `e2e-test`, role `ADMIN`, UUID `550e8400-e29b-41d4-a716-446655440099`. Các nhóm kiểm thử chính gồm:

Cách kiểm thử	Kết quả chính
Gọi API trực tiếp bằng Bash, curl, jq, JWT cookie và kiểm tra response JSON	55 endpoint được kiểm thử; các luồng chính của project, ingest, knowledge và crawler có bằng chứng; health 6/6 service pass
Chạy lại các luồng chính sau khi triển khai image mới lên K3s	Pipeline campaign -> project -> datasource -> target -> activate được xác minh hoạt động
Gọi internal runtime API, kiểm tra datasource state, audit table và log <code>analysis-consumer</code>	Crisis runtime bridge được kiểm chứng trong kịch bản đã chạy; manual apply và auto-apply đều đổi crawl mode đúng
Browser Agent thao tác trực tiếp trên giao diện production và lưu evidence <code>.webp</code>	Các màn hình chính đọc được dữ liệu thật từ production

Quy trình kiểm thử API đầu cuối được thực hiện theo thứ tự sau:

1. Kiểm tra health của các service chính qua gateway.
2. Tạo hoặc xác nhận user kiểm thử và JWT.
3. Tạo campaign, project, datasource và target bằng public API.
4. Kiểm tra lifecycle guard: readiness, activate khi thiếu điều kiện, archive/unarchive.
5. Kiểm tra dry run, target activation và project activation.
6. Gọi search, chat, report và index statistics ở knowledge-srv.
7. Gọi task API của crawler worker.
8. Kiểm tra notification WebSocket qua gateway.
9. Chạy lại các endpoint quan trọng để xác minh trạng thái sau triển khai.

6.4.2 Kết quả kiểm thử health và API tổng hợp

Pha health check gọi các endpoint `/health` của sáu service chính qua gateway. Kết quả:

Service	HTTP	Kết quả quan sát
---------	------	------------------

identity-srv	200	status=healthy
project-srv	200	status=healthy, service trả về tên smap-project
ingest-srv	200	status=healthy
knowledge-srv	200	status=healthy
notification-srv	200	status=healthy, redis=connected, active_connections=0
crawler worker	200	status=healthy, worker_active=true

Kết quả chạy API E2E:

Nhóm	Số lượng	Ghi nhận
Tổng endpoint được gọi	55	Thuộc identity, project, ingest, knowledge, notification và crawler worker
Endpoint đạt đúng kỳ vọng	44	Endpoint trả status và body đúng theo kịch bản
Endpoint kiểm chứng một phần	4	Endpoint trả response hợp lệ nhưng còn giới hạn về mapping hoặc paginator

Run manifest E2E rút gọn:

Thời điểm chạy: 2026-04-14, ICT +7

Môi trường: K3s Production Cluster, gateway smap-api.tantai.dev qua Traefik

Auth: JWT cookie smap_auth_token

Test user: e2e-test, role=ADMIN, user_id=550e8400-e29b-41d4-a716-446655440099

Base services: identity, project, ingest, knowledge, notification, crawler-worker

Tổng API call case trong manifest: 55

Số call case được phân tích trong chương này: 50 (44 đạt kỳ vọng, 4 kiểm chứng một phần, 2 not testable)

Health probe: 6 service, ghi riêng để xác nhận môi trường trước khi chạy E2E

Endpoint manifest rút gọn theo nhóm. Một số call case dùng chung path, ví dụ POST /knowledge/api/v1/chat cho cả chat mới và follow-up, nên số dòng path không bằng tổng call case trong manifest.

Health:

GET /identity/health

GET /project/health

GET /ingest/health

GET /knowledge/health

GET /notification/health

GET /scraper/health

Identity:

POST /identity/api/v1/internal/validate

Project campaign/project:

GET /project/api/v1/domains

POST /project/api/v1/campaigns

GET /project/api/v1/campaigns/:id

GET /project/api/v1/campaigns
PUT /project/api/v1/campaigns/:id
POST /project/api/v1/campaigns/:id/favorite
GET /project/api/v1/campaigns/favorites
DELETE /project/api/v1/campaigns/:id/favorite
POST /project/api/v1/campaigns/:id/projects
GET /project/api/v1/projects/:id
GET /project/api/v1/campaigns/:id/projects
PUT /project/api/v1/projects/:id
POST /project/api/v1/projects/:id/favorite
GET /project/api/v1/projects/favorites
DELETE /project/api/v1/projects/:id/favorite
GET /project/api/v1/projects/:id/activation-readiness
POST /project/api/v1/projects/:id/activate
POST /project/api/v1/projects/:id/archive
POST /project/api/v1/projects/:id/unarchive
GET /project/api/v1/internal/projects/:id
PUT /project/api/v1/projects/:id/crisis-config
GET /project/api/v1/projects/:id/crisis-config
DELETE /project/api/v1/projects/:id/crisis-config

Ingest:

POST /ingest/api/v1/datasources
GET /ingest/api/v1/datasources/:id
GET /ingest/api/v1/datasources
PUT /ingest/api/v1/datasources/:id
POST /ingest/api/v1/datasources/:id/archive
POST /ingest/api/v1/datasources/:id/targets/keywords
POST /ingest/api/v1/datasources/:id/targets/profiles
POST /ingest/api/v1/datasources/:id/targets/posts
GET /ingest/api/v1/datasources/:id/targets
GET /ingest/api/v1/datasources/:id/targets/:target_id
PUT /ingest/api/v1/datasources/:id/targets/:target_id
POST /ingest/api/v1/datasources/:id/targets/:target_id/deactivate
DELETE /ingest/api/v1/datasources/:id/targets/:target_id
GET /ingest/api/v1/datasources/:id/dryrun/latest
GET /ingest/api/v1/datasources/:id/dryrun/history
GET /ingest/api/v1/internal/projects/:id/activation-readiness

Knowledge:

POST /knowledge/api/v1/search
POST /knowledge/api/v1/chat
GET /knowledge/api/v1/conversations/:id
GET /knowledge/api/v1/campaigns/:id/conversations
GET /knowledge/api/v1/campaigns/:id/suggestions
POST /knowledge/api/v1/reports/generate
GET /knowledge/api/v1/reports/:id
GET /knowledge/api/v1/reports/:id/download
GET /knowledge/api/v1/chat/jobs/:id
GET /knowledge/api/v1/internal/index/statistics/:project_id

Crawler/notification:

POST /scraper/tasks/facebook
GET /scraper/tasks/:id/result
GET /scraper/tasks
GET /notification/ws

Request/response summary rút gọn:

POST /project/api/v1/campaigns
-> HTTP 200, campaign_id=19ed2b60-2656-4d07-9872-5f0b99984345

POST /project/api/v1/campaigns/:id/projects

```
-> HTTP 200, project_id=3a9698ad-9d68-4789-alda-afab3a29ed1a, status=PENDING

GET /project/api/v1/projects/:id/activation-readiness
-> HTTP 200, can_proceed=false, reason=DATASOURCE_REQUIRED

POST /project/api/v1/projects/:id/activate
-> HTTP 200, error_code=160026, project không được activate khi thiếu datasource/target

POST /knowledge/api/v1/search
-> HTTP 200, total_results=0, no_relevant_context=true, latency khoảng 13 ms

POST /knowledge/api/v1/chat
-> HTTP 200, model=gemini-2.0-flash, conversation_id được tạo, latency khoảng 2266 ms

GET /knowledge/api/v1/internal/index/statistics/:project_id
-> HTTP 200, total_indexed=0

POST /scraper/tasks/facebook
-> HTTP 200, queue=facebook_tasks, task_id=d5c8e9c1-d7f0-4fb2-837c-02a7466970e4
```

Điểm quan trọng của kiểm thử này là mỗi kết quả đều có response cụ thể. Ví dụ, POST /project/api/v1/projects/:id/activate khi project chưa có datasource trả lỗi nghiệp vụ 160026, tức hệ thống từ chối đúng theo điều kiện kích hoạt. GET /knowledge/api/v1/internal/index/statistics/:project_id trả 200 với total_indexed=0 khi chưa có dữ liệu indexed. GET /scraper/tasks/:id/result trả trạng thái chưa có kết quả theo kỳ vọng vì task bất đồng bộ chưa hoàn tất.

6.4.3 UC-01: Thiết lập chiến dịch theo dõi

Luồng thiết lập campaign/project, datasource, target và dry run được kiểm tra bằng API trực tiếp và bằng giao diện production.

Hạng mục	Cách kiểm tra	Kết quả thực tế
Campaign CRUD	Gọi GET /domains, POST /campaigns, GET /campaigns/:id, GET /campaigns, PUT /campaigns/:id, favorite/unfavorite	8/8 pass; campaign test được tạo với ID 19ed2b60-2656-4d07-9872-5f0b99984345; danh sách trả về 3 campaign trong phiên E2E
Project CRUD	Gọi POST /campaigns/:id/projects, GET /projects/:id, GET /campaigns/:id/projects, PUT /projects/:id, favorite/unfavorite	7/7 pass ở CRUD cơ bản; project test được tạo với ID 3a9698ad-9d68-4789-alda-afab3a29ed1a, trạng thái ban đầu PENDING
Datasource CRUD	Gọi POST /datasources, GET /datasources/:id, GET /datasources, PUT /	6/6 pass trong báo cáo E2E; datasource test có ID fd1f2ca3-de8a-4b3f-8e2b-d910bdc67400;

	<code>datasources/:id</code> , POST / <code>datasources/:id/archive</code>	<code>source_category=CRAWL</code> được tự suy luận đúng
Target setup	Gọi tạo keyword, profile, post target; sau đó list, detail, update, deactivate, delete	Các endpoint chính trả 200; delete làm số target giảm từ 3 xuống 2
Dry run history	Gọi GET <code>/datasources/:id/ dryrun/history</code>	200; history trả danh sách rỗng có paginator khi chưa có kết quả dry run

Kiểm thử giao diện production bổ sung bằng chứng người dùng cuối:

Test case	Cách thao tác	Kết quả thực tế
TC-02, TC-03	Mở campaign switcher và chọn campaign khác	Dropdown hiển thị đủ 15/15 campaign; URL cập nhật ? <code>camp_id=</code> và header đổi đúng campaign
TC-04	Chuyển campaign rồi quan sát dashboard	4 KPI card Total Mentions , Sentiment Score , Engagement , Audience Reach reload theo campaign mới
TC-05	Vào tab Projects của campaign Ahamove	Danh sách trả 1 project tên Ahamove, trạng thái active; mentions 1,016, sentiment 28.1%, badge No crisis config hiển thị đúng

Kết luận UC-01: API thiết lập campaign/project, datasource, target và dry run đạt trong phạm vi đã kiểm thử. Giao diện hiển thị được dữ liệu thật và cập nhật đúng context campaign.

6.4.4 UC-02: Vận hành chiến dịch theo dõi

Pha vận hành kiểm tra readiness, kích hoạt project và các thao tác lifecycle chính sau khi dữ liệu thiết lập đã tồn tại.

Hạng mục	Cách kiểm tra	Kết quả thực tế
Project readiness	Gọi GET <code>/projects/:id/ activation-readiness</code> khi chưa có datasource	200; <code>can_proceed=false</code> , lý do <code>DATASOURCE_REQUIRED</code>

Activate sai điều kiện	Gọi POST /projects/:id/activate khi project thiếu datasource/target	Request bị từ chối đúng bằng lỗi nghiệp vụ 160026, không phát sinh 500
Activation readiness	Gọi readiness sau khi có đủ datasource và target hợp lệ	can_proceed=true, missing_dryrun=0 trong kịch bản đã chạy
Project activation	Gọi POST /projects/:id/activate khi điều kiện sẵn sàng đạt	Thành công, datasource chuyển sang trạng thái vận hành tương ứng
Archive/unarchive	Gọi POST /projects/:id/archive và POST /projects/:id/unarchive	Archive chuyển PENDING -> ARCHIVED; unarchive chuyển về PAUSED theo state machine hiện tại

Kịch bản kích hoạt đầy đủ dùng dữ liệu đã thiết lập ở UC-01 làm tiền điều kiện. Với các tổ hợp không yêu cầu dry run, hệ thống dùng cơ chế IsDryrunRequired(sourceType, targetType) để xác định khi nào có thể bỏ qua dry run trước khi cho phép project đi vào vận hành. Một lần chạy ghi nhận luồng sau:

Bước kiểm tra readiness/activation	Kết quả thực tế
Tiền điều kiện: campaign	Thành công, campaign ID 54cf8af8-...
Tiền điều kiện: project	Thành công, project ID 14d8a0ab-...
Tiền điều kiện: datasource	Thành công, datasource ID 0acb2ab6-..., trạng thái PENDING
Tiền điều kiện: target	Thành công, target ID 8c4502e4-...
Tiền điều kiện: target sẵn sàng	Thành công, is_active=true, dry run được bỏ qua đúng với mapping không yêu cầu
Kiểm tra datasource	status=READY, dryrun_status=NOT_REQUIRED
Activation readiness	can_proceed=true, missing_dryrun=0
Activate project	Thành công, affected_datasource_count=1
Kiểm tra datasource sau activate	status=ACTIVE

Kết luận UC-02: readiness, project activation và lifecycle state change đã được kiểm chứng bằng API thật; các bước datasource/target/dry run chỉ đóng vai trò tiền điều kiện thiết lập từ UC-01.

6.4.5 UC-03: Tra cứu và hỏi đáp dữ liệu phân tích

Knowledge-srv được kiểm tra ở các nhóm search, chat, conversation, suggestions và index statistics. Đây là phạm vi cốt lõi của UC-03 theo Chương 4.

Hạng mục	Cách kiểm tra	Kết quả thực tế
Search	Gọi POST /knowledge/search khi chưa có dữ liệu indexed	200, 0 result, no_relevant_context=true, thời gian khoảng 13 ms
Chat mới	Gọi POST /knowledge/chat với campaign/project context	200; tạo conversation mới, model trả lời bằng gemini-2.0-flash, thời gian khoảng 2266 ms
Chat follow-up	Gọi tiếp POST /knowledge/chat cùng conversation_id	200; dừng lại conversation, thời gian khoảng 1632 ms
Conversation detail	Gọi GET /conversations/:id	200; có 4 message gồm 2 user và 2 assistant
List conversations	Gọi GET /campaigns/:id/conversations	200; trả 1 conversation
Suggestions	Gọi GET /campaigns/:id/suggestions	200; trả 2 gợi ý tiếng Việt
Index statistics	Gọi GET /internal/index/statistics/:project_id	200; total_indexed=0

Ngoài phạm vi UC-03 cốt lõi, Knowledge-srv còn có capability mở rộng cho report generation. Capability này được ghi nhận riêng để không làm lệch mapping use case đã định nghĩa ở Chương 4.

Capability mở rộng	Cách kiểm tra	Kết quả thực tế
Report generate	Gọi POST /reports/generate	Request được tiếp nhận, report lưu đúng user_id=550e8400-e29b-41d4-a716-446655440099 và trả trạng thái PROCESSING

Report not found	Gọi GET /reports/:id và /download với ID không tồn tại	Trả đúng trạng thái không tìm thấy
------------------	--	------------------------------------

Kết luận UC-03: search/chat đạt ở mức API contract và xử lý trạng thái không có dữ liệu. Capability report generation được ghi nhận riêng ở lớp mở rộng. Chương này không kết luận chất lượng nội dung câu trả lời hoặc báo cáo khi dữ liệu đầy đủ, vì bộ kiểm thử hiện tại chưa có benchmark định lượng cho chất lượng LLM/report.

6.4.6 UC-04: Theo dõi trạng thái và nhận cảnh báo

Kiểm thử giao diện production dùng Browser Agent thao tác trực tiếp trên domain `smap.tantai.dev`, có bộ ảnh minh họa được trích xuất và đưa vào thư mục hình của báo cáo. Các test case dưới đây cho thấy giao diện đọc được dữ liệu production thật qua API.

Các kết quả có dữ liệu thực:

Test case	Cách kiểm tra	Kết quả thực tế
TC-01	Mở app với session authenticated	App load đầy đủ giao diện authenticated, header có campaign context, navigation hiển thị đủ
TC-08	Mở trang Insights và quan sát chart qua analytics API	Charts render dữ liệu thật: YouTube 1,031 mentions, 12.3K engagement, 28% sentiment; sentiment gồm 71% Neutral, 29% Negative, 0% Positive
TC-09	Mở Reports, bấm Generate Report	Modal mở đúng, có lựa chọn From Campaign Data, Competitor Analysis, các toggle Overview/Sentiment/Trends/Recommendations, date range và project dropdown
TC-11	Mở MAP tab, notification panel và user menu	MAP tab load KPI 1.1K mentions, 26 sentiment, 12.6K engagement, 15.1M audience reach; notification panel hiển thị empty state; user menu mở được

Kết luận UC-04: dashboard và các màn hình chính đọc được dữ liệu production thật; notification panel và WebSocket route cũng có bằng chứng ở mức route/delivery nội bộ như trình bày thêm ở mục 6.4.8.

6.4.7 UC-05: Thiết lập và quản lý quy tắc cảnh báo khủng hoảng

UC-05 được kiểm tra trước hết bằng API CRUD cho crisis config gắn với project. Ngoài CRUD cấu hình, báo cáo cũng ghi nhận runtime apply như một luồng hỗ trợ để chứng minh trạng thái khủng hoảng có thể được áp dụng xuống crawl mode; phần phát hiện và quan sát cảnh báo thuộc UC-04, FR-09 và FR-11.

Hạng mục	Cách kiểm tra	Kết quả thực tế
Campaign/project dùng kiểm thử	Dùng campaign 496b8aa4-2e2c-43d9-a06b-abe2536fb2b2, project 398b42b6-dd0c-4bed-8778-12af0add91fb	Readiness trước kiểm thử có datasource_count=2, can_proceed=true
Crisis config CRUD	Gọi PUT, GET, DELETE / projects/:id/crisis-config	3/3 pass; sau khi xóa, GET trả lỗi Crisis config not found
Manual apply CRITICAL	Gọi POST /project/api/v1/internal/projects/:id/crisis-config/apply-runtime	Response error_code=0, crisis_status=CRITICAL, applied_crawl_mode=CRISIS, affected_datasource_count=2
Verify ingest state	Gọi GET /ingest/api/v1/datasources? project_id=...	Cả 2 datasource chuyển sang crawl_mode=CRISIS
Toggle NORMAL/ CRITICAL	Gọi apply-runtime với override NORMAL, sau đó CRITICAL	Mode đổi đúng CRISIS -> NORMAL -> CRISIS, mỗi lần ảnh hưởng 2 datasource
Audit database	Query bảng ingest.crawl_mode_changes	8 dòng audit cho 4 event manual, tương ứng 2 datasource mỗi event
Auto-apply từ analysis-consumer	Inject synthetic high-pressure logistics records vào topic smap.collector.output	Log ghi crisis_level=warning, composite_score=2.8, Crisis runtime applied, affected_datasources=2

Auto-apply audit	Query ingest.crawl_mode_changes	4 dòng audit cho 2 transition auto: NORMAL -> CRISIS và CRISIS -> NORMAL
Cleanup	Xóa crisis config và gọi lại apply-runtime	Trả error_code=160001 (Crisis config not found), đúng kỳ vọng sau cleanup

Evidence runtime apply rút gọn:

```
POST /project/api/v1/internal/projects/:id/crisis-config/apply-runtime
-> error_code=0
-> crisis_status=CRITICAL
-> applied_crawl_mode=CRISIS
-> affected_datasource_count=2
```

```
GET /ingest/api/v1/datasources?project_id=398b42b6-dd0c-4bed-8778-12af0add91fb
-> 2 datasource chuyển sang crawl_mode=CRISIS
```

```
DB audit ingest.crawl_mode_changes
-> manual apply: 8 rows cho 4 event, mỗi event 2 datasource
-> auto apply: 4 rows cho 2 transition NORMAL -> CRISIS và CRISIS -> NORMAL
```

```
analysis-consumer log
-> crisis_level=warning, composite_score=2.8, signals=2
-> Crisis runtime applied, crisis_status=WARNING, applied_crawl_mode=CRISIS, affected_datasources=2
```

Kết luận UC-05: crisis config CRUD đạt trong phạm vi đã kiểm thử. Runtime bridge analysis-consumer -> project-srv -> ingest-srv là bằng chứng hỗ trợ cho việc áp dụng trạng thái khủng hoảng trong cluster thật, với đối chiếu response API, trạng thái datasource, audit table và log consumer.

6.4.8 Crawler worker và notification

Crawler worker được kiểm tra bằng task API:

Hạng mục	Cách kiểm tra	Kết quả thực tế
Tạo task Facebook	Gọi POST /tasks/facebook	200; task được đưa vào queue facebook_tasks, trả task_id=d5c8e9c1-d7f0-4fb2-837c-02a7466970e4
Đọc result khi task còn xử lý	Gọi GET /tasks/:id/result	404 với thông điệp task có thể vẫn đang xử lý, phù hợp mô hình async
Liệt kê task	Gọi GET /tasks	200; danh sách rỗng vì kết quả lưu theo file và chưa

		có task hoàn tất trong phiên kiểm thử
--	--	---------------------------------------

Notification-srv được kiểm tra theo hai mức. Ở unit test, WebSocket delivery và Redis delivery đều pass như mục 6.3. Ở gateway E2E, endpoint WebSocket đã reachable và yêu cầu xác thực trước khi nâng cấp kết nối. Kiểm thử hai chiều bằng WebSocket client đầy đủ chưa được ghi nhận bằng số pass riêng, nên báo cáo chỉ kết luận route và delivery nội bộ đã được xác minh, chưa kết luận độ trễ hoặc chất lượng kết nối WebSocket end-to-end.

6.5 Đánh giá phi chức năng

6.5.1 Phạm vi và phương pháp đánh giá

Đánh giá phi chức năng được thực hiện bằng workload HTTP trên môi trường K3s, kết hợp thu thập metric từ request log, cluster snapshot, queue snapshot và database snapshot. Mục tiêu là đo các chỉ số có thể quan sát trực tiếp: latency, throughput, timeout, infra error, trạng thái pod, queue/backlog, health check và trace của runtime apply. Trong phạm vi báo cáo, kết quả định lượng cứng chủ yếu áp dụng cho API-lane; các snapshot về queue, database và runtime state được dùng để bổ sung ngữ cảnh vận hành, chưa dùng để khóa toàn bộ NFR định lượng cho cả hệ thống.

Nhóm đánh giá	Cách kiểm tra	Chỉ số thu thập
Performance	Chạy workload HTTP bằng script, đo thời gian từng request	Throughput, p50, p95, p99
Stability	Phân loại HTTP status, timeout và transport error	Timeout %, infra error %
Availability	Gọi health endpoint của service và kiểm tra trạng thái pod	Service health, pod running, restart state
Recovery	Chạy scenario restart một analysis consumer và tiếp tục workload	Throughput, timeout, infra error, queue/backlog snapshot
Resource	Thu snapshot Kubernetes bằng kubectl và kubectl top	CPU, memory, pod status
Data integrity	Đối chiếu unit test analytics contract, runtime state và audit record	UAP mapping, output contract, crawl mode change, audit rows

Observability	Thu log, request metrics, queue snapshot và DB snapshot	Trace, status, latency, queue lag, database evidence
---------------	---	--

6.5.2 Performance và stability

Bốn kịch bản workload được chạy trên API-lane gồm baseline, expected load, peak load và chaos consumer restart. Mỗi kịch bản thu p50/p95/p99, throughput, timeout và infra error. Infra error được tính riêng từ lỗi hạ tầng như 5xx, timeout hoặc status không nhận được; các response nghiệp vụ được tách khỏi nhóm này.

Kịch bản	Thời lượng	Throughput	p50 / p95 / p99	Timeout	Infra error
Baseline	20 phút	6.949 req/s	41.652 / 159.397 / 357.020 ms	0.0360%	0.0360%
Expected load	20 phút	7.265 req/s	41.760 / 171.699 / 285.326 ms	0.0000%	0.0000%
Peak load	20 phút	7.440 req/s	40.992 / 220.387 / 433.074 ms	0.0000%	0.0000%
Chaos consumer restart	12 phút	7.751 req/s	41.228 / 224.661 / 324.243 ms	0.0000%	0.0000%

Kết quả cho thấy API-lane giữ p95 trong khoảng 159.397-224.661 ms ở các kịch bản đã chạy. Throughput quan sát được nằm trong khoảng 6.949-7.751 req/s. Baseline ghi nhận infra error 0.0360% do timeout/status 000 theo định nghĩa 5xx + status 000; ba kịch bản còn lại ghi nhận infra error 0.0000%. Các số liệu này phản ánh hành vi API-lane trong workload 12-20 phút, không được diễn giải thành ngưỡng bảo hòa hoặc năng lực tải tối đa của toàn hệ thống.

6.5.3 Availability và recovery

Availability được kiểm tra bằng health endpoint và trạng thái runtime của các service chính. Trong cùng môi trường kiểm thử, sáu service identity-srv, project-srv, ingest-srv, knowledge-srv, notification-srv và crawler worker đều trả HTTP 200 ở health check. Notification-srv đồng thời trả trạng thái Redis `connected`; crawler worker trả `worker_active=true`.

Recovery được kiểm tra bằng kịch bản restart một analysis consumer trong khi workload vẫn chạy. Kết quả kịch bản chaos ghi nhận throughput 7.751 req/s, p95 224.661 ms, timeout 0.0000% và infra error 0.0000%. Queue/backlog snapshot vẫn được thu trong thời gian theo dõi để quan sát trạng thái xử lý bất đồng bộ sau sự kiện restart; tuy nhiên kịch bản này chưa chuẩn hóa MTTR hoặc backlog drain time thành chỉ tiêu định lượng cuối cùng.

6.5.4 Resource và khả năng mở rộng

Resource được đo bằng snapshot Kubernetes trong các kịch bản workload. Các node vẫn ở trạng thái Ready; CPU node quan sát trong khoảng thấp đến trung bình, khoảng 4-14%, memory khoảng 54-61%. Các pod chính của API-lane tiếp tục ở trạng thái running trong cửa sổ đo.

Nội dung	Cách kiểm tra	Kết quả quan sát
Node capacity	kubectl top node, cluster snapshot	CPU khoảng 4-14%, memory khoảng 54-61%
Pod runtime	kubectl get pods, restart/ pressure snapshot	Pod chính running trong cửa sổ workload
API throughput theo tải	So sánh baseline, expected load, peak load	Throughput tăng từ 6.949 lên 7.440 req/s; p95 tăng từ 159.397 lên 220.387 ms
Chaos workload	Restart một analysis consumer trong khi đo API-lane	Throughput 7.751 req/s; p95 224.661 ms

Evidence resource snapshot rút gọn:

Node resource:

```
k3s-01 CPU 4% memory 55%
k3s-02 CPU 13% memory 58%
k3s-03 CPU 4% memory 49%
```

Pod resource:

```
analysis-api          2m CPU    67Mi memory
analysis-consumer     4m CPU    1223Mi memory
identity-srv          1m CPU    18Mi memory
ingest-srv             1m CPU    19Mi memory
knowledge-srv          3m CPU    24Mi memory
notification-srv       2m CPU    6Mi memory
project-srv            1m CPU    15Mi memory
crawler-worker         2m CPU    61Mi memory
rabbitmq               71m CPU   143Mi memory
redis                  6m CPU    163Mi memory
redpanda               7m CPU    264Mi memory
```

Kết quả này phù hợp với thiết kế mở rộng theo lane ở Chương 5: control plane, ingest execution, analytics consumer, crawler worker và notification delivery được tách service, giúp theo dõi và mở rộng từng nhóm độc lập.

6.5.5 Security

Security được kiểm chứng ở hai mức: unit test của identity-srv và kiểm thử endpoint xác thực nội bộ. Bộ unit test identity-srv bao phủ authentication delivery, authentication use case, repository, audit và user module. Ở kiểm thử đầu cuối, endpoint internal validate trả 200 với token hợp lệ, kèm user_id và role ADMIN. Các service-to-service path dùng internal auth cũng được kiểm tra qua internal project lookup và runtime apply.

Nội dung	Cách kiểm tra	Kết quả quan sát
Token validation	Gọi internal validate bằng JWT hợp lệ	200, trả user_id và role ADMIN
Authentication logic	Unit test identity-srv	Các package authentication, audit, user và model pass
Internal auth	Gọi internal project lookup và runtime apply	Request hợp lệ được xử lý qua internal path

6.5.6 Data integrity

Data integrity được kiểm tra bằng cả unit test cấp module và bằng chứng runtime ở các lát cắt có thể đối chiếu trực tiếp. Ở analysis-srv, bộ 309 tests kiểm tra UAP ingest record, format detection, normalization, dedup, enrichment, sentiment/intent calibration, keyword extraction, crisis assessment, reporting, storage và output contract. Ở runtime apply, trạng thái crawl mode được đối chiếu giữa response API, danh sách datasource và bảng audit. Các chỉ báo toàn tuyến như duplicate, loss, orphan hoặc data completeness dài hạn chưa được khóa thành hard-result trong lần đo này.

Nội dung	Cách kiểm tra	Kết quả quan sát
UAP và output contract	Chạy bộ test analysis-srv	309 tests pass
Dedup/normalization/enrichment	Unit test analysis-srv	Các nhóm test tương ứng pass
Runtime state	Gọi apply-runtime rồi đọc datasource state	2 datasource đổi đúng NORMAL -> CRISIS và CRISIS -> NORMAL theo kịch bản

Audit trail	Query bảng <code>ingest.crawl_mode_changes</code>	Có audit row cho từng datasource trong các lần đổi mode
-------------	--	---

6.5.7 Modularity và observability

Modularity được đánh giá bằng cách đối chiếu kết quả test với ranh giới service. Các kiểm thử chức năng, unit test và NFR đều bám theo từng service: identity-srv chịu trách nhiệm xác thực, project-srv quản lý campaign/project/crisis, ingest-srv quản lý datasource/target/execution, analysis-srv xử lý analytics pipeline, knowledge-srv phục vụ search/chat/report, notification-srv phục vụ WebSocket/Redis delivery, crawler worker xử lý task crawling.

Observability được kiểm tra qua khả năng thu thập bằng chứng trong lúc workload chạy. Các artifact thu được gồm request metrics, status summary, cluster snapshot, queue snapshot, database snapshot, health summary và log runtime apply. Nhờ đó, các kịch bản NFR có thể đối chiếu request-level metric với trạng thái runtime của service, queue hoặc database ở mức snapshot; phần đối chiếu database đầy đủ nhất nằm ở baseline có DB evidence.

Nội dung	Cách kiểm tra	Kết quả quan sát
Service boundary	Đối chiếu test theo từng service	Test được tách theo identity, project, ingest, analysis, knowledge, notification và crawler
Request observability	Thu request metrics và status summary	Có p50/p95/p99, throughput, timeout và infra error
Runtime observability	Thu cluster, queue, DB snapshot và log	Có bằng chứng pod state, queue/backlog, database state và runtime apply

6.6 Đối chiếu tổng hợp với yêu cầu và kiến trúc

Yêu cầu / thiết kế	Bảng chứng kiểm chứng	Mức đánh giá
FR-01 Xác thực người dùng	Unit test identity-srv, endpoint validate 200	Đạt trong phạm vi token/internal auth

FR-02 Campaign/project	Campaign CRUD 8/8, project CRUD/lifecycle 12/12, unit test project- srv	Đạt
FR-03 Lifecycle	Readiness, activate rejection hợp lệ, archive/ unarchive, runtime apply	Đạt trong kịch bản đã chạy
FR-04 Crisis config	Crisis config CRUD 3/3, crisis assessment, runtime apply	Đạt
FR-05 Datasource	Datasource CRUD 6/6, ingest-srv datasource package đạt statement coverage ở package được đo	Đạt
FR-06 Target	Target keyword/profile/ post, list/detail/update/ deactivate/delete, ingest- srv use case test	Đạt trong kịch bản đã chạy
FR-07 Dry run	Dry run package đạt statement coverage ở package được đo, dry run history, cơ chế IsDryrunRequired	Đạt trong phạm vi mapping được kiểm chứng
FR-08 Điều phối crawl	Crawler task scenario, RabbitMQ lane, UAP Kafka package đạt statement coverage ở package được đo	Đạt trong kịch bản đã chạy
FR-09 Analytics	analysis-srv 309 tests pass	Đạt
FR-10 Search/chat	Search 1/1 pass, chat 5/5 pass	Đạt ở mức endpoint/ contract
FR-11 Notification	notification-srv unit test, Redis delivery, WebSocket route yêu cầu xác thực	Đạt trong phạm vi delivery và routing

FR-12 Internal interop	Internal validate, internal project lookup, project service client test	Đạt trong luồng đã kiểm chứng
------------------------	---	-------------------------------

Lane kiến trúc Chương 5	Test tương ứng	Nhận xét
Internal HTTP control plane	identity-srv, project-srv, ingest-srv, knowledge-srv	Có bằng chứng tốt ở API/control flow
RabbitMQ crawl runtime	ingest-srv dry run RabbitMQ packages, crawler task scenario	Có bằng chứng module và task scenario
Kafka analytics data plane	UAP Kafka package, analysis output contract, shared Kafka helper	Có bằng chứng tốt ở module
Redis notification ingress	notification-srv Redis delivery, miniredis-based test	Có bằng chứng unit test
Qdrant knowledge layer	Search/chat, index statistics	Có bằng chứng endpoint và nghiệm thu chức năng
PostgreSQL/Redis/MinIO state	Repository tests, database evidence, runtime state snapshot	Có bằng chứng theo module và scenario

6.7 Phạm vi diễn giải kết quả

Các kết quả trong chương này được diễn giải theo đúng phạm vi đã đo. Kiểm thử đầu cuối và NFR dùng các campaign/project cụ thể trong môi trường K3s tham chiếu, vì vậy kết luận tập trung vào những luồng và trạng thái đã có bằng chứng trực tiếp. Với các tổ hợp dữ liệu, platform hoặc rule chưa nằm trong kịch bản, hệ thống cần được mở rộng thêm test case khi đưa vào vận hành thực tế.

Các chỉ số performance hiện tại phản ánh API-lane trong cửa sổ workload 12-20 phút. Những chỉ số vận hành dài hạn như MTTR chuẩn hóa, backlog drain time theo nhiều dạng chaos, saturation threshold theo replica và soak nhiều giờ nên được tiếp tục đo trong các đợt kiểm thử vận hành sau. Vì vậy, phần NFR chỉ chốt kết quả định lượng cho latency, throughput, timeout, infra error và xu hướng tài nguyên sơ bộ trong phạm vi các scenario đã chạy; các chỉ báo toàn hệ về integrity, capacity resilience và long-running stability được để lại cho các đợt đo tiếp theo.

Đối với notification, báo cáo đã có bằng chứng unit test cho WebSocket/Redis delivery và route WebSocket qua gateway yêu cầu xác thực. Kiểm thử hai chiều bằng WebSocket client đầy đủ nên được bổ sung khi cần chốt thêm số liệu về độ trễ và độ ổn định kết nối thời gian thực.

6.8 Kết luận

Kết quả kiểm thử cho thấy các luồng thuộc phạm vi đồ án đã được nghiệm thu chức năng theo tầng xử lý: delivery, use case, repository, model, transport hoặc runtime boundary. Các bằng chứng ở mục 6.3 cho thấy các package/nhóm test được liệt kê của identity-srv, project-srv, ingest-srv, notification-srv, analysis-srv, shared-lib, knowledge-srv, crawler worker và smap-analyse đều pass trong lần chạy này. Với ingest-srv, báo cáo unit test trích nguyên văn cho thấy 15 package thuộc datasource, dry run, execution và UAP đều pass và đạt 100.0% statement coverage ở các package được đo.

Đối với yêu cầu chức năng, các luồng chính như campaign/project, datasource/target, activation, search/chat/report, crisis runtime apply, crawler task và notification delivery đều có bằng chứng kiểm thử cụ thể ở mức API, UI hoặc runtime state. Đối với yêu cầu phi chức năng, các kịch bản NFR cho thấy API-lane duy trì p95 dưới 225 ms trong các scenario đã chạy, throughput nằm trong khoảng 6.949-7.751 req/s và infra error nằm trong khoảng 0.0000-0.0360% trong cửa sổ đo. Nhìn chung, trong phạm vi kiểm thử đã thực hiện, hệ thống có bằng chứng phù hợp với các ranh giới service và kiến trúc microservices đã thiết kế; các chỉ số vận hành toàn hệ và dài hạn cần tiếp tục được mở rộng trong các đợt đo sau.

CHƯƠNG 7: TỔNG KẾT

7.1 Tổng kết kết quả thực hiện

Đề tài đã xây dựng SMAP theo một chuỗi nội dung tương đối đầy đủ từ khảo sát bài toán, phân tích yêu cầu, thiết kế kiến trúc đến kiểm thử và đánh giá hệ thống. Ở giai đoạn phân tích, báo cáo đã xác định bối cảnh của bài toán social listening, đối chiếu với một số hệ thống liên quan và rút ra nhu cầu cốt lõi: người dùng cần thiết lập phạm vi theo dõi, vận hành thu thập dữ liệu, khai thác kết quả phân tích và nhận biết các tình huống cần chú ý trong quá trình theo dõi.

Trên cơ sở đó, Chương 4 đã đặc tả hệ thống bằng mười hai yêu cầu chức năng, bảy yêu cầu phi chức năng và năm use case nghiệp vụ chính. Cách tổ chức use case tập trung vào mục tiêu của người dùng thay vì tên module kỹ thuật: thiết lập chiến dịch theo dõi, vận hành chiến dịch, tra cứu và hỏi đáp dữ liệu phân tích, theo dõi trạng thái và nhận cảnh báo, thiết lập hoặc quản lý quy tắc cảnh báo khủng hoảng. Các cơ chế như xác thực, xử lý analytics, kiểm tra nội bộ hoặc message bus được đặt đúng vai trò là capability hỗ trợ cho các use case được bảo vệ, không bị tách thành use case người dùng giả tạo.

Ở Chương 5, đề tài đã thiết kế SMAP như một hệ thống microservices có phân tách rõ theo capability và workload lane. `identity-srv` giữ security boundary; `project-srv` quản lý campaign, project, lifecycle và crisis configuration; `ingest-srv` cùng `scraper-srv` hình thành execution plane cho datasource, target, dry run và crawl task; `analysis-srv` gồm `analysis-consumer` cho analytics pipeline và `analysis-api` cho dashboard read path; `knowledge-srv` phục vụ search, chat, report và indexing; `notification-srv` giữ delivery boundary cho Redis/WebSocket; frontend đóng vai trò giao diện và backend-for-frontend cho browser. Thiết kế này giúp tách control plane khỏi các luồng xử lý nền nặng, đồng thời cho phép từng lane có cách scale, storage và transport phù hợp.

Về dữ liệu và giao tiếp, báo cáo đã làm rõ vai trò của PostgreSQL, Redis, MinIO và Qdrant trong mô hình polyglot persistence. PostgreSQL lưu metadata nghiệp vụ, insight và trạng thái xử lý; Redis hỗ trợ cache hoặc Pub/Sub notification; MinIO lưu artifact lớn; Qdrant phục vụ semantic retrieval. Về transport, hệ thống sử dụng internal HTTP cho các tương tác đồng bộ cần phản hồi ngay, RabbitMQ cho crawl execution runtime, Kafka cho analytics data plane và Redis Pub/Sub cho notification ingress. Việc chọn nhiều cơ chế giao tiếp làm tăng độ phức tạp vận hành, nhưng phù hợp với bản chất không đồng nhất của workload trong SMAP.

Chương 6 bổ sung lớp bằng chứng thực nghiệm cho các thiết kế trên. Các bộ kiểm thử unit theo service cho thấy những package/nhóm test được liệt kê của `identity-srv`, `project-srv`, `ingest-srv`, `notification-srv`, `analysis-srv`, `shared-lib`s, `knowledge-srv`, `crawler worker` và `smap-analyse` đều pass trong lần chạy được báo cáo. Các kịch bản kiểm thử

chức năng và đầu cuối đã xác minh nhiều luồng chính như quản lý campaign/project, datasource/target, activation, search/chat/report, crisis runtime apply, crawler task và notification delivery ở mức API, UI hoặc runtime state. Các kịch bản NFR cho thấy API-lane duy trì p95 dưới 225 ms trong các scenario đã chạy, throughput quan sát nằm trong khoảng 6.949-7.751 req/s và không ghi nhận infra error trong cửa sổ đo.

Nhìn chung, kết quả đạt được không chỉ là một tập hợp sơ đồ hoặc mô tả rời rạc. Báo cáo đã nối được yêu cầu, use case, activity diagram, service boundary, data design, sequence diagram, communication pattern, deployment view và kiểm thử thành một mạch diễn giải thống nhất. Nhờ đó, người đọc có thể truy vết từ nhu cầu nghiệp vụ ở Chương 4 sang quyết định kiến trúc ở Chương 5 và bằng chứng kiểm chứng ở Chương 6.

7.2 Đóng góp chính của đề tài

Đóng góp thứ nhất của đề tài là cách tổ chức lại bài toán social listening thành một mô hình yêu cầu có ranh giới rõ. Thay vì mô tả hệ thống theo từng màn hình hoặc từng service, báo cáo tách rõ mục tiêu người dùng, capability hỗ trợ và các concern kỹ thuật. Điều này giúp bộ use case cô đọng hơn, tránh nhầm lẫn giữa nghiệp vụ người dùng và luồng xử lý nội bộ như analytics, indexing hoặc queue processing.

Đóng góp thứ hai là thiết kế kiến trúc đa lane cho một hệ thống có nhiều kiểu tải khác nhau. SMAP không ép toàn bộ xử lý vào request-response đồng bộ mà tách riêng control plane, execution plane, analytics data plane, knowledge retrieval và notification delivery. Cách tiếp cận này phù hợp với các hệ thống cần vừa phản hồi thao tác người dùng, vừa xử lý crawl, phân tích NLP, indexing và phát thông báo trong nền.

Đóng góp thứ ba là mô tả được sự phối hợp giữa nhiều cơ chế lưu trữ và truyền thông theo trách nhiệm cụ thể. PostgreSQL, Redis, MinIO, Qdrant, RabbitMQ và Kafka không được liệt kê như các công nghệ độc lập, mà được đặt vào từng lane xử lý và từng loại dữ liệu. Điều này làm rõ vì sao hệ thống cần polyglot persistence và vì sao mỗi transport được chọn cho một nhóm tương tác khác nhau.

Đóng góp thứ tư là việc gắn thiết kế với bằng chứng kiểm thử. Chương 6 không chỉ nêu rằng hệ thống đã được kiểm thử, mà chỉ ra lớp kiểm thử, công cụ, phạm vi, kết quả quan sát và giới hạn diễn giải. Các kết quả này giúp kết luận về hệ thống có cơ sở hơn: những phần đã đo thì được nêu bằng số liệu hoặc evidence cụ thể; những phần chưa đo đủ thì được giữ ở mức phạm vi cần mở rộng sau.

7.3 Hạn chế

Hạn chế đầu tiên nằm ở phạm vi dữ liệu và kịch bản kiểm thử. Các kết quả E2E và NFR trong Chương 6 được đo trên những campaign/project cụ thể trong môi trường K3s tham chiếu, vì vậy chúng không nên được hiểu như cam kết tuyệt đối cho mọi tổ hợp dữ liệu, nền tảng, rule hoặc profile tải. Các chỉ số latency và throughput hiện phản

ánh API-lane trong cửa sổ workload 12-20 phút; những phép đo dài hơn như soak test nhiều giờ, saturation threshold theo replica, backlog drain time hoặc nhiều dạng chaos khác vẫn cần được bổ sung nếu hệ thống bước vào vận hành chính thức.

Hạn chế thứ hai là độ trưởng thành vận hành chưa đồng đều giữa mọi runtime. Hệ thống đã có health check, logging, probe và một số bằng chứng observability trong kiểm thử, nhưng metrics surface chưa được chuẩn hóa hoàn toàn giữa các service. Trace ID propagation và log correlation có giá trị khi chẩn đoán, nhưng chưa đi kèm một lớp visualization xuyên service hoàn chỉnh. Các nội dung như runbook, backup/restore, disaster recovery và chính sách retention cũng cần được hoàn thiện thêm để tạo thành một bộ vận hành nhất quán.

Hạn chế thứ ba liên quan đến notification và realtime. Báo cáo đã có bằng chứng unit test cho WebSocket/Redis delivery và route WebSocket qua gateway yêu cầu xác thực, nhưng kiểm thử hai chiều bằng WebSocket client đầy đủ chưa được ghi nhận bằng số pass riêng. Vì vậy, kết luận hiện tại chỉ nên dừng ở khả năng route, xác thực và delivery nội bộ, chưa nên mở rộng thành cam kết về độ trễ realtime end-to-end hoặc độ ổn định kết nối trong nhiều điều kiện mạng khác nhau.

Hạn chế thứ tư đến từ bản chất của bài toán thu thập và phân tích dữ liệu mạng xã hội. Crawl runtime phụ thuộc vào chính sách, quota, cấu trúc giao diện và thay đổi kỹ thuật từ các nền tảng bên ngoài. Analytics lane hiện cũng được kiểm chứng trong phạm vi mô hình, ontology và dữ liệu nhất định, chưa phải pipeline đa ngôn ngữ hoặc đa miền tổng quát cho mọi ngữ cảnh. Với knowledge layer, chất lượng câu trả lời và báo cáo LLM cần thêm benchmark hoặc tiêu chí đánh giá nội dung nếu muốn kết luận ở mức chất lượng tri thức, thay vì chỉ ở mức endpoint/contract.

7.4 Hướng phát triển

Hướng phát triển quan trọng nhất là tăng độ trưởng thành vận hành. Hệ thống cần được bổ sung metrics chuẩn cho các runtime chính, dashboard giám sát theo lane, trace visualization xuyên service, alerting vận hành và runbook xử lý sự cố. Các quy trình backup, restore, retention và disaster recovery nên được cụ thể hóa bằng cấu hình và rehearsal thay vì chỉ dừng ở định hướng thiết kế.

Hướng thứ hai là mở rộng kiểm thử phi chức năng. Các kịch bản hiện tại nên được bổ sung thêm soak test dài hơn, chaos test nhiều dạng hơn, đo backlog drain time cho RabbitMQ/Kafka, kiểm thử saturation khi tăng replica và kiểm thử WebSocket end-to-end bằng client đầy đủ. Những phép đo này sẽ giúp chuyển các giả định về availability, scalability và realtime delivery thành bằng chứng vận hành có độ tin cậy cao hơn.

Hướng thứ ba là nâng cấp analytics và knowledge layer. Analytics pipeline có thể tiếp tục được cải thiện ở các bước calibration sentiment, intent, keyword extraction, crisis assessment và domain ontology. Knowledge layer có thể được mở rộng bằng benchmark

chất lượng retrieval, đánh giá hallucination, đánh giá citation grounding và cải thiện report generation. Khi các capability này được đo bằng tiêu chí nội dung rõ ràng, báo cáo trong tương lai có thể kết luận không chỉ endpoint hoạt động đúng mà còn chất lượng phân tích đạt yêu cầu.

Hướng thứ tư là mở rộng execution plane một cách có kiểm soát. Việc thêm nền tảng crawl, kiểu datasource hoặc target mới cần đi kèm contract dữ liệu, dry run mapping, quota policy và test case tương ứng. Mỗi mở rộng nên giữ nguyên nguyên tắc của kiến trúc hiện tại: control plane không bị kéo vào workload crawl nặng, artifact lớn vẫn đi qua object storage, còn dữ liệu chuẩn hóa tiếp tục được publish vào analytics data plane bằng contract có thể truy vết.

7.5 Kết luận

SMAP đã đạt được mục tiêu chính của đề tài: xây dựng một hệ thống social media analytics có thiết kế nhất quán, có phân tách trách nhiệm rõ và có bằng chứng kiểm thử cho các luồng trọng yếu. Từ góc nhìn nghiệp vụ, hệ thống hỗ trợ các bước chính của một quy trình social listening nội bộ: thiết lập chiến dịch, vận hành thu thập dữ liệu, khai thác kết quả phân tích, theo dõi trạng thái và quản lý ngữ cảnh cảnh báo khủng hoảng. Từ góc nhìn kỹ thuật, hệ thống được tổ chức thành các service và lane xử lý phù hợp với đặc điểm tải của từng nhóm chức năng.

Giá trị nổi bật của đề tài nằm ở tính truy xuất nguồn gốc giữa yêu cầu, kiến trúc và kiểm thử. Các quyết định như tách service theo capability, dùng RabbitMQ cho crawl task, dùng Kafka cho analytics data plane, dùng Redis cho notification ingress, dùng Qdrant cho retrieval và tách `analysis-api` khỏi `analysis-consumer` đều có liên hệ trực tiếp với yêu cầu hoặc đặc tính chất lượng đã đặt ra. Chương 6 cho thấy nhiều phần quan trọng của thiết kế đã được kiểm chứng bằng test cụ thể, đồng thời cũng chỉ ra ranh giới của những kết luận chưa nên suy rộng.

Mặc dù vẫn còn các hướng cần hoàn thiện về vận hành dài hạn, observability, benchmark chất lượng tri thức và kiểm thử realtime sâu hơn, nền tảng hiện tại đã đủ rõ để tiếp tục phát triển thành một hệ thống vận hành thực tế. Theo nghĩa đó, đề tài không chỉ dừng ở việc mô tả một kiến trúc đề xuất, mà đã hình thành được một bản thiết kế có kiểm chứng, có giới hạn diễn giải rõ ràng và có lộ trình phát triển tiếp theo.

CHƯƠNG 8: TÀI LIỆU THAM KHẢO

1. Andi. (2024). *Andi - Social Listening and Media Monitoring*. Truy cập tại: <https://andi.vn/>
2. Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet Allocation. *Journal of Machine Learning Research*, 3, 993-1022.
3. Brandwatch. (2024). *Brandwatch - Consumer Intelligence Platform*. Truy cập tại: <https://www.brandwatch.com/>
4. Brown, S. (2024). *The C4 Model for Visualising Software Architecture*. Truy cập tại: <https://c4model.com/>
5. Buzzmetrics. (2024). *Buzzmetrics - Social Media Research*. Truy cập tại: <https://buzzmetrics.com/>
6. Cockburn, A. (2000). *Writing Effective Use Cases*. Addison-Wesley Professional.
7. Commsights. (2024). *Commsights - Media Intelligence Solutions*. Truy cập tại: <https://commsights.net/>
8. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019*, 4171-4186.
9. Docker Documentation. (2024). *Docker Documentation*. Truy cập tại: <https://docs.docker.com/>
10. Electron Documentation. (2024). *Electron Documentation*. Truy cập tại: <https://www.electronjs.org/docs/latest>
11. Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.
12. Facebook Graph API. (2024). *Graph API Documentation*. Truy cập tại: <https://developers.facebook.com/docs/graph-api>
13. FFmpeg Documentation. (2024). *FFmpeg Documentation*. Truy cập tại: <https://ffmpeg.org/documentation.html>
14. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.
15. Fowler, M. (2014). *Microservices: A Definition of This New Architectural Term*. Truy cập tại: <https://martinfowler.com/articles/microservices.html>
16. Gin Web Framework. (2024). *Gin Web Framework Documentation*. Truy cập tại: <https://gin-gonic.com/docs/>
17. Go Documentation. (2024). *The Go Programming Language Documentation*. Truy cập tại: <https://go.dev/doc/>

18. Hohpe, G., & Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional.
19. Hugging Face Transformers. (2024). *Transformers Documentation*. Truy cập tại: <https://huggingface.co/docs/transformers/>
20. IEEE. (1998). *IEEE Recommended Practice for Software Requirements Specifications*. IEEE Std 830-1998.
21. IETF. (2011). *RFC 6455 - The WebSocket Protocol*. Truy cập tại: <https://tools.ietf.org/html/rfc6455>
22. INCOSE. (2015). *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities* (4th ed.).
23. ISO/IEC/IEEE. (2011). *Systems and Software Engineering - Architecture Description*. ISO/IEC/IEEE 42010:2011.
24. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
25. Kompa. (2024). *Kompa - Data and Market Intelligence Platform*. Truy cập tại: <https://kompa.ai/>
26. Kubernetes Documentation. (2024). *Kubernetes Documentation*. Truy cập tại: <https://kubernetes.io/docs/>
27. Apache Kafka Documentation. (2024). *Apache Kafka Documentation*. Truy cập tại: <https://kafka.apache.org/documentation/>
28. Liu, B. (2012). Sentiment Analysis and Opinion Mining. *Synthesis Lectures on Human Language Technologies*, 5(1), 1-167.
29. Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
30. Meltwater. (2024). *Meltwater - Media Intelligence Platform*. Truy cập tại: <https://www.meltwater.com/>
31. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*.
32. MinIO. (2024). *MinIO Object Storage Documentation*. Truy cập tại: <https://min.io/docs/>
33. Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.
34. Next.js Documentation. (2024). *Next.js Documentation*. Truy cập tại: <https://nextjs.org/docs>

35. Nguyen, D. Q., & Nguyen, A. T. (2020). PhoBERT: Pre-trained Language Models for Vietnamese. *Findings of the Association for Computational Linguistics: EMNLP 2020*, 1037-1042.
36. OASIS. (2012). *Advanced Message Queuing Protocol (AMQP) Version 1.0*. OASIS Standard.
37. Object Management Group. (2017). *Unified Modeling Language (UML) Specification Version 2.5.1*.
38. Pang, B., & Lee, L. (2008). Opinion Mining and Sentiment Analysis. *Foundations and Trends in Information Retrieval*, 2(1-2), 1-135.
39. Playwright Documentation. (2024). *Playwright Documentation*. Truy cập tại: <https://playwright.dev/docs/intro>
40. Pontiki, M., Galanis, D., Pavlopoulos, J., Papageorgiou, H., Androutsopoulos, I., & Manandhar, S. (2014). SemEval-2014 Task 4: Aspect Based Sentiment Analysis. *Proceedings of the 8th International Workshop on Semantic Evaluation*, 27-35.
41. PostgreSQL Documentation. (2024). *PostgreSQL 16 Documentation*. Truy cập tại: <https://www.postgresql.org/docs/16/>
42. Qdrant Documentation. (2024). *Qdrant Vector Database Documentation*. Truy cập tại: <https://qdrant.tech/documentation/>
43. Pressman, R. S., & Maxim, B. R. (2014). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.
44. PyTorch. (2024). *PyTorch Documentation*. Truy cập tại: <https://pytorch.org/docs/>
45. RabbitMQ Documentation. (2024). *RabbitMQ Tutorials*. Truy cập tại: <https://www.rabbitmq.com/tutorials>
46. Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust Speech Recognition via Large-Scale Weak Supervision. *Proceedings of the 40th International Conference on Machine Learning*, 28492-28518.
47. Redis Documentation. (2024). *Redis Pub/Sub*. Truy cập tại: <https://redis.io/docs/interact/pubsub/>
48. Traefik Labs. (2024). *Traefik Proxy Documentation*. Truy cập tại: <https://doc.traefik.io/traefik/>
49. Reputa. (2024). *Reputa - Giải pháp giám sát và phân tích thông tin trên môi trường mạng*. Truy cập tại: <https://reputa.vn/>
50. Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

51. SQLBoiler. (2024). *SQLBoiler - A Go ORM Generator*. Truy cập tại: <https://github.com/volatiletech/sqlboiler>
52. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson Education.
53. Sprinklr. (2024). *Sprinklr - Unified Customer Experience Management Platform*. Truy cập tại: <https://www.sprinklr.com/>
54. Talkwalker. (2024). *Talkwalker - Consumer Intelligence Platform*. Truy cập tại: <https://www.talkwalker.com/>
55. TikTok for Developers. (2024). *TikTok API Documentation*. Truy cập tại: <https://developers.tiktok.com/>
56. VnCoreNLP. (2024). *VnCoreNLP: A Vietnamese Natural Language Processing Toolkit*. Truy cập tại: <https://github.com/vncorenlp/VnCoreNLP>
57. YouNet Media. (2024). *SocialHeat - Social Listening Platform*. Truy cập tại: <https://younetmedia.com/>
58. YouScan. (2024). *YouScan - AI-Powered Social Listening*. Truy cập tại: <https://youscan.io/>
59. YouTube Data API. (2024). *YouTube Data API v3 Documentation*. Truy cập tại: <https://developers.google.com/youtube/v3>

CHƯƠNG 9: PHỤ LỤC

Phụ lục A: Danh sách từ viết tắt

Từ viết tắt	Nghĩa đầy đủ
ABSA	Aspect-Based Sentiment Analysis
AC	Architecture Characteristic
ACID	Atomicity, Consistency, Isolation, Durability
ADR	Architecture Decision Record
AES	Advanced Encryption Standard
AI	Artificial Intelligence
AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
ASR	Automatic Speech Recognition
BI	Business Intelligence
BPMN	Business Process Model and Notation
BR	Business Rule
BSI	Buzzmetrics Social Index
C4	Context, Container, Component, Code (Model)
CDN	Content Delivery Network
CI/CD	Continuous Integration/Continuous Deployment
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
DAO	Data Access Object
DDD	Domain-Driven Design
DNS	Domain Name System
DRY	Don't Repeat Yourself
DTO	Data Transfer Object
ERD	Entity Relationship Diagram
ETL	Extract, Transform, Load
FFmpeg	Fast Forward Moving Picture Experts Group

FR	Functional Requirement
GDPR	General Data Protection Regulation
GIN	Generalized Inverted Index
gRPC	Google Remote Procedure Call
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IAM	Identity and Access Management
I/O	Input/Output
IP	Internet Protocol
JSON	JavaScript Object Notation
JSONB	Binary JSON (PostgreSQL format)
JWT	JSON Web Token
K8s	Kubernetes
KISS	Keep It Simple, Stupid
KPI	Key Performance Indicator
MinIO	Minimal Object Storage
ML	Machine Learning
MVC	Model-View-Controller
MVCC	Multi-Version Concurrency Control
NFR	Non-Functional Requirement
NLP	Natural Language Processing
NoSQL	Not Only SQL
NSS	Net Sentiment Score
OAuth	Open Authorization
OLAP	Online Analytical Processing
OOM	Out of Memory
ORM	Object-Relational Mapping
PDPA	Personal Data Protection Act
PII	Personally Identifiable Information
PM	Project Manager

PostgreSQL	PostgreSQL Database
RabbitMQ	Message Queue System (AMQP)
RAM	Random Access Memory
RBAC	Role-Based Access Control
RDBMS	Relational Database Management System
RDB	Redis Database (snapshot)
Redis	Remote Dictionary Server
REST	Representational State Transfer
RFC	Request for Comments
ROI	Return on Investment
S3	Simple Storage Service
SaaS	Software as a Service
SDD	Software Design Document
SMAP	Social Media Analytics Platform
SOLID	Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
SoV	Share of Voice
SQL	Structured Query Language
SRS	Software Requirements Specification
SSE	Server-Sent Events
SSL	Secure Sockets Layer
SSOT	Single Source of Truth
SSR	Server-Side Rendering
TCP	Transmission Control Protocol
TDD	Test-Driven Development
TLS	Transport Layer Security
TTI	Time-to-Insight
TTL	Time to Live
UC	Use Case
UI	User Interface
UML	Unified Modeling Language

URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
UX	User Experience
WebSocket	Web Socket Protocol

Phụ lục B: Bảng thuật ngữ

Thuật ngữ	Định nghĩa
Aspect Extraction	Trích xuất các khía cạnh được đề cập trong nội dung văn bản, ví dụ như giá cả, chất lượng, dịch vụ.
Batch Processing	Xử lý dữ liệu theo lô thay vì từng item riêng lẻ để tối ưu throughput và giảm overhead.
Bounded Context	Phạm vi trong đó các thuật ngữ nghiệp vụ được hiểu nhất quán, là đơn vị cơ bản trong Domain-Driven Design.
Captcha	Cơ chế xác thực người dùng thật của các platforms để chống bot tự động.
Circuit Breaker	Pattern ngăn chặn cascade failures bằng cách tạm dừng gọi service khi phát hiện lỗi liên tục.
Claim Check Pattern	Pattern tách payload lớn khỏi message, lưu vào object storage và chỉ gửi reference qua message queue.
Clean Architecture	Kiến trúc phần mềm tách biệt business logic khỏi infrastructure thông qua các layers và dependency inversion.
Core Domain	Miền nghiệp vụ trọng tâm tạo ra giá trị cốt lõi và lợi thế cạnh tranh của hệ thống.
Crawler Worker	Thành phần thu thập dữ liệu từ các nền tảng mạng xã hội thông qua web scraping hoặc API.
Crisis Detection	Phát hiện khủng hoảng truyền thông dựa trên sự tăng trưởng đột biến của thảo luận tiêu cực.
Dead Letter Queue	Hàng đợi lưu trữ các messages không thể xử lý được để phân tích và retry sau.
Dependency Inversion	Nguyên tắc thiết kế trong đó modules cấp cao không phụ thuộc vào modules cấp thấp, cả hai phụ thuộc vào abstractions.

Distributed State	Quản lý trạng thái phân tán giữa nhiều services sử dụng cache tập trung như Redis.
Dry-run	Chức năng kiểm tra chất lượng từ khóa bằng cách thu thập mẫu kết quả trước khi chạy project chính thức.
Event-Driven Architecture	Kiến trúc trong đó các services giao tiếp thông qua events qua message broker thay vì gọi trực tiếp.
Eventual Consistency	Mô hình nhất quán dữ liệu trong đó hệ thống đảm bảo dữ liệu sẽ nhất quán sau một khoảng thời gian.
Fanout Exchange	Loại exchange trong RabbitMQ phát sóng messages đến tất cả queues được gắn kết.
Fault Isolation	Khả năng cô lập lỗi để lỗi ở một service không ảnh hưởng đến các services khác.
Generic Domain	Miền nghiệp vụ phổ biến có thể sử dụng giải pháp có sẵn hoặc dịch vụ bên thứ ba.
Health Check	Endpoint kiểm tra trạng thái hoạt động của service, bao gồm liveness và readiness probes.
Hexagonal Architecture	Kiến trúc phần mềm sử dụng ports và adapters để tách biệt business logic khỏi infrastructure.
High Cohesion	Nguyên tắc thiết kế trong đó các thành phần trong một module có mối liên hệ mạnh mẽ với nhau.
Horizontal Scaling	Mở rộng hệ thống bằng cách thêm nhiều instances thay vì nâng cấp hardware của instance hiện tại.
Impact Score	Điểm số đánh giá mức độ ảnh hưởng của nội dung dựa trên engagement và sentiment.
Intent Classification	Phân loại ý định của nội dung văn bản thành các categories như review, question, complaint.
Job ID	Mã định danh duy nhất cho mỗi tác vụ thu thập hoặc phân tích trong hệ thống.
Load Balancing	Phân phối tải công việc đều giữa nhiều workers hoặc instances để tối ưu hiệu năng.
Loose Coupling	Nguyên tắc thiết kế trong đó các services có sự phụ thuộc tối thiểu vào nhau.
Marketing Analyst	Người sử dụng chính của hệ thống SMAP để theo dõi danh tiếng thương hiệu và phân tích đối thủ.

Master-Worker Pattern	Pattern trong đó Master node phân phối tasks đến các Worker nodes để xử lý song song.
Message Acknowledgment	Cơ chế xác nhận message đã được xử lý thành công trước khi xóa khỏi queue.
Message Queue	Hàng đợi tin nhắn cho phép các services giao tiếp bất đồng bộ.
Microservices	Kiến trúc phần mềm chia hệ thống thành các services nhỏ, độc lập, dễ mở rộng và bảo trì.
Monolithic Architecture	Kiến trúc phần mềm trong đó toàn bộ ứng dụng được triển khai như một đơn vị duy nhất.
Net Sentiment Score	Điểm cảm xúc ròng tính bằng công thức $(\text{Positive} - \text{Negative}) / \text{Total}$.
NLP Pipeline	Chuỗi xử lý ngôn ngữ tự nhiên bao gồm preprocessing, intent, keyword, sentiment và impact.
Object Storage	Hệ thống lưu trữ dữ liệu phi cấu trúc dưới dạng objects với metadata, sử dụng MinIO.
Observability	Khả năng theo dõi, giám sát và debug hệ thống thông qua logs, metrics và traces.
Orchestrator	Thành phần điều phối và quản lý luồng xử lý giữa các components hoặc services.
Partial Result	Kết quả không đầy đủ do lỗi, timeout hoặc hủy giữa chừng trong quá trình xử lý.
PhoBERT	Mô hình ngôn ngữ pre-trained cho tiếng Việt dựa trên kiến trúc BERT.
Pipeline Pattern	Pattern xử lý dữ liệu qua chuỗi các bước tuần tự, mỗi bước là một module độc lập.
Playwright	Framework tự động hóa browser cho web scraping và testing.
Polyglot Persistence	Chiến lược sử dụng nhiều loại database khác nhau cho các use cases khác nhau.
Port and Adapter	Pattern tách biệt business logic khỏi infrastructure thông qua interfaces và implementations.
Project	Dự án theo dõi thương hiệu bao gồm cấu hình từ khóa, đối thủ, nền tảng và thời gian.
Pub/Sub	Mô hình publish-subscribe cho phép gửi thông báo real-time đến nhiều subscribers.

Rate Limit	Giới hạn số lượng requests trong một khoảng thời gian để tránh quá tải hệ thống.
Repository Pattern	Pattern trừu tượng hóa data access layer qua interfaces để tách biệt business logic khỏi storage.
Retention Window	Khoảng thời gian lưu trữ dữ liệu trước khi archive hoặc xóa.
Rolling Deployment	Chiến lược triển khai cập nhật từng instance một để giảm thiểu downtime.
Routing Key	Thuộc tính của message được exchange sử dụng để quyết định routing đến queue nào.
Sentiment Analysis	Phân tích cảm xúc của nội dung văn bản thành tích cực, tiêu cực hoặc trung tính.
Share of Voice	Tỷ lệ phần trăm thảo luận về thương hiệu so với tổng thảo luận trong ngành.
Skip Logic	Logic bỏ qua các bước xử lý tốn kém cho nội dung spam hoặc không liên quan.
Social Listening	Quá trình theo dõi và phân tích các cuộc thảo luận trên mạng xã hội về thương hiệu.
Social Media Platform	Nền tảng mạng xã hội như YouTube, TikTok, Facebook cung cấp dữ liệu công khai.
Soft Delete	Xóa logic bằng cách đánh dấu deleted_at thay vì xóa vật lý khỏi database.
Spike Detection	Phát hiện sự tăng đột biến trong volume thảo luận để cảnh báo khủng hoảng.
Stateless Service	Service không lưu trữ state nội bộ, cho phép scale và restart dễ dàng.
Strategy Pattern	Pattern cho phép chọn algorithm tại runtime thông qua interface chung.
Supporting Domain	Miền nghiệp vụ hỗ trợ cho Core Domain nhưng không phải yếu tố tạo nên sự khác biệt.
Throughput	Số lượng items được xử lý trong một đơn vị thời gian, thường đo bằng items/phút.
Trend Detection	Phát hiện xu hướng tự động dựa trên engagement và velocity của nội dung.

Web Scraping	Kỹ thuật thu thập dữ liệu từ websites bằng cách phân tích HTML và tự động hóa browser.
Webhook	Cơ chế callback HTTP cho phép service gửi thông báo đến service khác khi có sự kiện.
WebSocket	Giao thức cho phép giao tiếp hai chiều real-time giữa client và server.
Worker	Instance xử lý tasks từ queue, có thể scale độc lập theo workload.

Phụ lục C: Bảng quá trình làm việc

STT	Tên công việc	Thời gian	Người thực hiện
Giai đoạn 1: Khảo sát và phân tích yêu cầu			
1	Khảo sát nhu cầu và thị trường Social Listening tại Việt Nam	15/08 - 22/08	Cả nhóm
2	Phân tích các hệ thống liên quan	22/08 - 29/08	Cả nhóm
3	Xác định actors và stakeholders của hệ thống	29/08 - 01/09	Cả nhóm
4	Thu thập và phân tích yêu cầu chức năng	01/09 - 05/09	Cả nhóm
5	Thu thập và phân tích yêu cầu phi chức năng	05/09 - 10/09	Cả nhóm
6	Đặc tả Use Cases	10/09 - 15/09	Cả nhóm
Giai đoạn 2: Thiết kế kiến trúc hệ thống			
7	Nghiên cứu và lựa chọn phong cách kiến trúc	15/09 - 20/09	Cả nhóm
8	Thiết kế nguyên tắc và phương pháp tiếp cận	20/09 - 23/09	Cả nhóm
9	Phân rã hệ thống theo Domain-Driven Design	23/09 - 27/09	Cả nhóm
10	Thiết kế System Context Diagram	27/09 - 30/09	Cả nhóm
11	Thiết kế Container Diagram	30/09 - 03/10	Cả nhóm
12	Lựa chọn và đánh giá Technology Stack	03/10 - 06/10	Cả nhóm
Giai đoạn 2.5: Thiết kế Activity Diagrams			
13	Thiết kế Sơ đồ hoạt động tổng quát	06/10 - 07/10	Nguyễn Chánh Tín

14	Thiết kế Activity Diagram cho UC-01	07/10 - 08/10	Nguyễn Chánh Tín
15	Thiết kế Activity Diagram cho UC-02	08/10 - 09/10	Nguyễn Chánh Tín
16	Thiết kế Activity Diagram cho UC-03	09/10 - 10/10	Nguyễn Chánh Tín
17	Thiết kế Activity Diagram cho UC-04	10/10 - 11/10	Nguyễn Chánh Tín
18	Thiết kế Activity Diagram cho UC-05	11/10 - 12/10	Nguyễn Chánh Tín
19	Hoàn thiện và chuẩn hóa bộ Activity Diagrams	12/10 - 13/10	Nguyễn Chánh Tín
Giai đoạn 3: Thiết kế chi tiết các dịch vụ			
20	Thiết kế Component Diagram cho Ingest Service	13/10 - 16/10	Đặng Quốc Phong
21	Thiết kế Component Diagram cho Analytics Service	16/10 - 19/10	Nguyễn Tấn Tài
22	Thiết kế Component Diagram cho Project Service	19/10 - 21/10	Đặng Quốc Phong
23	Thiết kế Component Diagram cho Identity Service	21/10 - 23/10	Nguyễn Tấn Tài
24	Thiết kế Component Diagram cho Notification Service	23/10 - 25/10	Đặng Quốc Phong
25	Thiết kế Component Diagram cho Knowledge Service	25/10 - 27/10	Nguyễn Tấn Tài
26	Thiết kế Component Diagram cho Frontend Application	27/10 - 30/10	Nguyễn Chánh Tín
Giai đoạn 4: Thiết kế cơ sở dữ liệu			
27	Thiết kế chiến lược Database	30/10 - 01/11	Nguyễn Tấn Tài
28	Thiết kế ERD cho Identity Service	01/11 - 02/11	Nguyễn Tấn Tài
29	Thiết kế ERD cho Project Service	02/11 - 03/11	Đặng Quốc Phong
30	Thiết kế ERD cho Analytics Service	03/11 - 04/11	Nguyễn Tấn Tài
31	Thiết kế ERD cho Ingest Service	04/11 - 05/11	Đặng Quốc Phong
32	Thiết kế Data Management Patterns	05/11 - 06/11	Nguyễn Tấn Tài
Giai đoạn 5: Thiết kế Sequence Diagrams			

33	Thiết kế Sequence Diagram cho UC-01	06/11 - 08/11	Đặng Quốc Phong
34	Thiết kế Sequence Diagram cho UC-02	08/11 - 10/11	Nguyễn Tấn Tài
35	Thiết kế Sequence Diagram cho UC-03	10/11 - 12/11	Đặng Quốc Phong
36	Thiết kế Sequence Diagram cho UC-04	12/11 - 14/11	Nguyễn Tấn Tài
37	Thiết kế Sequence Diagram cho UC-05	14/11 - 16/11	Đặng Quốc Phong
38	Tinh chỉnh sequence cho execution và analytics lanes	16/11 - 18/11	Nguyễn Tấn Tài
39	Tinh chỉnh sequence cho retrieval và notification lanes	18/11 - 20/11	Đặng Quốc Phong
Giai đoạn 6: Thiết kế Communication Patterns			
40	Thiết kế RabbitMQ Topology	20/11 - 22/11	Nguyễn Tấn Tài
41	Thiết kế Event Schemas và Event Catalog	22/11 - 24/11	Đặng Quốc Phong
42	Thiết kế REST API Design Principles	24/11 - 26/11	Nguyễn Tấn Tài
43	Thiết kế Real-time Communication	26/11 - 28/11	Đặng Quốc Phong
Giai đoạn 7: Traceability và Validation			
44	Xây dựng Traceability Matrix	28/11 - 01/12	Nguyễn Tấn Tài
45	Viết Architecture Decision Records	01/12 - 03/12	Nguyễn Tấn Tài
46	Phân tích Gaps và Mitigation Plans	03/12 - 05/12	Nguyễn Tấn Tài
Giai đoạn 8: Hiện thực hệ thống - song song với thiết kế			
47	Hiện thực Identity Service	15/09 - 30/09	Nguyễn Tấn Tài
48	Hiện thực Project Service	01/10 - 15/10	Đặng Quốc Phong
49	Hiện thực Ingest Service	15/10 - 31/10	Đặng Quốc Phong
50	Hiện thực Analytics Service	01/10 - 31/10	Nguyễn Tấn Tài
51	Hiện thực Scraper Worker Service	15/09 - 31/10	Đặng Quốc Phong
52	Hiện thực Knowledge Service	01/11 - 15/11	Nguyễn Tấn Tài
53	Hiện thực Notification Service	15/11 - 25/11	Đặng Quốc Phong
54	Hiện thực Web UI	01/10 - 30/11	Nguyễn Chánh Tín
55	Tích hợp và chạy thử MVP	01/12 - 10/12	Cả nhóm

Giai đoạn 9: Tháng 1/2026 - Khởi tạo repository và chuẩn hóa tài liệu nền			
56	Tách cấu trúc thành các repository/ service độc lập và chuẩn hóa hướng dẫn quản lý source (report: 0330108, 04260c8; ingest-srv: 554d19f)	11/01 - 25/01	Cả nhóm
57	Cập nhật sequence diagrams, đường dẫn hình ảnh và README diagram cho báo cáo kiến trúc (report: 2de68ca, e87418a, ab90a08)	11/01 - 26/01	Cả nhóm
Giai đoạn 10: Tháng 2/2026 - Xây dựng lõi service và contract dữ liệu			
58	Hiện thực nền tảng Backend cho Identity Service, JWT, token blacklist, internal admin và security enhancements (identity-srv: 570fedd, 4b3606c, 24aaf73)	09/02 - 15/02	Đặng Quốc Phong
59	Hiện thực Project-Campaign hierarchy, refactor package convention và Kafka consumer nền cho Project Service (project-srv: dde4514, 2e1359d, e0fa363)	18/02 - 19/02	Đặng Quốc Phong
60	Thiết kế contract UAP, data flow, ingest API specification và project/data source hierarchy (report: 3fafd06, eeaff7d, 5bcbf9e, b2913ae)	08/02 - 20/02	Đặng Quốc Phong
61	Hiện thực Analysis Service clean architecture, RabbitMQ/Kafka consumer, NLP domains, UAP và post_insight pipeline (analysis-srv: 86f6ea2, 2ad5b97, f0da7ce, b4448fc)	11/02 - 19/02	Nguyễn Tấn Tài
62	Hiện thực Knowledge Service với MinIO, Qdrant, Voyage/Gemini, indexing, semantic search, chat và report generation (knowledge-srv: 576478e, d5e6808, 049c817, ba95eb2, 46f43bc)	15/02 - 17/02	Nguyễn Tấn Tài
63	Hiện thực Notification Service WebSocket, alert dispatching và notification event contract (notification- srv: adaac4f, e5ec7d7, e7ab7d3, a094254)	17/02	Nguyễn Tấn Tài

Giai đoạn 11: Tháng 3/2026 - Tích hợp runtime, tracing và kiểm thử UI/API			
64	Hoàn thiện Ingest Service với dry-run, cron job, execution task, UAP parser và Kafka producer (ingest-srv: 03d1255, 69ceb62, da32036, 8c006d3)	06/03 - 08/03	Đặng Quốc Phong
65	Phát triển Scapper Service, RabbitMQ/MinIO dev mode, completion routing, YouTube logic và SDK release (scapper-srv: 96b7702, a8669d3, 7e3917f, b3ef730)	04/03 - 24/03	Đặng Quốc Phong
66	Chuẩn hóa shared-librs cho tracing, logging, HTTP/Kafka/Postgres clients, auth middleware và internal service auth (shared-librs: 3e6cdb4, cfe40ac, 891be8e, 350081f)	12/03 - 16/03	Đặng Quốc Phong
67	Docker hóa, tracing, JSON logging và cấu hình vận hành cho Analysis/Knowledge/Notification services (analysis-srv: 52e5539; knowledge-srv: ffc9a5c; notification-srv: d680afd)	10/03 - 16/03	Nguyễn Tấn Tài
68	Tích hợp NotebookLM automation, dynamic Qdrant collection, knowledge contracts và demo analyse CLI (knowledge-srv: 99c6ec0, 6fc03f4; smap-analyse: 2964617, 558892c)	24/03 - 31/03	Nguyễn Tấn Tài
69	Xây dựng UI test, kết nối prod API, các màn hình Knowledge Chat, Search, AI Reports, Scraper Task và WebSocket notifications (report: 81a6c31, 1140824, 66a9649)	12/03 - 18/03	Nguyễn Chánh Tín
70	Thiết lập deployment manifests, Fluent Bit, log viewer, Discord bot và pipeline triển khai ban đầu (smap-deploy: 91bfb73, a64e509, 21617eb)	10/03 - 12/03	Đặng Quốc Phong
Giai đoạn 12: Tháng 4/2026 - Hoàn thiện nghiệp vụ, UI và báo cáo chính thức			
71	Hoàn thiện lifecycle datasource/crawl target, UAP parser TikTok/YouTube/	08/04 - 14/04	Đặng Quốc Phong

	Facebook, dry-run guard và readiness contract (ingest-srv: 9c61837, 5489683, 8df317e, 3e6148e)		
72	Hoàn thiện Project Service với domain registry, lifecycle enum, internal campaign endpoint và HTTP 404 contract (project-srv: a149461, 34fc7fe, 343ca15, 43c22c5)	08/04 - 14/04	Đặng Quốc Phong
73	Chuẩn hóa Identity Service schema, middleware, Swagger gateway-aware BasePath và shared auth dependency (identity-srv: e814a1e, 5a6a40c, 90be4bc, c326826)	13/04 - 21/04	Đặng Quốc Phong
74	Tích hợp Analysis new core, ontology/ domain registry, Redis publication, batch processing, ONNX và language-filtered NLP flow (analysis-srv: 5b66695, d770dc0, 8eec279, 7b69688, b688b35)	08/04 - 23/04	Nguyễn Tấn Tài
75	Hoàn thiện Knowledge Service với multi-provider LLM, Qdrant per-project collections, search/chat bugfix, parallel LLM và citation quality (knowledge-srv: a3c296c, 0973c79, 606b99e, 3bcb0da, 7b7551e)	14/04 - 26/04	Nguyễn Tấn Tài
76	Ổn định Notification Service WebSocket sau Traefik, shared-libs dependency và production middleware (notification-srv: 50ecc47, 62f1cd6, ecede9e)	14/04 - 21/04	Nguyễn Tấn Tài
77	Hiện thực Web UI authentication, scope filter, onboarding nhiều project, dashboard chart, proxy/middleware, dry-run flow và responsive fixes (web-ui: 5ac7b98, 092191a, 3f2ba03, a5ddfa5, 57bb171)	15/04 - 26/04	Nguyễn Chánh Tín
78	Tập trung manifests vào smap-deploy, cấu hình infra UI, Redpanda/ RabbitMQ, Metabase, E2E assets và rollout hardening (smap-deploy: 8ebac1e, 3ec495a, 97772c9, 4163c02, cb76a71)	13/04 - 26/04	Đặng Quốc Phong

79	Biên soạn báo cáo chính thức chương 0-6, C4/ERD/sequence diagrams, database strategy và kiểm thử tích hợp (report: 7503206, 554e453, 597bc8b, ce57cd2, 981605b)	15/04 - 30/04	Cả nhóm
Giai đoạn 13: Tháng 5/2026 - Kiểm thử cuối, polish và triển khai bản hoàn thiện			
80	Bổ sung datasource lifecycle runtime và crawl source tagging cho Ingest Service (ingest-srv: bf8d629, e5bc847)	05/05 - 09/05	Đặng Quốc Phong
81	Bổ sung crisis response policy, project states, ontology rules configuration và access control/login domain (project-srv: 9910409, a48bcea; identity-srv: 1221155, 7a60d8d)	09/05 - 10/05	Đặng Quốc Phong
82	Tối ưu analytics crisis runtime, source filter, query performance, source link preservation và giảm nhiễu Redis log (analysis-srv: 211a463, 33a02e2, 09aa0d2, 7a35851)	09/05 - 10/05	Nguyễn Tấn Tài
83	Phục vụ reports từ Knowledge Service, cải thiện search relevance/citations và bridge crisis alerts sang realtime notifications (knowledge-srv: e29c688; notification-srv: 6468f61, 4acbeef, f886ed0)	09/05 - 10/05	Nguyễn Tấn Tài
84	Hoàn thiện SMAP dashboard workflows, source filters, post/report filtering, auth cookie và notification socket stability (web-ui: 73b3b7d, 0367922, 1f28347, c35eba4)	09/05 - 10/05	Nguyễn Chánh Tín
85	Cập nhật scapper handlers, SDK surface, local RabbitMQ/MinIO infrastructure và rollout service images/ source filters (scapper-srv: ad99ab7, caa0202, 9639a69; smap-deploy: ae544ec, 627ad34, 7b7e34b)	05/05 - 10/05	Đặng Quốc Phong

86	Hoàn thiện chương 5-6, rà soát evidence, bổ sung review findings và chỉnh sửa final-report sau kiểm tra chéo source code (report: 559d8bb, 8dc0eb6, 5d19007, d1db2aa)	01/05 - 10/05	Cả nhóm
----	---	---------------	---------